



The Open
University

TM354 Software engineering

Block 2 Units 5–8

From analysis to design

This publication forms part of the Open University module TM354 *Software engineering*. Details of this and other Open University modules can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom (tel. +44 (0)845 300 60 90; email general-enquiries@open.ac.uk).

Alternatively, you may visit the Open University website at www.open.ac.uk where you can learn more about the wide range of modules and packs offered at all levels by The Open University.

To purchase a selection of Open University materials visit www.ouw.co.uk, or contact Open University Worldwide, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a catalogue (tel. +44 (0)1908 858779; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk).

The Open University, Walton Hall, Milton Keynes MK7 6AA

First published 2014.

Copyright © 2014 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS (website www.cla.co.uk).

Open University materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 1 7800 7916 5

1.1

Contents

Unit 5	
Structural modelling	I
Unit 6	
Dynamic modelling – from analysis to design	73
Unit 7	
More on dynamic modelling – states and activities	139
Unit 8	
The case study: part 2	189
Index	249

Unit 5 Structural modelling

Contents

1	Introduction	5
2	Structural models	6
3	Domain structural models	8
3.1	The search for concepts	8
3.2	Classes and properties	11
3.3	Classes and objects	11
3.4	Classes and time	11
3.5	Class and object models	13
3.6	Object diagrams	13
3.7	Class diagrams	15
3.8	Identity	16
3.9	Attributes	18
3.10	Associations	18
3.11	The hotel chain revisited	27
3.12	Summary of section	34
4	From domain to analysis models	35
4.1	Interpreting attributes	35
4.2	The importance of time	36
4.3	Aggregation and composition	38
4.4	Navigability	39
4.5	Qualified associations	40
4.6	Attributes on associations	42
4.7	Derived associations	42
4.8	Summary of section	45
5	Generalisation and specialisation	46
5.1	Superclasses and subclasses	47
5.2	Interfaces	48
5.3	Levels of specialisation	49
5.4	Specialisation and substitutability	50
5.5	Summary of section	55
6	Constraining models	56
6.1	Constraints on classes	56
6.2	Constraints across associations	58
6.3	Constraints on related objects	59
6.4	Finding invariants by considering loops in associations	61
6.5	OCL summary	68

	6.6 Summary of section	69
7	Summary	71
	References	72

Introduction

So far we have looked at a business context (business processes and business rules) where a problem requires a software solution, and we discussed requirements for such solutions.

In this unit we look at the structure, first of the business context and then of the solution. This structure will be represented in class models – the conceptual model and then the analysis model. The conceptual model represents elements of the problem domain, the analysis model elements of a solution.

Some of the material that you will encounter in this unit may be familiar to you. We will identify the most basic, introductory material that you may want to skip over.

2

Structural models

This section introduces the main modelling elements that you will find in a **domain structural model**, also known as the **conceptual model**, which presents a view of the static structure of the problem domain. If you start by looking for the objects in the problem domain, then when you move to a software model for that domain you are more likely to design a system whose components remain useful as the problem domain evolves. With an understanding of the domain and its structure, you can build a structural model that is an acceptable solution to the problem posed by the requirements – this is called an **analysis structural model**, or simply an **analysis model** (discussed in Section 3). The main difference between the conceptual and the analysis models is that the former represents entities from the domain and the latter entities from the software solution. Both are class models describing the objects and the relationships between them.

The objects identified for the software solution, in the analysis model, will ultimately be responsible for the intended behaviour of the system, which has so far been described through use cases and detailed software requirements. It is therefore important to find the right objects and relationships between them. The analysis model will itself evolve into a **design model** when decisions are taken about how this structure fulfils the required behaviour of the system. Design will be discussed from Unit 6 onwards.

Building the structural models, also known as class modelling, goes hand in hand with use case modelling and they ‘feed off’ each other. You might choose to consider what a conceptual model should look like first, so that you can write use cases with a firm idea of what sort of relationships between classes are expressible. Alternatively you may start by understanding the behaviour of the processes in the domain and from them decide on the use cases for the system. In this case, use case modelling comes first and textual descriptions of use cases form the basis of drafting the conceptual and analysis models. There is, of course, no single ‘correct’ approach. In the early stages of analysis, you may swap between structural and use case modelling as you begin to understand more about the domain and the problem that your project intends to solve.

Models are always built for a purpose. You cannot just examine a model and judge whether it is good or bad. You need to establish whether the model represents what is required of it. The way you create a model to represent a person, for example, will depend on what you want to do with the information about the person. A medical system and a hotel system have very different concerns: the former might model blood groups, the latter smoking preferences. These two systems therefore require different models for the concept of a person. The question is whether the models are adequate for the purpose. This can only be answered by someone who understands the domain being modelled, for example a hospital administrator or a hotel manager. A **domain expert** understands the relevant part of the application domain rather than the technology used to automate it. Without a domain expert, your best chance of identifying a good concept to include in the model comes from an

understanding of the problem domain gained from discussions with the potential users.

Models are built for communication. Even if structural models are mostly diagrammatic, it is not always the case that they mean the same to everyone. It is therefore important to make sure they are clear to the intended audience and that the rationale behind a model is well accepted.

In this unit you will learn the detail of the techniques used in building structural models. However you should always keep in mind that what is important is to create a model that is comprehensible and that represents a shared understanding. The agile modelling principles are particularly relevant when thinking about how much precision and correctness should go into drawing diagrams. They stress that modelling is not the ultimate goal, software is (Ambler, 2002).

- Lightness and simplicity take precedence over completeness and complexity.
- Models should only be pursued while they are of use.
- Ease of change and adaptability also need to be part of the modelling process.

Naturally adapting to be more or less agile requires experience and good understanding of the techniques used.

3

Domain structural models

If you are familiar with class models, their characteristics and how to build them, you may want to skip this section. We advise however that you read it quickly but pay more attention to some aspects that may not be familiar to you, such as those in subsections 3.4, 3.8 and 3.11.

3.1 The search for concepts

The first step in building a conceptual model is to identify objects in the problem domain that may eventually be relevant to a solution. However identifying objects is not an easy task. A number of techniques have been suggested that address the problem but none of them should be applied mechanically.

Which objects should you look for and where can you find them? In Block 1 Unit 2, we identified the term *requirements elicitation* for the source of the requirements. Robertson and Robertson (2012) use the term *trawling* as an analogy to fishing with a net, catching some but maybe not all. In effect, you search through all the documents, forms and so on supplied by the customer and possibly talk with the customer as well. The following categories are useful sources of relevant objects:

- **tangible objects** – the physical things in the domain, such as rooms, bills, books and vehicles
- **roles** – the roles played by people in the domain, such as employees, guests and members of some organisation
- **business transactions** – the activities, episodes and interactions, such as room reservations, vehicle registrations, orders, deliveries and transactions
- **organisational units** – the groups to which people belong, such as accounts departments, production teams and maintenance crews.

Use these four categories for guidance. They are pointers to alert you to the objects or entities that might result in appropriate concepts for your models. It is worth stressing again that there is no guidebook or recipe for producing a model, nor is there an ideal model. Object-oriented analysis does not differ in this respect from other methods for software development.

Sometimes it is hard to decide which category is the right ‘home’ for an object or concept; you will see an example of this later in this section when we discuss attributes. The reason behind this difficulty is the potential for different interpretations of the same concept. Which interpretation is appropriate will usually depend on the particular circumstances. Furthermore, how you model a concept and its properties is affected by what you believe that concept represents and what its role is in the problem domain.

A document that needs to be created at the beginning of a project is an agreed **glossary** of terms, or a data dictionary. It cross-references the different words that are to be used on the project and explains how they interrelate – particularly important for establishing your understanding of key concepts in

the problem domain. For instance, what is a *sale*? Does it come into existence when hands are shaken, or when documents are signed, or when goods are delivered, or when payment is received? Where are the goods produced? Who delivers the goods? Is a payment to be made in one sum, or is it staged according to delivery? There will be many terms and phrases used in the problem domain, the majority of which will have some local meaning. Without the agreement on vocabulary that a glossary gives there is little hope of building a good model, where the meanings of the concepts need to be clear.

In this module you have seen that requirements can be expressed in the form of use cases or triggered by user stories. While looking for the use cases and/or user stories, you could simultaneously search through the different texts to find the objects that indicate candidate concepts.

A list of nouns from a written description of a problem makes an excellent starting point when considering candidate concepts for a conceptual model. The process of obtaining this list is known as a **grammatical parse**. However the list may initially contain many nouns and noun phrases that will not be appropriate. At some point you need to decide which ones to keep and which to discard. We suggest that you use the following guidelines.

- The same concept is often given different names and you should identify those that are redundant. For example, the terms ‘account’, ‘posting code’ and ‘ledger’ might all turn out to have the same meaning.
- The same name might be used for different concepts so you should rename them appropriately. Often a single noun has several distinct meanings in different parts of an organisation. In different contexts, ‘policy’ might mean a piece of paper, a legal agreement or a person’s insurance cover.
- Many names are not important or independent enough to be considered as concepts on their own. Some of these may be used as properties of other identified concepts or as a concept that connects two other concepts.
- Exclude names that refer to concepts that are not relevant to the situation being considered. They are outside the scope of the problem that you are trying to understand. This sounds obvious, but it can often be difficult to tell. A list of candidate concepts for a petrol station system is likely to include ‘customer’. At a later stage in modelling, it might become apparent that the system need not record information about the customers as individuals. The system is concerned with the fact that petrol was pumped and money was received. The unifying concept is probably something similar to a *PurchaseTransaction*, which involves dispensing and paying for the petrol.

Sometimes you may find that you cannot determine exactly what a particular word or phrase means. In such cases you must establish a clear definition with the domain experts. It is not your job as a modeller to tell domain experts how to describe their subject, but you must be wary of the complexities of their language.

Because you are not considering the behaviour of the domain, but just the things involved, you may be tempted to exclude some or all of the events

We use the convention of a single word for concepts. The name of a concept can be made by concatenating several words with initials capitalised.

(and operations) that you have found. However the presence of a name of an event – such as ‘marry’ or ‘reserve’, which are verbs – can be a useful clue to a thing about which something needs to be said. For instance, if you wish to record the date when someone marries, you might introduce a concept named *Marriage* (a noun). If you need to record when a room becomes reserved, rather than whether or not it exists, you might introduce a concept named *Reservation*. When there is relevant information associated with an event that may change with time, this may suggest the need to represent such event as a concept.

The easiest concepts to find are usually those that represent tangible, real-world entities, such as *Book* (in a library context) and *Room* (in a hotel context). Moving slightly deeper into the problem, the different roles in a domain may provide more ideas. A *Person* may at different times have the behaviour of a *Parent*, an *Employee*, a *Customer* and a *LibraryMember*, and so it may be useful to keep these role classes separate from *Person*.

Concepts that are easy to overlook are those that represent a transaction over a long period. For example *MortgageApplication* might be a useful concept that could represent the state of a complex interaction with a lifetime of months. This is a typical example of a concept corresponding to a business transaction – a concept that represents the corresponding business process (applying for a mortgage in this case) in its different stages.

When we looked at use cases as a way of representing requirements, we used the word actor for someone or something external to the system that can initiate interactions with it. In the case of a garage, the actors probably include the *Customer*, the *Cashier* and the *Manager*. When modelling the domain, you are likely to include concepts for each of these actors. However it is often not clear whether actors need to be represented in a software system. If the cashiers are merely the agents for accepting money and recording the payment of transactions, the software system may not need to model them. On the other hand, if you need to record information about cashiers or the credentials that they need to provide in order to be authorised to use the system, such as the hours they work or which transactions they handle, they must be modelled.

SAQ 1

- (a) Why do object modellers concentrate on nouns?
- (b) What are the main criteria for filtering a list of nouns in order to remove inappropriate ones and settle upon a suitable set of candidate concepts?

Answer

- (a) The nouns represent the things in the domain being modelled, and things are more stable than actions, which are expressed as verbs.
- (b) There are basic filtering criteria that can be applied as follows:
 - different concepts that are expressed using the same name
 - redundancy, the same concept, given different names

- not important or independent enough, such as an attribute of another concept rather than a concept in its own right
- lack of relevance to the problem domain, being either beyond the scope of the desired system.

In addition to these basic criteria, you should pay particular attention to events. Will an instance of that kind of event have state, behaviour and identity that are significant in the problem domain? A loan of a book from a friend might not be worth modelling, for example, but a library loan is significant – it has a time limit and may incur a charge if that limit is exceeded.

In all cases, you should clear up any ambiguity with a domain expert or the users themselves.

3.2 Classes and properties

In a class model each concept is represented by a class. A class will usually have properties that are important to the concept it represents. For example, the name of a *Person* may be important, or you might want to associate the *Person* with the *Company* the person works for. You have worked with attributes of classes when programming in Java – attributes are examples of properties of classes that are intrinsically connected to the class, such as a person's name. Associations are examples of another kind of property, one that represents a relationship between classes. Attributes and associations will be discussed later in this section.

You have seen associations in Block 1 Unit 3. To reiterate, associations are relationships between two types, in this case between two classes.

3.3 Classes and objects

In object-oriented modelling, you must carefully distinguish between objects and classes. An object stands for something real in the problem domain, such as a specific customer or a specific room in a hotel. The properties of an object have values that can be tested. A class describes all possible objects of that kind, defining what the objects have in common. If you talk about the class *Person*, anything you say about the class is a general statement about all possible persons, such as *jack*, *jill* and *jenny*. Properties of a class do not have values, but instead indicate the type of values that instances of the class (that is, objects) may have. A class describes what is known about all objects of that class at all times.

3.4 Classes and time

Whether or not something in the world should be modelled as a distinct class may depend on what happens over time. You should consider the life histories of instances (objects) and how their behaviour may change over time.

Life histories of instances

A class is a template for a set of objects. If you ask questions about when the objects are created and when they are removed, you may uncover complexities that you have failed to model. In a model of a hotel, you might have a class called *Guest*. When are instances of the *Guest* class created? In domain terms, this is the same as asking when does a hotel know about an instance of *Guest*? It might seem that the hotel should know about an instance when checking in a newly arrived person. That is, a *Guest* object would not exist unless and until a guest has actually checked in. Such a model may or may not be a good representation of the world imagined by the hotel domain experts. If the hotel accepts reservations, you may wish to record the reservation against a person who has not yet checked in. Do you now want to say that a guest represents either a person who has checked in or someone who has made a reservation but has not yet checked in? If so, you should ask whether the entity staying in a room and the entity making a reservation are members of the same class. A company can make a reservation and pay the bill, but a company cannot stay in a room. Perhaps you should restrict the word guest to refer to a person who is currently staying in the hotel, and use the word reserver for an entity that makes a reservation. In that case, reservations would be made by reservers and rooms would be allocated to guests.

Similarly you might ask when a *Guest* object should be deleted from the system. One choice is that any guest who has checked out and paid the bill should no longer be known to the hotel. In some hotels this would be an adequate model, but in other cases the hotel would want to keep records relating to a guest, the room that was occupied and the dates of occupancy, in case of, for example, lost property or disputes over bills. Another complexity comes from the fact that sometimes the guest is not the entity that pays the bill. Again you might split the notion of guest into those aspects to do with being on the premises and those to do with contractual obligations.

Whatever the meaning of the class *Guest*, it is vital that you document it. Merely choosing a good descriptive noun like ‘guest’ to name a class is not enough. You must be precise about what set of instances it actually represents. This sort of explanation does not belong in a UML diagram but in the project glossary, which defines every term to be used. For example, you might put the following entry into the project glossary.

Guest: a *Person* who is occupying or has occupied a *Room* in the *Hotel*. See also *Reserver* and *Payer*.

Change of behaviour over time

When modelling, you should assume that an object cannot change its class during its lifetime. A *Room* cannot become a *Hotel*, and a *Guest* cannot become a *Reserver*. An instance is created as an instance of a particular class and remains an instance of that class until it is destroyed. This may seem obvious, but it is actually a very strong restriction. For example, when modelling a hospital you might conclude that you need classes called *Patient* and *Doctor*. There would be instances of both classes – doctors would treat

Note that we are assuming single class membership. In some approaches, objects can be instances of several classes.

patients and patients would have appointments with doctors. However if a doctor were to become ill, the doctor would need to become a patient. As *Patient* and *Doctor* are different classes, you could not change the status of an instance of *Doctor* to become an instance of *Patient*. Rather you would need to have a separate instance of the ill doctor as a *Patient*.

Another example is a school administration, where you may represent the classes *Teacher*, *Parent* and *Child*. A person can be both a parent and a teacher, and teachers can be parents. Eventually children grow up and might become parents, teachers or both. It may be possible to be all three! These possibilities show that you must think carefully about what real-world entity an object represents, and whether having more than one object for a single entity is a problem. Also, bear in mind that the more a system needs to know about an object, the more complex it becomes.

3.5 Class and object models

A **class model** indicates which classes are in the system and describes the associations between those classes. In the same way that a class describes what is true for all objects of that class, a class model describes what is true about the entire system at all times. An **object model** is a collection of objects, where every object in the model is an instance of a class in the class model. The object model represents one specific configuration of the system – a **snapshot** of the system at one instant in time. No object can have attributes that are not in the class model and neither can objects have relationships that are not in the class model. Just as an object is an instance of a class, an object model may be thought of as an instance of a class model.

As the software that you want to produce will be expressed in terms of classes, it is tempting to jump straight from a list of candidate classes to an attempt to build a class model. However at this point the classes are really just a representation of all possible objects. If you generalise too quickly you can easily overlook particular cases that do not fit the generalisation. For this reason, you may find it useful to construct some object models that show interesting snapshots.

These snapshots consist of a number of concrete instances of relevant candidate classes and the relationships between them. All of the eventual object models must be instances of the eventual class model, without exception.

An object model is visually represented using an **object diagram**. A class model is visually represented using a **class diagram**.

3.6 Object diagrams

As mentioned in subsection 3.5, an object diagram is a visual representation of an object model. Figure 1 shows an object diagram representing a snapshot of a hotel system. Each object is represented by a rectangular box. Inside each box is a name chosen for the object, followed by a colon and the name of a class of which the object is an instance.

In this module we will be using object models to illustrate some specific cases. In reality, object models may not be drawn even if they exist in the mind of the modeller.

We will continue to use underlining to identify objects in diagrams, but not in the text.

The convention is to write object names starting with lower case, to distinguish them from class names that start with upper case.

Object names, such as *theRitz : Hotel* in Figure 1, are underlined to distinguish them as individual instances (rather than classes).

You can add a second compartment to each box to include the names and values of any useful attributes. In Figure 1, for example, the object *r401 : Room* has an attribute called *enSuite* set to *true*.

Lines drawn between objects are called **links**. They represent particular cases of one object ‘knowing about’ another object. Each link is an instance of an association or a relationship between classes. Thus in Figure 1 the object *theRitz : Hotel* knows about three *Guest* objects and three *Room* objects. Two of the *Guest* objects know about specific *Room* objects: *jack* knows about *r123* and *jill* knows about *r307*. At this stage *jenny*, the third guest, is not linked to a room.

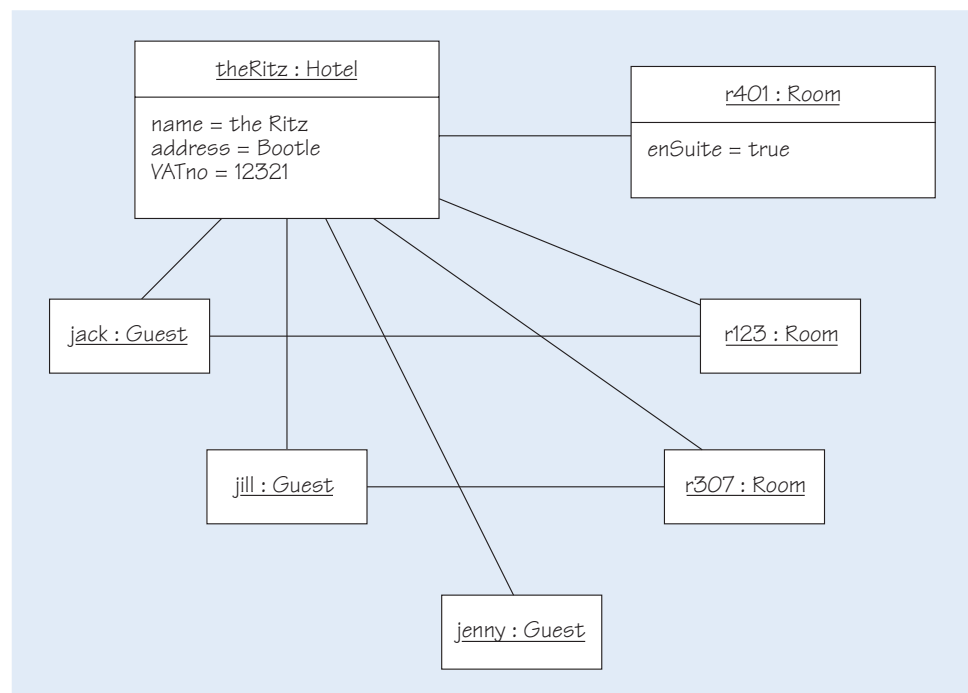


Figure 1 An object diagram

Object diagrams are extremely useful for checking your understanding of concepts and relationships. They are a fruitful source of special cases where requirements are not precisely clear and therefore require clarification from the domain expert and stakeholders. For example, you can use one or more of the object models to walk through the different scenarios for each use case.

Since an object diagram is a snapshot of the domain or the system at a particular moment in time, pairs of them can be useful in showing the domain or the system before and after some operation has been carried out. For example, in the hotel system you can use pairs of object diagrams to show how new *Guest* objects are created, and then show what happens when a guest checks in.

An object diagram can be very useful for determining whether some configuration of objects is valid. However, as noted above, it cannot express general structural properties. In Figure 1, it is not clear from the diagram whether the hotel always has exactly three guests and three rooms or whether this is simply the situation at the moment. A generalisation of all possible object diagrams is shown in a class model, which is visually represented by a class diagram.

For example, you can use object modelling to gain some understanding about the appropriate multiplicities for your class diagram. We will return to the subject of multiplicity in subsection 3.10.

SAQ 2

Explain why object diagrams cannot form the basis for a software specification.

Answer

Object diagrams represent particular states of the system at particular moments in time, whereas a specification must describe all valid states of the system at all possible times.

3.7 Class diagrams

Similarly to an object diagram, a class diagram consists of boxes representing classes and lines representing associations. However a box may contain up to three different parts, as opposed to the two parts found in an object diagram. It always contains a class name in the top section. It may have a second section containing attributes and a third section containing operations – this third section does not appear in object diagrams. A class diagram represents what all possible instances of the class have in common, rather than the particular values of any given instance. It acts as a template when creating instances, to ensure that the resultant objects all have the same structure and behaviour.

When you use a class diagram in a conceptual model, the main aim is to represent the information that exists in the problem domain. Hence the class diagram will show the key concepts as classes, their properties as attributes and their relationships as associations.

Figure 2 shows a class definition and some typical instances of it. The class defines the structure and behaviour that all the instances have in common. (Note that this class definition does not include a section containing operations – examples including operations are given in subsection 5.1.)

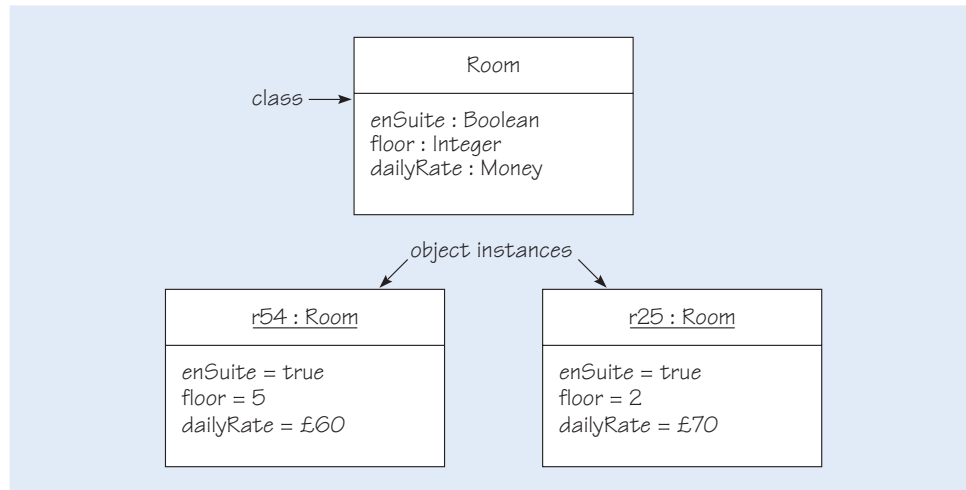


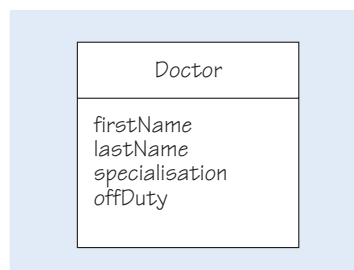
Figure 2 A class and some instances

3.8 Identity

One of the essential parts of the object-oriented way of looking at the world is the notion that every object is inherently unique and different from every other object. Every object has a unique identifier that does not depend on the current value of any of its attributes, or even on what class it is.

Consider the *Doctor* class shown in Figure 3. You can imagine numerous instances of such a class, each having its own values of the attributes. However there is no reason why you could not have two different instances that have identical values of all attributes. This situation would arise if two cardiac surgeons named John Wilson work at the same hospital. It would also arise if, for some reason, two objects were created for the same John Wilson. Even though all the attributes are identical, from the object point of view each instance is a distinct object, which may or may not be what is desired. The modeller faces two problems:

- providing some means of finding a desired object instance (for example, through an association)
- ensuring that enough information is included in the class to distinguish one object sufficiently from another.

Figure 3 The *Doctor* class

As an example, consider a patient–doctor relationship, where objects of the *Patient* class are linked with objects of the *Doctor* class. If the first case above – two objects each for a real-life doctor with first name John, second name Wilson and the same specialisation – were to occur, it would be difficult to distinguish one object from the other even if they represent two distinct people. In this case, the value seen for *offDuty* would depend on which instance was accessed. To permit the correct *Doctor* object to be associated with the patient, both problems described in the previous paragraph must be solved. The model, or perhaps the implementation, must include some way to find all the *Doctor* objects, and there must be enough information in the object to identify which real-life doctor the object stands for, such as for example an attribute with a unique identifier. You would also want, in most cases, to have a single object representing a single real-world entity.

SAQ 3

- (a) In a windowing system, a window may be converted to an icon and back to a full window. What operations can be performed on full windows but not on iconised windows? Would a model containing the classes *Icon* and *FullWindow* be able to capture the distinction adequately?
- (b) In connection with rooms, the hotel manager’s vocabulary includes the words ‘occupied’ and ‘free’. How might such words be represented in a class diagram?
- (c) In your model in part (b), will your decision about occupancy change if you have to include the fact that a room must be cleaned before the next guest occupies it?

Answer

- (a) Scrolling and maximising can be done on full windows but not on iconised ones. It would be difficult to model the distinction between a full window and its iconised version adequately by using two different classes, since an object (in this case the window) cannot dynamically change its class. One solution would be to have a single class with an attribute that distinguishes between a window being iconised or fully open.
- (b) Two ways come immediately to mind:
 - as an attribute of the *Room* class
 - as an association between the *Room* and *Guest* classes.
 Either is acceptable as a way of recording the information.
- (c) No. The cleaning of a room depends upon whether or not it is occupied, but not on how you choose to model occupancy. (You would include this requirement relating to cleaning in a dynamic model, such as a sequence diagram or state diagram. This will be discussed in a later unit.)

A Person might have an age attribute, which means that people know their own age, but this does not commit us to storing age explicitly – it might be computed from a stored date of birth.

3.9 Attributes

In an object-oriented program, **attributes** of a class are represented by stored data values held in each object instance. During modelling however we use the word ‘attribute’ in a more abstract way, to refer to important properties of the instances of a class. The attributes listed in a class diagram indicate which properties of the class are important, but no values are shown. When building a conceptual model of how a hotel works, the attributes of a class are just those parts of the domain expert’s vocabulary that apply to that class. Thus a hotel manager will talk about whether a room has en suite facilities or which floor it is on. You need a way to model these terms – the middle part of the class box is used to record these attributes. Similarly if a personnel system needs to include whether or not a person is married, there must be an attribute whose value differs between a married person and an unmarried one.

It may be difficult to decide what should be modelled as a class, and what should be represented as an attribute. Larman (2004) suggests the following: ‘If you do not think of some conceptual class X as a number or text in the real world, X is probably a conceptual class, not an attribute.’

SAQ 4

- (a) What characterises the state of an object at a particular point in time?
- (b) What does an attribute of a class represent?

Answer

- (a) The state of an object is characterised by the value of each of its attributes at that point in time.
- (b) An attribute represents a particular property (a named value) of the class that each instance of that class will have. At any one time they collectively define the state of an instance of the class.

3.10 Associations

A model containing only classes (things) does not give you a working system. You need the associations (connections) that provide the glue to hold it together.

We have said previously that an association is a relationship between two classes. More generally we might say that an association relates two different concepts by a third. UML defines an association to be a semantic relationship between two types that connects their instances. In the case of class diagrams, the types are classes and their instances are objects. A class model indicates what possible connections *may* exist while an object model indicates what connections *do* exist at that instant in time.

There is often a temptation to model relationships between concepts by using attributes. However if the concepts are sufficiently important to exist in a structural model then any relationship between them should, at this stage, be modelled by an association. Also use Larman’s guideline given in

subsection 3.9 to help you decide whether to use an association with a class or an attribute to model some information.

Object models are useful for deciding what associations should be added to the class model. To see what the connections might be between sets of objects, we will examine the relationships in the hotel chain example. We can simplify the hotel chain problem introduced in Block 1 Unit 4 and identify three core concepts, *Hotel*, *Room* and *Guest*, represented in UML as three classes. How might these three concepts be related?

In our model of the hotel reservation system we could define a *Guest* (that is, an object of the *Guest* class) to be a person who has contacted one of the hotels in the chain to make a reservation. So a *Guest* is a person who is known to the hotel chain. This may mean that the person is currently staying in one of the hotels, has stayed in a hotel previously, or has made a reservation for some time in the future. So a *Guest* is related to:

- a *Hotel* in the case of a reservation
- a *Room* in the cases of currently occupying a room or having stayed in a room previously.

In addition, a *Room* is related to a *Hotel* by being physically located in a hotel. These connections between objects are represented by three associations illustrated in the class diagram of Figure 4.

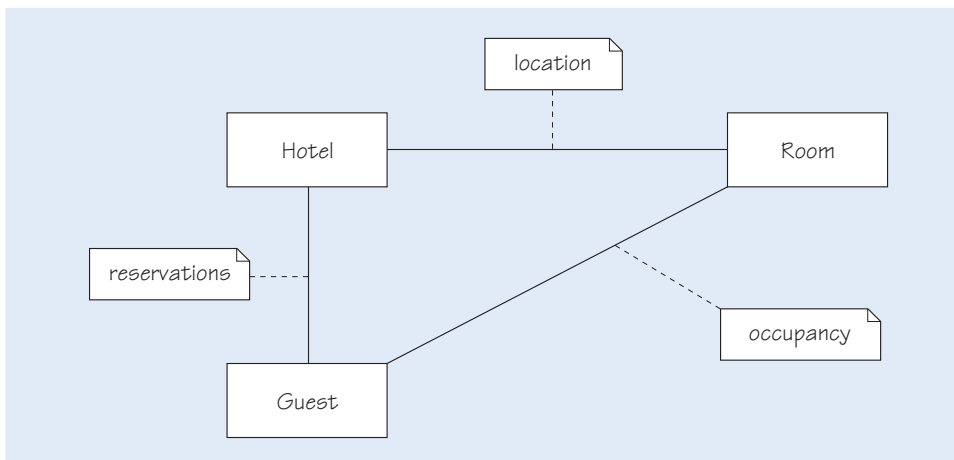


Figure 4 Some associations between *Guest*, *Hotel* and *Room*

As a temporary convenience, we have used notes to label the three associations as *reservations*, *occupancy* and *location*. Better forms of labelling will be introduced later in this subsection.

Multiplicities

Once you have decided what classes to have in a model, and their relationships, the next most important decision is to determine how many objects are involved in a given relationship. Does your system contain a single hotel object or several? Is there a limit on the number of rooms a hotel can

You should remember to check the associations in your class diagram with a domain expert. A good conceptual model brings no surprises to domain experts because it bears a strong resemblance to the relevant part of the world that they are familiar with.

have? Is a guest always associated with a room or are there guests who have several rooms or none at all?

UML provides a very rich language for modelling all such distinctions. One of the most important aspects of an association is its **multiplicity** – how many of one sort of thing can be related to another sort of thing. The annotations are shown in Figure 5.

1	exactly one, the same as 1..1
3..5	between 3 and 5, inclusive
1..*	1 or more
7, 12, 365	7, 12 or 365 (and no other)
*	the same as 0..*
	undefined

Figure 5 Annotations for associations

Note that * (0..*) is different from 1..*, which requires at least 1.

In the early stages of modelling, such as producing the class diagrams for a conceptual model, you may choose not to show any multiplicities until you are confident that you have understood the problem domain.

The general form for multiplicities uses two integers to express the range of allowable values:

minimum . . maximum

The symbol * is used in place of the second integer when there is no upper limit to the allowable values. Thus the range 3 . . * means ‘3 or more’. The symbol * on its own can be used instead of 0 . . *, which means ‘0 or more’ – often read as *many*. You can have several ranges in a multiplicity expression separated by commas – for example, the expression 0, 2 . . * means that there can be either no associated objects or two or more.

Each end of an association has a multiplicity. By stating a multiplicity at one end of an association you are saying how many instances (objects) of the class, at that end, can be linked with a single instance (object) of a class at the other end of the association. Said in another way, if there were an observer standing at the object on the other end of the association and looking across the line, the multiplicity indicates the possible number of objects that the observer would be able to see. An example of the use of multiplicities is shown in Figure 6. You should consider this to be an early attempt at identifying the appropriate multiplicities for the hotel chain example (although we have made some of the multiplicities deliberately unrealistic, as you will see).

Figure 6 has been derived from Figure 4 by adding a fourth association to represent the history of the occupation of a room. Notice that labels have been added to the ends of some of the associations. These labels are **role names**, and are used both to name the end of an association and to indicate the purpose of the association for the class at the opposite end of the association from the label. Role names are discussed further below.

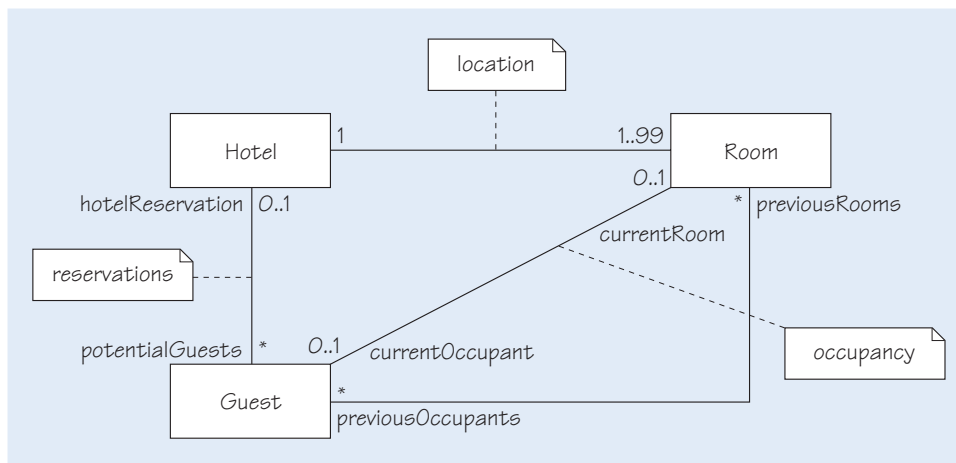


Figure 6 Examples of multiplicities

How do you interpret the multiplicities in such a diagram? It is easy to get confused by the generality, because each class represents all its instances. The best way to interpret the diagram is to think of it as saying something about each individual instance. For example, in the *location* association the multiplicity $1..99$ means that each individual hotel has at least one room and no more than 99 rooms. At its other end, the same association notes that each room is located in precisely one hotel. The *occupancy* association shows that the number of guests currently occupying each room is either 0 or 1 – either the room is unoccupied or there is exactly one person in it (not perhaps a realistic statement, and one that should be checked with a domain expert).

See Exercise 2 for a further discussion of the multiplicities and role names in Figure 6.

One way of arriving at the hotel model shown in Figure 6 is to draw a series of object diagrams in which you explore the potential links between pairs of objects. You need to take a number of snapshots before you can make a generalisation about the appropriate multiplicities for a class diagram. However you are not drawing object diagrams for their own sake, but rather to learn something about the domain. Be wary of overdoing it and stay focused on the reason for all those snapshots. Snapshots should be concise and contain no more information than is needed – you would not draw a snapshot with all the 99 rooms of a hotel just to show what happens to one room and one guest.

The hotel example in Figure 6 is still a greatly simplified model, used to illustrate UML notation. For example, it allows no more than one guest per room. In addition, instances of *Guest* can have no more than one reservation at a time. What happens when a guest wants to reserve a room at the same hotel for several different dates? Would this be possible with this model? What happens when the person, represented by that instance of *Guest*, wants to reserve a room at more than one hotel in the chain? Depending on the context, you might choose to change the multiplicity at the *Hotel* end of the *reservations* association from $0..1$ to $0..*$, although you would need to resolve the problem of date-overlapping reservations.

Association names and role names

As associations are such important parts of models, you will frequently discuss them and so need to be able to name the associations. In Figure 4, we named the associations with a comment, which is not the best way. UML provides two mechanisms for naming associations:

- placing an association name in the middle of the association line
- using role names.

Association names are usually verbs. They represent the use that objects at one end of the association make of objects at the other end of the association. A potential difficulty with a verb is knowing which way to read it. Does an *Employee* work for a *Company*? Or does a *Company* work for an *Employee*? Both make sense in different models. In UML a little black triangular arrowhead added to the association name indicates which way to read it, as illustrated in Figure 7.



Figure 7 Showing the direction in which to read a named association

Often you need to be able to read the association in both directions, so it would be nice to have two different association names. Unfortunately UML only allows one name per association. Role names overcome this restriction.

Role names allow you to name the ends of associations. The name you give to one end of an association depends on the purpose – or role – of the association from the point of view of the object at the other end. Consider an association between a *Room* and a *Guest*. There are two roles: the role the guest plays for the room, and the role the room plays for the guest. As roles are from the point of view of an object at one end of an association, the role names at the two ends are usually different, often two ‘ends’ of a related concept. Figure 8 shows a fragment containing role names for *Guest* and *Room* classes in a model of a hotel.

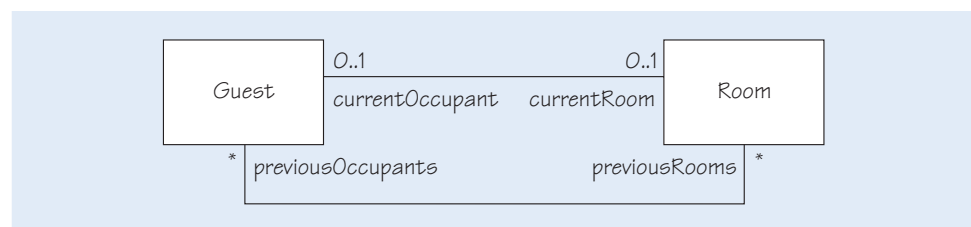


Figure 8 Role names

The placement of role names can be confusing. A role name for an associated object is placed at the opposite end of the association in which the object plays the role. For example, in Figure 8 one of the roles that a guest plays for

These role names may subsequently become the names of variables in an implementation model.

a room has been named *currentOccupant*. From the point of view of the room, the guest at the end of the association is the *currentOccupant*. As you can see in the figure, the role name is at the *Guest* end of the association.

Assumptions about meaning

As with all parts of the notation, it is important to give a symbol precisely the meaning that is defined, rather than to make assumptions and read some other interpretation into it. It is extremely easy to give meanings to symbols that were not intended. For instance, consider the model in Figure 9, which says that each *Person* has a single associated *birthCity* and a single associated *workCity*.

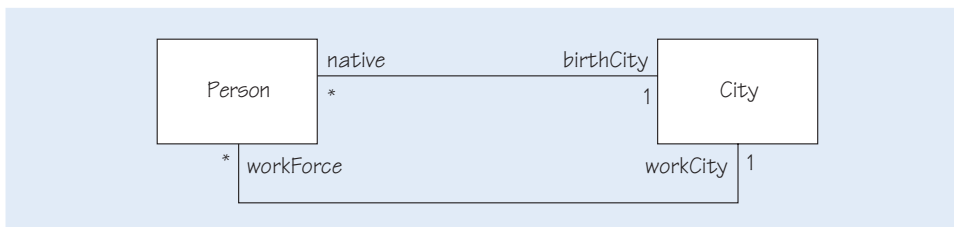


Figure 9 Two associations

If the model had not included a *workCity* but had mentioned only the *birthCity*, there is a strong possibility that you would have used your knowledge of English and your understanding of life to assume that the model would not allow the birth city to change during the lifetime of a person. There is no such temptation with the association of the *workCity* since people take jobs in different cities at different times. However there is nothing in the model that permits you to make either assumption. Both of these associations have been represented with the same notation. It is because of the meaning that we as humans attach to the role names that you are tempted to interpret one as a fixed association and the other as variable. If you want the model to capture the unchangeable aspect of an association, something more than multiplicity is required. You will see how to express such constraints later.

Navigation expressions

Once you have assigned role names it is easy to refer to attributes of related objects, because roles can be used in a fashion similar to attributes. Figure 10 shows a simple class model and an object diagram corresponding to it.

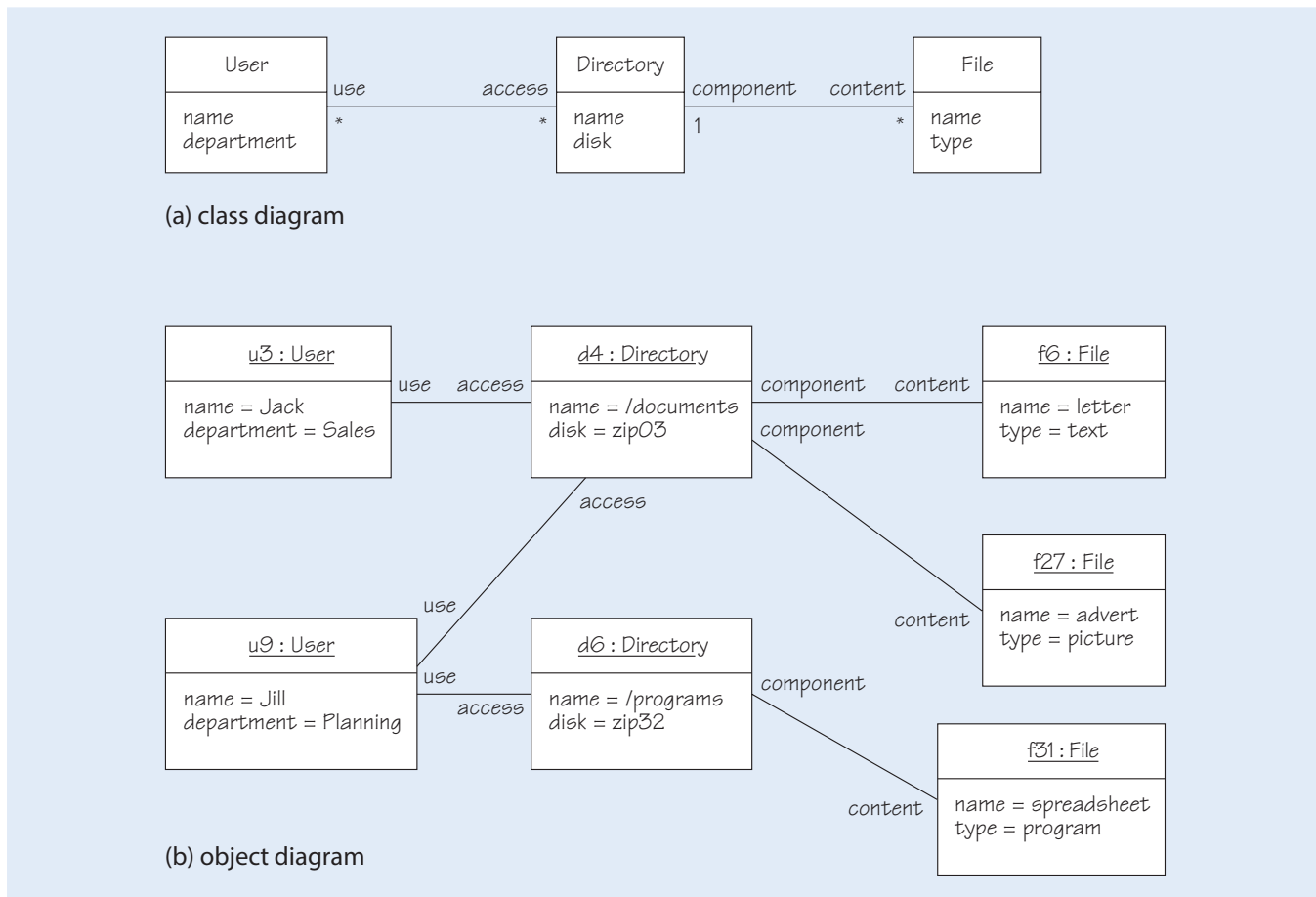


Figure 10 Role names in (a) class diagram and (b) object diagram

Navigation expressions are part of OCL, which is introduced more fully in Section 6.

In this figure, the identifier *f27* refers to an object of the class *File*, so *f27.name* will refer to the *name* attribute of that specific object. Using role names to navigate between objects, *f27.component* refers to the *Directory* object *d4* and *f27.component.disk* refers to the *disk* attribute of the *d4* object of the class *Directory*, using the *component* role name. The term **navigation expression** is used for this kind of notation, in which you name an object or object attribute by starting at some object and then hopping from that object to another, following a path. Navigation expressions are naming conventions – it may be that the actual objects do not know about each other or contain attributes permitting the navigation. When using a navigation expression, you start at one object, and then use role names to identify the next object or attribute of interest. If there is no role name at the other end of an association, you can use the name of the class at that end of the association, but with the first letter in lower case; for example, instead of *f27.component* we could have written *f27.directory*. Later we will see how to deal with the case when there is more than one object at the end of a navigation expression.

The first part of a navigation expression identifies the object where the path is to begin. The middle parts, if any, are role (or class) names that you use to indicate which objects connected by associations are next. The expression may end with the name of an attribute, in which case you are referring to that

attribute, or it may end with a role (or class) name, in which case you are referring to the object (or set of objects) connected by the association being used. The expression may consist of a single object name, in which case you are referring to that object.

Here are some examples from the hotel models (see Figures 1, 2 and 8):

- `jack` – the starting object of the class *Guest*
- `jack.currentRoom` – a *Room* object reached by following the *currentRoom* role name
- `jack.currentRoom.floor` – an attribute of the *Room* object.

Several associations between a pair of classes

There can be more than one association between any two classes, as illustrated in Figures 9 and 11. Figure 11 can be interpreted as saying that each shop knows about one set of customers called *allCustomers*, another set of customers called *thisWeeksCustomers*, yet another set called *valuedCustomers*, and a single customer called *bestEverCustomer*.

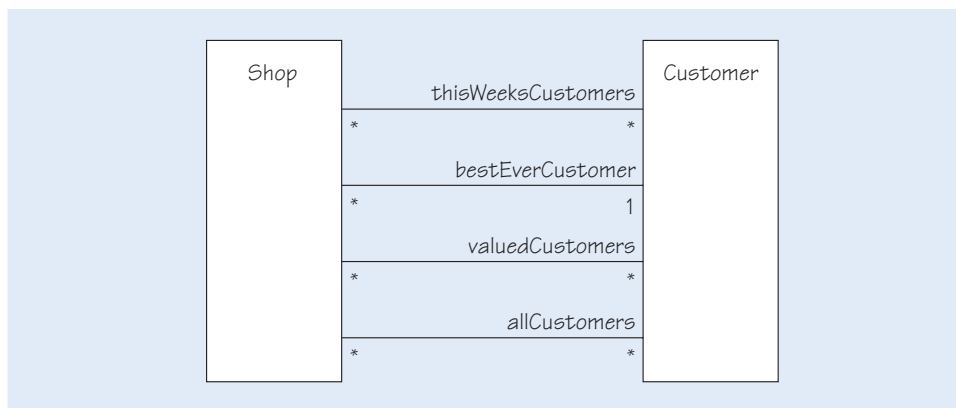


Figure 11 Several associations between two classes

As a modeller, you will want to represent the structure of the domain. If there are several associations between two classes in your model, you will need to include them all. Do not allow implementation considerations to sneak in at this point. You might be tempted to describe the *bestEverCustomer* as an attribute because you believe that is how it will be implemented, but from a modelling perspective that would be incorrect. You are modelling the relationships that a customer might have with the shop, not how they are to be implemented. It is vital that you do not let your programming knowledge of *how* to represent things corrupt your representation of *what* it is that you are representing.

Associations between a class and itself

So far all associations have been between different classes, but you often need to represent an association between a class and itself. A *Person* can be friends

with other people (more than one *Person*) or work for another *Person*. Such associations are called **recursive**. Figure 12 shows a recursive association.

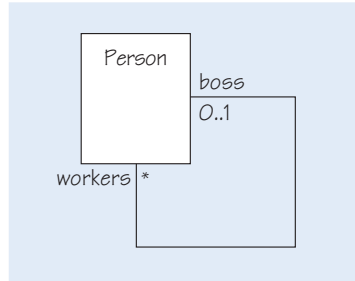


Figure 12 A recursive association

The figure shows that a person can have at most one boss and that a boss may be responsible for several workers. Note that the role names are necessary to distinguish the meanings of the ends of the association. As the ends have different multiplicities, you must distinguish them – otherwise the model could be interpreted as allowing multiple bosses and at most one worker. Note that although common sense might tell you that a person cannot be his or her own boss, there is nothing in the model to rule this out. If this is not an allowable state of affairs, you will need to express this constraint in some way other than by using multiplicities on associations. You will see how to do this in Unit 6.

Recursive associations may or may not be symmetrical. If *jack* is a colleague of *jill*, *jill* is also a colleague of *jack*. However *jack* can like *jill* without *jill* liking *jack*. Being a colleague is symmetrical. Liking a colleague is not necessarily so.

SAQ 5

- Does a multiplicity of 1 indicate that there can be no change in the object to which the multiplicity relates?
- If an airline system models flights and pilots, and each flight needs two pilots, would you use a multiplicity of 2?
- Suppose that a shoe shop has a number of cupboards and each cupboard contains an even number of shoes. How could you model the evenness of the shoes?
- If a model contains role names, do you also need to use association names?
- What is a navigation expression used for?
- What is a recursive association?

Answer

- No. It merely means that at any one time there will be exactly one object at that end of the relationship. The attributes, or even the identity, of this object may change over time.

- (b) Probably not. There are probably times during the life of a *Flight* object when fewer than two pilots are allocated, such as when the flight has been scheduled but crew details have not yet been settled.
- (c) You might use a multiplicity on the association between the classes *Cupboard* and *Shoe*, indicating that valid values are 0, 2, 4, 6, 8 and so on up to some reasonable limit. Alternatively you could say that a *Cupboard* contains an arbitrary number of instances of a class called *ShoePair*, where each *ShoePair* contains one left shoe and one right shoe. This approach generalises more easily to situations where the groups are not homogeneous. For example, a table setting contains one knife, one fork and one spoon.
- (d) No, but it is sometimes convenient to have a name for the association as a whole. For example, you might focus on what is meant by *works for* rather than the need to consider both the role *employer* and the role *employee* (at the same time).
- (e) It provides a way of naming another object or its attributes relative to a starting object by referring to intermediate role names.
- (f) A recursive association is an association where both ends terminate at the same class.

3.1.1 The hotel chain revisited

You have already seen how noun identification can be used as a means of identifying concepts that will be used in your early class diagrams. Earlier in this section we suggested that a wider view of the problem description is needed to get at the key concepts that will help you construct your first conceptual model, to understand the domain, en route to specifying a software solution. Some concepts will become the key domain abstractions – the classes; others will define the relationships between the classes – the associations.

Figure 13 shows the key concepts that have been identified so far in the hotel system. (Be aware that from here onwards we will not use arrowheads in association names unless they are needed. We will also be replacing association names by roles in most figures.) However it is not clear how we might deal with the fact that guests must have a reservation before they can check in.

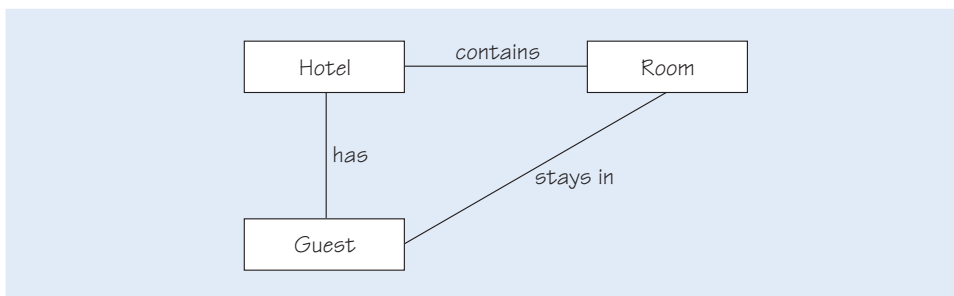


Figure 13 A set of key hotel concepts

One possibility is to make the concept of reservations into a class and make it the focus of a more elaborate class diagram. Figure 14 shows how we could refine our first attempt and include the new information. The new class, *Reservation*, makes the connection between a guest and a possible room at a particular hotel. We use a class because we want to model a relationship involving three things: *Guest*, *Room* and *Hotel*. This cannot be done with individual associations between *Guest*, *Room* and *Hotel*, because associations relate two classes. If we had three associations they would be independent of each other, which is not how reservations behave. *Also, there is some specific information to a reservation that does not belong to any of these three classes, such as the dates reserved.* Note that the change has resulted in three roles for the class *Room*: *allRooms*, *allocatedRoom* and *currentRoom*.

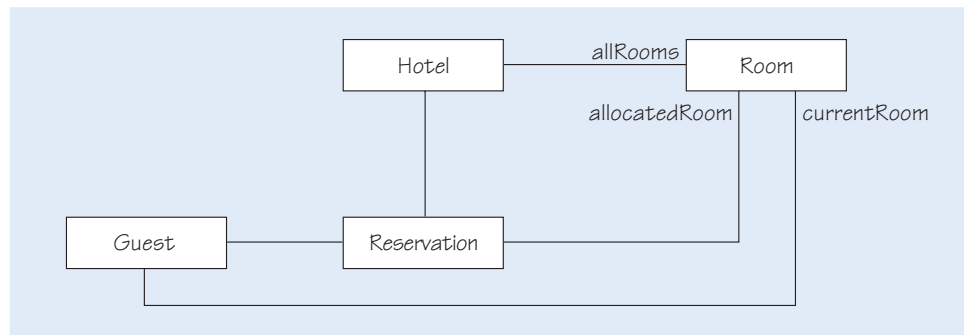


Figure 14 A first refinement of the hotel concepts

Figure 14 does not capture the fact that we expect each guest to pay for their stay. One way to do this is to create two new classes – *Bill* and *Payment* – and create new associations between them and *Guest*. However a large hotel chain is likely to offer services other than simply providing rooms for guests to sleep in. For example, each hotel would like its guests to use the restaurant facilities. Consequently each guest's bill may have the charges for several meals on it as well as the cost of any particular room. This is probably as far as you will want to go when building a model of the domain, the conceptual model. However as soon as you start thinking about the structure of the solution, instead of the structure of the domain, architectural concerns come into play. You would probably follow well-accepted principles, such as modularisation, and would plan to deal with bills and their payment in a separate area of the software system. These architectural concerns will be discussed at length in future units.

We still need some way of representing the cost of any given stay and the invoicing methods for the different rooms at a particular hotel. One way to do this is to create a new class, *RoomType*, to deal with the common characteristics or properties of different rooms. We will assume for the purpose of discussion that *RoomType* has an attribute called *dailyRate* to help us generate an offer and open a new bill when a guest checks in.

Figure 15 shows some further refinements of the concepts shown in Figures 13 and 14. It includes the multiplicities for each association as well as

some potential attributes. Notice how the class model has changed with respect to the concept of reservations.

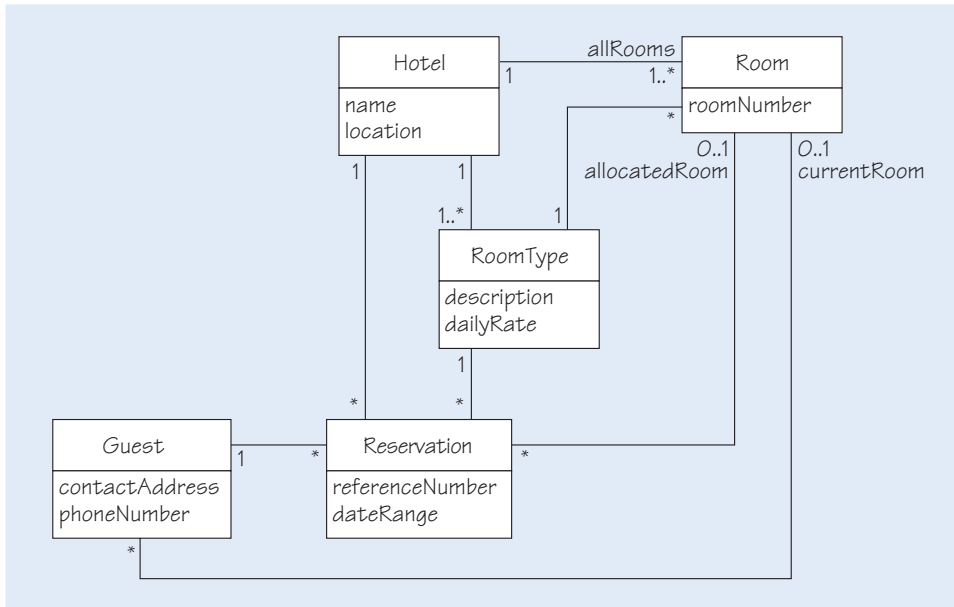


Figure 15 The classes for a hotel chain that requires reservations

As you learn more about the problem domain and requirements, you will review and revise your models. For example, you could decide to change the class diagram shown in Figure 15 if there is no need to differentiate between the planned and the actual occupancy of a room. In the next section you will see how such additional information can be shown on a class diagram.

Exercise 1

Imagine you are building a conceptual model of a petrol station to represent the administration of the dispensing of petrol and its associated billing. Consider all the interactions that are involved between driving into the petrol station and departing. What makes the petrol flow? How does the cashier know how much to charge? How does the manager know when to refill the storage tanks?

Make a list of all the nouns that you use in describing how various people make use of the petrol station. After you have done that, filter your list of candidate concepts. You need to be clear about the reasons for rejecting them.

Solution

Here is our list, but yours may be different.

arrival	bill	car
car’s petrol tank	cashier	change
cheque	credit card	customer
delivered volume	departure	fuel cost
fuel delivery	fuel quantity sold	fuel type
holster	manager	nozzle
petrol tank cap	price display	pump
sale	signature	storage tank for fuel
trigger	unit fuel cost	volume display

You can group the rejected nouns from the candidate concepts list using the following reasons:

- out of scope – cashier, customer, holster, nozzle, petrol tank cap, price display, pump, signature, storage tank for fuel, trigger, volume display
- probably out of scope – car, change, cheque, credit card
- an operation – arrival, departure, fuel delivery
- attributes of something else – car’s petrol tank, delivered volume, fuel cost, fuel quantity sold, fuel type, unit fuel cost.

You should be left with concepts such as bill, manager, sale.

Exercise 2

Table 1 records some of the associations and their multiplicities contained in the class diagram in Figure 6. Complete the table with the remaining associations, stating what their multiplicities might mean (look at the role names for clues). Are there any multiplicities that you think are unrealistic and that you should review with the help of a domain expert?

Table 1 Some associations from the hotel model in Figure 6

Association	Class A	Multiplicity for class A	Class B	Multiplicity for class B	Comment
location	Hotel	1	Room	1..99	Every hotel in the chain has at least 1 but no more than 99 rooms. Each room is in precisely 1 hotel.
occupancy	Guest in the role of currentOccupant	0..1	Room in the role of currentRoom	0..1	Each room is either empty or has a guest in it. Each guest is either currently occupying a room or not.

Solution

Our solution is shown in Table 2.

Table 2 Further associations from the hotel model in Figure 6

Association	Class A	Multiplicity for class A	Class B	Multiplicity for class B	Comment
<i>reservations</i>	<i>Hotel</i> in the role of <i>hotelReservation</i>	0..1	<i>Guest</i> in the role of <i>potentialGuests</i>	*	Each hotel has many guests (not all of whom have actually stayed in the hotel). Each guest may have made a reservation with a single hotel or may not yet have made a reservation.
<i>history of occupancy</i>	<i>Guest</i> in the role of <i>previousOccupants</i>	*	<i>Room</i> in the role of <i>previousRooms</i>	*	Each guest has previously occupied <i>many</i> rooms (some guests may not have occupied a room at all – yet). Each room has had <i>many</i> guests occupying it (although some rooms may not yet have had anyone staying in them).

The idea that each room can be occupied by at most one person seems rather restrictive. Generally a hotel has different types of room, such as single, double and twin rooms, and suites, some of which could hold more than one guest. Some hotels do restrict the number of guests per room to a maximum of 4, so this is a good area for further investigation.

We think that a *Guest* should be allowed to have reservations for more than one hotel (think of booking a multi-centre holiday or reserving rooms for a sales representative who travels a great deal). Similarly a *Guest* should be able to make several reservations for the same hotel at different times. Beware of the temptation of including a restriction that a *Guest* should not have multiple reservations that overlap in time, since this assumes that the person who makes the reservation is the person who finally occupies the room. The notion of *Guest* being a person who has contacted one of the hotels in the chain to make a reservation needs further investigation.

On reflection, we think that an individual could have several roles in relation to a hotel, for example as a customer who makes reservations and pays for the service, as a lodger who occupies a room and as an enquirer who has asked about the hotel chain's services but has not yet entered into an agreement to receive a service. Therefore a single class *Guest* is probably not sufficient in a model of a real hotel system.

Exercise 3

In Figure 1, suppose that Jack checks out of the hotel. What changes would you make to the diagram?

Solution

The link between *jack* and room *r123* should be deleted. Whether or not the link between *jack* and *theRitz* should be removed needs to be checked with the domain expert, because the hotel may wish to retain some link with its guests after they have checked out.

Exercise 4

Give examples of different possible interpretations of classes with the names *Room* and *Guest*. When would new instances be known to *Hotel* (created) and when would they no longer be known to *Hotel* (destroyed)?

Solution

A *Room* class might represent physical rooms, which might be created and destroyed as building operations change the number of physical rooms. The instances might represent lettable rooms, whose existence might be related to whether the room was unlettable because of cleaning or repair works. A *Guest* class could represent a person currently staying in the hotel, which would be created at check-in and destroyed at check-out. It might represent a person who has stayed at least once, so that they would be created on the first check-in and not destroyed on check-out, but kept on the books. It might represent a potential guest, such as someone who has reserved but perhaps did not stay. Instances would be created on first contact with the hotel and perhaps would not be destroyed, or not until they had generated no business for some years – the hotel would keep information on past guests for some time at least.

Exercise 5

Figure 16 presents a conceptual model for the administration of a number of orchestras. Study the model and make a list of possible problem areas that would need to be resolved with the domain experts – presumably the orchestra managers.

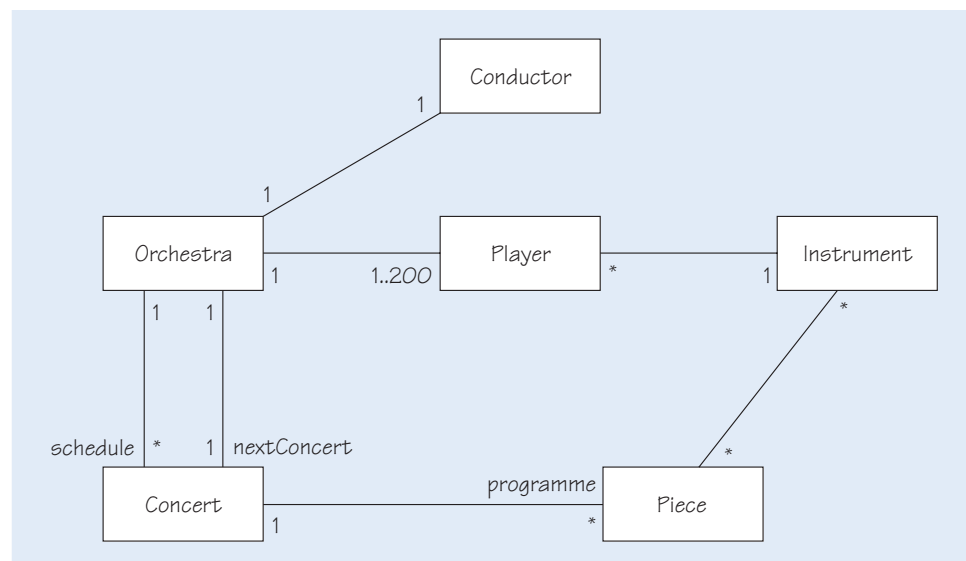


Figure 16 A class model for an orchestra

Solution

You may have identified other problem areas, but here are nine that we found using some domain knowledge.

- 1 A player can play for only one orchestra.
- 2 The model cannot represent an orchestra that currently has no players, such as during the gap between forming the orchestra and hiring the first player.
- 3 Is there always a ‘next concert’ – even immediately after a new orchestra has been created?
- 4 Does any player ever play multiple instruments?
- 5 What does ‘instrument’ mean? Does it refer to an instrument type, such as a violin, or to a particular instrument, such as Susan’s Stradivarius violin? If it means the latter, do we need the concept of ‘instrument type’?
- 6 The model currently just shows that pieces of music need instruments. Do you need to say how many of each instrument will be needed?
- 7 Does a single orchestra give each concert?
- 8 Is there only one conductor associated with an orchestra? Do you need to distinguish chief conductors from guest conductors? Should *Concert* have an association with conductors?
- 9 What is meant by ‘piece’? Can an orchestra only play each piece in one concert?

Exercise 6

Build a model to show the relationships between a person and their natural parents. Does your model prohibit someone from being his or her own mother? Is your model true for the whole of humanity or just for the people represented in some software system?

Solution

Figure 17 shows one possible model. There is nothing to say that the people at the end of the relationships are different. Since the non-identity of parent and child is part of the meaning of being a parent, you will need to capture that constraint in some other way. We have shown that every person has a father and a mother. That is true if we include dead people as instances of *Person*, but is not true if *Person* represents a living person or a person in some finite set such as those represented in a computer. This is another example of how vital a project glossary is to relate terms in the class model, such as *Person*, to one particular meaning in the world. Simply naming a class *Person* is never enough.

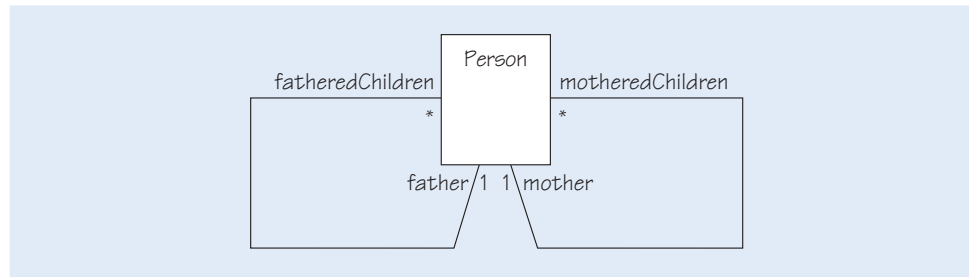


Figure 17 A class model of families

3.12 Summary of section

This section introduced the basic ideas of conceptual modelling. Techniques such as noun identification can produce a list of candidate concepts from a description of the requirements. Such a list should be filtered to remove any inappropriate concepts, such as those that are beyond the scope of the domain problem.

Once a suitable list of concepts is identified and recorded in a glossary, UML can be used to represent them with classes, their attributes and their associations in a model. The multiplicity of the classes involved in a given association is used to indicate how many of one class can be related to another class, but it cannot express all the constraints concerned with an association. Associations can be identified using either directional verbs or role names, with the latter generally being more useful.

Considerations of the lifetimes of objects and their links often reveal inadequacies in the class model. In particular, the fact that the class of an object is fixed for its lifetime when it is created, and can never subsequently change, places considerable constraints on how to use classes to model a domain.

From domain to analysis models

This section introduces some additional concepts that will allow you to increase the precision and rigour of a model by focusing on the associations between classes.

By now you have seen enough features of UML to do some useful modelling of domain problems. The conceptual models you have been building may evolve into models for a potential software system and there are many aspects of a model that have not yet been expressed. If they are not expressed in some way, either in UML or in English, different readers of the model will form different assumptions, with the risk that the wrong software will be built or that the software will not work at all. You will therefore need more precise modelling concepts, which are the subject of the remainder of this unit. You will also start building models of the structure of a software solution, rather than of a domain problem – analysis models rather than conceptual models. These analysis models will eventually evolve into precise descriptions of how the software will address the problem solution – design models. The techniques introduced in this section are mostly used in structural models of the software, particularly design models.

This assumes an approach where modelling will be done to a level of detail and precision that will make it possible to hand models over to a different group of people to carry out implementation, or even to have some automatic generation of code from the models. In an agile development team you would probably stop refining models earlier, as soon as they are no longer useful to those building them – in this case, the model builders and software developers are likely to be the same people.

UML has a rich vocabulary of notation that different groups of modellers will use in different ways. Some prefer to use as much of the language as possible, while others prefer to use a small subset. Agile modellers, as you are aware by now, will probably only use a small amount, just enough to agree on a shared mental model of what is to be done. They will also use different modelling notations to represent aspects that not one single modelling language will cover – for example, they may use something other than UML to model user interfaces (Ambler, 2002). Our aim is to introduce some of the techniques of UML and explain their role in software development. It is up to you to decide which techniques to use in the context of a given project.

4.1 Interpreting attributes

At different stages in modelling, you will use classes to represent different kinds of information. During conceptual modelling you might be building a model to explain how a complete hotel works, with no particular computing system in mind. The classes would represent pure concepts – the more tangible business objects and their properties – rather than bits of software. The attributes of such classes represent parts of the domain expert's vocabulary. For instance, a hotel manager will use terms like daily rate,

occupied and en suite when discussing rooms. These terms must be modelled somehow.

When you decide to build some software to automate part of the hotel's operations, the analysis model you build will again include classes, but now the attributes will represent something that is defined in a software object.

SAQ 6

When considering attributes, what is the effect of moving from a conceptual model to an analysis model?

Answer

The conceptual model records attributes of classes that will be familiar to a domain expert. For example, a hotel manager will be familiar with the daily rate for a room and whether or not it is occupied. In the analysis model, the developer must consider the representation of attributes within a software system. For instance, daily rates for rooms involve money and you can use a true/false (Boolean) expression to represent the occupancy of rooms.

4.2 The importance of time

A class model is meant to express all valid configurations of the system. It is possible to build a model that appears to be true most of the time, only to find that you have overlooked and accidentally excluded certain valid states.

Figure 18 shows a fragment of a model that expresses the fact that an invoice has at least one entry (known as an invoice line).

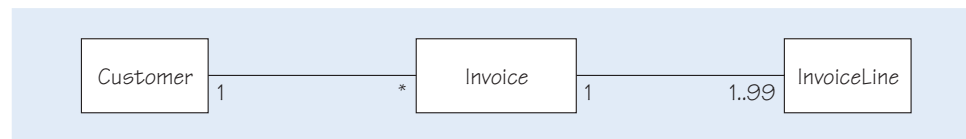


Figure 18 A class model for invoices

This might look sensible, as a company would not normally send out an invoice that had no lines. However if you consider the entire life history of an invoice you may discover that there are circumstances in which it contains no invoice lines. An invoice may be created when an order is placed, but the invoice lines are only populated as the items are picked or manufactured. It could be that a zero-value invoice is created when an account is closed. Similarly during editing all lines might be deleted before new lines are added and the invoice may remain in this state for some days. It seems that the model is probably wrong.

Very often, consideration of the life history of a relationship reveals other associations that should have been modelled. Figure 19 represents part of a library system that maintains an association between members and books.

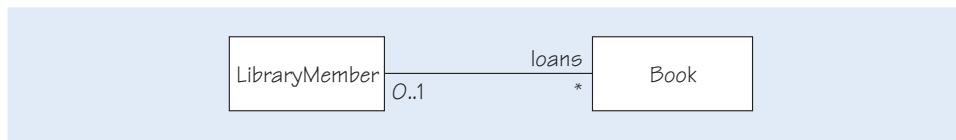


Figure 19 A class model with a single association

If you consider the association with the role name *loans* and ask about its life history, some issues arise. When are instances of such an association (links) created and when are they broken? A link is probably made when the member borrows the book, but should it be broken when the book is returned? If the purpose of the association is to know which books are currently on loan, the association will need to be broken when the member no longer possesses the book. However if you want to record which members have borrowed which books in the past, the association should never be broken. Such a quandary usually reveals that you have omitted an association from the model. In this case you probably need two associations, with roles such as *currentLoans* and *pastLoans*. When a book is returned, it might be removed from one association and added to another. Figure 20 shows a slightly richer model, which has captured this distinction (although it does not show when the associations are created and destroyed – you need dynamic models to do this).

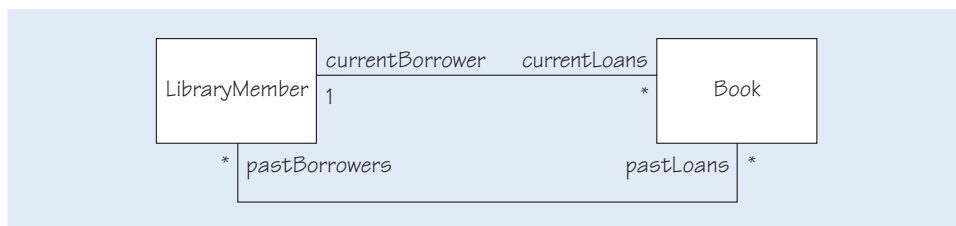


Figure 20 A class model with multiple borrowing associations

A useful check on any model is to examine every element – every class and every association – and ensure that there is agreement with domain experts on *when* instances should be created and destroyed.

SAQ 7

Why is a class model not sufficient to describe a system?

Answer

A class model is a static model that describes the elements of a system (the classes) and their relationships (the associations) but does not describe the behaviour of the system over time. For this you will need one or more dynamic models. In particular, you need to model the life histories of objects and the interactions between them. The model needs to capture when instances of classes should be created and destroyed.

4.3 Aggregation and composition

An association represents the fact that an instance of one class is linked with a set of instances of another class. Figure 18 showed that an *Invoice* can have several instances of *InvoiceLine*. That is, there can be several invoice lines associated with a single invoice. We can describe this situation as a whole–part relationship: the whole (an invoice) is made up from the parts (the invoice lines). A similar **aggregation** association (whole–part relationship) holds between an honours degree and the courses that were studied. In both of these examples it would be possible to add, remove or replace one of the parts and still have a meaningful relationship. An invoice would still be an aggregation of invoice lines if another invoice line were added.

Aggregations have one very important additional constraint compared with normal associations: aggregations must be free of cycles. If you arrive at an object by traversing through an aggregation association, there must be no way that you can continue to traverse associations away from the object and arrive back at that same object through any aggregation. More formally, aggregations must form a graph without cycles. Less formally, aggregations form a whole–part relationship and no object may be part of itself, directly or indirectly.

There is a stronger form of aggregation called **composition**. In a composition the composed objects are part of at most one composing object. If the composing object is deleted, all of the composed objects must be deleted. For example, consider a *Window* object (for a window on your screen) containing a *ScrollBar* and a *Button*. If the window is destroyed, the scroll bar and button must be destroyed as well. Compare this situation with a model for describing a *Product* consisting of instances of *Part*. One example might be the parts that make up a particular model of television set. A part, such as the screen, might appear in more than one model of television, or in none at all. Destroying one of the television model descriptions does not cause the description of the screen to be destroyed.

UML provides a notation to record the nature of a whole–part relationship. For aggregation, you simply add an open diamond to the end of the association to indicate the class that is to act as the whole. Composition, the stronger form of aggregation, is shown using a solid black diamond. Both notations are illustrated in Figure 21.

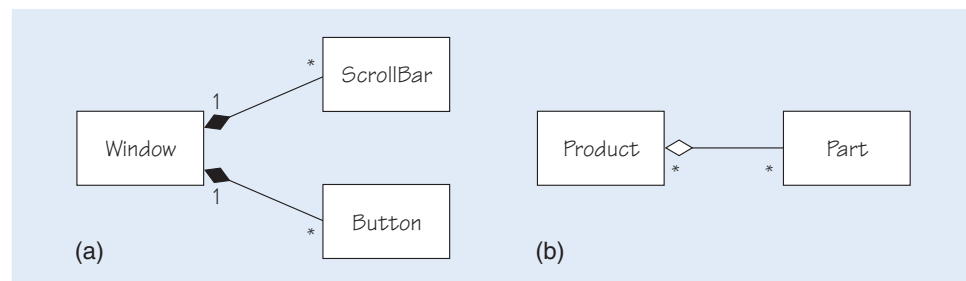


Figure 21 Notation for (a) composition and (b) aggregation

4.4 Navigability

So far we have not said anything about the direction of potential messages between instances of one class and instances of another where there is an association between the two of them. You do not have to think about this in a conceptual model where the direction of an association does not really make sense and they are all, at this stage, bidirectional. However in a structural model for a software system you need to start thinking which objects access other objects – which objects send messages to which other objects. In Figure 19, for example, should a *LibraryMember* object be able to send messages to the *Book* objects that it is linked to? Or should it be the other way around, or should messages be able to pass in both directions?

In UML, you can put an arrowhead at one or both ends of an association to indicate that it is possible to reach one class from another following the direction of the arrow. In doing so, you are restricting access to instances of the class at the end of the association where you placed the arrowhead. If you put no arrowheads on the association, **navigability** in both directions is assumed. Figure 22 is a revised version of Figure 18. Now an *Invoice* can send a message to an *InvoiceLine*, but not the other way round.

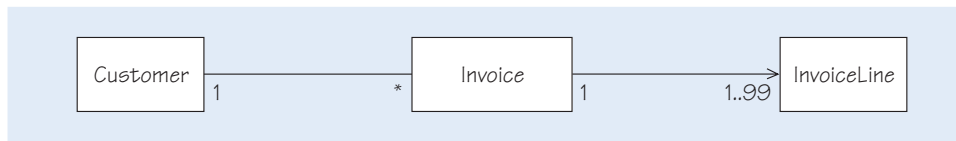


Figure 22 Constraining the navigability of an association

Note that navigability has no effect on navigation expressions. Navigation expressions are names used to indicate specific objects or attributes of objects, and do not take navigability into account. The notion of navigability is mainly of use during implementation, when you have to decide how to write the code that represents an association. If navigability can be restricted to one direction only (which is not always possible) the code will usually be simpler.

SAQ 8

- What is meant by navigability? When is this idea useful?
- In a multi-user operating system, users are allocated passwords. Draw a fragment of a class model to represent this association. Bear in mind that you do not want to be able to identify the corresponding user for a given password. What does this tell you about the representation of the association?

Answer

- Navigability means that it is possible to identify (or ‘reach’) objects in one class from objects in an associated class. The usefulness of this idea is realised during implementation, when navigability in one direction alone (unidirectional navigability) can lead to simpler code.

- (b) Users will want to ‘know about’ their passwords, not the other way round. Figure 23 shows that each instance of the class *User* will have a collection of references to the appropriate *Password* objects.

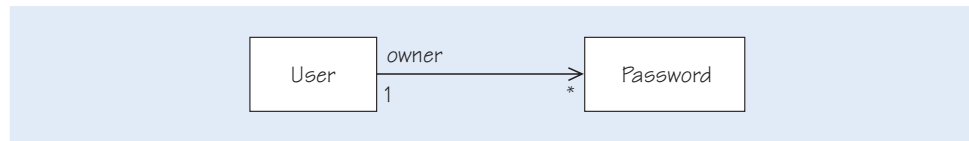


Figure 23 Navigability from *User* to *Password*

4.5 Qualified associations

A suitable time to generate a unique account number is when a new account is created.

Suppose that one of your classes has an attribute that acts as a unique identifier. For example, a bank usually identifies an individual bank account by a unique account number. Figure 24 shows how you might initially represent banks and their accounts, with an attribute to represent a bank account number. To ensure that the assumption that account numbers uniquely identify accounts is documented, this use of account numbers would be noted in the glossary.

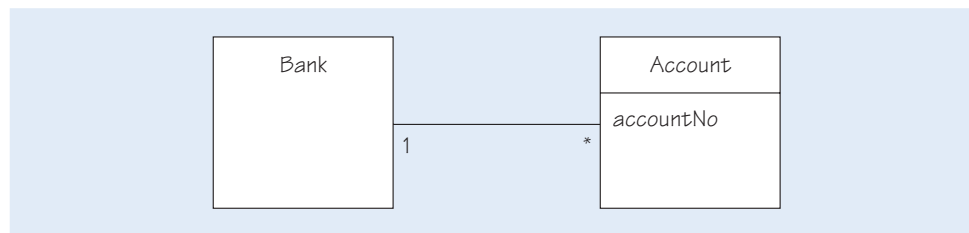


Figure 24 Account with account number

There is an association between the two classes, with a multiplicity of 1 at the *Bank* end and a multiplicity of * at the *Account* end.

UML allows this situation to be modelled by using what is called a **qualified association**. Instead of a *Bank* instance being linked to many *Account* instances, as in Figure 24, it is shown as linked to at most one *Account* per account number. Figure 25 shows such a qualified association. The *accountNo* attribute has been moved out of the *Account* class and has become a qualifier. The qualifier is represented by a rectangular box at the far end of the association from the class that the qualifier is an attribute of.

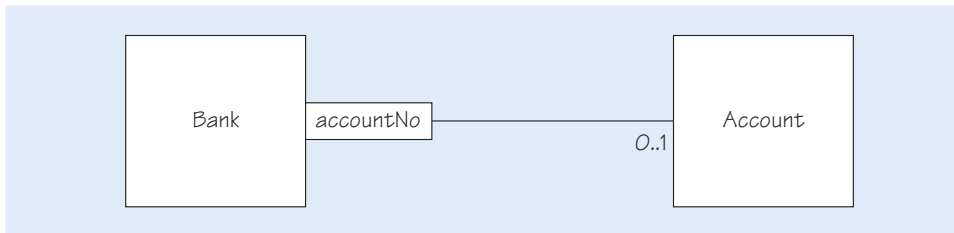


Figure 25 Using a qualified association

The reason why the multiplicity at the *Account* end is $0..1$ and not 1 is that there may be account numbers that do not correspond to any actual account.

You can now read the model as saying that for a given combination of a bank and an account number there is either one associated account or none. It thus tells you something extra about the *Bank–Account* association. The advantage of this approach is that you have put more information into the model and have placed less reliance upon people using the glossary.

This illustrates the general point that a qualified association often replaces an association having a multiplicity of $*$ with one having a multiplicity of either 1 or $0..1$.

In UML, a qualifier is considered to be an attribute of an association. It tells you about a property of the concept that relates the classes at the ends of the association.

In cases where objects can be uniquely identified by combinations of two or more attributes, all those attributes should move into the qualifier. For example, in a bank with many branches an account may be uniquely identified by a combination of the account number and the branch number. The UK system of sort codes and account numbers works this way.

SAQ 9

- What is a qualified association?
- Suppose that, in the invoices example shown in Figure 18, invoices have unique numerical identifiers known as invoice numbers. How would you capture this information in a class diagram?

Answer

- A qualified association is an association at one end of which there is a qualifier, consisting of one or more attributes. The values of the attributes (taken together) uniquely identify the objects in the class at the other end of the association.
- You could use a qualified association whose attribute is named *invoiceNo* in a manner similar to that shown in Figure 25, by attaching the qualifier to the *Customer* end of the association between *Customer* and *Invoice*.

4.6 Attributes on associations

In some situations it is convenient to put attributes directly on an association, instead of on one of the classes at the ends of the association. Consider a model for a chain of stores and the items on the shelves in each store. The owners of the stores want the price for an item to depend on the store it is in and want to know the quantity of each item in each store. You cannot put the price and quantity in the store, because that would imply a price and quantity for all items. You cannot put them in the item, because that would imply a price and quantity for all stores. The solution is to put the information on the association, as illustrated in Figure 26.

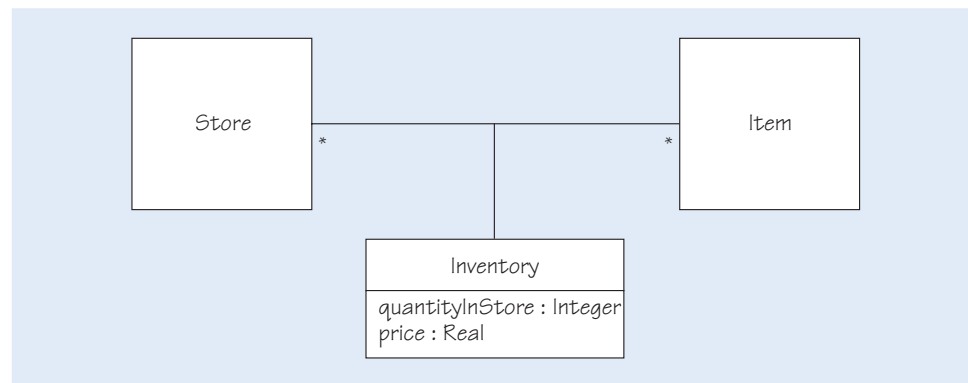


Figure 26 An association class

An **association class** has the distinctive feature that it has properties of its own and thus it becomes a first-class citizen. It also imposes the restriction that there can be at most one instance of the association class per pair of objects linked by the association. In the example shown in Figure 26, there can be at most one instance of *Inventory* for a given instance of *Store* (for example, the London store) and a given instance of *Item* (for example, a Tower of London souvenir cup). Each link between an instance of *Store* and an instance of *Item* has the attributes *quantityInStore* and *price*.

4.7 Derived associations

As you work with domain experts to develop class models you may find that you want to add an association to the model because it is important for understanding, but the association is redundant because other associations in the model can be combined to achieve the same navigation. To avoid this redundancy, UML provides **derived associations**.

Figure 27 shows some derived associations, indicated by preceding the role name (or association name) with a slash. Starting from a *Family* object, the association with the role name *allChildren* would produce a set of *Child* objects representing the children in the family. However because the *Child* object contains the attribute *age*, you could easily compute a set of objects representing the teenagers in the family by first getting the set of all children, and then removing the children who are not between 13 and 19 years old. You

could do something similar to compute the set of children living at home. So the *teenagers* and *childrenAtHome* associations are derivable from *allChildren* and information in the object.

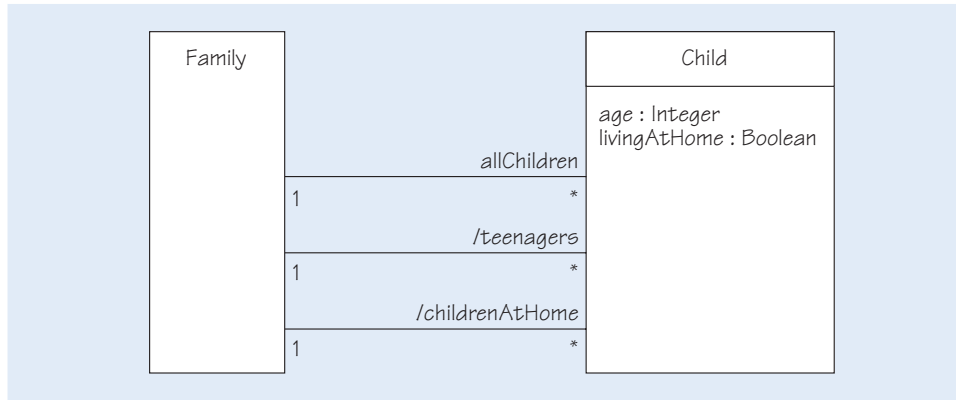


Figure 27 Some derived associations

Although an association is shown as derivable, the model does not show how it is derived. This needs to be put in the project glossary.

It is not only associations that may be derived. UML allows you to derive a new element from the other elements in your model. Derived associations are probably the most common kind of derived element, closely followed by derived attributes. Any attribute can be marked as a derived attribute using the slash notation. For instance, a *Person* might have the attributes *dateOfBirth* and *ageInYears*. These attributes are not independent – if you were given *dateOfBirth*, you could calculate *ageInYears* – so the age could be marked as a derived attribute: */ageInYears*. Typically the rule (the computation) by which one attribute is derived from another would be held in the glossary, but you could also express it in a note attached to the appropriate element in a model. For example, the value of *ageInYears* is obtained by comparing *dateOfBirth* with the current date.

There are several reasons for choosing to identify a derived element. As noted above, during analysis a derived attribute or association may be used to identify and define a useful concept. The disadvantage is a redundancy in the overall model that requires an ‘overhead’ to check that the derivation rules are understood, possible and valid after other changes to the model. Whatever the reason for choosing to identify a derived element, its presence in a model implies that there is a responsibility to consider changing it if there is a change in the elements used to derive it.

Example 1

Subsection 3.11 describes a class diagram that represents the main concepts in the hotel domain for reservations and checking in and out. Figure 15 shows a direct association between *Hotel* and *RoomType* and an indirect association

through *Room*. As part of your design optimisation you might find that the direct association is readily derived from the indirect association.

SAQ 10

- (a) Under what circumstances would you want to show an association that is not independent of others in the diagram?
- (b) Which of the two following models would you recommend: representing *age* as an attribute in Figure 27, or having an attribute *dateOfBirth* and *age* being a derived attribute?

Answer

- (a) If a word describing an association is part of the natural vocabulary of the domain expert, it will be sensible to include it in the model, as otherwise a linguistic gulf will start to open between the domain expert and the system designer. However if you know that an association is not essential, because it can be derived from other associations, you will also need to record that fact.
- (b) Having an attribute *dateOfBirth* and *age* being a derived attribute would be a more reasonable model, as otherwise *age* would need to be updated on a regular basis.

Exercise 7

Build three class models:

- (a) associating companies with employees
- (b) associating libraries with library members
- (c) associating families with family members.

For each model, identify the associated people (employees, library members and family members) by suitable qualified associations. When considering a family, be sure that your model handles twins (multiple children with the same date of birth) and sharing of names between generations (for example, mother and daughter with the same name).

Solution

Looking from a company's and a library's perspective, you can find a way to identify a single person who may be a worker or a member, respectively, as shown in Figure 28. In the case of a family, two or more members of that family may have the same birthday or the same name, but the combination of name and date of birth should be unique. Using two qualifiers (*name* and *birthDate*) should uniquely identify a person.

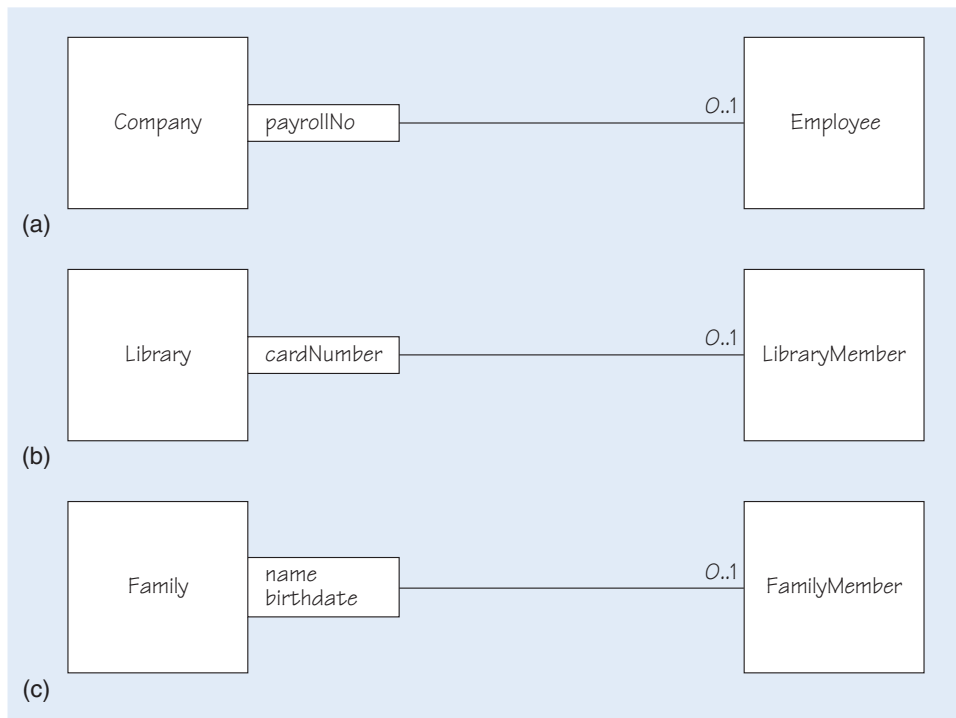


Figure 28 Models with qualified associations

4.8 Summary of section

You have seen in this section that class models are limited because they only show the overall structure of a proposed system. That is, a class model records the elements (classes) of a system and their relationships, but does not show the importance of time.

Other techniques can greatly increase the precision and rigour of models.

- The navigability of an association can be used to indicate whether it is possible to traverse an association to obtain the object or set of objects of a given class.
- Qualified associations capture the uniqueness of some means of identifying objects, as in the case of account numbers in a bank.
- It is possible to show additional, derived associations that are deducible from other associations in a class diagram.

5

Generalisation and specialisation

This section will introduce you to additional modelling constructs that allow you to deal with specialisation and generalisation among classes. You will also see how the concept of an interface helps to control the dependencies between classes. These concepts are only needed when modelling software and designing the implementation, not in domain models.

Generalisation/specialisation is a relationship between two types. If type *B* specialises type *A*, an instance of *B* has all the features of an instance of *A* but also has additional features special to type *B*. In this case, *B* is a **specialisation** of *A* and *A* is a **generalisation** of *B*.

Figure 29 illustrates the concepts of specialisation and generalisation. There are some general properties and behaviour shared by all trees: they all have leaves and they tend to grow upwards (and outwards). Oak and pine are two specific kinds of tree that have properties specific to the type of tree. For example, they have differently shaped leaves and oak trees have acorns, pine trees cones. Their behaviour is also different – an oak tree drops its leaves in autumn and grows new ones in the following spring, whereas a pine tree keeps its needles all year round. ‘Tree’ is therefore a generalisation of ‘oak’ and ‘pine’, while ‘oak’ and ‘pine’ are specialisations of ‘tree’.

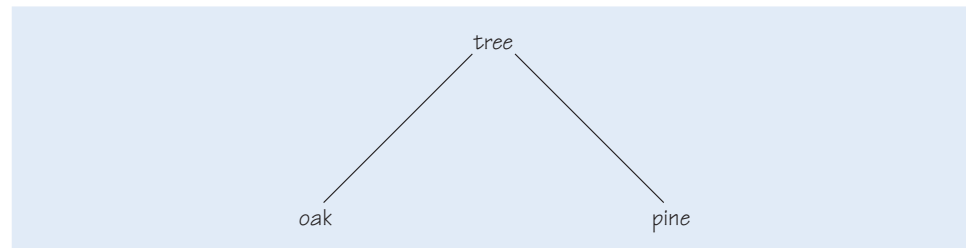


Figure 29 Different kinds of tree

Note that we can only talk in more abstract terms about the tree at the top of the hierarchy in the figure. A tree is always a tree of some species. We do not see instances of a generic tree in our woods and forests, but we do see instances of oak and pine trees. We will return to the abstract nature of types at the top (or root) of a generalisation hierarchy later on.

Notice that statements may be made about a general property, such as the direction of a tree’s growth, no matter whether it is an oak or pine tree. This illustrates, in a very simple way, that an instance of the more specific element can be used in place of the more general element. This is called **substitutability**: a more specialised element can be substituted for a less specialised element in the same hierarchy.

5.1 Superclasses and subclasses

In object-oriented software development we call the more general element a **superclass** and the more specific element a **subclass**. A subclass has the same operations and attributes as the superclass, although it may define some additional ones. The natural time to decide on operations is not until the requirements for the whole system have been agreed and decisions are being taken on which parts of the functionality of the whole system belong with each class. We need some operations and attributes so we can discuss specialisation – we want to be able to show subclasses having extra behaviour and properties – so for this purpose we have supplied a plausible set of operations and attributes. How we decide what operations are appropriate in practice is something we explore in a later unit.

Figure 30 shows a typical use of specialisation. There is a class *Account* that represents the basic notion of something that maintains a balance, and supports operations to credit and debit the account.

You may also be familiar with the terms *parent* and *child* for a superclass and a subclass respectively.

In Unit 6 we introduce the concept of ‘design by contract’, which will help you decide on which operations can guarantee substitutability.

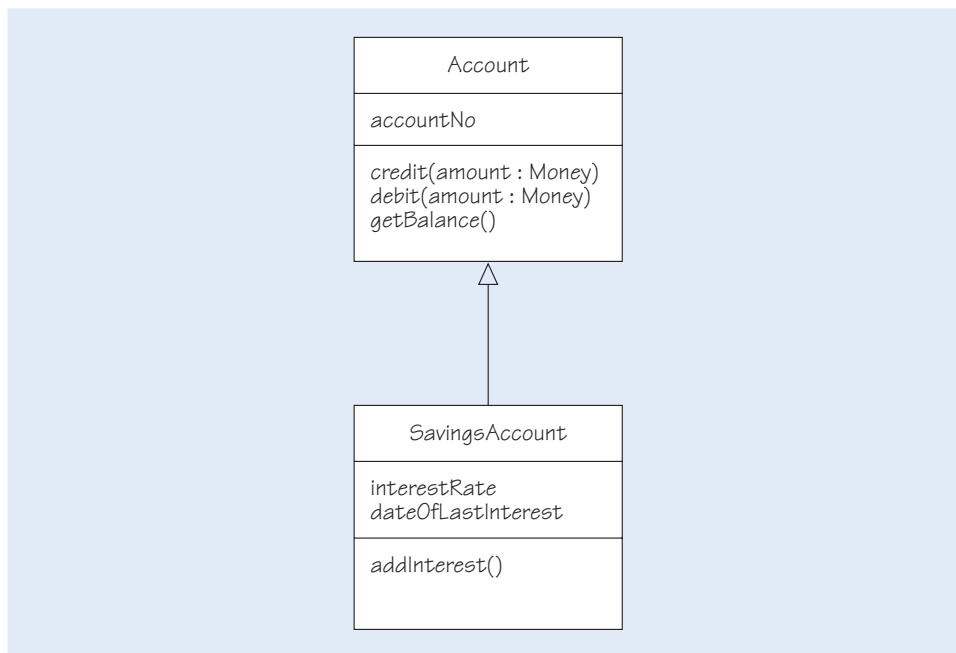


Figure 30 A specialisation of *Account*

Suppose that you already have an *Account* class and want to define a *SavingsAccount* class, with operations to credit, debit, get a balance and add interest. You could define a completely new class with all the necessary operations. If you did so, you would end up with two apparently unrelated classes. What would be missing is the reason why both classes have so many operations and attributes in common: a *SavingsAccount* really *is* an *Account*; it is just a special form of *Account* that can also handle interest. It is therefore better to model this connection directly. Another advantage of defining *SavingsAccount* as a specialisation of *Account* is that *SavingsAccount* will automatically share any changes that you make in *Account*, therefore avoiding

duplication of effort. In a class model specialisation is indicated by a line with an open arrowhead at the superclass end of the line, as illustrated in Figure 30.

When deciding whether to model some distinctions by using specialisation, you need to remember that an object cannot change its class dynamically. Figure 31 looks like a reasonable model for representing the fact that children attend a school and that adults are allocated to a tax office. It uses two different subclasses of *Person* and puts the aspects that differ into those subclasses. However this model will not be able to represent the change when a child becomes an adult. This may or may not be a problem. If the model is intended for short-term use, say in a school, it may never have to cope with such changes. Once again you can see that whether a given model works or not can depend on the life cycles of the objects in the system.

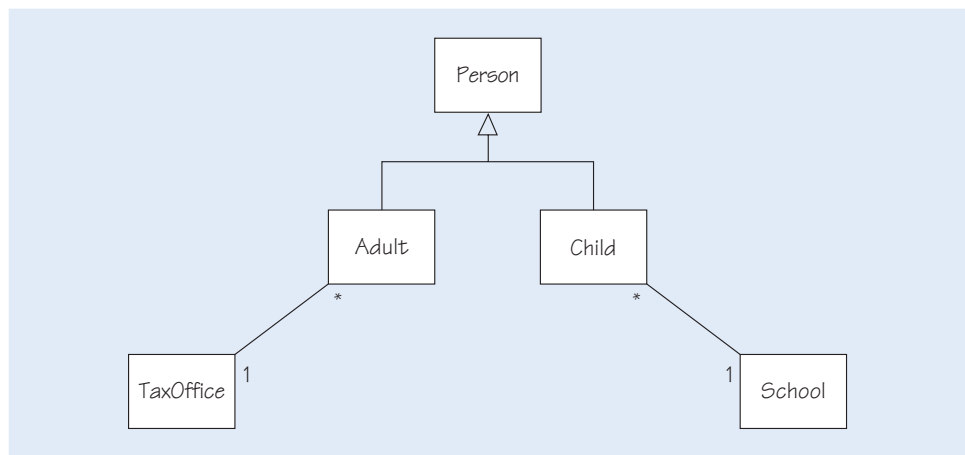


Figure 31 Specialisation used to distinguish roles

5.2 Interfaces

The **interface** of a class is the set of operations that specify the service that it provides. The class *Account* has the interface {*credit(amount : Money)*, *debit(amount : Money)*, *getBalance()*} while the class *SavingsAccount* has the interface {*credit(amount : Money)*, *debit(amount : Money)*, *getBalance()*, *addInterest()*}. Note that the interface of the subclass always contains the interface of the superclass, although in general the subclass will have extra operations as well. There is no mechanism for removing operations. You cannot define, say, a *DepositOnlyAccount* class as being a subclass of *Account* that does *not* have the *getBalance* operation. This fact is vital for substitutability because it means that you can guarantee that all subclasses have the attributes and operations of the superclass, although they may have more.

Subclasses can however redefine operations that are in the superclass. Figure 32 shows a different subclass of *Account*, called *GoldAccount*. This defines an extra operation, *addBonus*, which is not present in *Account*, but it also redefines the *credit* operation, which was present in the parent. Perhaps it

redefines it to reward depositors by crediting an extra £10 to the account each time a deposit is made. The interface of *GoldAccount* therefore contains the operations *credit*, *debit*, *getBalance* and *addBonus*. The class still supports all the operations of the superclass but it does so in a slightly different way, in that the behaviour of the *credit* operation has been changed.

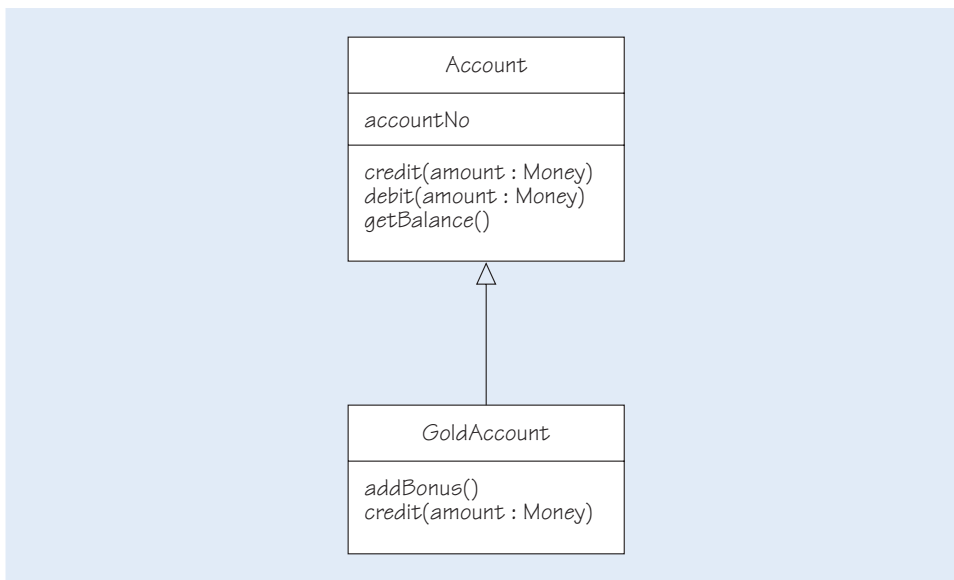


Figure 32 Another specialisation of *Account*

5.3 Levels of specialisation

Specialisation can occur at various levels. Figure 33 represents a word-processed document containing various parts that are specialised at two levels.

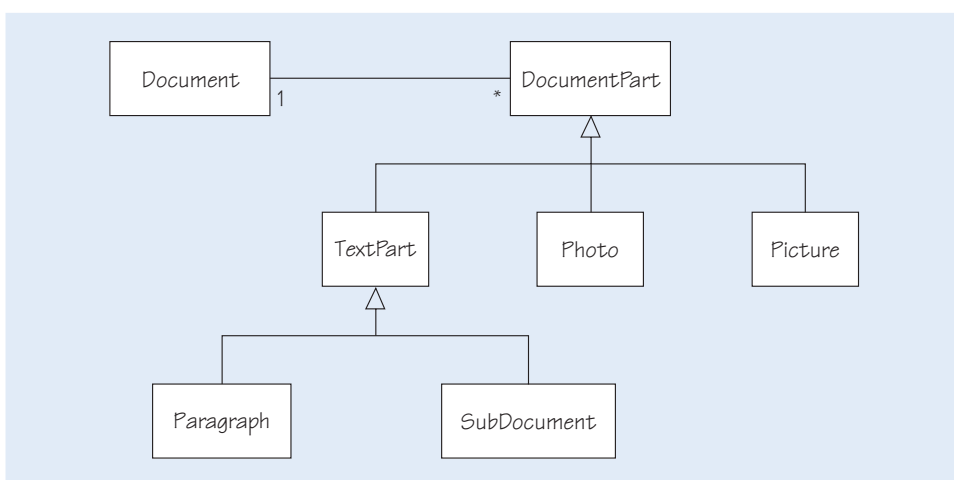


Figure 33 Multiple levels of specialisation

If you have several levels of specialisation, some associations in the model may be with the subclass and others with the superclass. Figure 34 shows

such a situation, where clubs know all about their members but teams are related to playing members only. Both subclasses of *Member* inherit the association with *Club*, so an instance of either subclass can be linked to an instance of *Club*. However only an instance of *PlayingMember* can be linked to an instance of *Team*. An *AssociateMember* instance cannot be so linked.

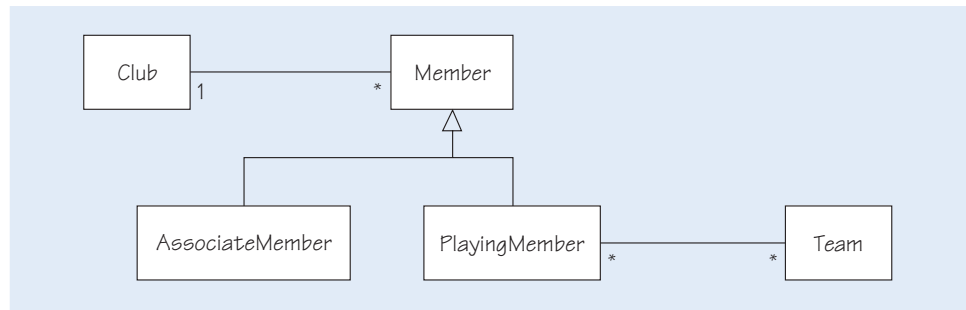


Figure 34 Clubs and members

5.4 Specialisation and substitutability

The substitutability test can indicate whether a use of specialisation is correct. You should use specialisation only when an instance of a subclass is substitutable for an arbitrary instance of a superclass. Suppose a club is looking for a treasurer with the only constraint being that the treasurer must be a member. Any instance of *PlayingMember* would do, because a *PlayingMember* has at least all the properties of an ordinary *Member*. However if a club is looking for a player, instances of *AssociateMember* would not do as they cannot be linked with instances of *Team*. If you want to play for the club, you have to be a *PlayingMember*. Similarly if the club wants an *Account* to keep its funds in, a *GoldAccount* will do because this satisfies all the expectations one can have of a simple *Account*.

Cases where the substitutability principle does not hold tend to arise when the use of specialisation is motivated by a desire to unclutter a model or by implementation concerns, rather than by analysis of the relationship between concepts.

Suppose you have an existing class *Car*, defined as in Figure 35. You now have a requirement to define a class *Bicycle*. You might notice that most of the desired attributes of *Bicycle* are also attributes of *Car*, so you decide to use specialisation to save some time. However some attributes of *Car*, such as *usesDiesel*, are inappropriate for *Bicycle*. There is no mechanism for removing attributes in a subclass, so you leave them there and just hope that no one uses them. You have violated the substitutability principle because it is not true that a bicycle can be substituted for a car in all contexts. If you try to fill a bicycle with fuel, you will discover the differences.

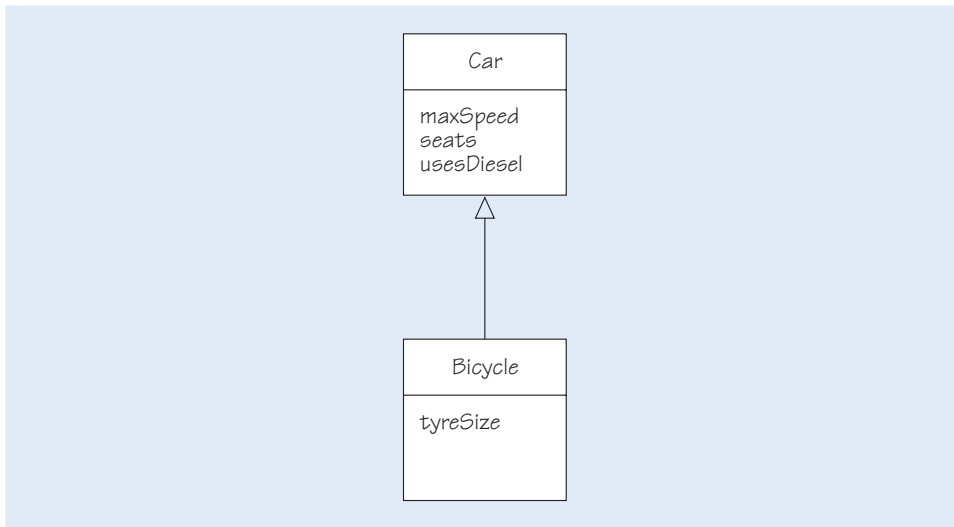


Figure 35 Improper specialisation

When tempted to use specialisation in this way, you should always investigate whether you should add a superclass to the model. A better solution is shown in Figure 36.

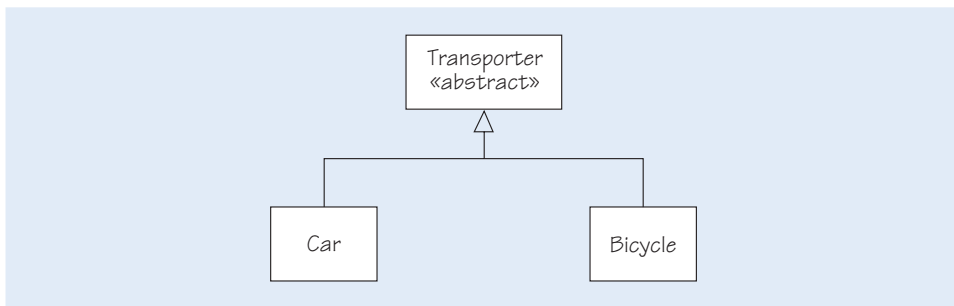


Figure 36 Inventing a superclass

You have invented a superclass, *Transporter*, but do not intend there to be any instances of it. There are no objects that are purely transporters – just cars and bicycles. The superclass exists just because it collects common concepts together and documents the similarity between the subclasses. The domain experts may not have thought of the superclass but they should agree that it represents something, or at least does not break anything. Such a superclass will never be used to create instances, and in UML it is represented with the name in italics or tagged with the stereotype «*abstract*».

SAQ 11

- What is the difference between inheritance and generalisation?
- Look again at Figure 30. Will an instance of *Account* support the *addInterest* operation?
- In a computer graphical user interface, how might you represent the relationship between a full window and an iconised window from the

A stereotype is just a label that may be added to a model element such as a class or an association to convey extra information.

classes *FullWindow* and *Icon* respectively? List the operations that are common to both and those that are peculiar to each.

Answer

- (a) Inheritance is the consequence of a generalisation/specialisation relationship; a subclass inherits the features of the superclass.
However the term inheritance tends to be used by programmers, whereas the terms generalisation and specialisation are used by analysts and modellers.
- (b) No. *addInterest* is not part of the interface for the class *Account*. It is however part of the interface for the class *SavingsAccount*, which is a specialisation of the class *Account*. Objects of the superclass cannot replace objects of any subclass.
- (c) You could introduce a new, abstract class named *Window* and let the other classes inherit from it as follows:
 - abstract class *Window* with operation *display*
 - subclass *Icon* with additional operation *maximise*
 - subclass *FullWindow* with additional operations *iconise* and *scroll*.

Exercise 8

Figure 37 shows a simplified model of a lending library that holds copies of publications for loan by its members.



Figure 37 A lending library

- (a) Using generalisation, draw a fragment of a class model to illustrate the fact that books and journals are both kinds of publication in a lending library.
- (b) A typical lending library will stock copies of both books and journals. However not all of these copies will be available for the members of the library to borrow. Some copies will be for reference only and must not be taken out of the library. Copies for loan will have an attribute that indicates when they are due back in the library. After the due date, a fine is charged on a daily basis. Redraw Figure 37 to show the different kinds of copy that are held in stock, incorporating your answer to part (a).

Solution

- (a) Books and journals can be represented as different specialisations of publication, as shown in Figure 38.

- (b) Following the same use of generalisation as in part (a), you can use a hierarchy to show that copies for reference and copies for loan have different properties that you wish to model.

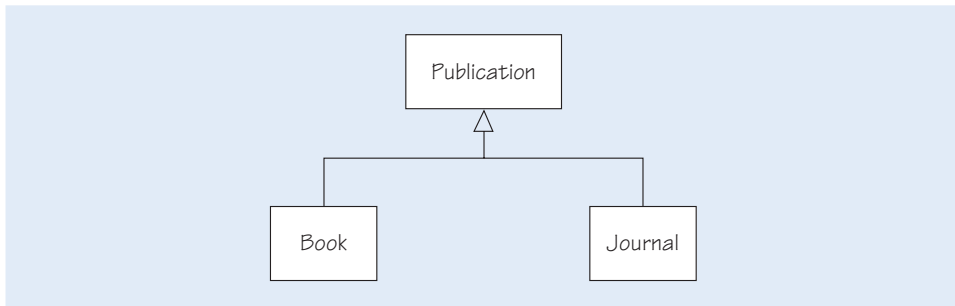


Figure 38 Class model for publications

Our solution is shown in Figure 39, where we have added a *returnDate* attribute to the class *LoanCopy* to indicate one of the differences between loan and reference copies.

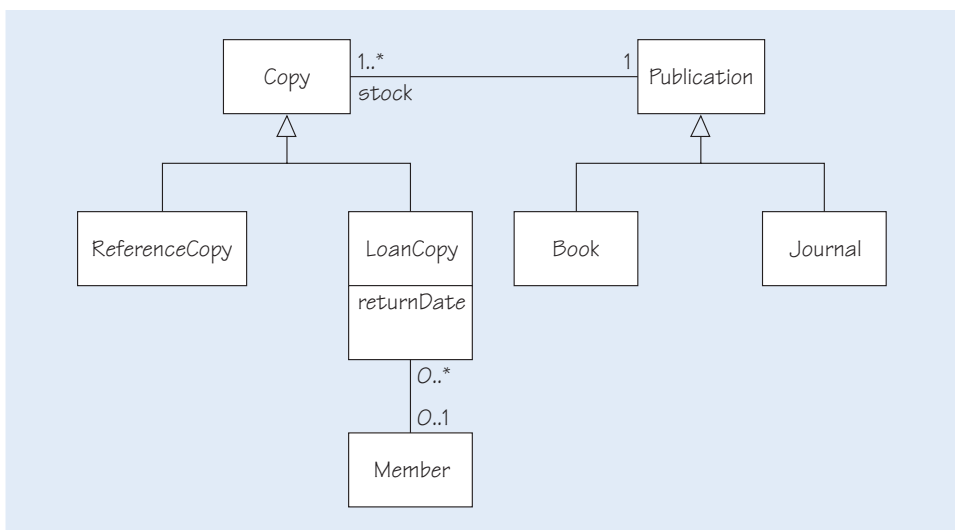


Figure 39 Class model for a library

As we have drawn the *Copy* hierarchy, it is possible to have instances of *Copy* that are neither for loan nor for reference. It is possible that people working in the library use copies of publications for their own purposes. If we wanted to exclude this possibility, we would make *Copy* an abstract class, showing this with the name in italics or with the stereotype *«abstract»*.

The substitutability rule indicates a potential problem with the class model in Figure 39. For example, if we wanted to lend a reference copy to someone (for a special purpose such as a research project), this model would not allow it. If that is not what the domain experts want, then the model must be changed.

Exercise 9

The fragment of a class model in Figure 40 shows that there is a payment associated with a sale, which represents a common activity when purchasing goods.

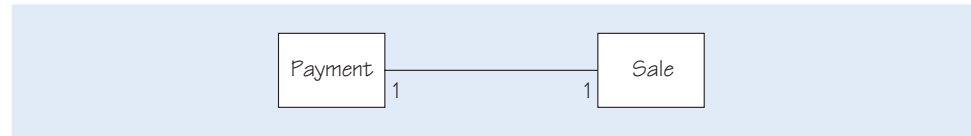


Figure 40 Fragment of a class model for making a sale

You are asked to model two different kinds of payment that are allowed at a supermarket checkout:

- cash
 - a debit or credit card, both of which require a PIN or signature.
- (a) Draw a fragment of a class model to show the two different ways of paying an amount of money for a sale at the supermarket.
 - (b) Describe how you would model the details of the different payment methods. (Hint: think about subclasses.)
 - (c) There are three general reasons that you might have for adding subclasses to a model. See if you can work out what these three reasons are, by examining your answers to part (b) above.

Solution

- (a) We have chosen to include a common attribute, called *amount*, in the superclass *Payment*. Figure 41 shows the two subclasses.

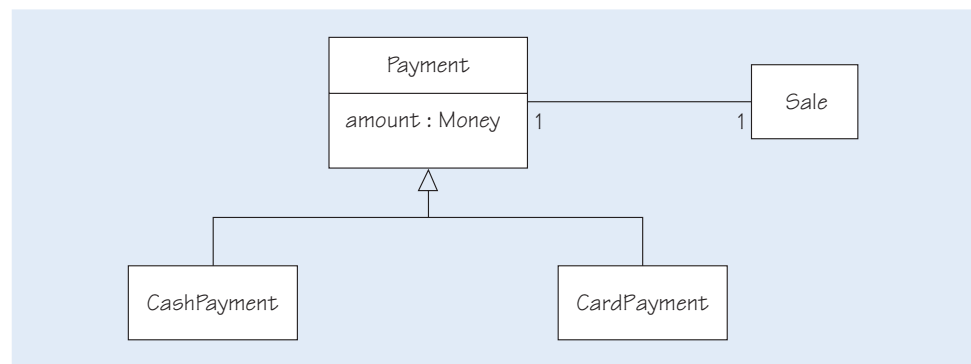


Figure 41 Class model for payments

- (b) Although there are common features, each subclass of *Payment* needs to be dealt with in a different way. A cash payment is the simplest. A customer uses a combination of bank notes and coins to pay for the sale. Often a customer hands over more cash than is required to cover the cost of a sale so there is a notion of change to be handed back, which may have to be shown on the till receipt. The other form of payment is

associated with the exact amount required, so no change is required (we will ignore facilities such as ‘cash back’).

Payments made by a card require the supermarket to be confident that they will receive the required amount of money and will require, for example, that the last four digits of the card’s number are shown on the till receipt.

- (c) The above discussion illustrates that there are three situations that could lead to the decision to include additional subclasses in a model:
 - (i) there may be additional attributes of interest
 - (ii) there may be additional associations of interest
 - (iii) there may be different kinds of behaviour that ought to be represented.

That there are precisely three situations is related to the fact that classes have attributes, operations and associations.

5.5 Summary of section

This section has illustrated the power and dangers of generalisation and specialisation. The creation of subclasses is a way of expressing the fact that elements share concepts. The subclass always contains the attributes and interface of the superclass, although the subclass may have additional attributes and operations. The substitutability principle – whereby an instance of a subclass can be substituted for an arbitrary instance of a superclass – should not be violated when considering generalisation and specialisation.

6

Constraining models

This section will introduce additional UML notation that makes it possible to capture aspects of the domain that cannot be expressed by the notation you have met so far. We look at how a model can represent facts about the domain that cannot be expressed just by classes, their operations, their attributes and associations with their multiplicities.

6.1 Constraints on classes

The UML notation we have described so far simply cannot capture all the static information about a domain. There can be many extra restrictions that should somehow be expressed. Although all the necessary extra information could be added to the project glossary, in practice it is better to incorporate it in the models if possible.

First, we will investigate how to add these extra restrictions, or **constraints**, to a model. Then we will go on to look at how constraints can be identified. A constraint can be expressed in a natural language, such as English, or using a special notation defined by UML. It is also useful to introduce the concept of an **invariant** – something that must be true about the system at all times – because the constraints we will study are concerned with ensuring that the invariants are not disobeyed.

Some common kinds of constraint are:

- constraints on the values of attributes
- constraints on associations
- uniqueness constraints (which can sometimes be converted to qualified associations).

To say that a *Person* has an integer *age* attribute is to assert that any integer value is permissible, such as 123 or 999. The attributes of the *Journey* class record the start and finish of a single journey of a train.

The simplest way to record constraints on a model is to attach a description of the constraint to the appropriate part of the model. In UML you can attach a constraint to a class by putting it within curly braces, as shown in Figure 42. The contents of the braces can be in any appropriate notation. English is the most common and for most purposes is perfectly adequate.

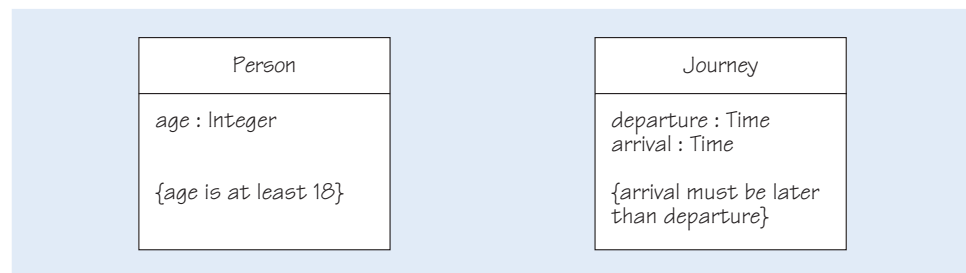


Figure 42 Constraints on classes

When considering constraints it is best to think of them in terms of particular instances, just as you did when you used object diagrams to consider

multiplicities. You could read the constraint on the *Journey* class in Figure 42 as, ‘for every journey, the departure time is earlier than the arrival time’. However often the times may be modified as they are rescheduled, this model tells us something about how the modification must affect the times.

A constraint can be applied to any model element or list of elements. Each invariant is a constraint upon the possible configurations of objects in a software system. In general a constraint defines a restriction that must be satisfied in any subsequent implementation. If your implementation breaks a constraint, it is incorrect with respect to the design.

Constraints are also passed on from superclass to subclass, which introduces a dependency in a model. A subclass cannot ignore an inherited constraint, but it can modify the constraint by strengthening it, that is, by adding further restrictions.

UML defines a formal notation called the **Object Constraint Language (OCL)**, which combines logical expressions with set notation to allow a more rigorous specification of a constraint. As you will see below, certain constraints are predefined while others may be defined by the modeller. In this module we will usually express our constraints in natural language, but occasionally we will use OCL.

Finally you should beware of the risk of introducing complexity into your class diagrams. If you show too much detail on a diagram, it becomes hard to read. It may be better to record only the most important constraints on a class diagram and record the remainder in your glossary or other form of project documentation.

In Unit 6 we introduce the concept of ‘design by contract’, which will allow you to control and coordinate the constraints in your model and apply them to a subsequent implementation.

There is a simplified summary of OCL in subsection 6.5.

The question of how many constraints you need is context-dependent. Try not to get carried away and create constraints simply because you can!

SAQ 12

- Figure 24 shows the class *Account* with an attribute called *accountNo*. Write an appropriate invariant that limits the values of this attribute to a number composed of a 6-digit sort code (to identify the branch) followed by an 8-digit account number.
- When is an invariant on a class true?
- What are the risks involved when you try to record all the possible constraints on a model?

Answer

- We are not told anything about the range of either branch codes or identifiers, but we do know that a valid account number has 14 digits as follows:
 $\{accountNo \text{ is a 14-digit number, with a 6-digit number at the beginning for the sort code followed by an 8-digit identifier}\}$
- The invariant on a class must be true for every object of that class from the time that object is created to the time it is deleted.
- The first risk relates to the complexity of the resultant model. If too many constraints are recorded on a model, that model will become difficult to read or comprehend. For each case you should decide

whether or not a given constraint adds value to your particular model. It may be more appropriately recorded in the glossary.

The second risk relates to the potential increase in the number and complexity of any dependencies that would arise for each additional constraint, especially those among two or more model elements.

6.2 Constraints across associations

Very often the constraint is not about a class, but is about the associations between classes. You have seen an example already in the discussion about qualified associations in subsection 4.5. There are many different kinds of constraint on associations that you may want to express, and UML provides special-purpose notations for the more common ones. We will begin with the two standard constraints that describe the properties of pairs of associations.

Figures 43 and 44 show two ways of expressing the fact that the *companySecretary* must be a member of *staff*. Figure 43 uses the special notation `{subset}` (attached to a dashed arrow) to express this constraint.

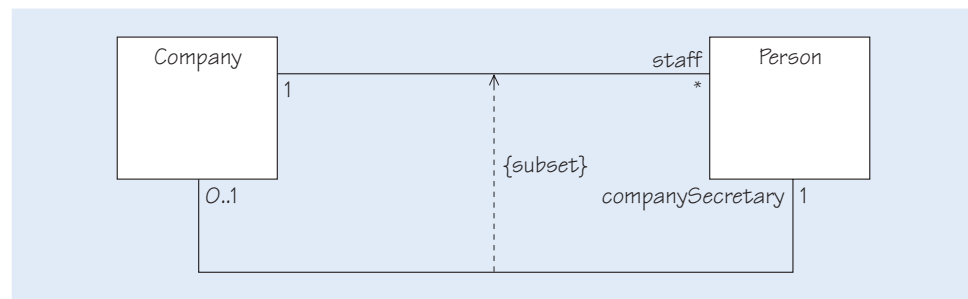


Figure 43 A formal constraint using `{subset}`

If you take into account *who* is likely to read the class model, it may be more appropriate to add a note to the model explaining the constraint rather than to use a special notation. Figure 44 shows an alternative, informal approach in a class diagram that may be shown to a user. It uses a note to explain the constraint.

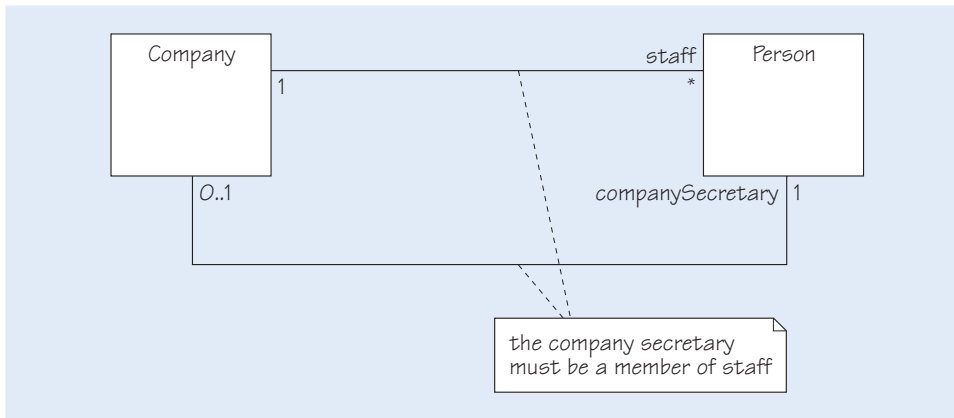
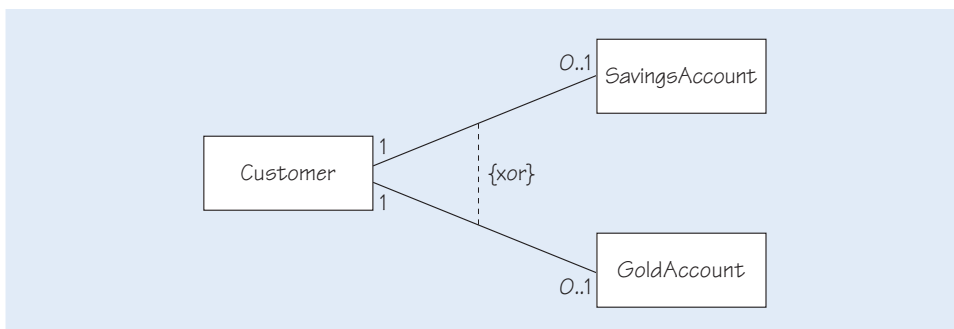


Figure 44 An informal constraint using a label

Another standard kind of constraint applies to associations from one single class (let's call it the base class) to two other classes. The special `{xor}` notation is used to specify that an instance of the base class must participate in exactly one of the associations grouped together by the `{xor}` constraint. Exactly one of the associations must always be active.

Figure 45 shows how this exclusive-or constraint can be used to express the fact that although the class *Customer* has two different associations, one with *SavingsAccount* and the other with *GoldAccount*, any particular instance of *Customer* will only have, at any time, one single link corresponding to these associations – a *Customer* cannot have both a *SavingsAccount* and a *GoldAccount*.

Figure 45 Controlling the use of different accounts using `{xor}`

6.3 Constraints on related objects

You have seen constraints on a single class, such as the examples in Figure 42, and constraints on associations, such as the examples in Figures 43, 44 and 45. Often constraints combine these elements. You might want to constrain the attributes of objects that are related by an association. In a family, we might want to capture the constraint that the ages of the parents must be greater than the ages of the children. In a company, there might be a constraint that an employee has a lower salary than that person's boss. You

`xor` is a Boolean operator that gives true as a result only if its two operands are one true and the other false.

If your modelling tool does not support constraints directly, place them inside a note and attach the note to the affected model elements.

In a different context, you might choose to model such relationships using inheritance.

can express constraints such as these informally with notes, following the example in Figure 44. However if there are a lot of associations on the diagram it is useful to have a name for the objects related by each association. This is where role names can be particularly helpful.

Figure 46 shows one way to represent relationships between children and adults – as father, mother and teacher. Children and adults both have an *age* attribute.

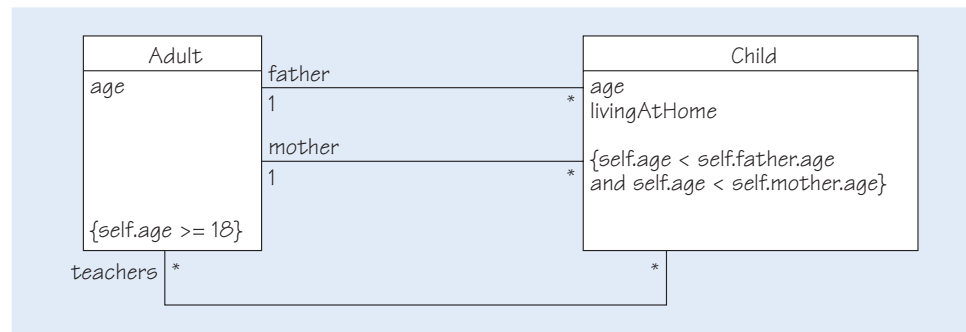


Figure 46 A constraint on related objects

To capture the common invariant that a child must be younger than both father and mother, we have written the following invariant for the class *Child*:

context Child **inv**:

```

self.age < self.father.age and self.age < self.mother.
age

```

The context of the invariant determines the starting point for the navigation expressions, *Child* in this case. Notice how navigation expressions, introduced in subsection 3.10, are used to distinguish the terms or elements in a model. Given a typical object of the class – referred to as *self* – we can refer to its attributes by using a navigation expression, as in *self.age* in the class *Child*. We can also refer to objects related to *self* by using a role name, as in *self.father*. We can therefore refer to attributes of related objects, for example *self.father.age*.

In Java, this is used instead of *self*.

SAQ 13

- In a model that contains the classes *Adult* and *Child*, where each *Child* has a mother and father that are instances of *Adult*, what constraints might you impose on relationships between the classes? Express the constraints in English.
- In the UK, it is a legal requirement that both parties to a marriage are at least 16. Should this be modelled as an invariant?
- How would you model the constraint in a hotel system that every bill must be paid with either a debit or a credit card? How would you extend your model if cash were to be allowed?

Answer

- (a) Every *Child* must have a mother and father that are instances of *Adult*. The father cannot be the mother. A father must be male and a mother must be female. Both adults must be older than the child.
- (b) Almost certainly not. A model is meant to express what the case *is* about rather than what it *ought* to be about. So unless the domain expert agrees that illegal marriages need not be represented, the model should allow them.
- (c) Figure 47 shows one way is to use the $\{xor\}$ notation. However this can relate only two associations. If you need to express a three-way constraint in order to allow for the addition of cash payments, for example, you will have to abandon the graphical notation of $\{xor\}$ and instead write a textual constraint. Alternatively you could use generalisation to create an abstract *PaymentMethod* class with an association to the class *Bill*, following the examples in Section 5. Then each payment method, such as debit or credit card or cash, would become a specialisation of the parent, abstract class *PaymentMethod*.

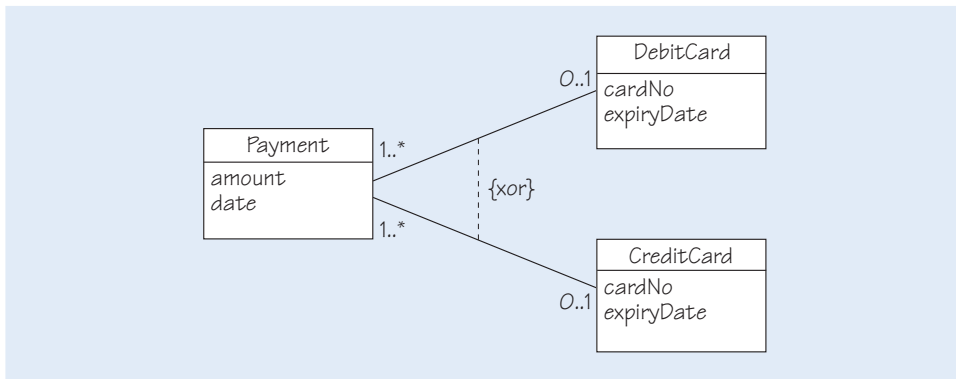


Figure 47 Payments using debit or credit cards

6.4 Finding invariants by considering loops in associations

You have seen several examples in which a plausible-looking initial model turned out to be too general. The model was under-constrained and needed an invariant to constrain it to allow only valid configurations. It can be extremely easy to overlook the need for an invariant. Everyone tends to interpret a model in their own way and to remain oblivious to other interpretations. When the model is passed to someone else to implement, that person may not share the same assumptions.

A particular type of invariant can arise from loops formed by associations. If there is a loop, there are two paths for getting from one class to another. Are these paths really independent or should there be a restriction? Look again at

Figure 6, reproduced in Figure 48, which has three loops – two loops linking *Hotel*, *Guest* and *Room*, and one linking *Guest* and *Room*.

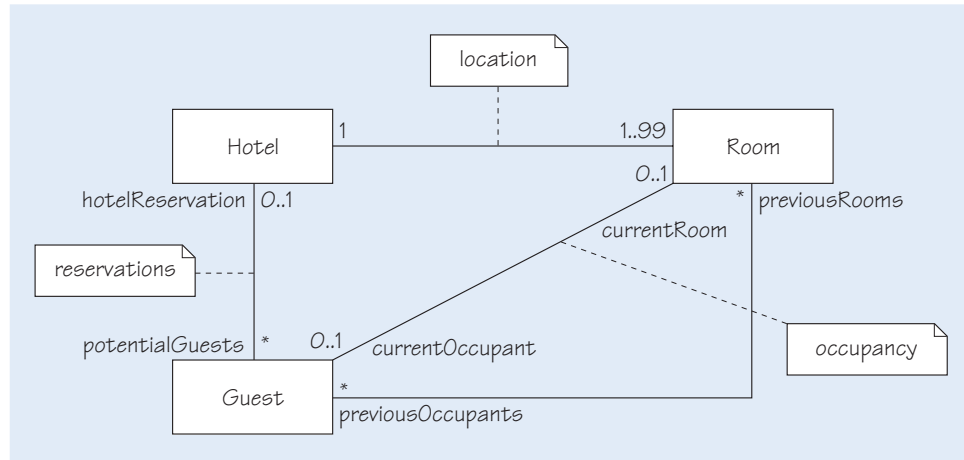


Figure 48 A class diagram with association loops

In this model the hotel owns rooms and guests occupy rooms. It is a slightly unrealistic example, just for the purpose of this explanation, where a guest can have no more than one reservation at a time. There are three ways of starting at a guest and reaching a hotel – directly from a guest to a hotel, or via a room (and there are two ways of doing this). You can ask which hotel a guest has a reservation in, or which hotel owns the room that the guest is occupying. These routes must yield the same hotel when both paths reach a hotel. Note that a guest may have no reservation or may not be occupying a room. A guest cannot stay in a room belonging to *theRitz* while at the same time having a reservation at *theSavoy*. Once you have spotted the need for a constraint it does not matter how you record it. English is satisfactory, but you could write an invariant for the class *Guest* in OCL as follows:

```

context Guest inv:
    self.currentRoom.notEmpty() implies self.currentRoom.
hotel = self.hotelReservation

```

Notice that, as mentioned in the discussion of navigation expressions in subsection 3.10, the default role name for an association end is the name of the class at that end of the association, but beginning with a lower-case letter. From Figure 48, for example, we have derived the default role name *hotel* at the *Hotel* end of the association between *Room* and *Hotel*.

One of the simplest forms of loop is when there are two or more associations between a pair of classes, or when a class is related to itself. There may well prove to be a constraint between the related objects. Figures 49 and 50 show some examples.

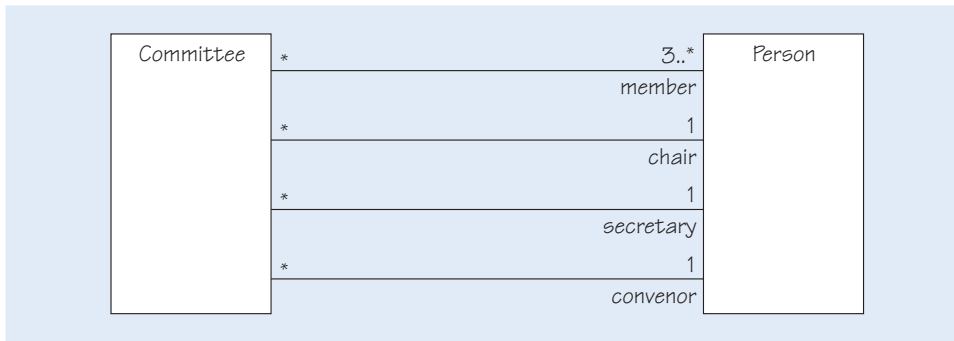


Figure 49 Some loops in a committee model

In Figure 49 the person who chairs a committee cannot be the same person as the secretary, although both must be members of the committee. However the chair might also be the convenor. Some loops or cycles in a model may be harmless, in which case there will be no need to add a constraint. The loop involving the chair and the secretary however reveals that the model allows illegal situations, so you must constrain the model accordingly by, for example, placing the following constraint in the class *Committee*:

context Committee **inv**:

```
self.chair <> self.secretary
```

In Figure 20 you saw that a library member could be associated with past loans and current loans. Figure 50 shows the same situation – this time with an intermediate class called *Loan*. Presumably no loan can be both a past and a current loan simultaneously. In other words, the sets of loans associated with a member do not overlap. You can express this constraint in OCL by saying that the intersection of *pastLoans* and *currentLoans* for a *LibraryMember* is the empty set, as follows. (Refer to Table 3 in subsection 6.5 for a description of OCL operators.)

context LibraryMember **inv**:

```
(pastLoans->intersection(currentLoans))->isEmpty()
```

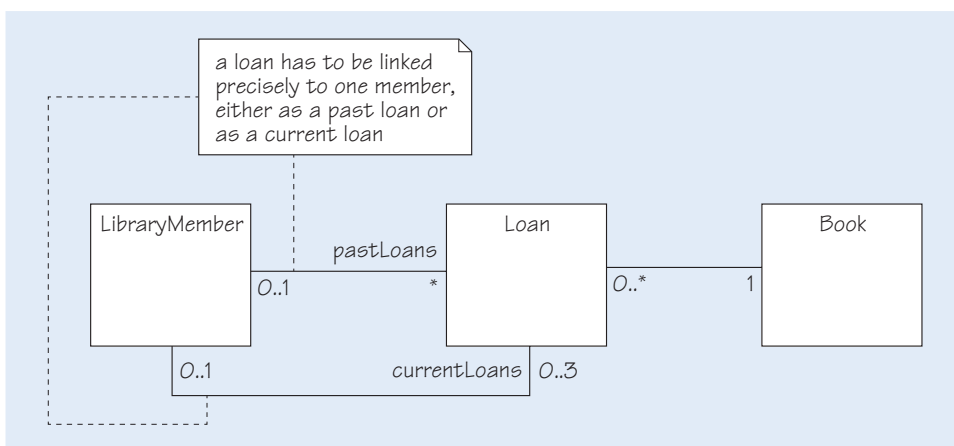


Figure 50 A possible loop in a class model for a lending library

The symbol `<>` should be read as 'not equals'.

When there is no confusion `self.attribute` can be written just as `attribute`.

We also need to say that, if a loan exists, it must be linked to exactly one instance of *LibraryMember*. The multiplicity of 0..1 on both associations leading from *Loan* to *LibraryMember* does not enforce this because this would allow a *Loan* to be linked both as a past loan to a member and as a current loan to the same or a different member. This constraint is written as a note in the figure. You could also express this with an *xor* constraint. In this case the base class is *Loan*. An instance of the base class (*Loan*) must participate in exactly one of the associations with *LibraryMember*.

We have used the equality operator (=) without discussing what it means. You might assume that the equality operator in

context Guest **inv**:

```
self.currentRoom.notEmpty() implies self.currentRoom.  
hotel = self.hotelReservation
```

is comparing equality of two objects. This is not necessarily the case as a navigation expression does not always return a single instance of an object, but instead may return a set of objects. In addition the object identity is being compared, not the object value. For example, using the model shown in Figure 48, the expression `aHotel.room` (where `aHotel` represents an instance of *Hotel*) will return a set of instances of *Room* objects, (in theory) one for each room in the hotel. For an equality expression to be true, the navigation expressions on each side of the operator must return the same set of instances. The sets are not equal if different instances are returned, even if the instances contain the same room information.

The inequality operator (<>) tests whether two sets are *not* identical, that is, that there is at least one object that is in one of the sets but not in the other. Again, note that what is compared is instance identity, not value.

The includes operator (as in `aHotel.room->includes(aGuest.currentRoom)`) tests whether the object returned by the expression on the right-hand side – (`aGuest.currentRoom`) – is an element of the collection on the left-hand-side expression (`aHotel.room`).

Notice that `aHotel.room` is a collection because the multiplicity at the *Room* end of the association between *Hotel* and *Room* is 1..99. In OCL, when the multiplicity at the end of an association has a maximum of 1 (0, 0..1, or 1), that association end is an object, otherwise it is a set. But even when the multiplicity has a maximum of 1, that association can still be used as a set, for example to test whether it is empty, e.g. `self.currentRoom.notEmpty()`.

SAQ 14

The model in Figure 51 expresses the fact that a train must be associated with two employees: a driver and a guard. By considering the loop of associations identify a problem with the model, and suggest a constraint that solves the problem.

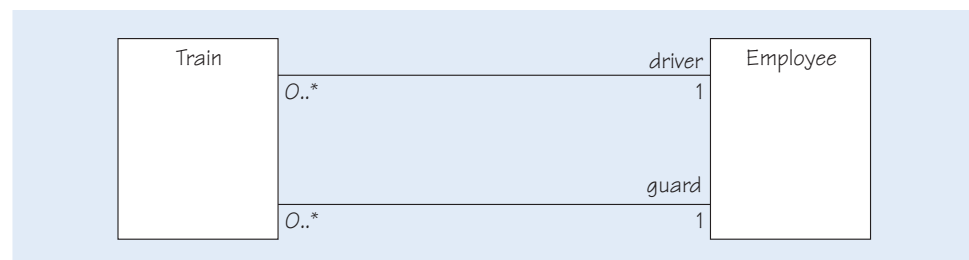


Figure 51 A loop that requires a constraint

Answer

You need to capture the fact that a given employee cannot be both the driver and the guard for a particular train. You can formulate this as a constraint on any class round the loop.

For example, the following constraint can be located in the class *Train*:

```
context Train inv:
    self.driver <> self.guard
```

SAQ 15

The model in Figure 52 shows that a person takes out a mortgage to buy a house. A person must offer that house as security against the mortgage. A person can take out more mortgages, each secured by a house. By considering the loop of associations, identify a problem with the model and suggest a suitable constraint using OCL.

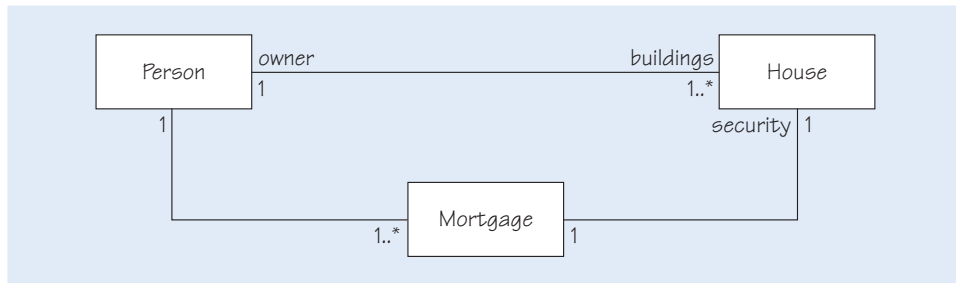


Figure 52 A loop that has a complex constraint

Answer

The model would allow one person to have a mortgage that uses someone else's home as security. You need to capture the fact that a house's owner is the same person who offered that house as security for a given mortgage. When there are multiplicities in the loop that can be greater than one, you need to take care in writing the invariant. In one direction around the loop an individual house has only one owner. In the opposite direction each house is security for only one mortgage, and that mortgage is associated with only one person. This makes it easy to place an appropriate constraint on either the *House* class or the *Mortgage* class. For the *House* class:

```
context House inv:
    self.owner = self.mortgage.person
```

Similarly for the *Mortgage* class:

```
context Mortgage inv:
    self.person = self.security.owner
```

However expressing an equivalent constraint on the class *Person* is more problematic because you would have to find the intersection of two sets: the houses owned by a particular person and the houses secured by

mortgages taken out by the same person. We suggest that you use English to formulate a suitable constraint in such circumstances. You could place the constraint as text inside a note and attach it to the appropriate class.

Exercise 10

Suppose that there is a class *Person* in a model, and you have identified the attributes

title

firstName

lastName

gender

houseNumber

streetName

city

postcode

all of which are represented as strings. Write an appropriate invariant that will constrain the attribute values to exclude invalid values.

Solution

You might constrain some fields to have appropriate capitals and constrain others to contain only values from a fixed set as follows:

{*title* must be spaces, or one of ‘Mr’, ‘Mrs’ or ‘Ms’}

firstName and *lastName* are written in lower case, but upper case is applied either to the first character of a word or after a space

sex must be ‘M’ or ‘F’ or an empty string

houseNumber must contain only digits, forming a number greater than zero, optionally followed by a letter

streetName and *city* are written in lower case, but upper case is applied to both the first character of a word and any character after punctuation or spaces

postcode may contain upper-case letters and digits, and a single space}

The invariants are certainly all things that must remain true, but they may not be the whole truth. For example, the constraint given above is *necessary* for a postcode to be valid but it is not *sufficient*: a string could conform to it but still not be a postcode that actually exists. However the constraint *will* always be true, if the system is implemented correctly.

Unfortunately the invariant for *title* is probably incorrect. There are other commonly approved titles such as ‘Dr’ and ‘Sir’.

Exercise 11

Figure 53 shows a fragment of a class model for a chain of video libraries. It includes an attribute called *location* for the class *VideoLibrary* to show where each branch is located. The class *Member* has been given two attributes: *name* and *address*. Identify a problem with the loop of associations in this model. How could an object diagram help you identify the problem?

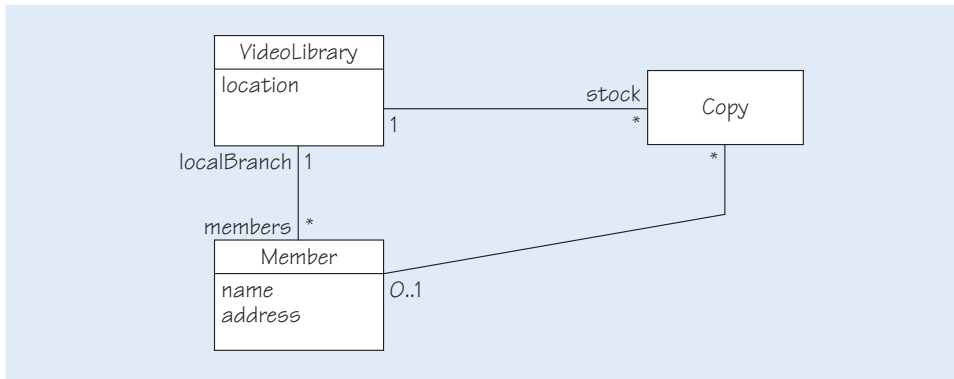


Figure 53 A class model for a video library

Solution

The loop in the associations for the video library's class model is similar to that for the hotel's class model that you saw in Figure 48. That is, there are two ways of starting with a member and ending with a library. You can ask the member either which library he or she has borrowed the stock copy from, or which library owns the stock copy that the member is borrowing. Both routes must yield the same library.

In Section 2 you saw that an object diagram can be useful for determining whether a class model is correct and sufficiently constrained, because an object diagram represents a particular configuration at a particular moment in time. For example, you could show a particular valid configuration of video library objects as in Figure 54. In another object diagram, you might (try to) show an invalid configuration. If you can show an invalid configuration, then you might want to add constraints or change the model to prevent it.

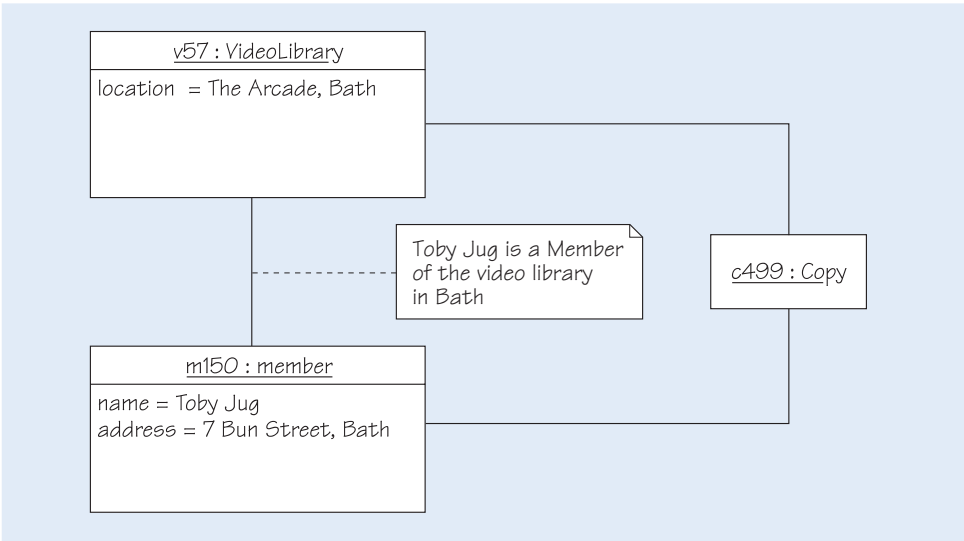


Figure 54 An object diagram for the video library

6.5 OCL summary

OCL is a query language. It allows the user to write queries about a model and to express constraints on a model.

The basic OCL types include Integer, Boolean and String. Table 3 lists some OCL operators, some of which you will only encounter later. This table should be used as a reference whenever you encounter an OCL operator. Do not worry if you have not yet encountered many of the operators below.

Table 3 OCL operators

Type	Operator	Examples
Comparison	=, <, >, ≤, ≥, <>	(see examples above)
Logical	and, or, xor, not, implies	a and b, a implies b, ...
Integer	+, -, *, /	
String	concat(string): String	s1.concat(s2) – the string consisting of s1 concatenated with s2
Collection (operators on collections are separated from the collection by ‘->’) A collection can be a set, a bag or a sequence	size(): Integer	s1.size() – the size (length) of s1
	isEmpty(): Boolean	c1->isEmpty() – true when c1 is empty
	notEmpty(): Boolean	c1->notEmpty() – true when c1 is not empty
	count(): Integer	c1->count() – number of elements of c1
	includes(object): Boolean	c1->includes(o1) – true when o1 is an element of c1
	includesAll(collection): Boolean	c1->includesAll(c2) – true when all the elements of c2 are members of c1
	any(booleanExpression): Element	c1->any(e1) – one element of c1 for which e1 is true
	select(booleanExpression): Set	c1->select(e1) – all elements of c1 for which e1 is true
	forAll(booleanExpression): Boolean	c1->forAll(e1) – true if e1 is true for all elements of c1
	exists(booleanExpression): Boolean	c1->exists(e1) – true if e1 is true for at least one element of c1
Set	= (set): Boolean	s1 = s2 – true when s1 and s2 contain the same elements
	union(set): Set	s1->union(s2) – the set containing all elements of s1 and s2
	intersection(set): Set	s1->intersection(s2) – the set containing all elements that are in both s1 and s2
	- (set): Set	s1 - s2 – the set containing all elements that are in s1 but not in s2

6.6 Summary of section

In the early stages of analysis, class models may be too general because they are not constrained by the rules that govern the domain in question. Most models require constraints to limit them to valid configurations, and there are various notations for expressing them.

- In UML, attributes of a class can be constrained using an invariant in order to express one of the rules of the domain.
- UML provides a number of ways of expressing constraints on associations. There are special notations such as {xor}, as well as the formality of OCL

or the informality of natural language. When choosing which notation to use, you should consider the skills of those who will read the model.

- Associations in a class model can form loops. Such loops may be totally independent, but can often hide a restriction that should be expressed as a constraint.

Summary

7

This unit has introduced the parts of UML needed to build class models of some sophistication. You have seen some of the shortcomings of modelling using classes. From the basic notation for classes and associations, you have developed more precise notations such as qualified associations. You have seen the need for constraints on over-general models and a way of constraining a model to reduce dependencies among classes.

At this point, you should be able to describe the static structure of a set of classes and their associations for your potential software system. At the conceptual modelling stage, your focus will be the key domain abstractions, their properties and their relationships. As your focus shifts to the specification of a solution, you will begin to associate conceptual elements with software elements in your model.

On completion of this unit you should be able to:

- create a list of candidate classes from a requirements document
- model the classes and their attributes using UML
- identify relationships between classes
- understand the difference between class and object models
- identify conditions that will limit the potential behaviour of model elements
- identify the main limitations of class models
- model associations between classes
- identify and represent the navigability of associations in a class model
- express generalisation/specialisation relationships in UML
- recognise the need for constraints on class models
- express constraints within a single class
- express constraints on associations
- test models with loop associations for the need for an invariant
- be aware that modelling should only be carried out while it is useful.

References

- Ambler, S. (2002) *Agile Modeling* [Online], Hoboken, NJ, John Wiley & Sons. Available at <http://libezproxy.open.ac.uk/login?url=http://open.ebilib.com/patron/FullRecord.aspx?p=131031> (Accessed 2 June 2014).
- Larman, C. (2004) *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* [Online], Upper Saddle River, NJ, Prentice Hall. Available at <http://libezproxy.open.ac.uk/login?url=http://proquestcombo.safaribooksonline.com/book/software-engineering-and-development/uml/0131489062> (Accessed 2 June 2014).
- Robertson, S. and Robertson, J. (2012) *Mastering the Requirements Process* [Online], Upper Saddle River, NJ, Addison Wesley. Available at <http://libezproxy.open.ac.uk/login?url=http://proquestcombo.safaribooksonline.com/book/software-engineering-and-development/software-requirements/9780132942850> (Accessed 2 June 2014).

Unit 6 Dynamic modelling – from analysis to design

Contents

1	Introduction	77
2	Design by contract	78
	2.1 Contracts to produce quality software	79
	2.2 How can design by contract help to improve quality?	80
	2.3 Contracts in the real world	81
	2.4 Contracting, subcontracting and inheritance	84
	2.5 Summary of section	89
3	Starting dynamic modelling	90
	3.1 Focusing on a solution	90
	3.2 Starting to build a sequence diagram	92
	3.3 Assigning responsibilities in design	96
	3.4 Sequence-diagram notation	97
	3.5 Making design decisions	101
	3.6 Class-responsibility-collaboration cards	108
	3.7 Summary of section	109
4	Working with interaction diagrams	110
	4.1 Communication diagrams	110
	4.2 Message results and parameterised interactions	114
	4.3 Creation and deletion	117
	4.4 Levels of detail	119
	4.5 Summary of section	123
5	More on design decisions	124
	5.1 Managing associations	124
	5.2 Link manipulation	125
	5.3 Forks and cascades	129
	5.4 Summary of section	134
6	Summary	136
	References	137

Introduction

The aim of this unit is to show you how to make the transition from the *analysis* of a software solution consisting of use cases and a class model, indicating *what* is intended of the software, to a *design* of how this solution is going to be implemented, detailing the responsibilities (operations) of classes in the class model.

The approach we illustrate here, while requiring you to make careful choices, is to a large degree systematic. With experience, and depending on how critical the application you are working on is, a more agile approach may be appropriate. Indeed you may find that the diagramming techniques presented are often not needed, or that once you have worked on the design of one or more classes you have enough insight to directly code the other classes. Or it may be appropriate to use them mainly for particularly complex behaviour. However, by studying a systematic approach you will gain knowledge of the key issues that need to be addressed, whether explicitly through models or implicitly guided by experience. In other words, from a professional perspective, the key thing to understand is the issues this unit addresses rather than the particulars of how we address them.

By being clear about the way a system is designed, we can ensure that it has an understandable structure. This will help with the system's maintainability and make it easier to trace from the requirements to the design.

As mentioned in Block 1 Unit 1, one of the key things to determine early in design is the overall system architecture. In this unit we will assume, for simplicity, that we are dealing with a layered model and working on the applications layer.

2

Design by contract

Perhaps somewhere in your studies you have asked yourself the question, ‘What is it about objects that works?’ There is a simple answer to this question: ‘Object orientation works because it uses “real-world-shaped pieces” to build systems.’ Objects exist in the real world – you see and interact with them every day, and use them to build other things; in this way they are familiar to use. You gain the ability to work with objects very early in your life – the notion of an object, and your ability to work with objects, very quickly becomes part of your intellectual equipment.

Object-oriented software development builds on this familiarity by providing a traceable path from the conceptualisation of real-world objects during analysis and their construction during design, to their use during implementation. In the end, the objects that you implement become the software system. Of course this doesn’t mean that the classes we deal with during design do not significantly differ from those during analysis.

So what can be done to make sure that the objects implemented in software will behave according to the users’ requirements? One way to ensure that a software system performs its intended functions (its specification) is based on the real-world concept of a contract. When an object sends a message to another object, a form of contract exists. The receiver is being asked to perform a service for the sender. In this sense, the sender is viewed as a client that requests a service from a supplier (the receiver). Whenever a service is provided, a contract comes into play: the client expects the supplier to perform the service correctly, and the supplier expects to be asked to perform only those services it knows how to supply. If either of these expectations is not met, the contract has been broken.

In real life, contracts are very often formal and binding agreements between two parties; in object-oriented software development, contracts are used to produce more formal descriptions of objects and the services they provide. Identifying roles in terms of clients and suppliers enables designers to specify the responsibilities of objects more precisely. This allows clearer software to be built and, in turn, leads to greater confidence in the correctness of the software – that is, confidence that the users’ requirements are being met.

Throughout this module we have concentrated on how to build software that satisfies the customer. You have studied how a customer’s requirements provide the impetus for software development, and how important a good understanding of them is to the delivery of a product that truly meets the customer’s needs.

In Unit 5 we looked at static modelling representing the structure of the entities, be it a domain model or a model of a software solution. We now turn to the representation of behaviour, using dynamic modelling, during which the operations appropriate to each class in a structural model are identified. Precisely what each operation expects and should achieve should be specified by pre- and postconditions, using the principles of **design by contract (DbC)**. In this unit however we will concentrate on the identification of the

responsibilities of each class, and how these are carried out in terms of its operations; that is, what a class is responsible for doing and how. We will defer the detailed discussion of contracts until Block 3 Unit 10, where you will see how pre- and postconditions, together with class invariants, can be incorporated into an implementation.

2.1 Contracts to produce quality software

Earlier in this module you saw how difficult it can be to obtain a correct set of requirements, and how regular meetings with the customer and users are necessary to clarify their requirements. In effect, this process develops a contract between the developer and the customer.

Use cases provide a language for describing requirements so that they are understandable by both technical people (developers) and non-technical people (customers and users). A significant part of a use case is the preconditions and postconditions that constrain it. Preconditions, postconditions and invariants are collectively known as **assertions**, which are statements that are, in theory at least, either true or false. Assertions specify details of the requirements that cannot be represented graphically. In specifying the services the software should provide, assertions embody the contract between customer and developer by stating (we hope unambiguously and completely) what the customer wants, and hence what the developer must provide.

We have shown how assertions can be introduced early in the development process, during analysis. As development progresses, the assertions will be refined, with more detail being added. In this and subsequent units, you will see how assertions can be carried through into design and then into implementation. Importantly, you will see how some assertions (ones that can be expressed in code) can be included in the final code to be checked both by the compiler and by the run-time system. The use of such explicit contracts helps to provide a traceable pathway from analysis, through design, to implementation, that shows how the assertions were developed and relates the code directly to the requirements. This process is known as design by contract, or DbC for short.

At the implementation level, if the software representing the client and supplier meets its contract, we can say that the software is **correct** with respect to its specification.

There are four main advantages to the DbC approach for adding assertions:

- 1 Assertions provide accurate documentation for the implemented classes so that a programmer knows how to use the classes and what to expect from them.
- 2 Provided that they are executable, assertions are an important aid to testing, but without being an obstacle to efficiency.
- 3 Assertions provide a way of controlling inheritance in which substitutability and redefinition of methods are allowed.
- 4 Provided that the programming language has an exception mechanism that accords with the principles of DbC, assertions together with the exception-

Units 3 and 5 showed how pre- and postconditions and invariants can be used in analysis and design.

In Block 3 Unit 10 you will see how such contracts can be expressed during implementation.

You may have used JUnit when writing Java code. When using JUnit you can write assertions in the special test classes.

However, quite independently of JUnit, you can put `assert` statements into methods to check the input and output conditions. These `assert` statements are then executed every time the method is run, verifying that the contracts are being respected. You can use compile-time options to turn off the assertions when a system is released to production.

handling mechanism can be an important aid to developing mission-critical systems.

SAQ 1

- (a) In the analysis phase of system development that you studied in Units 3 and 5, what were the assertions (preconditions, postconditions and invariants) used for?
- (b) Explain why pre- and postconditions express a contract between a client object and a supplier object.
- (c) What is meant by the term design by contract (DbC)?

Answer

- (a) Assertions were used for placing constraints on the relationships between classes.
- (b) The contract is expressed by:
 - the precondition requiring something from the client object that is of benefit to the supplier object
 - the postcondition requiring something from the supplier object that is of benefit to the client object.
- (c) DbC is the process of developing software based on the notion of a contract between objects.

2.2 How can design by contract help to improve quality?

DbC complements many of the other tools that the software engineer uses to improve quality. In this subsection you will explore where DbC fits into the object-oriented technology toolkit.

Most modern software-development environments include tools that analyse code to detect defects. That is, the tools analyse the structure of the code before it is executed. Such tools are collectively known as **static analysis tools**, as they do not execute the code. For example, certain tools can estimate the complexity of code using the number of lines of code and rules about the chosen implementation language, such as the number of branching statements.

Perhaps the most effective static analysis tool in use at this time is the **compiler**. A compiler will detect *all* syntax errors in a piece of code. Indeed, a compiler can be made more effective in detecting defects by careful choice of programming language features: languages that use strong typing (in the way that Java does), for instance, constrain the programmer more but allow the compiler to detect a wider range of defects than just syntax errors.

An example of a **dynamic tool** is testing, which you will study in Unit 11. Testing can easily occupy 40 per cent of total project effort (Pressman and Ince, 2000). Block 3 Unit 11 also introduces **formal technical reviews (FTRs)** which, at all stages of the development process and on all of its

In Block 3 Unit 11 a **defect** is defined as a verified lack of conformance to requirements, that is, something that should be improved.

products, complement static analysis tools and testing. Like testing, FTRs can cost a great deal.

There is a group of software systems known as critical systems, whose failure would have catastrophic consequences such as loss of life (safety-critical systems), loss of business (business-critical systems) and failure to meet significant objectives (mission-critical systems). It is vital that such systems be as error-free as possible, and so even the most expensive development methods and tools can be justified. It is common in the development of such systems to use formal methods, which are based on mathematical notations, logic and proof.

Other than formal methods, which are generally used only in critical-system development, the analysis tools mentioned above work only on the products of the development cycle and do not link its stages – they do not provide a traceable pathway from requirements through to code. In the quest for quality this is a major problem. In Block 3 Unit 11 you will see that ensuring the mutual consistency of system descriptions at different levels is a very important aspect of quality throughout development. DbC is a cost-effective way to develop software products whose features are traceable from the customer's requirements.

SAQ 2

To what extent can DbC help with traceability, and hence be used to improve the quality of a software system?

Answer

DbC allows the development of a software system to be traced from requirements through to code.

2.3 Contracts in the real world

In business, **contracts** are specifications of the obligations of, and benefits to, the parties involved in an agreement. For example, suppose that you (the client) wish to take advantage of a low-cost holiday package from a local travel agent (the supplier). The holiday must be paid for in full at least 1 month in advance of travel, and the payment is non-refundable. Both you and the travel agent benefit from such a deal but you both have obligations. These are shown in Table 1.

Table 1 Contract – obligations and benefits

	Obligations	Benefits
Client	pay full cost of holiday in advance	obtain a low-cost holiday
Supplier	provide the advertised holiday (for example, air travel, hire car, accommodation)	make profit on the sale, whether or not the client is able to travel – no need to worry about customers who have not paid

Making the various reservations with a hotel, airline and car-rental company involves the travel agent in subcontracts. That is, in discharging the obligations to you the travel agent becomes the client in other contracts. In this scenario we can say that the travel agent *uses* other service providers in order to honour the contract with you. Moreover, a subcontracted supplier might (in this case, for reasons of economy) try to satisfy many customers' obligations at the same time. For instance, the travel agent might have subcontracted a hotel to supply 100 rooms over the holiday period. However, in order that the contract between you and your supplier (the travel agent) is satisfied, the subcontractor (the hotel) must supply you with a room at the time when you want to take your holiday.

The obligations of a 'small' contract can, therefore, be discharged by a 'larger' contract. Another example will better illustrate what 'enlarging' a contract means.

Imagine that you have set up a dot-com company to supply email addresses to junk mail companies. You currently have four million email addresses but your list grows every night as your robot trawls the web to find more. As you might capture the same email address many times, you need a way of removing duplicates from your list. You decide that the best way to do this is to sort the email addresses; this will enable the duplicates to appear together, and they can then be removed easily. Now, although you are able to develop code to delete duplicates from the sorted list, you decide to buy an off-the-shelf component that will sort the large number of email addresses quickly.

You determine that you need to sort at least 10 million email addresses at a time. You also determine that, because you need your list updated daily, you want the sorted list to be output from the component in less than 10 minutes. The first requirement contains the precondition of the sort component: it must allow at least 10 million email addresses. The second requirement contains the postcondition of the sort component: it must complete in less than 10 minutes.

Sorting algorithms are well known as components, and so you contact four componentware suppliers who send descriptions of their sorting products to you. You draw up Table 2 to help you decide which sorting component to choose.

Table 2 Description of sorting products

Company	Precondition: number of email addresses $< N$	Postcondition: takes at most time M to sort	Cost
Microsort	$N = 1$ million	$M = 5$ seconds	£250
ABDC Sorting plc	$N = 10$ million	$M = 20$ minutes	£300
AeLPP Componentware	$N = 15$ million	$M = 10$ seconds	£1000
DictoSort	$N = 12$ million	$M = 8$ minutes	£100

Which one would you choose? Well, one requirement is that the component must be able to sort an input list of not fewer than 10 million email addresses,

so you can exclude the Microsort product because the requirement exceeds the number permitted by the component's precondition. All the other companies' components satisfy this requirement on inputs. However, the ABDC Sorting component's postcondition does not satisfy the requirement as it would take too long to sort 10 million email addresses, and therefore it too can be excluded. The other two components, however, both satisfy the requirements.

You will notice that both of the products that satisfy your requirements are capable of doing *more* than is required. They both have a precondition that is weaker than (contains) the requirement and both have a postcondition that is stronger than (contained within) the requirement. This illustrates a general case: a supplier's services will satisfy your request if their constraints on inputs (preconditions) are a weakening of (or identical to) your requirements and their constraints on outputs (postconditions) are a strengthening of (or identical to) your requirements.

The same is true in object modelling terms. An object offering a service will satisfy your requirements if its service's preconditions are weaker than (or identical to) your precondition and its postconditions are stronger than (or identical to) your postconditions.

Actually, as in the real world, being able to supply a product that satisfies weaker conditions on inputs and stronger conditions on outputs is very convenient for the client, as it greatly facilitates reuse. A number of contracts can be satisfied by the same product, and thus the necessity to build a new product to satisfy a new customer is removed.

SAQ 3

What is the relationship between the client's and supplier's obligations and benefits?

Answer

A client's obligations to constrain inputs provide benefits to a supplier in that fewer input cases need to be considered. A supplier's obligations to produce outputs satisfying certain constraints mean that a client can expect to receive a clearly defined service.

SAQ 4

- (a) What does weakening a precondition mean in terms of the provision of a service?
- (b) Similarly, what does strengthening a postcondition mean?

Answer

- (a) Weakening a precondition means generalising the situation in which a service can be provided. In general, this means that it is easier for the client to satisfy the precondition because there are 'fewer' conditions to be satisfied.
- (b) Strengthening a postcondition means making the service that is requested 'better' in terms of time, precision or some other

measurable item. The precise notion of ‘better’ is not fixed, but must be considered in terms of the contract of which the postcondition is part. It can, however, make the postcondition more difficult for the supplier to satisfy because there are ‘more’ conditions to be satisfied.

2.4 Contracting, subcontracting and inheritance

In Block 1 Unit 3 you recorded use cases that described what tasks a software system will carry out. One way of describing what each task, activity or operation commits to achieve is to write a form of contract as follows:

- a description of the task, activity or operation in question
- a set of constraints that are assumed to be true before you can start – known as a precondition
- a set of constraints that must be true afterwards – known as a postcondition.

The aim is to describe the state changes required of an operation, activity or task without having to say how they might be achieved. Each contract tells you about the constraints on the expected behaviour of an operation. In effect, the design can be deferred while you focus on what must happen.

In the context of software development, a contract is between two objects: the client object and the supplier object. The contract comes into effect when the client uses one of the services that the supplier object provides – when the client object sends the supplier object a message to invoke one of its methods. As the client object wants the server object to do something for it, you can view this relationship as a contract, similar to the one we looked at in subsection 2.3, with conditions on inputs (preconditions) and outputs (postconditions).

As an example of how a contract works, consider a banking system in which there is a method for withdrawing an amount of cash from an account. Assume that there is a limit on the amount by which the account can be overdrawn. We can specify what the method should achieve, and what should be true before the method is invoked, as a contract with the following pre- and postconditions.

Precondition: there must be sufficient funds in the account to permit the operation to complete without exceeding the overdraft limit.

Postcondition: the account will have been debited by the requested amount.

Such pre- and postconditions place obligations on both client and supplier, and in return they provide benefits for both.

- To satisfy the precondition, the client must establish that the balance of the account allows the amount to be debited without exceeding the overdraft limit; the supplier does not need to check the balance. (In the case of software, if the client does not respect the precondition, the supplier’s behaviour is undefined.)

- To satisfy the postcondition, the supplier must have debited the amount from the account, and the client must have received the service that it required. If the postcondition is not satisfied, the supplier has not provided the advertised service and the client has the right to complain. (In software terms, a supplier that does not meet its postconditions when all preconditions are met is deemed incorrect.)

In practice, you might discover the appropriate contract for an operation by asking two basic questions.

Precondition: what is needed for this operation to be allowed to start?

Postcondition: what will have happened as a result of this operation?

In the process of answering these questions you may discover new classes, attributes or associations and will have to amend your models accordingly. For example, for a client of the above-described bank account class to respect the precondition it must be able to find out what the overdraft limit and current balance for the account are, which might imply the addition of an attribute, operation or association.

Subcontracting and inheritance

When we examined the generalisation relationship in Unit 5, we emphasised the use of the substitutability test to determine whether one class was a generalisation of another. If an object of one type can be substituted for an object of another type in all circumstances, then the type of the first object is a subtype of the second; that is, the type of the second object is a generalisation of the type of the first.

In contracting terms this means that if the object *objA* is an instance of the class *A* and the object *objB* is an instance of the class *B*, where *B* is a subclass of *A*, then *objB* can be substituted for *objA* (*objA* can be replaced by *objB*) but must satisfy any contracts that *objA* may be involved in, without clients being affected. A client of *objA* will be expecting a certain contract – it expects that if the precondition is met, the postcondition will be met. The substitution of *objB* for *objA* must not affect this contract – it must still be honoured. (Note that overridden methods should be given careful attention.)

When we come to examine the idea of subcontracts in software, we must bear in mind that classes relate to one another in two basic ways:

- through associations
- through generalisations.

It turns out that an important relationship must exist between the operations of a class and the operations of its subclass if the objects of the subclass are to be substitutable for those of the parent class. This relationship can be expressed in terms of the pre- and postconditions associated with the equivalent methods in the two classes. The subclass must neither strengthen the precondition (make the precondition more restrictive or, in other words, work for a smaller set of inputs) nor weaken the postcondition (deliver less service) in order to guarantee substitutability. You will explore this idea in

more detail in Block 3 Unit 10 when we move from design to implementation – where operations become methods.

Constraints on behaviour

In the domain of a lending library you might identify two basic use cases:

- borrow a book
- return a book.

In the case of borrowing a book, you expect to record a new loan while making sure that the member does not exceed the allowance of three books on loan. The ‘contract’ between the member (the client) and the library (the supplier) to ‘borrow a book’ is constrained as follows.

Precondition: the number of books currently on loan to this library member is less than 3 (otherwise the book cannot be taken out).

Postcondition: the number of books currently on loan to this library member will have been increased by 1; the library member will have been linked to a new loan; the book will have been linked to the same new loan.

A software system for such a lending library would be expected to support the borrowing and returning of books. During the design activity you would develop a class model that meets the contractual requirements of each use case. Each pre- and postcondition must be translated into your design. For object-oriented systems, this means that you would be constraining the behaviour of the system in terms of:

- objects that are created and/or destroyed
- links between objects that are created and/or destroyed
- attributes whose values are changed.

For example, if you chose to have a class called *Loan* to represent the relationship between library members and the books that they borrow, you would have to consider the consequences of creating each new *Loan* object in terms of how it relates to other objects.

SAQ 5

- (a) Under what circumstances is one object, *obj1* of class *A*, say, substitutable for another, *obj2* of class *B*, say?
- (b) Once a class model that meets the contractual requirements of each use case has been developed, what three sets of items in this class model should you examine to help you find suitable postconditions when identifying the possible operations for a class?
- (c) Figure 1 shows a class model for the lending of books where there is a requirement to record both past and current loans. Describe the contract to borrow a book in terms of objects and links created. Your answer should differentiate between the pre- and the postconditions.

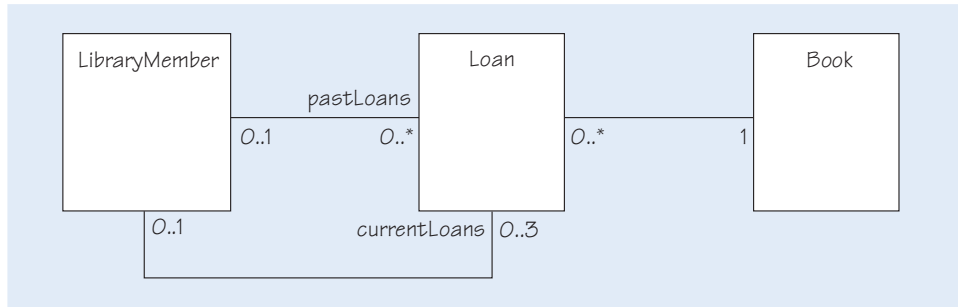


Figure 1 Class model for a lending library

Answer

- (a) Class *A* must be a subclass of class *B*, and class *A* must respect all contracts agreed to by class *B*.
- (b) You should investigate the following three sets of items when searching for the possible postconditions for an operation:
 - instances of a class (its objects) that have been created or deleted
 - instances of associations (links) that have been formed or broken
 - attributes that have been modified.

It is crucial of course that the postconditions reflect the requirements, and some practitioners would refer more directly to the requirements.

- (c) The contract between the library member and the library to borrow a book is constrained as follows.

Preconditions:

there must be an instance of the class *LibraryMember* that corresponds to the real-world member

there must be an instance of the class *Book* that corresponds to the real-world book that the member wants to borrow

the instance of the class *LibraryMember* must be linked to fewer than 3 instances of the class *Loan* in the role of *currentLoans*.

Postconditions:

a new instance of the class *Loan* will have been created and the instance of the class *LibraryMember* will have been linked to the new instance of *Loan* in the role of *currentLoans*

the instance of the class *Book* will have been linked to the same new instance of the class *Loan*.

Exercise 1

Consider the contract for a premium-rate, 24-hour courier service in which compensation is paid for non-delivery within 24 hours.

- (a) Show in a table the obligations and benefits of the client and supplier of this service.
- (b) Give the pre- and postconditions.

Solution

- (a) The obligations and benefits are shown in Table 3.

Table 3 Obligations and benefits for a 24-hour courier service

	Obligations	Benefits
Client	pay cost of premium service	delivery within 24 hours, or compensation for non-delivery
Supplier	make delivery or pay compensation	ability to charge a premium over ordinary service

- (b) The pre- and postconditions are as follows.

Precondition: the service is paid for.

Postcondition: either the package will have been delivered within 24 hours or compensation will have been paid.

Exercise 2

Look at Figure 1 again and identify the pre- and postconditions for returning a book to the library. Include the fact that there are two significant dates for each loan in relation to the possible payment of fines:

- an expected return date
- an actual return date.

Assume that the value of the expected return date will have been set as part of borrowing a book. Also assume that the actual return date will be recorded as an attribute of a loan (to enable you to deal with charging for overdue books at some later stage of the development).

Solution

In Figure 1 the class *Loan* has two associations with the class *LibraryMember*, to differentiate between current and past loans. The contract for returning a book can be expressed as follows.

Precondition: the instance of the class *LibraryMember* is linked to an instance of the class *Loan* in the role of *currentLoans*.

Postcondition:

the link between the instance of the class *LibraryMember* and the instance of the class *Loan* in the role of *currentLoans* will have been broken
the instance of the class *LibraryMember* will have been linked to the same instance of the class *Loan* in the role of *pastLoans*
a value will have been set for the attribute *actualReturnDate* in the same instance of class *Loan*.

Note that the association between the class *Loan* and the class *Book* is not affected by returning a book. The instance of the class *Book* is linked to the instance of the class *Loan* before and after its return.

2.5 Summary of section

In this section we:

- reviewed how assertions are used in classes and operations at the analysis and design stages of development
- described how using assertions in this way can be seen as expressing a client–supplier contract between the objects of an object-oriented system
- described how the process of using assertions in this way, called design by contract (DbC), can be applied during analysis
- examined the notion of a contract, and its benefits and obligations to clients and suppliers, in both business and software terms
- saw how preconditions and postconditions must be respected when one type is a subtype of another
- saw that the behaviour of a task or operation can be constrained by identifying the conditions that must be true before and after the given task or operation.

3

Starting dynamic modelling

This section shows you how to model the dynamic behaviour of objects from your static class models, so that you can decide which operations will be allocated to which classes.

3.1 Focusing on a solution

So far you have seen how to build structural models and how to capture requirements in the form of use cases and activity diagrams. Although we covered these by looking at use cases first, we explained that they are naturally developed in parallel. A structural model alone merely describes the structure of things; it says nothing about how they should behave. A set of use cases without a structural model to constrain the vocabulary used will suffer from all the problems of informal specification, open to numerous interpretations of what the words mean.

However you order the development process, you should eventually reach a stage of having a precise structural model identifying the classes and their relationships, together with some use cases describing what a system is intended to accomplish. What you do not yet have is a way of linking the use cases to the classes that are going to carry them through; this is the **use case realisation**. The classes do not contain complete lists of operations and attributes, and the use cases do not say which classes should be responsible for which parts of the computation. You have concentrated on the static modelling of the system. Now you need to do some **dynamic modelling**, to show how the objects interact by sending messages to implement the required functionality of the software system.

Dynamic modelling requires you to make some decisions about which classes have the responsibilities (operations) needed to carry out which parts of the overall use case. The point where you start making these decisions is the beginning of design. This certainly fits the suggested distinction that analysis is about understanding a real-world problem and defining what we want from a solution to the problem, not how we are going to build the solution. In contrast, when we start design, we are talking about software and programs. There are many decisions (choices) to be made during design.

Communication diagrams are also known by their UML 1.5 name: collaboration diagrams.

Sequence diagrams and **communication diagrams** (known collectively as **interaction diagrams**) are notations to help make and record decisions relating to the behaviour defined for each class. On an interaction diagram, objects are shown exchanging messages. The dispatch of a message from one object to another is represented by an arrow labelled with the message. A sequence diagram shows the flow of messages from object to object as time passes by. A communication diagram shows the objects and their links (the structure). It shows the flow of messages as they pass along the links between objects. The two diagrams are equivalent: what can be shown in one can be shown in the other.

Although interaction diagrams are sometimes presented as simply documenting the mechanics of an implementation, it is the act of building them that is at the very heart of object-oriented design. Every time you draw an arrow on an interaction diagram, showing one object requesting services from another, you are committing the class of the receiver to provide an operation with a particular name. When you have completed a set of interaction diagrams you will have built interface specifications for the classes in almost enough detail to hand over for coding.

Such a use of interaction diagrams is one, often effective, way to model a system, but it is not the only process that can be used in design. An alternative is to start by modelling classes and their operations and then model using sequence diagrams. This has the potential to make consistency between class and sequence diagrams easier to establish, but requires experience in deciding where to place operations.

Communication diagrams and sequence diagrams are used for similar purposes, although they emphasise different features. Communication diagrams make the interconnections between objects clear but the actual sequence of messages can be difficult to see. In practice, a numbering system is needed to identify which message comes first, which comes second, and so on. Sequence diagrams make the time relationship extremely clear, but it may not be easy to see overall patterns of message flow because each diagram shows only the objects involved in a particular sequence. We will consider sequence diagrams first.

Communication diagrams are discussed in detail in Section 4.

To show how you might construct a sequence diagram, we will examine the *check in guest* and *check out guest* use cases (see Figure 2) for the simplified hotel reservation system that was introduced in Block 1 Unit 3.

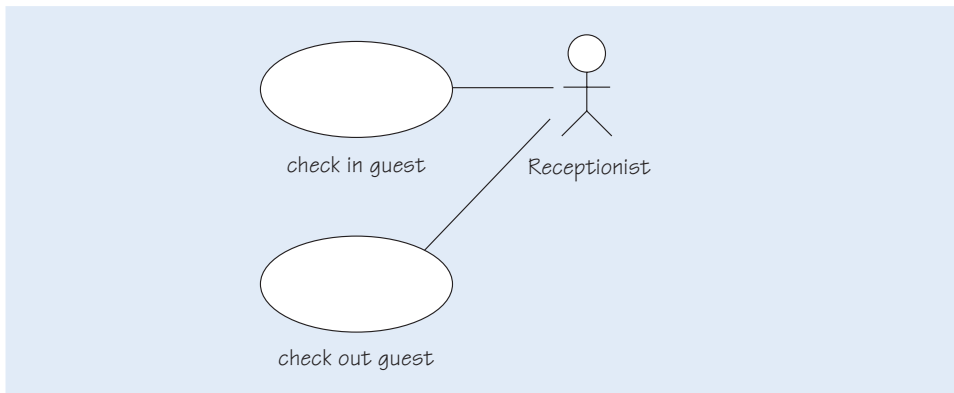


Figure 2 Use case model for checking in and out of a hotel

Figure 3 shows a class diagram that identifies all the rooms in a hotel and those that are free (that is, not currently occupied). We assumed that, before checking in, a guest would already have a reservation. This implies that an appropriate instance of the class *Guest* will already exist for the person who wishes to check in.

At some later point we might need an association between *Hotel* and *Guest* to identify those who are currently resident in any particular hotel. We will assume that it can be derived from the set of potential guests and the rooms that they occupy.

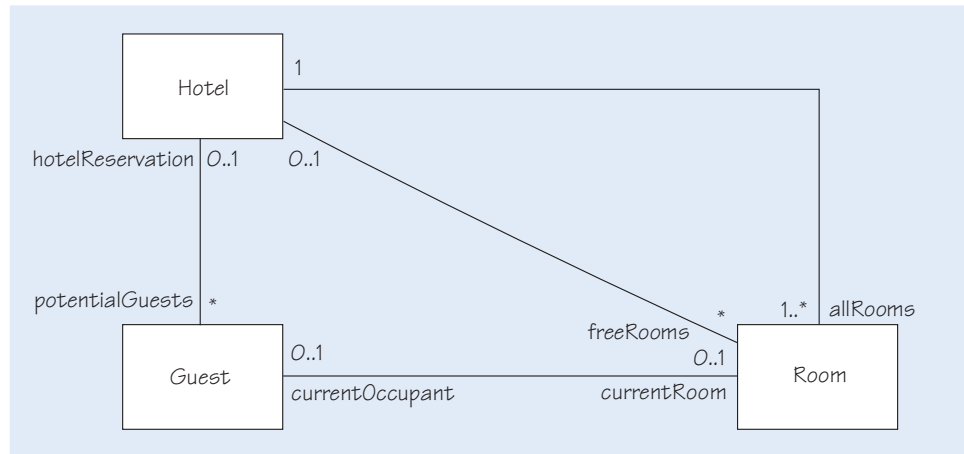


Figure 3 Simplified hotel class model

In Block 1 Unit 3 we developed a contract for the *check in guest* use case as follows.

Precondition: there must be a reservation for the guest, there must be at least one room available (of the desired type), and the guest must be able to pay for the room.

Postcondition: the guest will have been allocated to a room for the period identified in the reservation, the room will have been identified as being in use for a specific period, a bill will have been opened for the duration of the stay, and a key will have been issued.

We will assume that the receptionist issues the key, and not deal with it further here. If we also assume that bills are part of a separate accounting package, we can use the simplified class model in Figure 3 to define the *check in guest* use case as follows.

Preconditions:

The instance of the class *Guest* is linked to an instance of the class *Hotel* in the role of *hotelReservation*.

For the same instance of the class *Hotel* there is at least one link to an instance of the class *Room* in the role of *freeRooms*.

Postconditions:

The link between the instance of the class *Hotel* and the instance of the class *Room* in the role of *freeRooms* will have been broken.

The same instance of the class *Room* (in the role of *currentRoom*) will have been linked to the instance of the class *Guest* in the role of *currentOccupant*.

3.2 Starting to build a sequence diagram

Although not all interaction diagrams correspond to use cases (some are produced simply in the design of complex methods), we begin by looking at

interactions implied by use cases. A use case often requires a change of state of a system to take place. A sequence diagram represents the ripple of messages that brings about that change of state.

A use case is usually written to describe a general business-level operation on the system. For instance, the use cases for checking in and checking out are shown (simplified) in Figure 2. Although you must build software that can check in *any* guest, it is very often much more helpful to consider scenarios for specific situations. For example, how would your system allow Jill to check in and then occupy room 401? Once you have a detailed understanding of how you propose to handle some specific cases, you will be in a better position to claim that you can handle the general case. Choosing which scenarios to handle is similar to choosing test data. You choose some typical situations and some borderline ones. When exploring how guests check in to a hotel you might explore the scenarios where *jill* checks in to *theRitz*, which has three rooms, of which:

- one is occupied
- two are occupied (she takes the last room)
- all three are occupied (there is no free room).

Once you are clear that you can handle these specific situations, you might be in a position to claim that you can handle the general case. In practice, however, experienced practitioners embark directly on the specification of generic scenarios and return to the object level only at the point of deriving test cases from scenarios.

Before starting to consider message flow you must be absolutely clear what it is you are trying to establish. It is the *postcondition* of the use case that describes how the system must change. If you are constructing a sequence diagram for a concrete case, it may be useful to show an object diagram of the system before and after the use case. Figure 4 shows the state of a system before and after checking *jill* in to *theRitz* when only one out of the three rooms is occupied initially.

You may already have discovered the specific scenarios and cases if you have used activity diagrams to investigate the scenarios for the use cases.

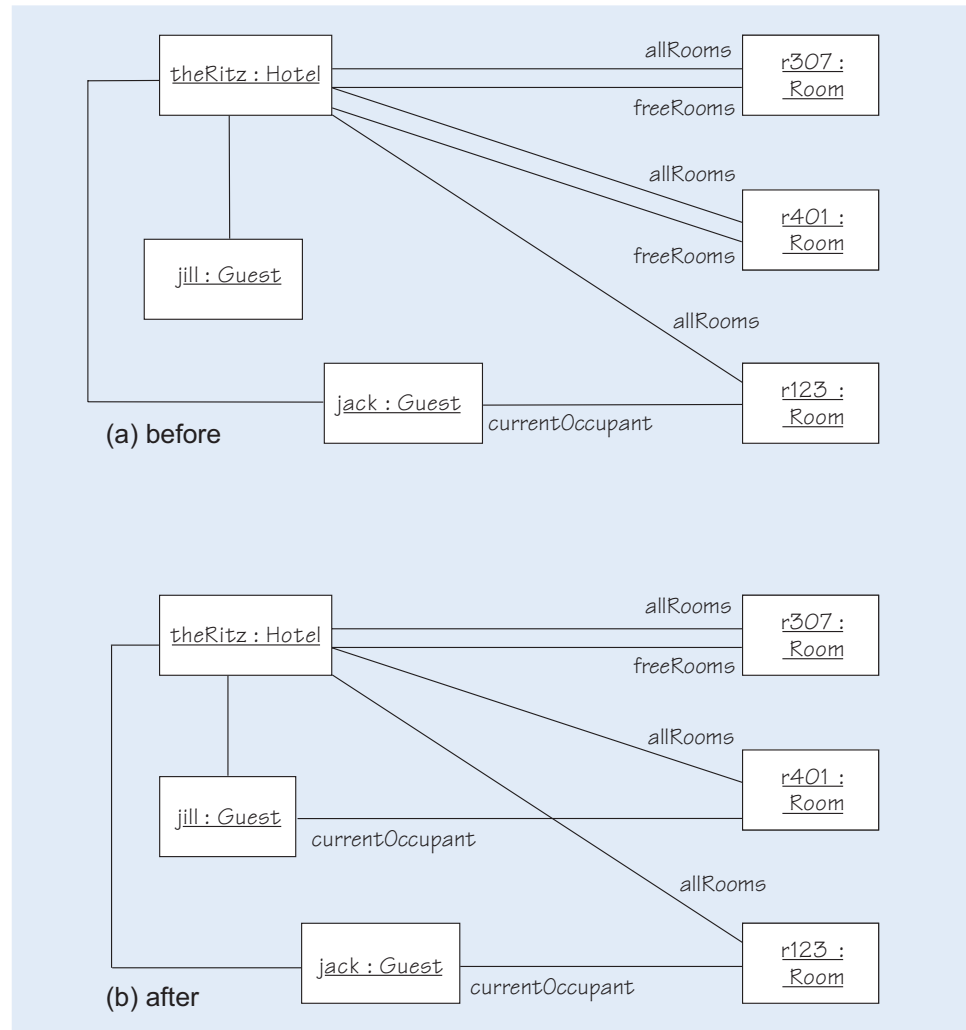


Figure 4 Object diagrams define a required change

Be aware that the use cases are not written at the level of particular styles of user interface. They are in terms of business-level changes on the business objects. They specify how some hotels, rooms and guests should change, rather than the details of what the user has to do to achieve this. This is clearly vital if you are to specify systems that are independent of the current means of interacting with them.

Notice that, while the steps in the main success scenario are one way to get from the before to the after object diagrams, at this stage we can simply work with the pre- and postconditions as we haven't decided which classes do what.

You need to capture what checking in is *about*, regardless of whether interacting with the system is via a keyboard, a microphone or the internet. Figure 5 is an example of an early model that you might draw when thinking about the eventual form of the user interface. It shows that the business model (*Hotel*, *Room* and so on) has very little connection to the user interface. You are about to design the business-model interactions, but you have to be aware that an interaction is triggered by a message from a user interface.

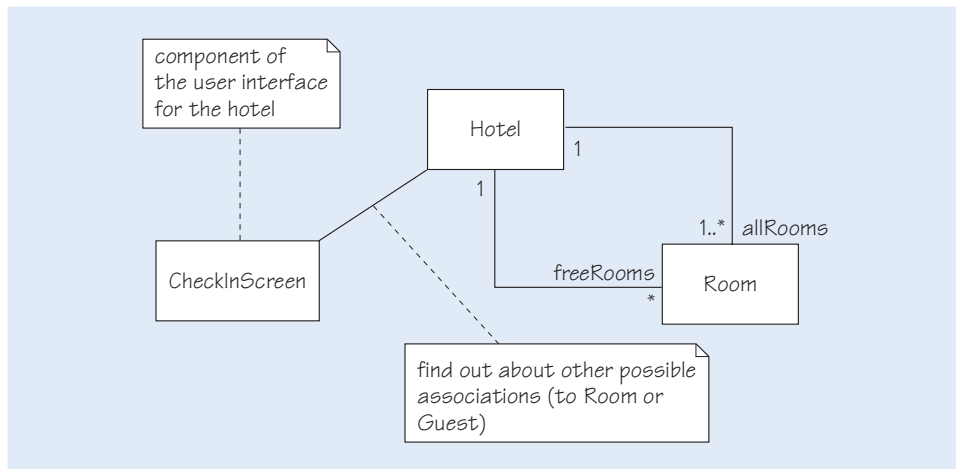


Figure 5 Separation between the business model and the user interface

To construct a sequence diagram, we need to know where the first message in the use case originates. It will normally come from a user interface. At this point we are not concerned with any of the details of this interface; we just need to know that it exists and that a user can initiate a use case from it, causing a message to be sent to one of the business objects. Figure 5 shows the class diagram with a hypothetical user-interface class that can originate messages.

SAQ 6

- How does the use of a pair of object diagrams help you prepare to build a sequence diagram?
- Is the initial message on an interaction diagram always sent from an object representing the user interface?

Answer

- The aim is to show how a given postcondition can be achieved in a sequence diagram. A pair of object diagrams, showing the states before and after the operation in question, identifies the changes in system state that take place in order to meet the postcondition.
- No – we are not constrained to showing interactions with the user interface. Message sequences can originate from any object. The user interface is the origin for those messages that relate to a use case scenario, which we have described in the case of checking guests in to a hotel. However, interaction diagrams can become very complex if we try to show all the possible messages for a given configuration of objects. Following the principle of modularisation, we would split up a complex interaction into a number of smaller ones. In the new diagrams the starting point need not be the user interface.

3.3 Assigning responsibilities in design

In design we must make choices about which responsibility should be assigned to which class. Ultimately any choice should be justified in terms of guiding design principles such as encapsulation, low coupling and high cohesion (see Block 1 Unit 1, subsection 2.5).

Craig Larman has defined a collection of patterns based on such principles, to help designers assign responsibilities in commonly occurring design scenarios (Larman, 2004). There are many other collections of design patterns and you will encounter others later in this module. Design is a difficult activity that benefits from experience, and design patterns are a means to systematically record and reuse best practice. While the design principles provide general guidance, the patterns provide practical advice on how to solve design problems based on those principles. Larman calls such patterns **GRASP**, an acronym for **general responsibility assignment software patterns**. Here we present two of these patterns: *Expert* and *Creator*.

GRASP Expert

The **GRASP Expert** (also known as *Information Expert*) pattern addresses the problem of distributing responsibilities for knowing across the system, in particular for information that should be derived from object properties. The fulfilment of a responsibility within a system often requires the consolidation of information that is distributed among several different objects, each being knowledgeable or being an ‘expert’ on some aspect of such information. The Expert pattern allows you to distribute responsibilities among ‘information experts’ in a way that encourages cohesive class definitions, which are easier to understand and maintain.

The Expert pattern, like many aspects of object technology, has a real-world analogy. Within organisations, responsibility to fulfil a task is commonly given to teams of individuals who collectively have the necessary expertise. And just as software objects may need to collaborate because information is distributed among classes, so it is with team members who need to interact and cooperate with one another to fulfil their task.

The pattern can be summarised as follows.

Name. Expert

Intent. To assign responsibility based on object properties.

How it works. The responsibility is assigned to the class that has the information necessary to fulfil that responsibility – the ‘information expert’; this information is represented by the properties of the object of the class.

When to use it. Use this pattern when you need to decide which of a number of interacting objects a responsibility should be assigned to. The pattern maximises encapsulation, as objects use their own information to fulfil a task. Therefore it enhances low coupling and high cohesion within a system.

Example. An information expert could be an object representing a sale, to which the responsibility of calculating the total based on the sale items within that sale can be assigned.

GRASP Creator

The fulfilment of a responsibility will often require the creation and initialisation of new objects. The **GRASP Creator** pattern is used to assign responsibilities related to the creation of objects. Its objective is to find a creator that needs to be linked to the created object, because the creator aggregates, maintains or records, or contains the created object. Sometimes we find a creator class by looking for the class that has the responsibility for providing the initialising data that will be used during creation.

The pattern can be summarised as follows.

Name. Creator

Intent. To assign responsibility for creating objects.

How it works. The responsibility for creating an instance of some class is assigned to the class that aggregates, maintains or records, or contains instances of the class of the newly created object, especially if the creator class provides the data required to initialise the newly created object.

When to use it. Use this pattern whenever you need to assign responsibilities for object creation. Low coupling is maintained between the creator class and the class of the created object, because the latter is probably already linked to the creator class owing to the existing associations that motivated its choice as the creator.

Example. A creator could represent a bank, with the newly created object being a bank account. In this case the creator might be responsible for assigning the account number to the newly created bank account. Another example is a creator representing an order, with the newly created object being an order item. In this case the creator and its associated created objects might be viewed as a composition.

You will have an opportunity to apply GRASP in subsection 3.5. First, however, we need to introduce sequence diagrams. If you are familiar with the notation for sequence diagrams you can go straight to subsection 3.5.

3.4 Sequence-diagram notation

Figure 6 is a sequence diagram showing how a particular guest called *jill* has been allocated a particular room upon checking in to *theRitz* hotel. We will show you how such a diagram can be constructed in subsection 3.5, but first we will consider UML's basic notation for sequence diagrams. As time passes, you can see the messages flowing from object to object.

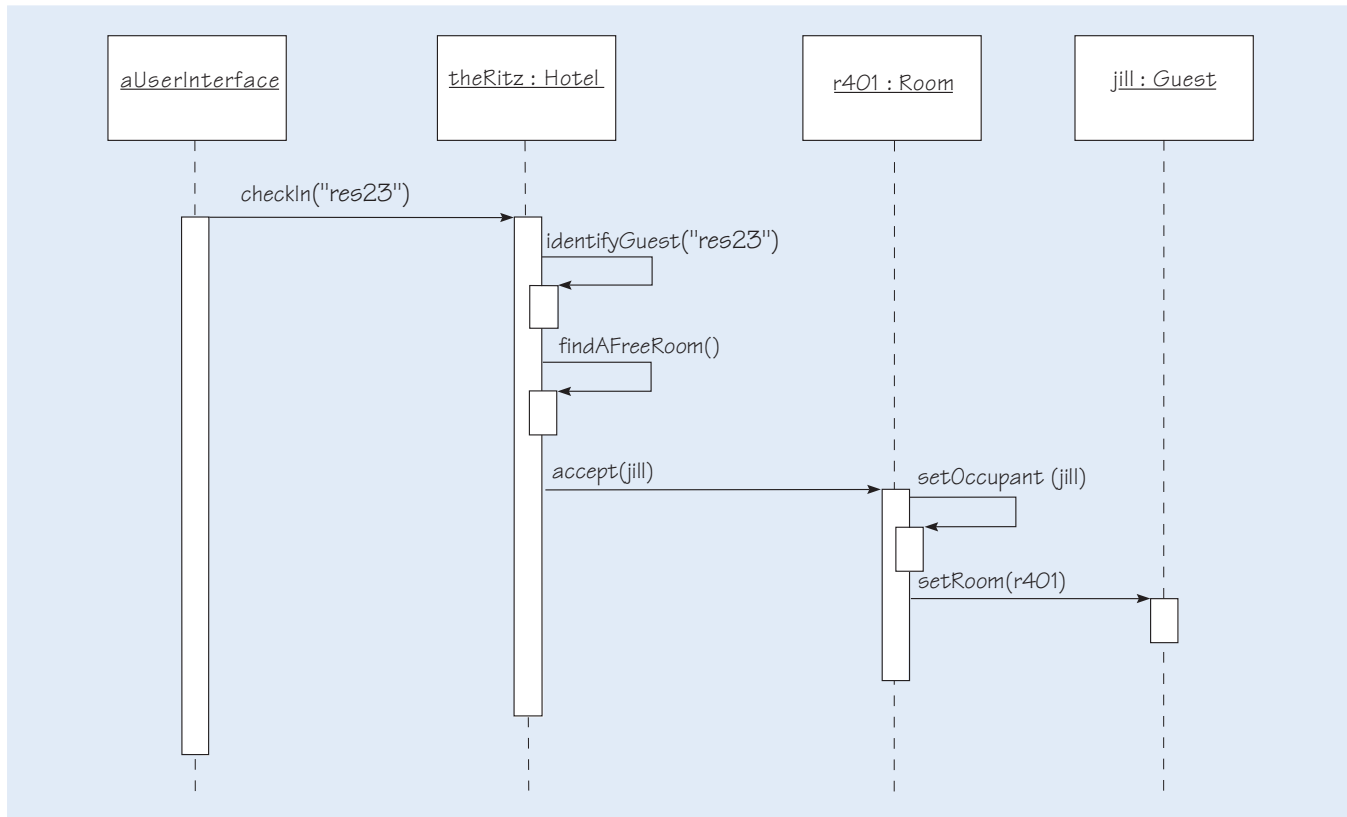


Figure 6 One way to check a guest in to a room

The main elements in the sequence diagram in Figure 6 are:

- the objects involved, represented by the rectangles at the top of the diagram
- messages between the objects, represented by arrows labelled by text strings
- the objects' lifelines, represented by dashed lines (with time passing as you move down the page)
- indications of when particular objects are 'active', represented by long rectangles overlaid on objects' lifelines.

It is also possible to identify each message with a number. This reinforces the ordering that is expressed by the vertical positioning of messages, but the numbering is not necessary and is usually omitted.

In a concrete sequence diagram, such as Figure 6, each lifeline represents a *specific* instance, so instance names that emphasise this particularity may be preferred – *theRitz* rather than *theHotel*, for example. There are two ways to identify specific instances of an object in a sequence diagram. Figure 6 shows both of them:

- using a colon to separate the name of a class from the distinguishing name of an object, and underlining the text, as for *theRitz : Hotel*
- using a distinct object name with underlining, but without identifying the class to which it belongs, as for *aUserInterface*.

There is also a way to identify a generic, rather than a specific, instance of a class. This form uses a colon in front of the underlined class name, for example : *Hotel*.

Every arrow represents the sending of a message. The solid arrowhead used in Figure 6 indicates a synchronous message. If all messages in a sequence are synchronous, only one object can be computing at any one time. The sender of a message does nothing until the flow of control returns from the receiver. The long rectangle, which is coincident with the lifeline of an object, shows the entire period when an object is active, known as an **activation**. The activation includes the times when the object is not computing because it has sent a message to some other object that is computing.

So the sequence diagram in Figure 6 shows a *procedural interaction* in which each activation is the execution of a procedure that includes the time spent waiting for nested procedures to complete their execution. A new activation begins on receipt of a message, which is indicated by the message arrow pointing to the top of the activation rectangle. When an object sends a message to itself – for example, *theRitz* sends the message *findAFreeRoom()* to itself in Figure 6 – the receipt of that message is also a signal to begin a new activation. Each new activation is overlaid upon the original activation.

Notice that the return of flow of control from the receiver is often not shown, to avoid cluttering up the diagram. It is usually obvious from the activation of the receiver. In Figure 6, *r401 : Room* sends the *setRoom(r401)* message to *jill : Guest*. While *jill : Guest* is computing, *r401 : Room* is not. It only resumes when *jill : Guest* finishes; when this happens, it is obvious from the diagram because it is at the point where the activation box of *jill : Guest* ends. If you need to be more explicit, you can show the return using an open-headed arrow with a dashed shaft.

An object cannot send a message to another object unless it has a reference to it. This means that either there is a link from the sender to the receiver (in which case the corresponding classes will be associated in the class diagram, because a link is an individual instance of an association) or the sender has obtained a reference to the receiver in some way during the course of the interaction so far (as the answer to an earlier message, or as a message parameter, or because the sender was responsible for the creation of the receiver object).

Whenever you add a message arrow to a sequence diagram, you should look at the class diagram to see whether an association exists. If it does not, you should ask yourself how the message can be sent. Where has the sender obtained the necessary reference? If the sender does not have the required reference, you must rethink the message sequence.

An object may send messages to itself as well as to other objects. It is possible to use an interaction diagram to show those messages that an object sends to itself in response to some other message. However, if we showed every message from an object to itself, we might soon have a very busy diagram in our attempts to model a particular computation. Therefore we usually restrict ourselves to those messages that are traceable to the pre- and postconditions in the use cases.

In UML 2, instance names are often not underlined when the lifeline represents a generic instance of a class instead of a specific one. Under this convention *theRitz : Hotel* would be underlined because it is a specific instance, but *theHotel : Hotel* would not be underlined because it is generic.

When an activation ends in a receiver then the flow of control returns to the sender. See Section 5.

Figure 6 shows two examples that are traceable to the pre- and postconditions for the *check in guest* use case. In the first example, the operation *findAFreeRoom* shows which class will be responsible for finding a suitable room for a new guest – part of the precondition. In the second example we see that each instance of the class *Room* keeps a reference to the instance of *Guest* using the operation *setOccupant(jill)* – part of the postcondition.

In the next subsection you will see how the sequence diagram in Figure 6 was constructed.

SAQ 7

- (a) In an interaction diagram, which class must provide the operation indicated by a message passed from one object to another?
- (b) What does a lifeline represent?
- (c) What does the box at the top of a lifeline include?
- (d) What sort of arrowhead is used on an arrow depicting synchronous message sending?
- (e) What is a procedural interaction? With what might it be contrasted?
- (f) In a sequence diagram, what does the widening of a lifeline into a tall, thin rectangle mean?

Answer

- (a) The class of the receiver object must provide the appropriate operation.
- (b) A lifeline represents the portion of the life of an object covered by the sequence diagram.
- (c) It includes an instance name, optionally followed by a colon and a class name, or, for a generic object, just a colon and a class name. For example, *objectname*, *objectName : ClassName* and *: Classname* are all allowed.
- (d) A solid black arrowhead, which indicates that the interaction is procedural. An arrowhead with a dashed shaft is used to signify the method's return, although it is often omitted for clarity.
- (e) It is an interaction in which the sender of a message is blocked until the receiver of the message has finished processing. This is exactly the same as what is sometimes called *subroutine semantics*. It is the usual policy when a single thread of control is allowed. If multiple threads are allowed, we might not want the sender of the message to block, in which case we could start a concurrent activity.
- (f) This shows that the object is active. An object is said to be active if it is either performing an operation or awaiting completion of an operation that it has requested another object to perform.

3.5 Making design decisions

Table 4 Textual description of the *check in guest* use case

Identifier and name	UC2 <i>check in guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest takes up a reservation and occupies a room at the desired hotel.
Precondition	There is a reservation for the guest for an available room of the desired type, and the guest can pay for the room.
Postcondition	The guest will have been allocated to a room for the period identified in the reservation, a bill will have been opened for the duration of the stay and a key will have been issued.
Assumptions	The guest is already known to the hotel's software system. The hotel is confident that the guest can pay, for example the guest has a valid credit card.
Main success scenario	
1	The guest provides a reservation reference number to the receptionist.
2	The receptionist enters the reference number to find the reservation.
3	The hotel system provides the details of the requested reservation.
4	The receptionist confirms the details of the room type and duration of stay with the guest.
5	The hotel system allocates a room to the guest.
6	The hotel system opens a bill for the guest. (It could be that there is a separate billing package, which must be notified upon check in.)
7	The receptionist issues a key to the guest.

We now need to implement the main success scenario of the *check in guest* use case developed in Block 1 Unit 3. We will need to implement each of the steps that mention the hotel system. Step 1, for example, 'The guest provides a reservation number to the receptionist' does not mention the hotel system as it doesn't involve the system itself but is part of the wider context. We begin by considering step 2 'The receptionist enters the reservation number to find the reservation'. This will be represented by a message *checkIn*.

The first two design decisions to be made relate to this first message, which starts the internal sequence of messages that produces the required postcondition for the use case. The same applies whether you are constructing a sequence diagram or a communication diagram. Figure 6 shows the initial message coming from an object identified as *aUserInterface*. This message will have been initiated in some way by a hotel receptionist. There must be some way of identifying the reservation involved, so that the software can locate the corresponding *Guest* object, and we have assumed for now that this will be done by passing a string identifier from the user interface. In Figure 6 the identifier is *"res23"*.

When making the first design decision, namely *what the very first message should be*, we are committing whoever builds the user interface to ensuring that the program sends that message. The interaction shown in Figure 6

commits whoever builds the user interface to making it send a message with the form *checkIn(String)*.

The second decision to be made relates to *where the initial message should be sent*. In Figure 6 the initial message is sent to the object *theRitz*. By sending this message to a *Hotel*, we commit the class *Hotel* to implementing this functionality. Of course, we could have chosen another object to receive the message. For example, we might introduce a class to represent each use case. Our choice, the one shown in Figure 6, is consistent with the GRASP Expert pattern: the hotel is the overall expert in how the hotel is managed. Notice that now we are dealing with message sequences we have altered the format of use case names to express the corresponding operation names. For instance, *check in* has become *checkIn*.

We can now start creating a sequence diagram. Figure 7 shows the initial step.

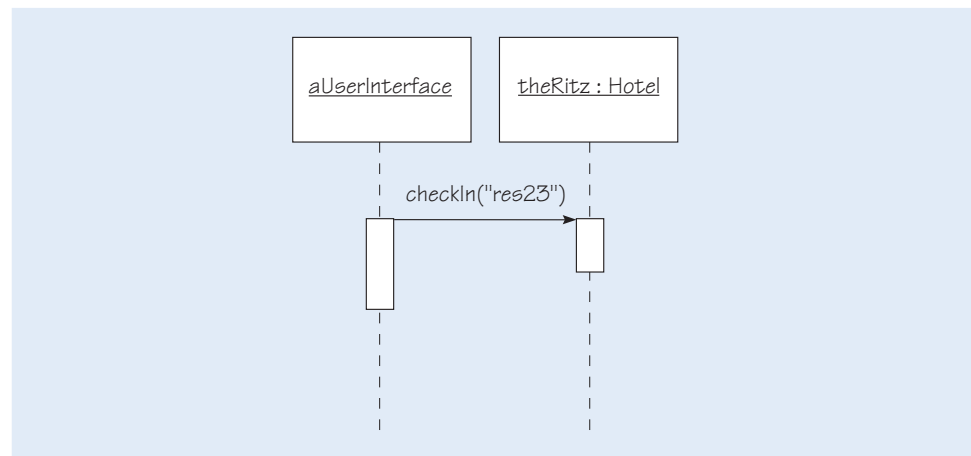


Figure 7 Starting the *checkIn* sequence diagram

We have now committed the *Hotel* class to implementing a method with the signature *checkIn(String)*, so we add this fact to the operations part of the *Hotel* class in the model straight away. You should get into the habit of building up the class model incrementally, as you make and record each successive decision about an operation and its location.

Now consider steps 3, 4 and 5 of the use case, which do the main work of the use case from the perspective of this section:

- step 3 – the hotel system provides the details of the requested reservation
- step 4 – the receptionist confirms the details of the room type and duration of stay with the guest
- step 5 – the hotel allocates a room to the guest.

As we are considering the main success scenario, we assume in step 4 that the confirmation of details is positive. While a room is associated with a reservation we will assume that for operational reasons we may wish to substitute an equivalent, and make the simplifying assumption for this example that all rooms are equivalent. So we have to identify the guest associated with a particular reservation identifier, find a room that is empty,

and allocate it to that guest. We could assign the overall responsibility for this to the *Hotel* object, which would coordinate the search for a room and its allocation to the intended guest, namely the object *jill* : *Guest*. Another approach would be to delegate some of the behaviour to the *Room* objects, perhaps giving them an operation *attemptToHouse*(*g* : *Guest*). Each room would know whether or not it was free and how to accommodate a *Guest*. If a given room could not accommodate a *Guest*, it would return an appropriate indicator so that the next room could be tried.

However, let us suppose we decide that the *Hotel* should do the work of finding the guest and allocating the room. As the *Hotel* needs to identify the guest associated with a particular reservation, we now have to decide how that will be done. The class model in Figure 3 shows that a *Hotel* has an association with a *Guest*, identified by the role names *hotelReservation* and *potentialGuests*. According to the GRASP Expert pattern, *Hotel* would be the expert for guests associated with the hotel, so we assign the operation to *Hotel*. At this stage we probably would not want to describe in detail a particular algorithm for identifying the actual *Guest* object, but we commit *Hotel* to providing an operation *identifyGuest*("res23"), which returns a single instance of the class *Guest*, without making any statement for now about how this method will be implemented. In the sequence diagram of Figure 6 we can show *theRitz* : *Hotel* sending a message to itself to invoke this operation.

If the hotel needs to find a room in order to house a guest, we must decide how to find a room. As the class model (Figure 3) shows that *Hotel* has an association with *Room*, identified by the role name *freeRooms*, we know that the hotel is the expert in free rooms at the hotel and can use that information. As with identifying the guest, we might not want to describe in detail a particular algorithm for choosing and allocating a free room. So we simply commit *Hotel* to providing an operation *findAFreeRoom*, which returns one room – a single instance of the *Room* class. In the sequence diagram we can show *theRitz* : *Hotel* sending a message to itself to invoke this operation. We need to specify what this method does – its pre- and postconditions. For example, does this method just find one of the free rooms, or does it also remove the room it found from the *freeRooms* association? Suppose that it does the latter. While drawing the sequence diagram we can add the pre- and postconditions for the operation, as well as comments for the developers who will implement the design.

Now that the *Hotel* object has both a room and a guest to occupy it, which object will be responsible for linking the guest and the room? All the processing could be done in the *Hotel* class, or in the *Room* class, or in the *Guest* class. Suppose we choose to make the *Room* responsible. The *Hotel* can send the message *accept*(*jill*) to *r401* : *Room*. But what should the *accept* operation do? According to Figure 6, *r401* needs to know the guest who occupies it (*jill*) and *jill* needs to know the room that is allocated (*r401*). We can represent this by making the *Room* invoke an operation on itself to set its occupant, and then send a message to inform the *Guest* object which room has been allocated.

As we are not dealing with the billing package no messages are needed for step 6, and step 7 is the responsibility of the receptionist.

For now we are working with two assumptions: first, that the association between *Room* and *Guest* is navigable in both directions; second, that the operation *findAFreeRoom* removes the room it finds from the *freeRooms* role, as shown in Figure 4.

All the design decisions that led to the final sequence diagram shown in Figure 6 have now been discussed. As a result of all these decisions, we are now committed to the operations shown in Table 5.

Table 5 Some operations required for the *check in use* case

Class	Operations
<i>Hotel</i>	<i>checkIn(s : String)</i> <i>identifyGuest(s : String)</i> <i>findAFreeRoom()</i>
<i>Room</i>	<i>accept(g : Guest)</i> <i>setOccupant(g : Guest)</i>
<i>Guest</i>	<i>setRoom(r : Room)</i>

You should build sequence diagrams for all other interesting scenarios of checking in, such as when there is no room available, and repeat the dynamic modelling for each of the other use cases for the software system. You will end up with a rich collection of operations on each class, and will also have verified that the model can in fact perform the needed operations.

Notice that at every stage we decide on the class that most naturally houses some piece of functionality. One reason for using GRASP is to avoid a common design mistake: putting all the functionality into a single class, with the other classes acting simply as data holders. A *Hotel* class that does every single operation involved in running one type of hotel will contain too much specific detail about the way things are done for the class to be reusable. In a context where, say, room allocation is done differently, such a class would be useless. By dividing the functionality between the classes, we hope to end up with a set of behaviours that naturally belong with *Room* and *Guest*.

One design guideline suggests that whenever a decision is made, it should be encapsulated in a separate class. For example, suppose that each hotel manager in the chain decides whether to allocate the free rooms that are nearest the reception desk (in order to minimise the walking distance) or those that are as far away as possible from occupied rooms (in order to minimise noise disturbance). Such a choice could be encapsulated in a class named *GuestIdentifier* or *RoomAllocator*. The *Hotel* would then *delegate* this decision to a *GuestIdentifier* or *RoomAllocator* object. The allocation strategy can readily be changed by plugging in a different allocator. Thus the *Hotel* class has not had to hard-wire a particular strategy. This is an example of the *Strategy* pattern.

It is not enough just to decide that a class must have an operation of a certain name, such as *allocateFreeRoom*. You must be certain that you understand what you intend the operation to accomplish. Many sequence diagrams are built assuming that the name chosen for the operation is sufficient definition, but the scope for different interpretations of a name is enormous. For example, you cannot tell from the name *findAFreeRoom* whether the operation just identifies a free room or also removes it from the set of free rooms. You should specify what each newly introduced operation does by adding preconditions and postconditions to the model, either in informal English or,

Patterns are studied in more detail in Block 3 Unit 9.

more formally, in OCL. You should not use complexity as an excuse not to describe the contract. If you cannot describe it, how can a developer implement it?

As you build more sequence diagrams for a system, you may come to realise that the choices you have made have unexpected consequences. Had you defined the operations differently, later design choices might have been better able to respect good design rules, or the classes might have been reusable in other models. For example, ‘violations’ of the GRASP patterns may be corrected by moving an operation to another class so that information is where you need it. It is normal to revisit completed diagrams to redesign them, in order to make the entire set cleaner and more consistent.

During the construction of a sequence diagram, you will frequently find weaknesses in the class model. For instance, in building a sequence diagram for printing a guest’s bill you may realise that you need to know what type of room the guest had occupied. As we have not modelled room types, you would have to return to the class model and decide how to represent room types before continuing with the sequence diagram. Object-oriented design is very much an iterative process, alternating between improving the structural model and deciding on the dynamic behaviour.

You may also discover classes that make implementation easier and more flexible but are not part of the specification model. For instance, we noticed earlier that inventing a *RoomAllocator* class might give a more flexible hotel system. It is during the creation of sequence diagrams that many new implementation classes are invented. They must be added to the implementation class model, which slowly grows to contain considerably more classes than the specification class model.

Notice that there is a potential problem with the design in Figure 6. The *checkIn* method has a *String* parameter to identify a particular reservation, which is used to identify the *Guest* object. Although for simplicity we have had the user interface pass in an external identifier, in practice a preferred approach would be for the user interface to obtain a collection of the *Reservation* objects involved then display the collection and have the user pick the right one from the list. The message from the user interface would then have a parameter that is the object identifier of the actual business object concerned, rather than just a string identifier. Of course, having a *Reservation* object to pass in raises the question of where that object is created and how the user interface finds the object. The simple answer is ‘somewhere else in the system’ – recall that here we are dealing with designing only part of the system and we do not expect it to solve all the issues that arise. This kind of situation is explored further in Section 3 of Unit 7.

SAQ 8

- (a) What does to *delegate* mean in the context of object-oriented design?
- (b) Would you describe a sequence diagram as a programming notation?

Answer

- (a) One object is said to delegate behaviour to another when, instead of implementing some behaviour, it sends a message to another object that implements that behaviour. We considered the possibility that instead of the *Hotel* doing all the work of finding a free room and housing *jill* in it, the *Hotel* might delegate the work to a *Room* by passing it the message *attemptToAccommodate(jill)*.
- (b) Not really, because all it shows is the inter-object message traffic. It does not represent the algorithms needed by senders in order to decide when and to which objects messages should be sent, or those needed by receivers in order to act on the messages. When you have finished drawing sequence diagrams you know the interfaces of the various classes but not how those interfaces will be implemented.

Exercise 3

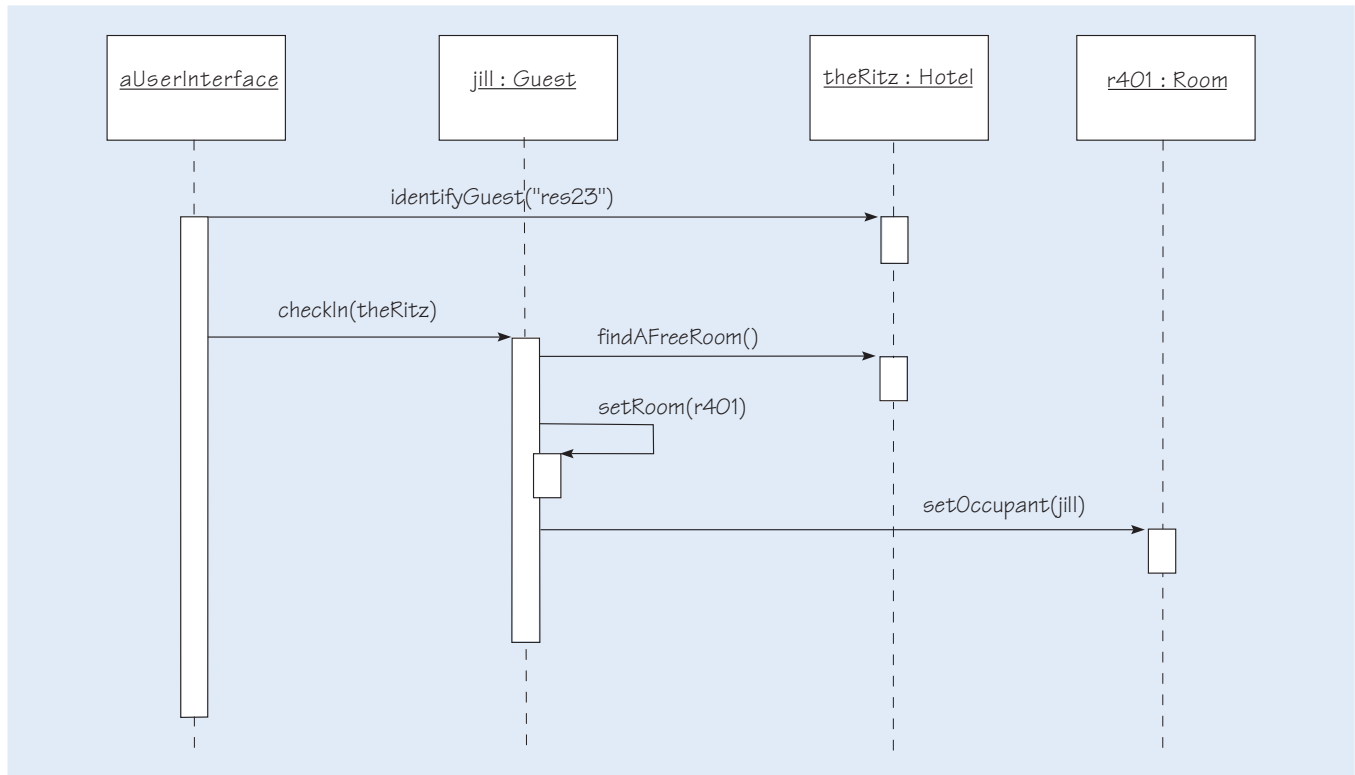
Construct another sequence diagram for checking *jill* in to *theRitz*, in which the initial message from the user interface is sent to *theRitz*. Then complete the sequence diagram by sending the second message to *jill*. Use the list of classes and associated methods given in Table 6. Why must the first message go to a *Hotel* object rather than a *Guest* object?

Table 6 Alternative operations for the *check in use* case

Class	Operations
<i>Guest</i>	<i>checkIn(h : Hotel)</i> <i>setRoom(r : Room)</i>
<i>Hotel</i>	<i>identifyGuest(s : String)</i> <i>findAFreeRoom()</i>
<i>Room</i>	<i>setOccupant(g : Guest)</i>

Solution

Our sequence diagram for checking *jill* in to *theRitz* is given in Figure 8. As you saw in Block 1 Unit 4, guests are given a reservation number to identify themselves when checking in, and the sequence diagram in Figure 6 shows how that reservation number is used to identify the *Guest* object in the software system. In the alternative interaction shown in Figure 8, we still need to identify the *Guest* object before that object can be used to complete the process of checking in. We are dealing with a single hotel, *theRitz*, but if we were to have several hotels we would need to identify the hotel as well.

Figure 8 Checking in using a *Guest* object**Exercise 4**

Imagine constructing some sequence diagrams for checking out a guest. What new classes might suggest themselves as candidates for inclusion in the class model?

Solution

Checking out requires the completion of a *Bill*, which might contain a number of separate items. A bill relates to the length of the stay, which has not been modelled. Perhaps we need attributes to record arrival and departure. These might fit most naturally in a *Stay* class. Payments may or may not be settled immediately, so it is wise to separate a *Bill* from any *Payment* that might arise. Collected instances of *Bill* and *Payment* might naturally be grouped into instances of an *Account* class.

All the above are extra classes that might have arisen when developing the use cases, but might arise later in the design process. Classes that are inherently design classes, which could not arise from writing use cases, might include a *CheckerOut* class to encapsulate the procedures for checking out, or a *RoomAllocator* to centralise decisions on how the set of free rooms is to be handled.

Exercise 5

The sequence diagram in Figure 6 shows the user interface sending a message *checkIn("res23")* to *theRitz*. How might the user interface discover that this is the correct hotel to send the message to?

Solution

This would be handled in another use case, probably *log on*, shown in Figures 16 and 17 of Block 1 Unit 3. (This is the use case performed by the receptionist on start-up, in which they identify the hotel that they are serving.)

This illustrates that, although you may think of issues when developing a particular use case, those issues may properly be resolved in other use cases.

3.6 Class-responsibility-collaboration cards

A more lightweight approach to modelling how classes interact is the use of class-responsibility-collaboration cards (CRC). (The approach is also called a class-responsibility-collaborators model.) CRC cards are used, for example, in XP, and can be used by a group of people, which can help ensure good communication. The key idea is very simple: a card has three boxes that can be filled in with the name of the card, its responsibilities (what data it holds and what it can do) and its collaborators (other classes it interacts with directly). See Figure 9.

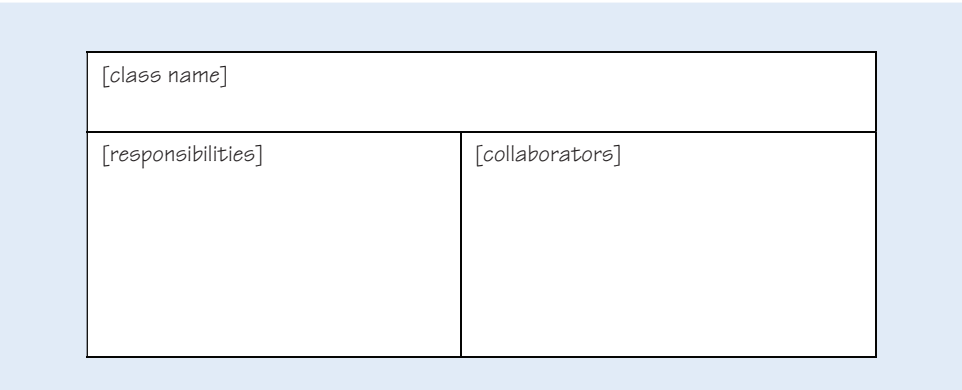
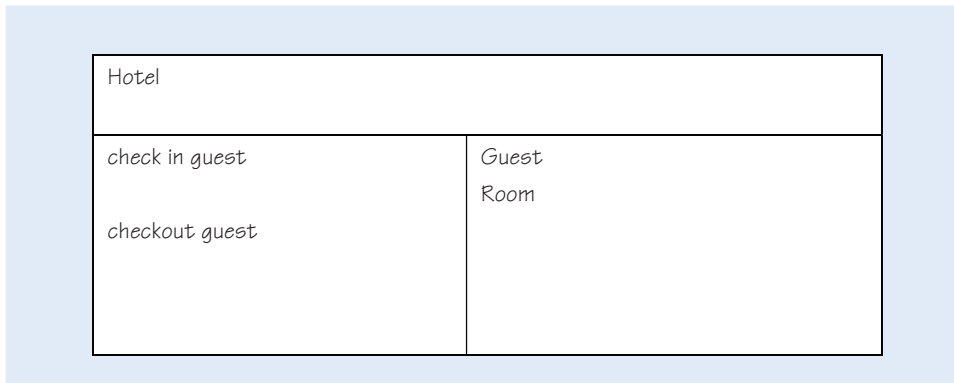


Figure 9 Generic CRC card

Figure 10 shows a specific card for a tentative *Hotel* class.

Figure 10 *Hotel* class

Typically the cards are used for identifying both classes and their responsibilities and collaborators. In one approach, one person reads through some text describing the system in general terms and the group members fill in the cards as they get ideas about what the classes might be and how they might be used. Alternatively the process might be driven by a discussion with client representatives.

The cards can be laid out on a table so that cards that collaborate are placed near to each other and the distances between them are indicative of how closely they are coupled (how much interaction they have). The key goal is usually to get a general idea about how the classes will divide functionality and collaborate, so the lists of responsibilities and collaborators may be very partial during the early stages.

3.7 Summary of section

Sequence diagrams are one kind of interaction diagram. They are used when moving from a specification to a design as a way of helping to decide which parts of the overall system behaviour should be allocated to which classes. So far, we have used sequence diagrams only for specific scenarios of use cases – *jill* checking in to *theRitz* rather than *aGuest* checking in to *aHotel*.

Sometimes a more lightweight approach based on, for example, CRC cards may be appropriate. The key thing is that the appropriate concerns are taken into account.

4

Working with interaction diagrams

Some CASE tools can generate a communication diagram from a sequence diagram and vice versa.

If you are familiar with the notation for communication diagrams you can go straight to subsection 4.2. A sequence diagram represents a number of instances by using vertical lines. Another notation that we have frequently used to show a number of instances is a *snapshot* or *object diagram*. Some designers prefer to add message sending to the familiar object diagrams rather than using the special notations of sequence diagrams. An object diagram with added message sends is called a *communication diagram*. As you saw in the previous section, both communication diagrams and sequence diagrams are types of interaction diagram.

4.1 Communication diagrams

Figure 11 shows a communication diagram that is equivalent to the sequence diagram of Figure 6; both show how a particular guest checks in to a hotel room.

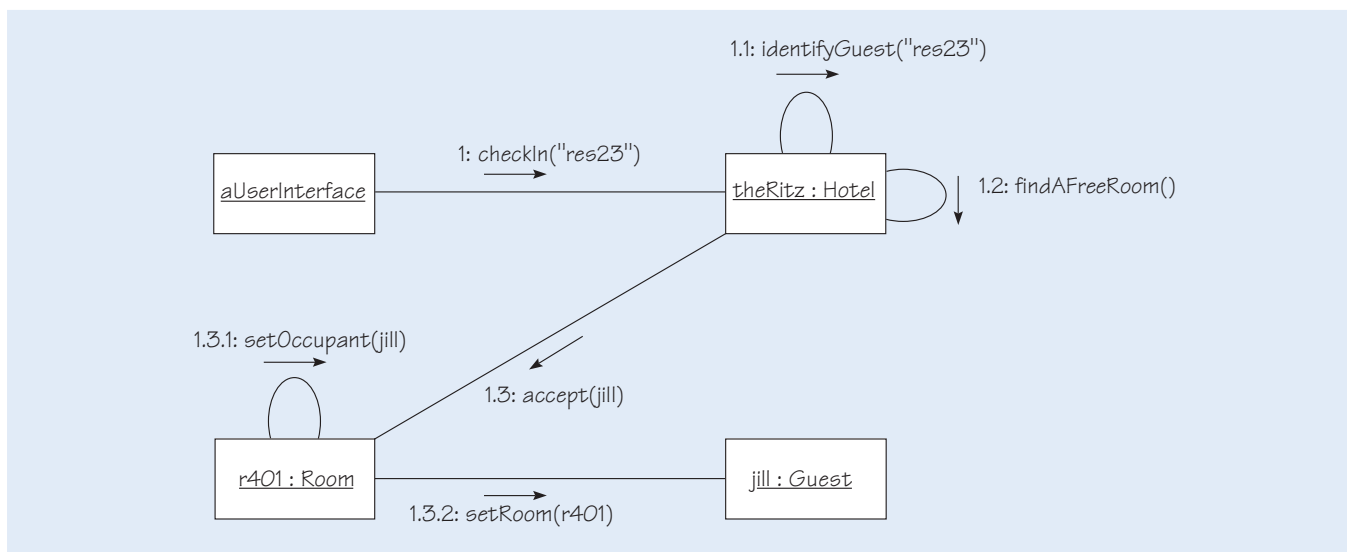


Figure 11 Equivalent communication diagram to Figure 6

Both types of diagram use the same notation for objects and use solid arrows for synchronous messages. There are two main differences between communication diagrams and sequence diagrams.

- A communication diagram shows in one place all the links of interest between objects, whereas a sequence diagram does not. You can see the messages flowing along the links in Figure 11. In a sequence diagram you must scan down the diagram to find which links are being used.
- The time-ordering of messages is clear in a sequence diagram; time passes by as you travel down the page in Figure 6. Some form of numbering is

needed in a communication diagram to show the time-ordering of messages, and the numbering can become very complicated.

If we did not have a sequence diagram for comparison, how would we arrive at the numbering that is shown in Figure 11?

In a communication diagram every message must be numbered, otherwise the sequence would not be fully defined. A simple numbering system such as 1, 2, 3, 4, ... would sometimes be adequate to avoid ambiguity, but not in the case of direct or indirect recursion.

Consider the two sequence diagrams in Figure 12. In the first sequence diagram (Figure 12a), $:B$ becomes active processing the message p sent by $:A$. As part of this processing $:B$ sends a message q to $:C$, which in turn sends the message r to $:B$. Because of the indirect recursion from $:B$ to $:C$ back to $:B$, at this stage $:B$ is ‘doubly active’ – it has to deal with both p and r . (Notice there is no parallelism: there is a single thread of control.) Once the processing of r is complete, $:C$ can resume its processing of q . Once a reply has been received, $:B$ can resume its processing of p which includes sending the message s to $:D$.

In the second sequence diagram (Figure 12b), $:C$ sends r to $:B$ but this time $:B$ sends s to $:D$ as part of its processing of r , rather than as part of its processing of p .

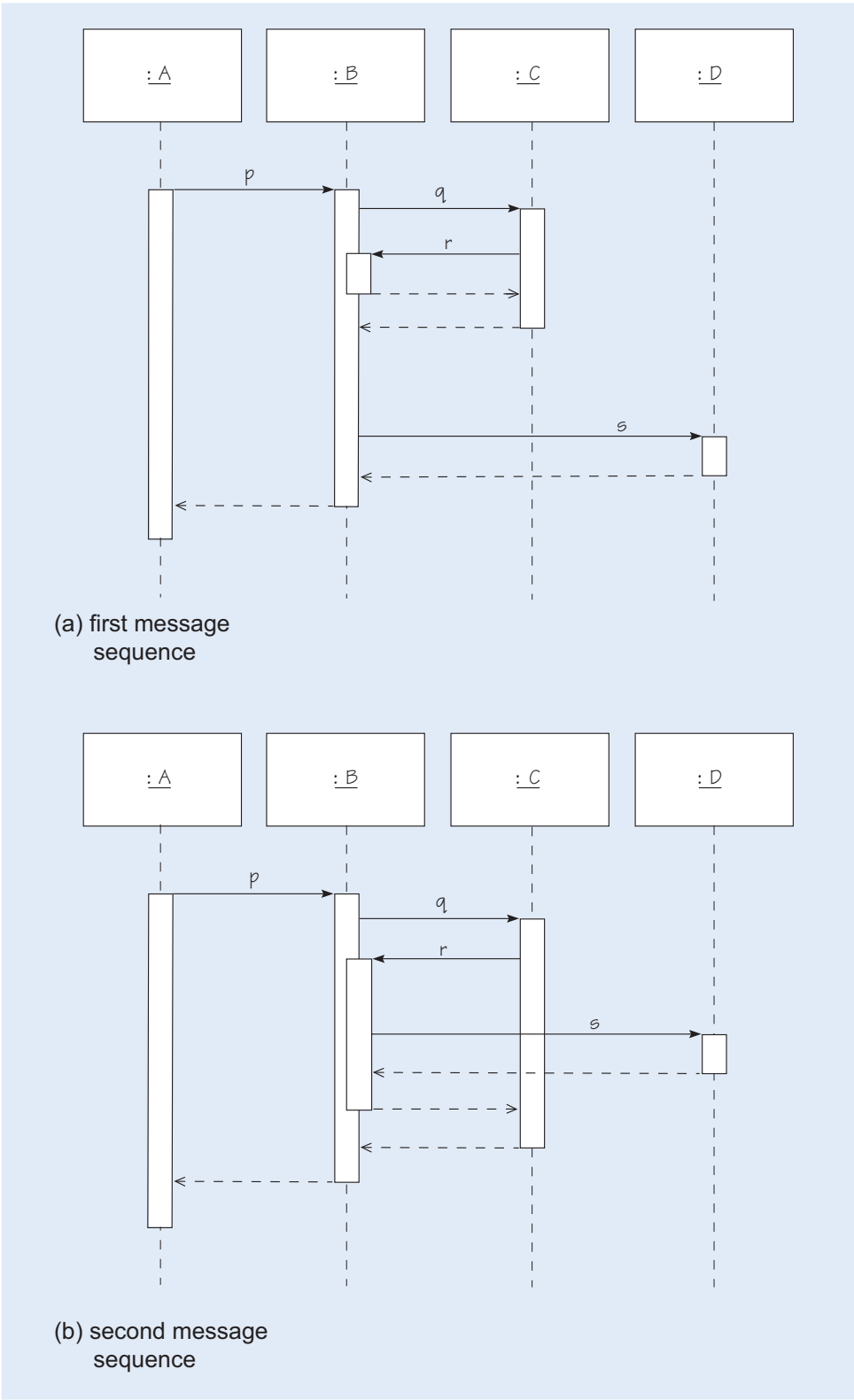


Figure 12 Two different sequence diagrams with the same time-ordering of messages

If you adopt a method of numbering the messages as they arise, you will end up with p labelled 1, q as 2, r as 3 and s as 4 in both cases, even though the processing is different. The ambiguity is resolved in UML by the use of a multi-stage numbering system.

The multi-stage numbering system consists of a varying-length number, with the parts separated by decimal points, as in 2.6.1. A **message number** always contains as its prefix the number of the message that gave rise to it. For example, if an object receives a message numbered 2.6, any messages it sends in the course of processing that message will be numbered 2.6.1, 2.6.2 and so on. Figure 13 shows how this resolves ambiguity problems similar to the one above, because the numbering of message s will indicate whether it is being sent as part of processing message p , or as part of processing message r .

Notice how more than one activation can be shown on a given object's lifeline, as shown by the receipt of message r in both sequences in Figure 12.

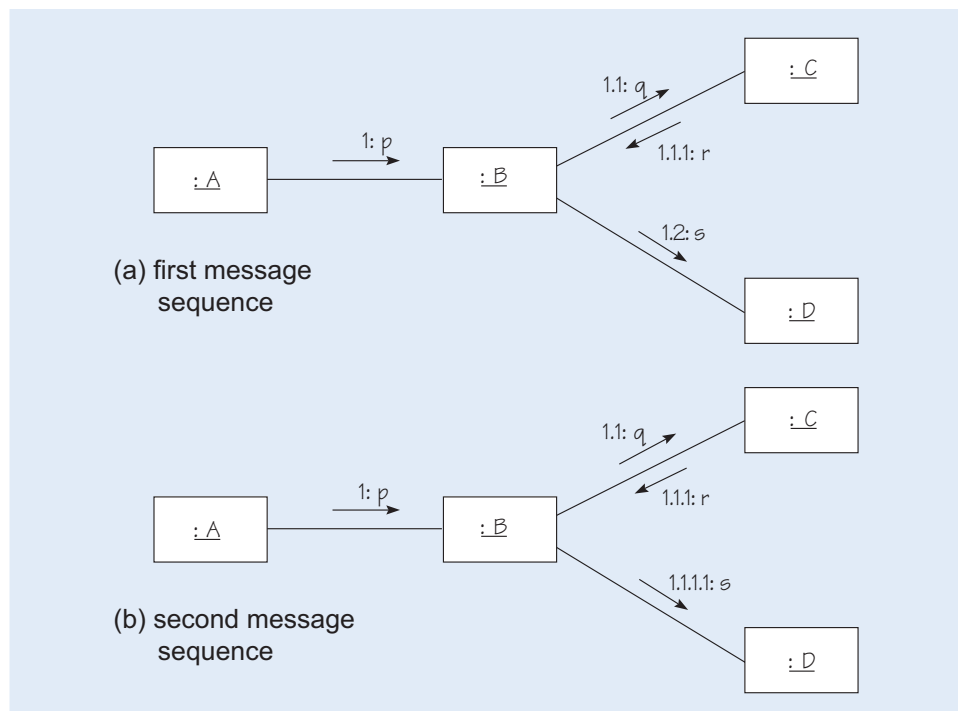


Figure 13 Solving the ambiguity using multi-stage numbering

SAQ 9

- How is the sequencing of messages represented in a communication diagram?
- Sequence diagrams and communication diagrams show almost equivalent information. What are their respective strengths and weaknesses?

Answer

- Every message has a multi-stage number. The numbers specify the sequencing, replacing vertical position in a sequence diagram.

- (b) Sequence diagrams make the relative order of messages extremely easy to see by presenting time vertically. Communication diagrams are an extension of object diagrams, so there is less new notation. They make it easy to see the links and to show role names, at the cost of making the relative ordering of messages less immediately clear.

SAQ 10

- (a) What is the difference in emphasis between sequence diagrams and communication diagrams?
- (b) How is time represented in a communication diagram?
- (c) What obligation is placed on an object that is sent a message?

Answer

- (a) Sequence diagrams emphasise the flow of messages from object to object over time. Communication diagrams emphasise the message traffic across the links in a particular configuration of objects.
- (b) Time is represented by the sequential numbering of messages.
- (c) The class of the receiving object is committed to implement an operation with a particular name and parameter signature.

4.2 Message results and parameterised interactions

So far you have learnt about interaction diagrams constructed for very concrete scenarios, such as *jill* checking in to *theRitz* when it has three rooms, of which only one is occupied. This is a good way to start: it avoids having to handle too much generality. A series of new diagrams is constructed for a series of well-chosen scenarios. Each diagram is easy to understand but the disadvantage can be that you end up with a lot of separate diagrams.

One level of abstraction above handling every concrete case separately is to try to capture some generality by using parameters. This is the ‘prototypical’ style used by some authors, where the objects involved have names such as *aGuest* or *theRoom*, with the identifying information supplied either as parameters or as attributes in the objects. The names for objects are chosen to suggest that the interaction will be the same whichever copy or member is involved.

There will be times when you will want to show conditional behaviour on an interaction diagram, and we will cover this in Unit 7. However, there are many designers who prefer to avoid conditional behaviour by constructing separate unconditional diagrams for a range of different concrete scenarios. In the case of the hotel check in, you might show two diagrams for two attempts at checking in – one for *jill*, who is able to occupy a room, and one for *jack*, who is not. The advantage of constructing only unconditional diagrams is that

A single activity diagram can show the alternative paths through a use case, whereas a set of concrete sequence diagrams shows the actual paths travelled.

each on its own is easier to understand. The disadvantages are that you have to draw more diagrams, and commonality and differences between the diagrams may not be evident. In any case, you should use your judgement to decide how many diagrams are needed; and they should be drawn only if they serve a well-justified purpose.

The purpose of sending a message is to request an action, to change the internal state of the recipient, or to get it to return a value. Operations such as *setOccupant* change the receiver's state, while those such as *getOccupant* return values. We need some way to show that a message returns a value. The returned value may be an object – and during the early stages of design it makes sense to treat everything as an object, even if subsequently you use a simple type to represent it. You will probably want to send messages to this newly discovered object later. In the hotel check in example of Figure 6, we made a *Hotel* object responsible for finding a free room for the guest to occupy. In UML you can give a name to the value returned by a message. You turn the message name into an assignment statement as follows:

```
r := findAFreeRoom()
```

The use of the name *r* is simply a mechanism in UML that enables you to refer to the resultant object (the message result) in subsequent messages. It is not meant to suggest that the sender is required to have a local variable called *r*. The mechanism is perfectly adequate when the returned value is subsequently used as a parameter to other messages. For example, you can use *r* to record the result of sending a message, and then reuse it as a parameter to other subsequent messages, such as:

```
setRoom(r)
```

However, there is a problem for a concrete scenario if the subsequent use of the name *r* is not as a parameter but as the receiver of other messages. Which object on the sequence diagram in Figure 6 is *r*? Somehow the name *r* must be associated with the object in the sequence diagram that is receiving the messages, which may be difficult if the objects have names like *jill* : *Guest*. The problem arises because you can describe object interactions in a number of ways – from the concrete (for example, *jill*) where the name of the object implies something about the identity of the object, to the generic (for example, *r*) where the name of the object says nothing about the identity of the object. To make things more difficult, use of a concrete name as a returned value can be misleading, because it implies that the operation can return *only* that concrete instance. For example, the statement *room201* := *findAFreeRoom*() might be interpreted as implying that *findAFreeRoom* can find only room 201!

Figure 6 corresponds to a concrete test case that you might use as a developer. A change to include parameters, such as *r* above, is a move towards the prototypical style and, eventually, the generic form, where you can model more than one scenario on the same diagram. Figure 14 shows how the concrete sequence diagram of Figure 6 can be generalised to describe a **prototypical interaction**. The identity of each newly arrived guest has been preserved using *g*. You could use a **prototypical object** such as *theHotel* : *Hotel*, but this adds no new information to the diagram. Also, you

Short names such as *r* may be convenient in a busy or large diagram rather than more expressive names such as *firstAvailableRoom*, which may better express the intended usage.

UML describes two forms of object interaction: one that applies to an actual sequence of messages and one that applies to all possible sequences.

might want to replace the name *aUserInterface* with *aCheckInScreen : UserInterface* to reflect a further design decision about the nature of the user interface for the software system.

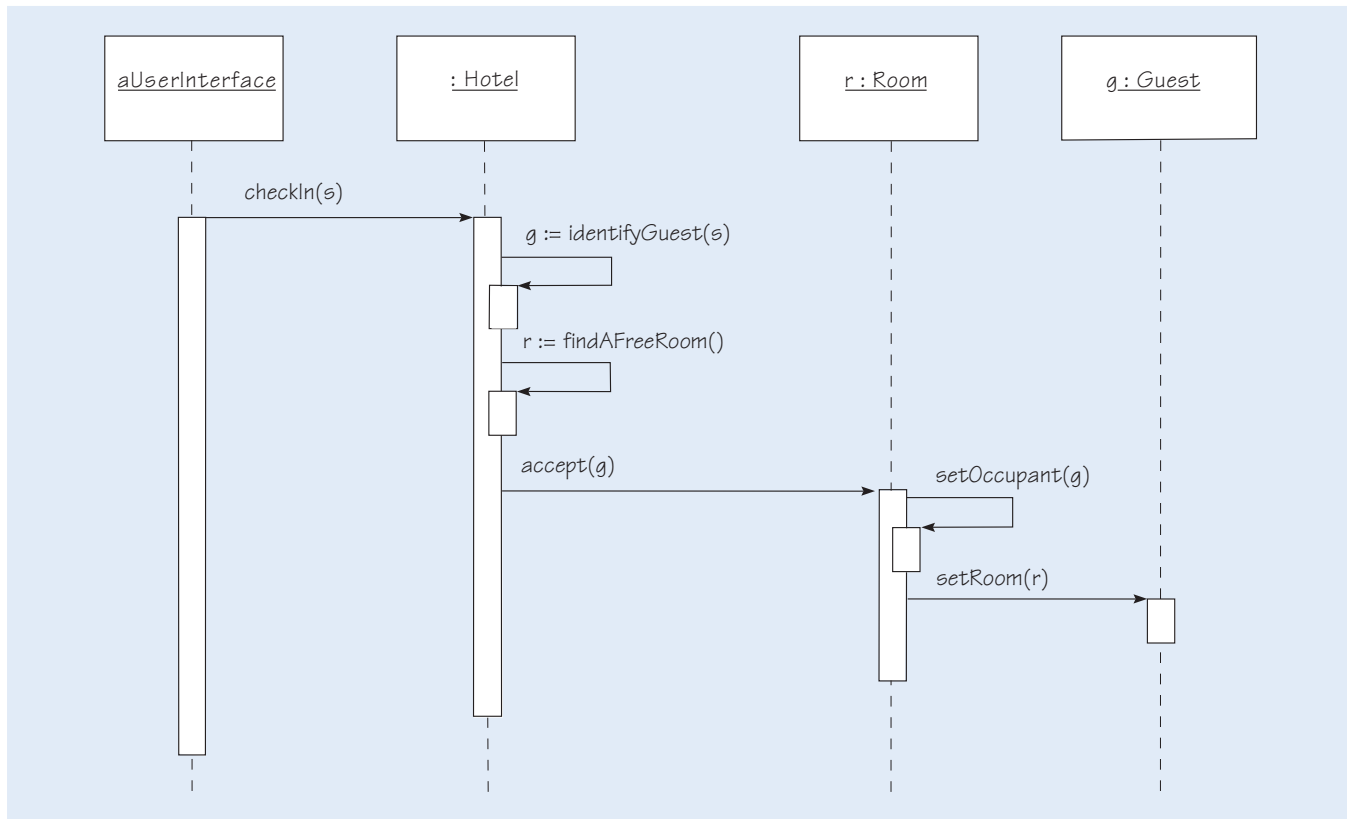


Figure 14 Generalised sequence diagram for checking in to a hotel

The scenario in Figure 14 does not apply in the case where a guest is not known to the hotel in question; nor does it apply where there is no room available at the hotel. To be precise, it does not apply if there is no link between a particular instance of *Hotel* and the object *g*; nor does it apply if *findAFreeRoom()* fails. (See, for instance, the precondition that was introduced in Section 2 of this unit.)

Whenever there is more than one association between classes, you have to choose the appropriate one that corresponds to the scenario with which you are dealing. In Unit 5 you saw how role names were useful when describing the associations between classes. In a communication diagram you can simply identify the link with the role name from the class diagram. However, in a sequence diagram the links between objects are not shown. One way to overcome this problem is to use the role name to identify an object as follows:

objectname / rolename : classname

In Figure 12, for example, you can see examples of *objectname : classname*, such as *g : Guest*. If there were two associations between *Guest* and *Room* (for example *currentOccupant* and *previousOccupant*), you would need to

UML allows you to use the different names in a variety of permutations. You have already seen examples of them in this unit, such as *aUserInterface* and *: B*.

show which link is being used. In the example from Figure 12, the result would be *g / currentOccupant : Guest*, because the interaction is about checking in to a hotel. This can be entered in the box at the top of the lifeline.

SAQ 11

How does an assignment statement help construct a prototypical sequence diagram?

Answer

In a programming language, an assignment statement allows you to store the result of one message in a variable and then send messages to whatever object is currently referenced by that variable. You can use the same mechanism in UML, although the name used to store the message result is not the same as a programming variable. The name can then be used as a parameter to other messages. In a bank, for example, you might want to credit an account with an amount of money. You can use a name such as *ac* to store a reference to an *Account* object (at a particular branch) and then send a message, such as *credit(amount)*, to that object.

4.3 Creation and deletion

Object-based programs frequently need to create new objects. Some objects will have a lifetime that is close to the lifetime of the software system, for example rooms and the hotels to which they belong. Other objects will have a shorter lifetime, for example objects representing guests currently occupying a hotel room and objects representing reservations.

It must be admitted that the notation for object creation and deletion is one of the least elegant parts of UML, for the simple reason that creating an object is not usually done by sending a message. It is in Smalltalk, but in most languages there is something more like the `new` statement in Java. It is therefore not surprising that trying to fit object creation into diagrams that are designed to show message traffic leads to inelegancies.

Figure 15 shows the creation of an anonymous *Guest* object, apparently by sending the message *newGuest()* to a *Guest* instance, but of course we cannot send messages to an instance that does not yet exist. The fact that it does not exist can be shown by not starting the object's lifeline at the top of the diagram with those of existing objects, but only some way down the diagram, level with the *new* message. What is actually meant by this notation is that the sender creates an instance by some appropriate means, such as a *new* statement, and then sends the constructor message to initialise it.

Some CASE tools do not support this part of UML notation and may force you to send the *new* message to an object's lifeline.

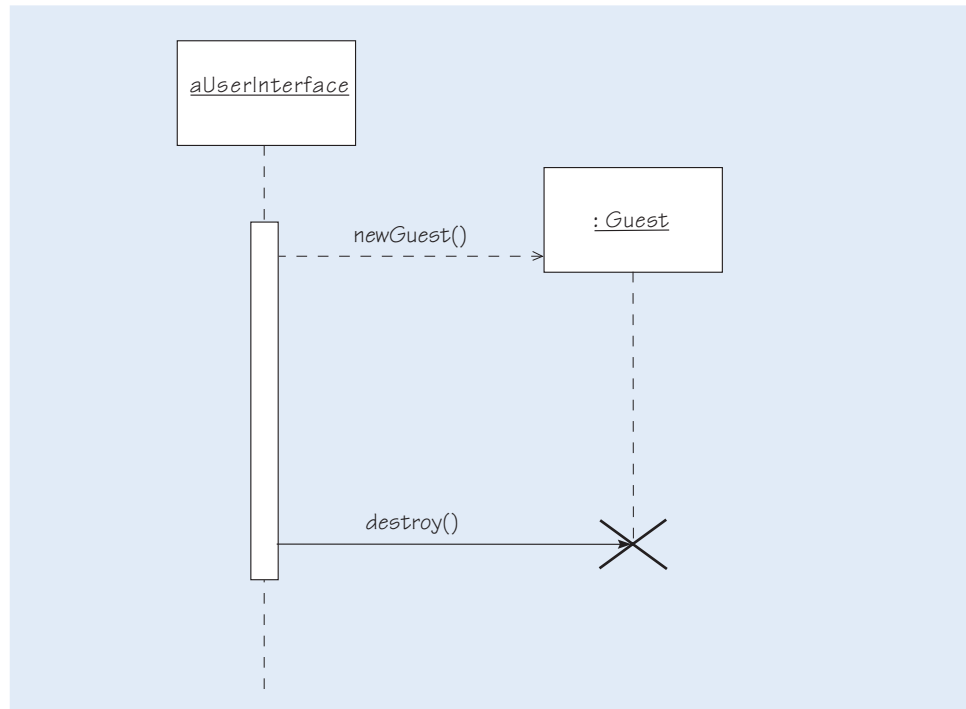


Figure 15 Creating and destroying an object in a sequence diagram

Deletion raises similar although less important issues. Some languages have a `delete` statement and some do not. Those that do may or may not allow an object to delete itself. In some languages, such as Java, it is neither possible nor necessary to delete objects. In Java any object that is no longer referenced by others is treated as deleted and will be garbage-collected in due course, so it is effectively destroyed.

How much a design should be totally language independent and how much it should include language-specific features such as deletion depends on the context. As Figure 15 illustrates, UML does provide a notation. You can put a cross at the bottom of the lifeline to indicate that, as far as the model is concerned, the object is to be considered no longer existent, and will not subsequently be used. Whether a developer has to use a specific `delete` command will depend on the target language. Even in a garbage-collected language, where explicit deletion is never required, it can be a helpful piece of documentation to capture the fact that no further links to an object should exist, and that the object can be ‘deallocated’. Leaving this assumption unexpressed can lead readers to miss the fact that no further reference should be made to the object.

In a pair of object diagrams it is easy to see which objects have been created and which have been destroyed. We can follow the change and look for the boxes that have been added and/or removed. In a communication diagram, which can be viewed as an object diagram with messages on it, object creation and destruction are shown by the special constraints `{new}` and `{destroyed}` respectively, the former as illustrated in Figure 16. It is important

that any such constraints are placed inside the ‘objects’ to show that they refer to objects rather than links.

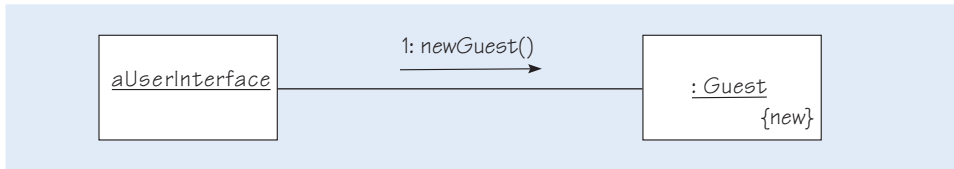


Figure 16 Object creation in a communication diagram

You can use the same notation for links. For example, if an interaction creates a new link between two objects, use the `{new}` constraint on the communication diagram.

SAQ 12

Figure 16 shows UML notation for creating new *Guest* objects in a communication diagram. Has this cured the problem of apparently sending messages to objects that do not yet exist?

Answer

No – it is still a notation based on message sending. As Figure 16 shows, marking the instance `{new}` has not captured exactly when it comes into existence. We are still dependent on the pseudo-message `newGuest()` to express creation.

4.4 Levels of detail

At every stage of a design you have to decide either to specify what is required in just enough detail for a programmer who understands the domain, or to spell out in detail how some goal is to be achieved. For example, the hotel check in scenario led to a requirement for an operation called *findAFreeRoom*, which returned a *Room* object. You could go into the precise details of how this might work, or you could simply specify the necessary operation by defining a precondition and a postcondition and leave the coding to a programmer. There is no one correct level at which to stop designing and start trusting programming professionals. A common mistake is to include more details than are necessary, spelling out unimportant details. This has been described as *writing Java in pictures*, and you can easily get details wrong or introduce inconsistencies. If you really want to specify low-level details, Java is a reasonable way to do it. Your job as a designer is to work at a higher level of abstraction, capturing structural aspects that cannot easily be expressed in code. If you are part of an agile team you will probably use some diagrams to help the whole team share the same mental models but you will stop as soon as the diagrams no longer serve a purpose for communication.

Another reason for not adding further detail to an interaction diagram is to ensure that the diagram does not get too big and therefore difficult to

understand. You can always stop at a given level of detail by showing the sending of a message but not showing the implementation of the method in the receiver. Then, on a new diagram, you can show how that operation is actually implemented. In effect, you are using an interaction diagram to show how something as small as a single step in a scenario can be achieved.

Software components allow the developer to perform repeatable tasks, which may be as small as one step in a scenario or as large as a use case – or bigger. If you treat the component as a single class or package, an interaction diagram can show how an instance of that component should behave when other external objects send messages to it. A component might need various operations to be invoked in a particular sequence in order to achieve some goal. An interaction diagram can document a typical interchange of messages.

Other uses of interaction diagrams are as follows.

- They show how a class provides an operation. This is sometimes called a *subcommunication*. An example might be documenting how *findAFreeRoom* works in the hotel example of Figure 6, as this is just one operation on a class rather than a particular use case scenario.
- They show how a design pattern works. We described the possibility of a *Hotel* delegating its room allocation to a *RoomAllocator* class and mentioned that this is a case of the *Strategy* pattern in which behaviour is moved out to a separate object (more about patterns in Block 3 Unit 9).
- Alternatively, we could describe this pattern in general terms using a sequence diagram. The instances would be general (prototypical) objects with names such as *theServer* and *theDelegate*.

SAQ 13

- (a) How does a sequence diagram drawn for a use case scenario differ from one drawn to show how an internal operation, such as *findAFreeRoom* in Figure 14, is carried out?
- (b) Would you expect to use sequence diagrams in a conceptual model?

Answer

- (a) A sequence diagram of a use case scenario will always have the very first message originating from an object which has been stimulated, directly or indirectly, by an actor (usually a user interface object). An internal operation will be invoked by a message that has been identified in the use case scenario. You can show how the recipient of that message achieves the required behaviour, for example how a free room is found. In all other respects they are the same.
- (b) Sequence diagrams are about message passing between software objects and conceptual models are about things in the world, where the language of message passing makes no sense.

Exercise 6

Recall Exercise 3, where you were asked to model the checking in sequence. Using the multi-stage numbering scheme, draw a communication diagram equivalent to the sequence diagram in Figure 8.

Solution

Our solution is shown in Figure 17.

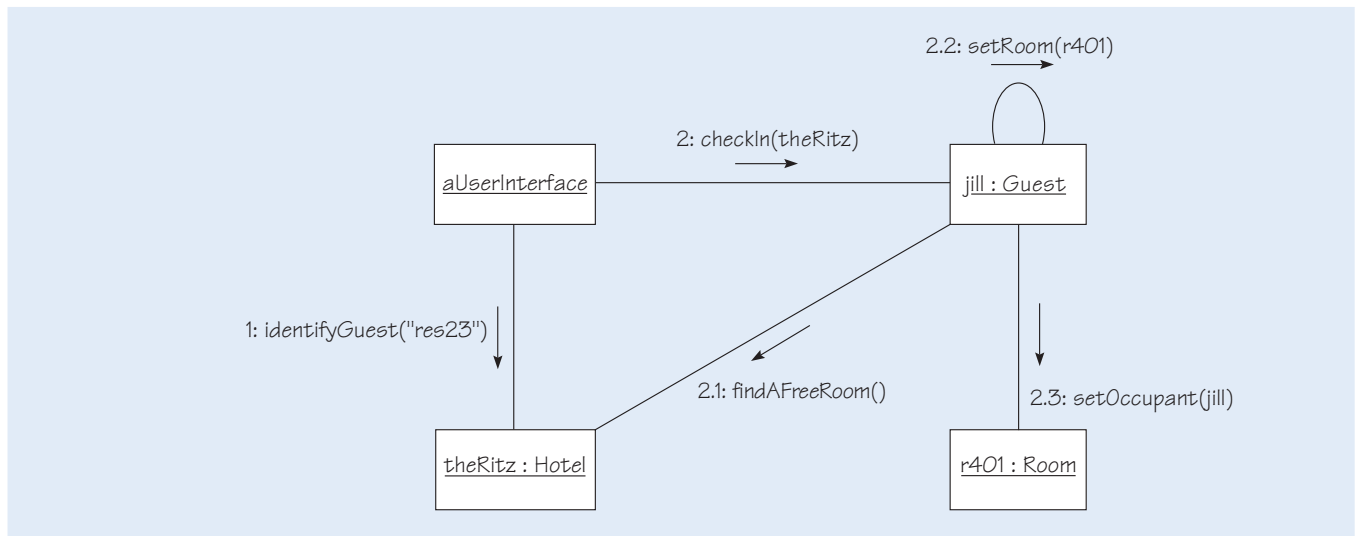


Figure 17 Checking in

Exercise 7

Use Figure 18 as the basis to represent a prototypical interaction for the borrowing of a book, which you saw in SAQ 5(c). Draw both a sequence diagram and a communication diagram that send the message *borrow(b)* to an instance of *LibraryMember* from *aUserInterface*, where *b* is a reference to the object representing the book that the library member *m* wants to borrow. At this point we are not concerned about where the object *b* came from.

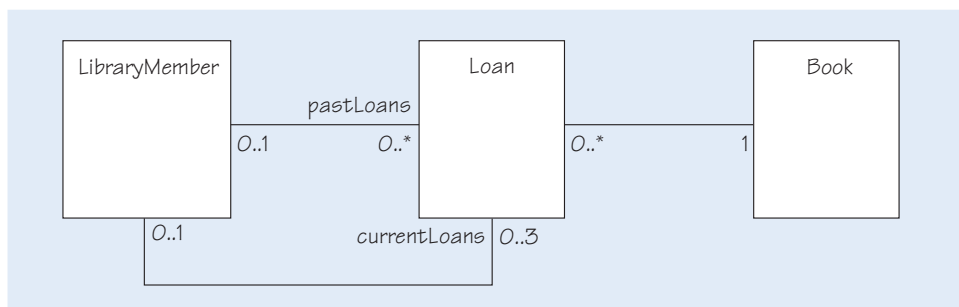


Figure 18 Class model for a lending library

Solution

We can assume that the precondition for borrowing a book, which was given in the solution to SAQ 5(c), has been established. Our aim for each interaction diagram is to show that the corresponding postcondition has been met. Figure 19 shows the sequence diagram and Figure 20 shows the communication diagram.

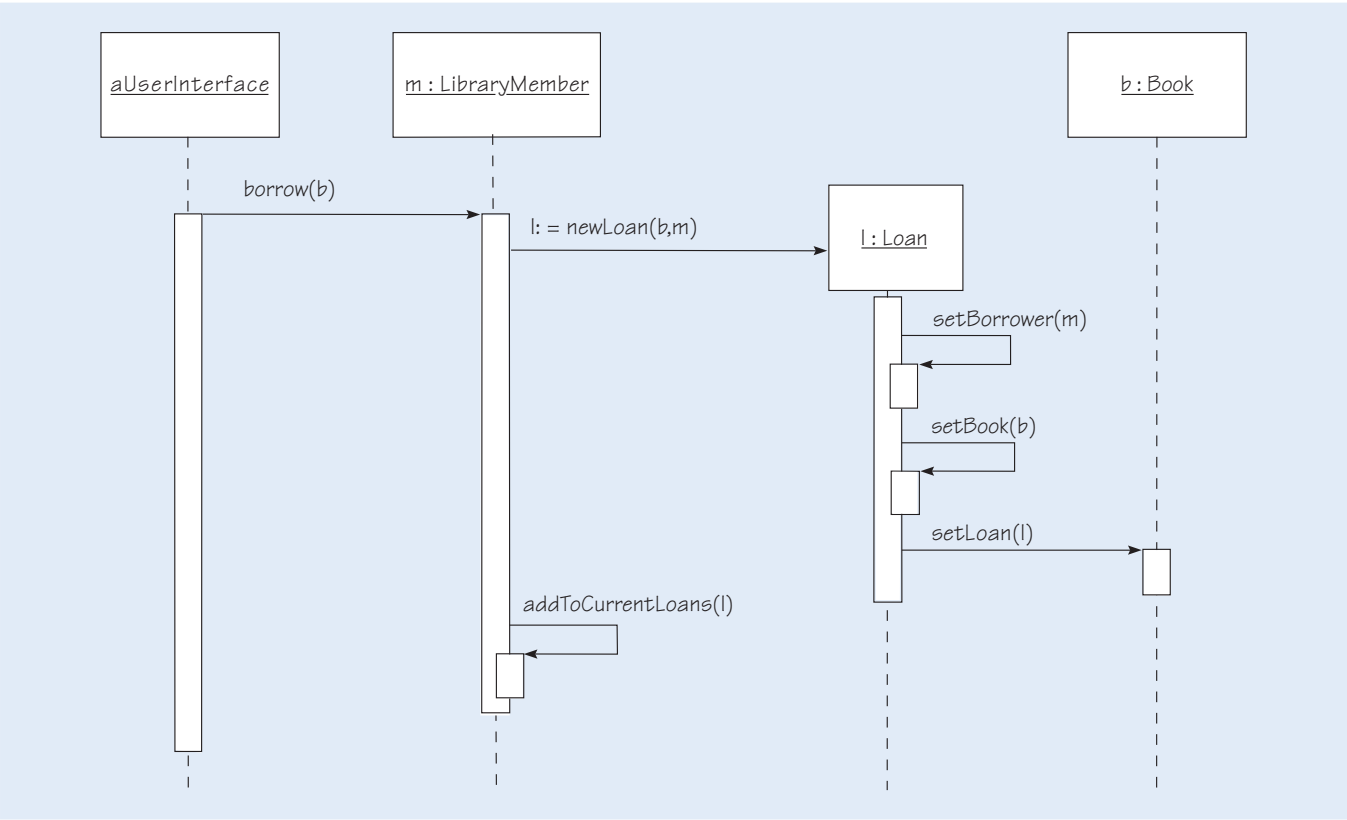


Figure 19 Generalised sequence diagram for borrowing a book

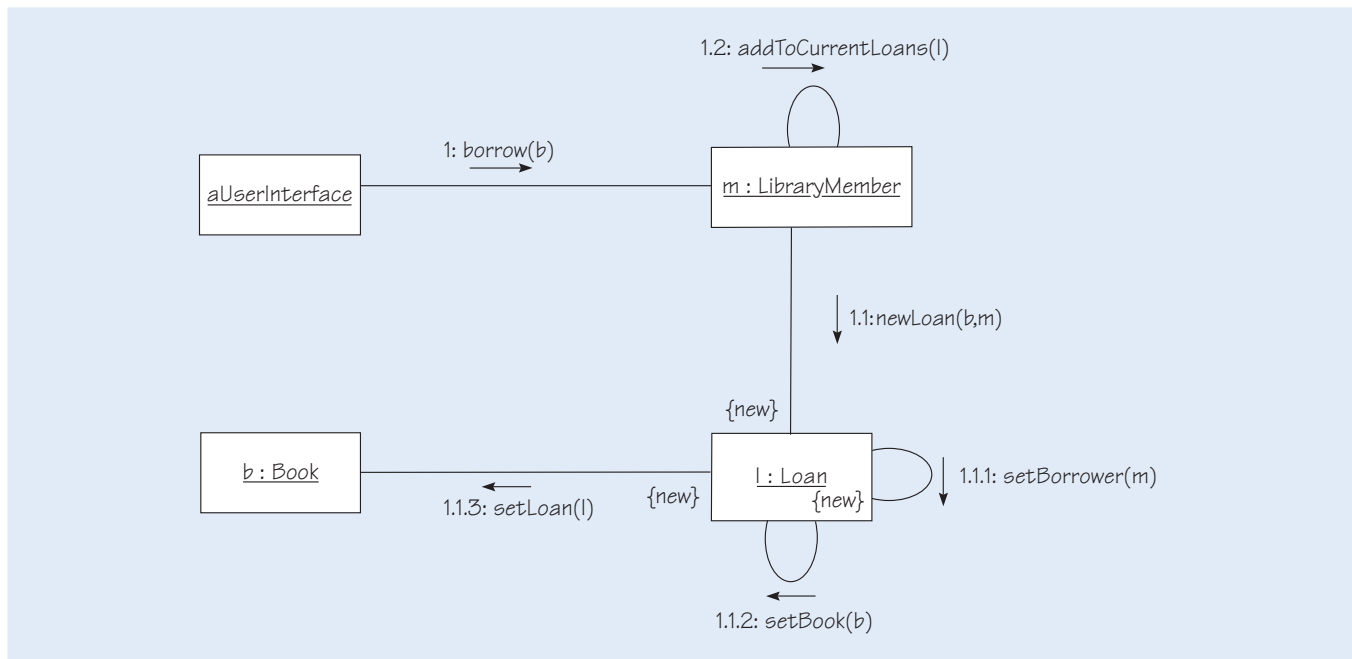


Figure 20 Generalised communication diagram for borrowing a book

In both diagrams we could have used a role name in the object identifier to reflect which link has been created, such as $l / \text{currentLoans} : \text{Loan}$.

4.5 Summary of section

This section has introduced communication diagrams as a different kind of interaction diagram to deal with the dynamic behaviour of a software system, and you have seen examples of how to add detail to your interaction diagrams as follows.

- Communication diagrams provide an alternative notation to sequence diagrams, emphasising message routes rather than timing.
- Communication diagrams require a complex numbering system in order to make the actual sequencing of messages clear.
- Sequence diagrams can be enriched by notation to assign the result of a message to a local variable.
- Sequence and communication diagrams have different notations relating to the creation and deletion of objects.
- Sequence and communication diagrams can be drawn at any level of detail for a software system. They can show how a use case is turned into object interactions, how a class provides an operation, how a component can be used and how a design pattern works.

Sequence diagrams can be drawn at many different levels of detail, ranging from concrete scenarios about individual objects to general ones operating on prototypical objects such as *theMember*.

5

More on design decisions

This section looks at examples of the kinds of **design decisions** that you might take when modelling the behaviour of a software system. These decisions can have a profound influence on implementation.

5.1 Managing associations

When you have completed interaction diagrams for the relevant use cases that define the functionality of the complete system, you will have a rich set of operations on all the classes. By examining the diagrams you can also see the overall pattern of message traffic. In particular, you can collect information on the directions of the messages. For example, you can see whether *Room* objects send messages to *Hotel* objects, or *Hotel* objects just send messages to *Room* objects. If you find an association that is one-way, you can indicate this in the class diagram by adding an arrow to the association. Figure 21 shows a class model to which arrows have been added to show that the associations are used in just one direction.

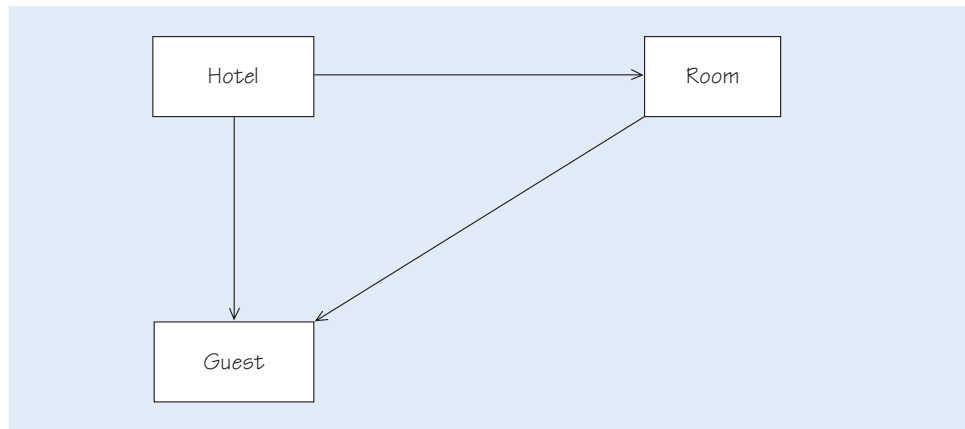


Figure 21 Associations used in a single direction

See also Unit 5, subsection 4.4.

The direction in which an association is used is called its **navigability**, as illustrated in Figure 21. If navigability is not shown, it is assumed that the association must be navigable in both directions. In UML it is permissible to add arrows to *both* ends of an association as a reminder that both directions are used. However, it is more common just to follow the convention that, if navigability is not shown explicitly, it means that both directions are available.

The direction in which an association is navigated is significant when implementing the association. For example, if no *Guest* object sends messages to any *Room* object, there is no need to implement the association in this direction. In this example, *Room* will need to represent the association to *Guest* but the class *Guest* will contain no reference to a *Room*.

Each navigability arrow on an association identifies a dependency. In Figure 21, for example, *Hotel* depends on both *Room* and *Guest*. In contrast, *Guest* depends on neither *Hotel* nor *Room*. A bidirectional association introduces a cyclical dependency, which is harder to maintain, as you will see in subsection 5.2.

In the early stages of development you should avoid design decisions that would reduce the number of possible software solutions. As you progress, you should add notes to your class diagrams about each association, and you should consider the need to navigate in both directions. As decisions are made, each class diagram should record the preferred navigability direction for each association.

SAQ 14

What are the advantages of recognising when an association is unidirectional?

Answer

Specifying an association in just one direction simplifies the implementation of the classes at each end and avoids the need to worry that both ends of the link are consistent when dealing with instances of those classes. But be aware that the overall flexibility is reduced in comparison with bidirectional associations.

5.2 Link manipulation

Often programmers come to act as though a given language provides a single natural representation for a link between two objects. For example, in Java one might imagine that every link was represented by a field containing a reference to another object. While this is often the easiest representation, there are many other possibilities. One object can hold some sort of identifier that characterises the other object, but that must be looked up in a mapping; qualified associations imply such a scheme.

For example, a `Company` could store the national insurance (NI) numbers of its employees and then every time it needed to send a message, it would look up the NI number in some mapping of NI numbers to objects representing an `Employee`. Another representation might be to hold a database key corresponding to some database data representing an `Employee`. This time a mapping from database keys to objects would be required. Finally, you might not store anything in the `Company` itself but, instead, hold all association information in some other object, such as an `AssociationManager`. When using this scheme you could easily add new implementations of associations without having to modify and recompile the `Company` or `Employee` classes to hold new references, because the links represented by the new associations are not stored in those classes.

When we were constructing conceptual and analysis models, we did not need to discuss the representation of associations: drawing a line between two classes was sufficient. Now that we are moving into detailed design and

There is no need to have a parameter to the operation *removeOccupant* because each room has only one guest.

taking decisions about code structures, we need to decide how associations are to be represented and write the appropriate code. We could draw sequence diagrams that assume a particular representation. However, even at this stage it is useful to have some independence from the representation in case it needs to be changed later.

The simplest way of being independent of the association representation is to hide it behind operations that do the actual manipulation. For instance, for the *Guest–Room* association, rather than publish a decision about whether to store pointers or something else, *Room* could be provided with *setOccupant* (*aGuest* : *Guest*) and *removeOccupant* operations, and *Guest* could be provided with the operations *setRoom*(*aRoom* : *Room*) and *removeRoom*.

Figure 22 shows a sequence diagram for the *changeRoom(newRoom)* use case.

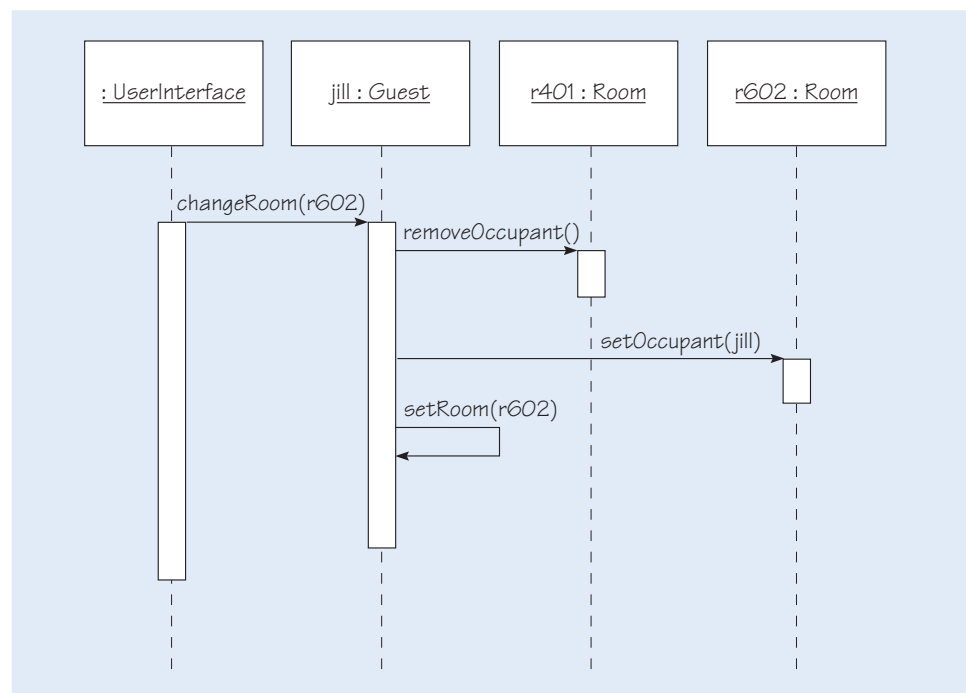


Figure 22 Sequence diagram manipulating links

Clearly, the methods that implement the operations *setRoom* and *setOccupant* will have to be in terms of an actual concrete representation of links, but the rest of the system can use the association between *Guest* and *Room* in total ignorance of how it is represented.

If the association that you are modelling has a multiplicity greater than 1, you will need a representation of the association that allows more than one link (for example, a collection or a database) and you will probably also need link-manipulation methods that will allow you to add or remove more than a single link at a time. For a *Company–Person* association representing employment you might give *Company* a collection of references to all the *Person* objects involved, and provide it with operations such as *addEmployee*(*p* : *Person*) and *removeEmployee*(*p* : *Person*).

Naming the link-manipulation methods

Once you have decided to hide the link representation behind some methods, it is sensible to consider how to name the methods. In the example of Figure 22, `Guest` has a method `setRoom` named after the class of the link that is being handled. This does not work in general because there might be several associations between `Guest` and `Room`. In Unit 5 you saw how useful role names are. These same role names can be used to name the methods that manipulate the associations, such as in `setOccupant`.

Figure 23 shows a class model to which we have added link-handling methods, whose names directly reflect the roles. A `Guest` has two different associations with `Room`, so it is given `setAccommodation` and `setFavourite` methods to handle them. The occupant–accommodation link is two-way, so `Room` must have a `setOccupant` method. As the `Guest`–`Room` association with the role name `favourite` is marked as unidirectional (because none of the use cases for the system have sequence diagrams that need to traverse the link from `Room` to `Guest`), there is no need to provide `Room` with a method to set the `Guest` whose favourite room it is. In fact, such a method should *not* be provided, because it would permit an implementer to violate the assumption contained in the model that the `Guest`–`Room` association goes only from `Guest` to `Room`.

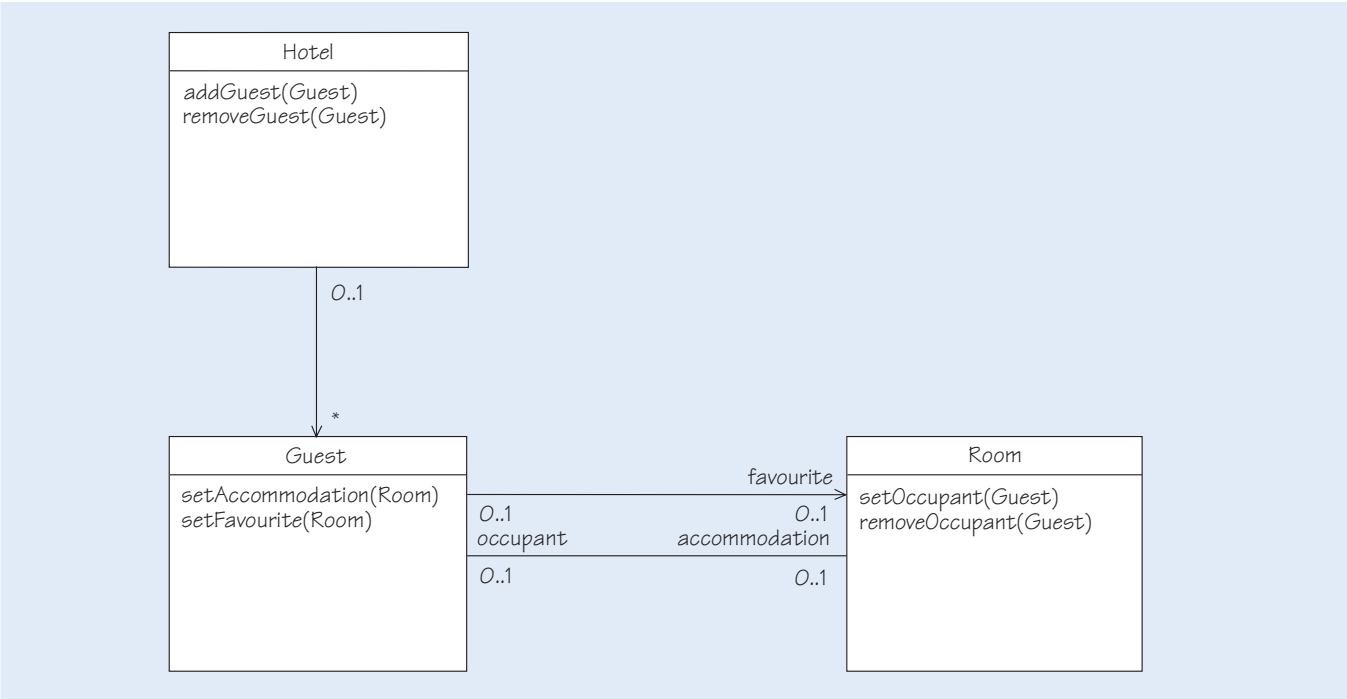


Figure 23 Role names used to name methods

Similarly, because the `Hotel`–`Guest` association is marked as unidirectional, we have provided methods `addGuest` and `removeGuest` only at the `Hotel` end; there are no methods provided on `Guest` to say which hotel is being used.

If you have a **bidirectional association**, there is a consistency requirement on the two ends. For example, if the guest `jack` is represented as occupying room `r23`, then `r23` must have `jack` as its occupant. If it has any other occupant, the system is in an inconsistent state.

If you allow unlimited access to the methods that manipulate the links, you are open to programmer errors that update one end and not the other. Rather than just relying on programmers not making a mistake, it is better to capture this need for consistency by putting in a single place the responsibility for maintaining consistency. You can do this by making just one of the classes involved in the association responsible for all manipulations, as suggested by the GRASP patterns. The rest of the system then always requests changes to links by sending messages to this particular class. In the above example you might give responsibility for the occupant–accommodation links to the `Guest` class.

Any part of the system that is required to modify the association would always invoke `Guest::setAccommodation`, and the implementation of `setAccommodation` would invoke `Room::setOccupant`, but no other method would. Therefore, as long as the implementation of `setAccommodation` is correct, the two ends of the association cannot get out of step.

If the target programming language has the facilities, the `Room::setOccupant` method could be visible only to the `Guest` class. In Java, package access restrictions might be used to restrict visibility to these methods, and hence reduce the possibilities of misuse.

The use of the double colon in `Guest::setAccommodation` indicates that the method `setAccommodation` is a method of `Guest` (this notation avoids ambiguity when the same method name is used in more than one class).

SAQ 15

Suppose that, in a Java implementation, a `Company` class represents the employment association with the `Person` class as a `Vector` of `Person` objects. What would be the disadvantage of providing a method `Company::getAllEmployees` that returned the `Vector`?

Answer

If `Company::getAllEmployees` is defined as returning an instance of the class `Vector`, changing the internal representation creates either a maintenance problem or a data-type conversion problem. Suppose the internal representation is changed to be an array. You then have to decide whether `getAllEmployees` should be changed to return the array (affecting all existing clients of the class) or a new `Vector` should be constructed from the array.

In addition, whichever Java class is used, encapsulation could be breached because some other object could modify the `Vector` object without using the methods supplied by the class `Company`, creating false links (the links would be in the copy of the `Vector` but not known by the `Company`). If clients of `Company` need to iterate over all employees, an interface should be provided that allows iteration without revealing internal implementation

decisions, and that has defined semantics for what happens if the links change during iteration.

5.3 Forks and cascades

When designing an interaction diagram to implement a use case, you often need an object to send messages to another object with which it has no direct association. Figure 24 shows a class model in which a `Company` has an association with a number of instances of `Job`, and each `Job` has an association with a `Person`. A `Company` has no direct association with a `Person`.

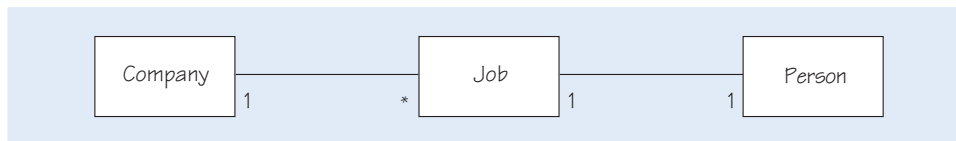


Figure 24 Sending indirect messages

Suppose that the company needs to collect information about the ages of all its employees. There are two ways to design this. You might make the `Company` ask the `Job` for its `Person`, and then have the `Company` send messages directly to the `Person` to ascertain its age. Alternatively, you might get the `Company` to send a message to each `Job`, requesting it to ask its `Person` for its age. These alternatives are shown as communication diagrams in Figure 25.

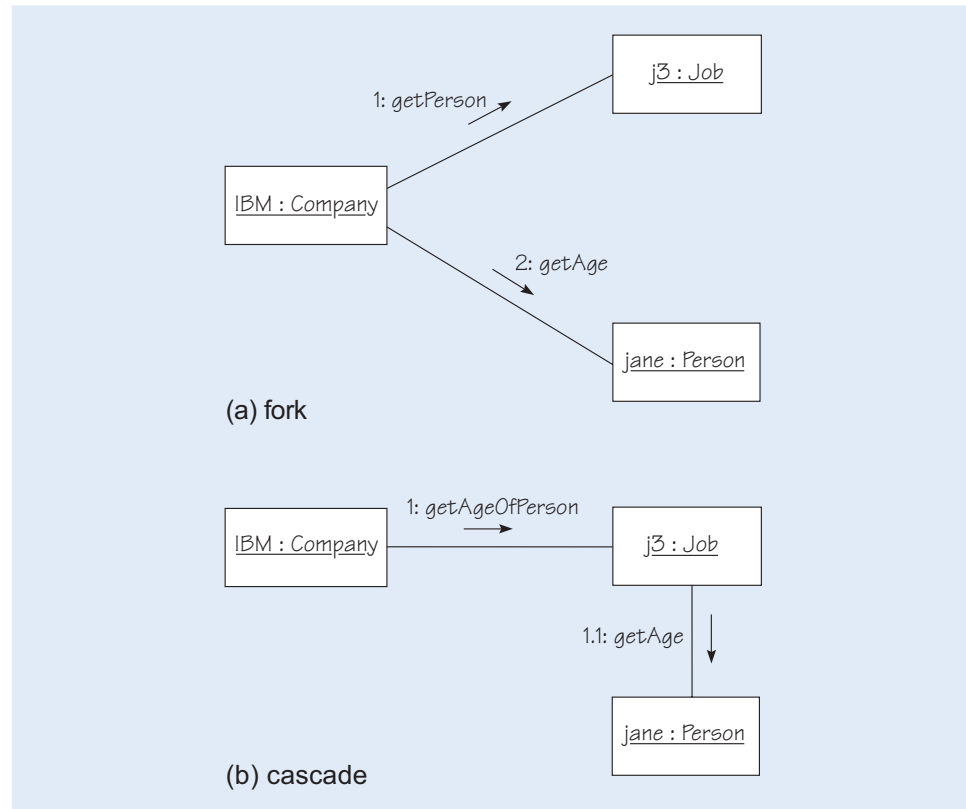


Figure 25 Fork and cascade

The first pattern is called the **fork pattern** and the second is called the **cascade pattern**. Their names describe the shapes of the communication diagrams. In the fork pattern the `Company` sends all the messages, which means that the `Company` must know the interface of both `Job` and `Person`. It represents a centralised form of control. `Job` must have a method `getPerson`, and `Person` must have a method `getAge`, but `Job` does not need any age-specific methods itself.

In the cascade pattern the `Company` delegates the task of finding a `Person`'s age to the `Job`. The `Company` does not need to know the interface to a `Person`, because it never sends messages directly to a `Person`. However, `Job` now needs to have an extra method `getAgeOfPerson`. You might not like this decision: does the class `Job` really need to know a `Person`'s age? A reasonable position is that `Job` should need to know only about the existence of `Person`, not particular attributes such as age.

Note that if `Company` knows the interfaces of both `Job` and `Person`, you also need to change the class diagram in Figure 24 (see the Law of Demeter below).

Thus you have a choice between keeping `Company` independent of `Person` but having to widen the interface to `Job` to include a `getAgeOfPerson` method, and keeping `Job` free from the need to know a `Person`'s age at the cost of making `Company` know the interfaces of both `Job` and `Person`. One choice makes `Company` dependent on additional classes, making it harder to maintain in the long term because of the coupling. If `Company` is changed, must the other classes change? If `Person` changes, must `Company` or `Job` change? The other choice involves adding some methods to `Job` that are not

really about a `Job`, but are there to support a particular need of `Company` in this specific application.

Building an interaction diagram (whether a sequence diagram or a communication diagram) forces you to make many such choices, which are the essence of object-oriented design. How should you choose from such patterns? One suggested guideline is the Law of Demeter, which says that messages should be sent by one object only to a very restricted set of other objects. The goal is to reduce coupling, which in turn simplifies both implementation and subsequent maintenance.

The **Law of Demeter** states that a method `m` in a class `A` should not send a message to another object unless that object is one of the following:

- an instance variable used in the method `m`
- a parameter to the method `m`
- an object created in the method `m`
- a global variable (the nearest thing to this in Java is a public static variable) used in the method `m`.

Note that an object should avoid invoking methods on another object returned by some method, where the returned object does not qualify under one of the above situations.

There is nothing magical about the Law of Demeter. It is just guidance for avoiding certain patterns of message sending that can lead to over-coupled and inflexible systems. The fundamental idea is that an object needs to know only about its immediate associated neighbours. What the neighbours know should remain private to them. Instead, objects should confine themselves to asking their immediate neighbours to do something for them, and should remain ignorant of their neighbours' representation of further relationships. The cost of using the law is that you must usually use the cascade pattern rather than the fork pattern. This means adding extra methods to the intermediate classes, such as `Job` in Figure 24, in order to forward requests down the chain.

SAQ 16

- (a) According to the Law of Demeter, an object should send messages only to a certain set of objects including the object itself. List the other objects.
- (b) In Figure 25 which pattern would be outlawed by the Law of Demeter?

Answer

- (a) The Law of Demeter allows an object to send messages to:
 - any objects communicated as parameters of the current method
 - any new objects that the object has created in the current method

- any objects to which the object has direct links – its neighbours
- itself.

Notice that the law does not allow sending messages to objects that are returned as a result of sending other messages.

- (b) The fork pattern would be outlawed. Here it would involve sending a message to the `Job` to get a `Person`, and then sending messages to the resultant `Person` object. The law bans the sending of messages to objects that are returned as a result of sending other messages. The cascade implementation involves the `Company` talking only to `Job`, and `Job` talking only to `Person`. Both of these are direct associations, so the Law of Demeter will allow this. One advantage of using the cascade pattern is that the `Company` is independent of how the age of an employee is represented. You could store the age directly within `Job`, or you could store it in an associated `Person` object. The `Company` will be oblivious to such choices.

Exercise 8

Figure 26 shows a class model for a `Company`. Notice the directionality of one of the associations. For each association, decide which end should manage the association and what methods must be defined. Mark each new method to show whether it is publicly available or just a helper method to assist the implementation of a public method.

In Java, publicly available methods are defined as `public` and helper methods are defined as `private`.

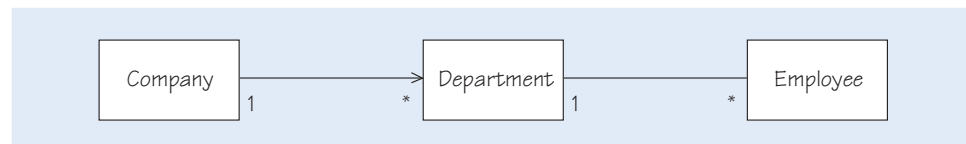


Figure 26 Company model

Solution

In Table 7 we have decided that `Company` should be responsible for the `Company–Department` link. Because the link is unidirectional, we need methods only at the `Company` end.

The `Department–Employee` association is bidirectional. We have decided to make `Department` responsible, so the only methods to manipulate links intended for public consumption are those on the class `Department`. However, the methods in `Department` will need the assistance of a helper method on `Employee`.

Table 7 Methods for managing a company

Class	Methods
Company	addDepartment (Department d) removeDepartment (Department d)
Department	addEmployee (Employee e) removeEmployee (Employee e)
Employee	setDepartment (Department d)

Exercise 9

Figure 27 shows a fragment of a hotel system, extended to give each Guest a Bill. Suppose we wish to implement a use case `printBill (Guest jack)`. It has been decided that the user interface will send the initial message `printBill (jack)` to the Hotel.



Figure 27 Hotel with Guests and their Bills

Draw two sequence diagrams, showing fork and cascade solutions. For each solution, list the methods on each class that this use case requires.

Which of your solutions would the Law of Demeter exclude, and why?

Solution

Figures 28 and 29 show the two sequence diagrams.

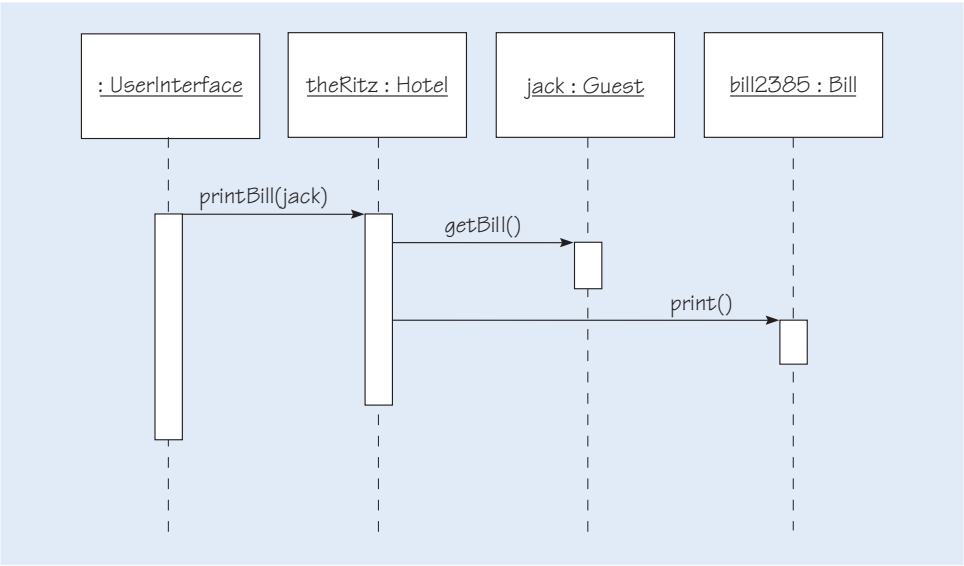


Figure 28 Fork solution for billing a guest

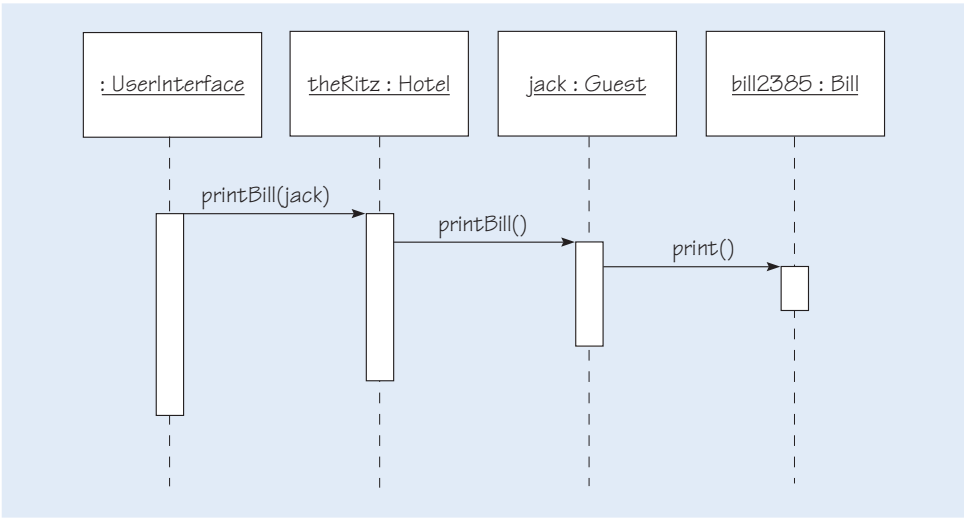


Figure 29 Cascade solution for billing a guest

Table 8 shows the methods required for the fork solution and Table 9 shows the methods for the cascade solution.

Table 8 Methods for the fork solution

Class	Method
Hotel	printBill (Guest g)
Guest	getBill ()
Bill	print ()

Table 9 Methods for the cascade solution

Class	Method
Hotel	printbill (Guest g)
Guest	printBill ()
Bill	print ()

The guidelines identified in the Law of Demeter would exclude the fork solution, because it involves the `Hotel` sending the message `print()` to the `Bill`. The `Bill` is not an object known to the `Hotel` through a permanent association; it is neither created in this method of the `Hotel`, nor passed to the `Hotel` as a parameter to the current method.

5.4 Summary of section

A set of completed sequence diagrams can be examined to gain information about the directionality of messages between classes, and the class diagram may be amended to represent this. However, interaction diagrams tell you more than the possible location of methods in classes. This section has shown examples of the kinds of design decisions that you might record in your

interaction diagrams; these can have a profound influence on the eventual software system. You have learnt the following.

- The actual implementation of links is best encapsulated within methods that hide the chosen representation. In the case of a unidirectional association, the class from which links will be navigated should be responsible for those links. In the case of a bidirectional association, making the class at one end of the association responsible for managing the links in both directions is a way of avoiding the danger of the two ends becoming inconsistent.
- During design you often have to decide how to send messages to an object that is not known directly. A fork centralises control in the sender, whereas a cascade delegates responsibility to another object.
- The Law of Demeter offers guidelines on the paths of communication between objects; these guidelines tend to favour cascades.

6

Summary

Dynamic modelling is used to decide on and document the message traffic that is intended to implement the system changes required by use cases, and results in assigning operations to the classes in the class model. In this unit you have seen how pre- and postconditions (assertions) can be used to specify operations, and that such assertions can be identified by applying a design approach based on the notion of contracts.

Sequence diagrams and communication diagrams (both types of interaction diagram) describe the dynamic nature of a design in terms of the messages passed between objects. They capture the same kind of information but have different advantages in what they explain visually.

During this stage of design, decisions must be made on where the different parts of the system functionality should be placed. At the detailed level we have to decide on detailed patterns of messages, and the Law of Demeter offers some suggestions, which can minimise dependencies between classes.

On completion of this unit you should be able to:

- explain the benefits of using a design approach based on the idea of contracts
- make informed choices between different possible designs
- apply some principles that will help you to decide the classes that should implement system functionality
- use sequence diagrams to show how use cases can be realised in a class model
- use communication diagrams to show the same information.

References

Larman, C. (2004) *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd edn, Upper Saddle River, NJ, Prentice Hall. Also available online at <http://libezproxy.open.ac.uk/login?url=http://proquestcombo.safaribooksonline.com/book/software-engineering-and-development/uml/0131489062> (Accessed 28 February 2014).

Pressman, R. S. and Ince, D. (2000) *Software Engineering: A Practitioner's Approach*, 5th edn (European adaptation), Maidenhead, McGraw-Hill.

Unit 7 More on dynamic modelling

– states and activities

Contents

1	Introduction	143
2	Capturing more complex interactions	144
2.1	Conditional message sends in sequence diagrams	144
2.2	Iterative message sends in sequence diagrams	147
2.3	UML fragments	147
2.4	Summary of section	151
3	Design issues	152
3.1	Communication between the user interface and business model	152
3.2	Checking preconditions	153
3.3	Strategies for implementing use cases	155
3.4	Summary of section	159
4	Introduction to state machines	161
4.1	State machine examples and diagrams	162
4.2	Guards	168
4.3	Exceptions	171
4.4	Entry and exit events and actions	172
4.5	When to use state machines	174
4.6	Final note on state diagrams	177
4.7	Summary of section	179
5	Overall design process	180
5.1	Packages	181
5.2	Summary of section	185
6	Summary	187

Introduction

In this unit you will extend your ability to model the dynamic behaviour of a software system. You will be introduced to a technique for modelling the life histories of objects. This unit will also review design considerations, discuss structure, and show how model elements can be grouped into packages.

The techniques introduced in this unit enable you to model complex behaviours within a range of systems. Where the objects you are dealing with are relatively well understood, you will not necessarily need to use these techniques and the associated diagrams explicitly – you might take a more agile approach.

However in a more complex system the complexity is often a source of failure. In such cases, proceeding quickly to code may be a false economy. Building diagrammatic models and reasoning about them in order to be clear about the details may be a worthwhile investment of effort. Such modelling is also useful where verification of the behaviour of a class of objects and its ability to satisfy a given set of properties is required prior to system implementation.

In general you should strive to keep your designs simple, but often complexity cannot be avoided. A quote often attributed to Einstein is ‘Everything should be made as simple as possible, but no simpler’, although this may be a simplification of something more complex he said, and in a different context.

2

Capturing more complex interactions

In Section 3 of Unit 6 you saw that it is possible to represent behaviour with sequence diagrams at different degrees of generality. Some designers prefer to have a number of diagrams, each denoting a concrete scenario. Examples might be *jill* checking in to *theRitz* when it has two free rooms, and *jill* checking in to *theRitz* when it has no free rooms. Each diagram is totally unconditional because, starting from a concrete state, the software system can behave in only one way when it receives a message. In effect, a diagram for a concrete scenario specifies a test case for that scenario.

Other designers prefer to put all these cases onto a single diagram that captures a range of possible behaviours. Such a diagram will have to include conditional behaviour because the message flow will depend on the initial state in some way. The diagrams are more complex but there are fewer of them. This is an example of moving from specific to more generic diagrams.

You saw in Block 1 Unit 3 how activity diagrams can represent a user's routine. They also help to confirm and/or identify use cases. In the next section we will discuss the benefits of turning your use cases into classes.

In practice you are likely to use both views, depending on the complexity of each use case and its accompanying scenarios. But you should try to keep your diagrams easy to read and understand. If a particular interaction requires a diagram that is hard to read, it may be a sign that the use case you are modelling should be split into two or more use cases.

2.1 Conditional message sends in sequence diagrams

Extra notation is needed for conditional behaviour on sequence diagrams. You need to show the condition that must be satisfied before a particular message can be sent. In Block 1 Unit 3 you saw how transitions between activities were guarded to constrain a workflow. The same notation can be applied to the messages in an interaction diagram.

In the hotel check-in example (see Figure 6 of Unit 6), we made a *Hotel* object responsible for finding a free room for the guest to occupy. Figure 1 shows a more generic sequence diagram that will put a guest into a room only if a free room has been found (we defer the issue of the value of *r* when no room is available until implementation). The message *accept(jill)* has been prefixed with the guard *[room available]*, displayed in square brackets. You can use the same notation to attach a guard to the messages in a communication diagram.

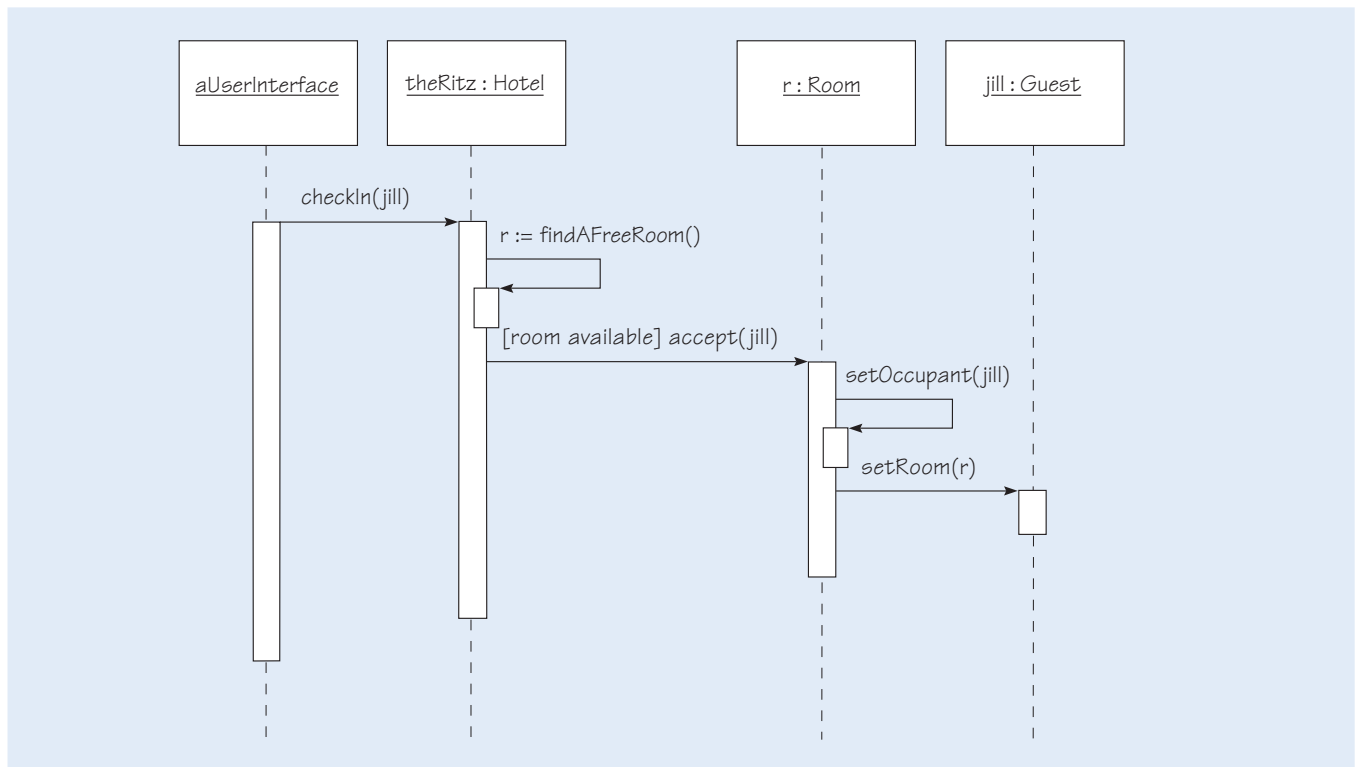


Figure 1 Sequence diagram showing a guard

Even more generality than that shown in Figure 1 is needed to enable you to represent the alternative courses of action required for different scenarios. For example, suppose that a particular room is unoccupied but still needs to be cleaned. Different messages would be sent depending on the state of the *Room* object, as discovered by the operation *findAFreeRoom()*.

This is illustrated in Figure 2, where the *Room* object *r* performs a different operation depending on whether or not it is ready for occupation. Notice that further activity will be required by the system and/or user to ensure the room is allocated once cleaned. Notice how UML allows you to add a branch to the lifeline of an object in a sequence diagram.

Recall that *r* is simply a placeholder for the reference to the room object found by the *findAFreeRoom()* message. It is not a programming variable. See Unit 6.

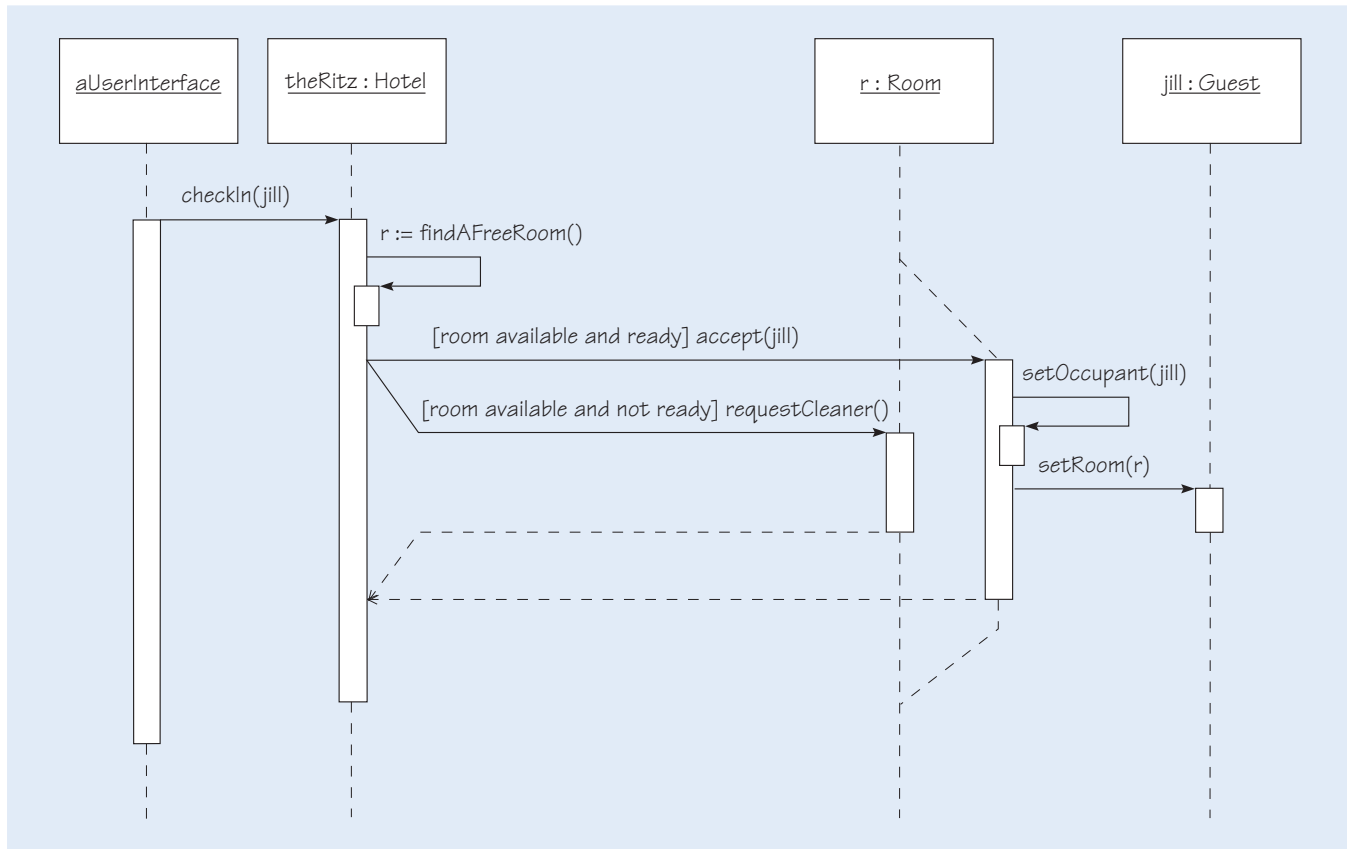


Figure 2 Alternative course of action in a sequence diagram

The return messages have been shown to emphasise the flow of control coming back to the same point in the activation of the sender.

If more than one conditional message originates from the same point on the lifeline of an object in a sequential system, the conditions must be mutually exclusive. In Figure 2 the room is either ready or not – it can never be both at the same time.

In order to avoid complexity, we recommend that you place each scenario into a single sequence diagram. If you attempt to show all the things that could happen in each of the scenarios in a single use case, the resultant sequence diagram can become crowded – for example, trying to show the maintenance of a room as well as its cleaning on the same generic sequence diagram.

Partitioning the scenarios in such a manner also allows you to explore different designs. You may, for example, find a more suitable solution by dealing with maintenance and cleaning checks within the operation *findAFreeRoom*. A room is available for occupation by a guest or it is not. UML does not specify the syntax of the conditional expression in a guard.

However, whatever syntax you use, you must represent a Boolean expression that results in either true or false. The expression must only involve attributes available either throughout the system (for example, time and date) or from the current operation – the attributes of the sender or attributes of any parameters of the invoked operation. You could use English, your chosen programming language or some other notation, such as OCL, as long as some consistency is maintained during the project, and possibly across projects.

2.2 Iterative message sends in sequence diagrams

Occasionally you may want to send a sequence of messages to the same receiver. If the messages only differ by some simple parameter, such as a counter, your diagram could become cluttered with repeated messages. UML uses the `*` notation for a repeated message send, which may be attached to a guard, generating an iteration clause such as `*[i := 1..10]` to send one message ten times. It is assumed that the messages are executed sequentially. Any nested messages in a sequence will be repeated in accordance with the iteration clause that you define.

Figure 3 shows the identical message `moveForward(10)` being sent twice. Each time an instance of the *Car* class receives that message, it sends the message `sound()` to an instance of the *Horn* class.

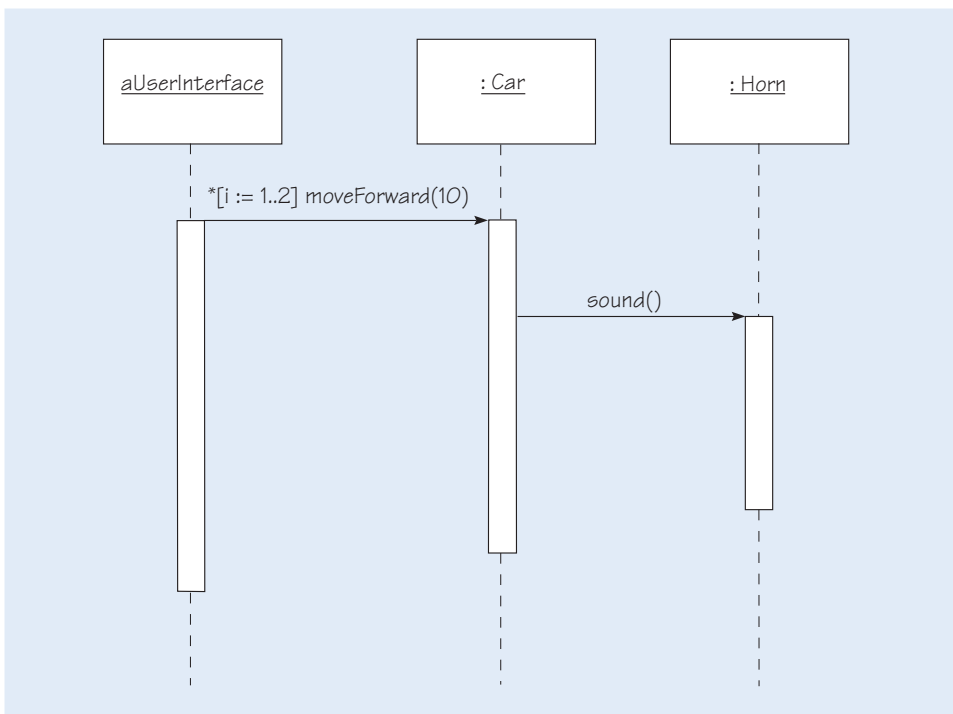


Figure 3 Sequence diagram showing iteration

A more common use of iteration is to make at least one parameter of the message vary between sends. A company might print a year's reports by sending to an instance of a *Department* class the message `printReportForMonth(i)`, using an iteration clause `*[i := 1..12]`.

2.3 UML fragments

UML has a more structured way of indicating conditional, alternative and iterative behaviour. This is based on the use of a more general notion of what is called a **fragment**, which is sometimes referred to as a **frame**. A fragment is a way of encapsulating behaviour inside a labelled box. The label can be,

for example, **opt**, **alt** or **loop**, standing for optional, alternative or looping behaviour.

While in this module we will use the approach shown above, Figures 4, 5 and 6 give examples of the optional, alternative and loop fragments.

- **Optional fragment.** The behaviour specified inside the fragment is only executed if a guard, shown at the top of the box, is true (Figure 4).
- **Alternative fragment.** This is used where we want to choose from a number of mutually exclusive pieces of behaviour. The box is divided, with alternatives in sub-boxes along with appropriate guards (Figure 5). One sub-box can be labelled with the guard *else*, to indicate it contains the default behaviour. (Hence the *opt* fragment is like a simplified alternative fragment, with one guard and no default behaviour.)
- **Loop fragment.** The behaviour is repeated while a guard, shown in the top left-hand corner, remains true or a variable moves between two bounds (as in a *for* loop, shown in Figure 6).

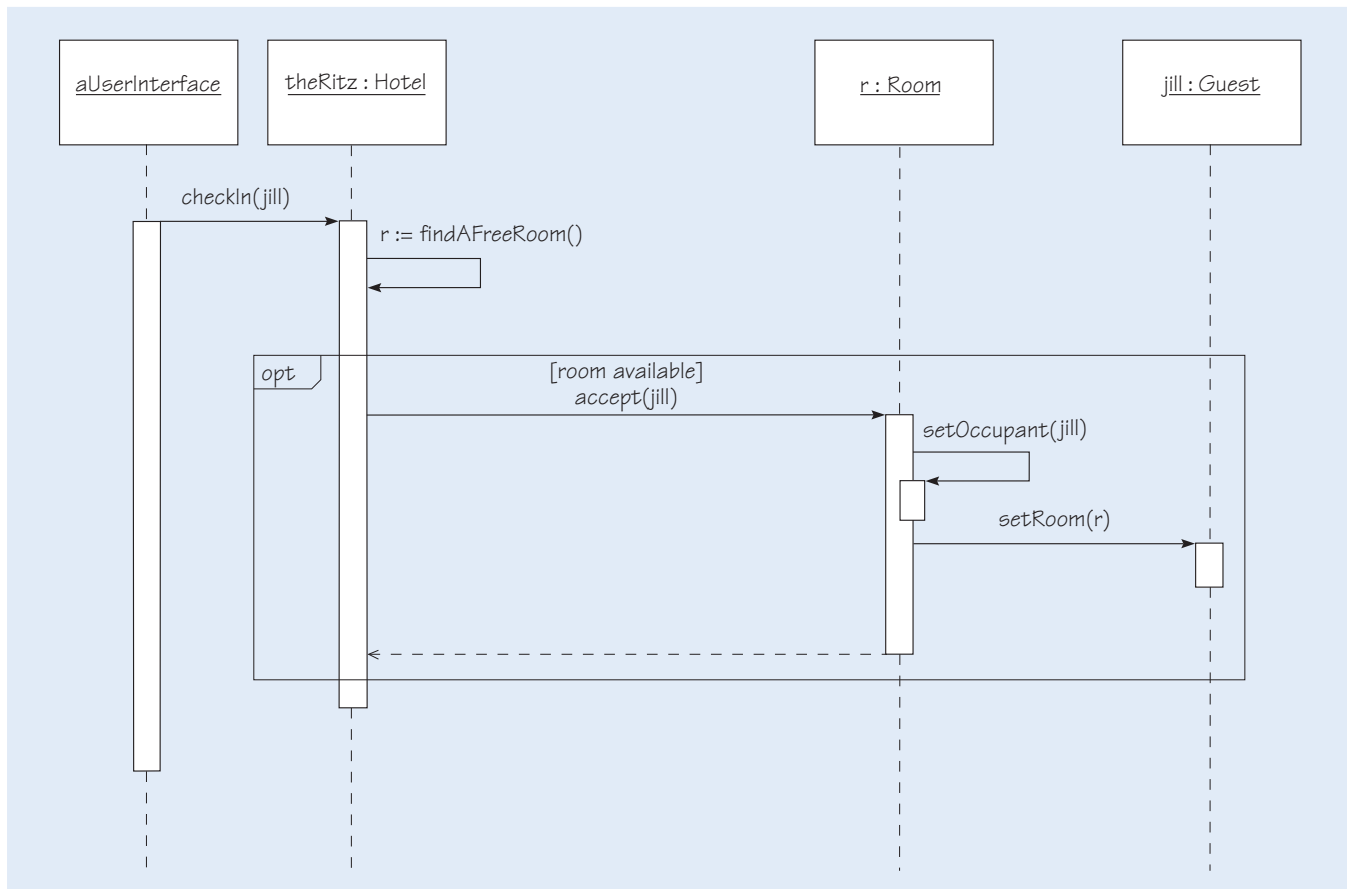


Figure 4 Sequence diagram showing an optional fragment

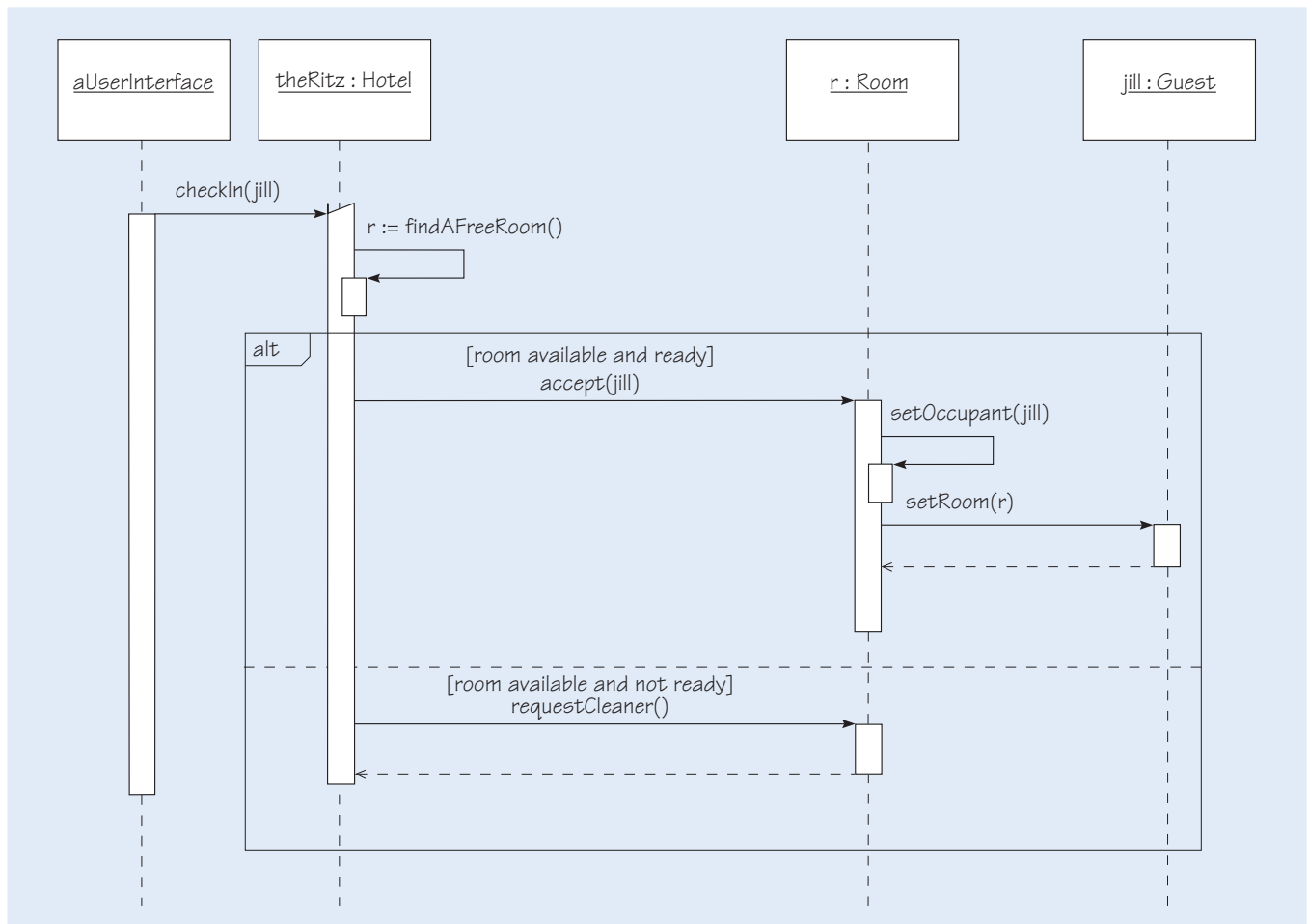


Figure 5 Sequence diagram showing an alternative fragment

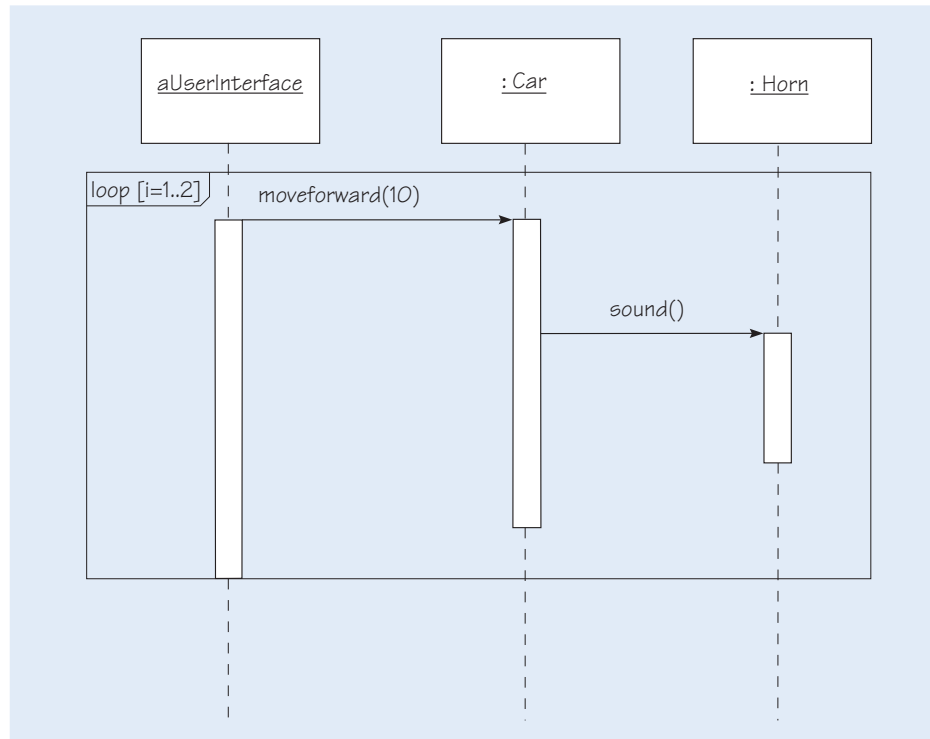


Figure 6 Sequence diagram showing a loop fragment

SAQ 1

- Why must the conditions on a message send be mutually exclusive in a sequential system?
- Figure 2 shows two conditional message sends originating at the same moment on the timeline, meaning that all the guards are evaluated before any message is sent. Could the same behaviour be implemented if the messages (and their conditions) were separated on the time line?
- Suggest a reason why an object's lifeline must branch in cases such as Figure 2. How is this handled in Figure 5?
- How would you change the sequence diagrams in Figures 3 and 6 to sound the car's horn five times each time it moved forward?

Answer

- If conditions were not mutually exclusive, more than one condition might be true. This would mean that multiple messages would be sent at once, resulting in multiple receiving objects being active at the same time – leading to the possibility of **non-deterministic** behaviour.
- Yes, provided that the conditions were mutually exclusive, so that the first message could not affect the state of the receiver before the second message arrived. We might use code such as the following:

Non-deterministic behaviour is behaviour where one of a number of outcomes is possible, for example when a dice is thrown. It is something we wish to avoid introducing where it is not intended.

```

if (room.ready) room.accept(jill);
if (!room.ready) room.requestCleaner();

```

- (c) The *Room* object will receive different messages depending on which branch is executed in the *Hotel* object. It makes no sense for the messages to arrive at just one lifeline, as that would imply that the *Room* object received all of the messages in a single interaction. Instead, we show a fork in the lifeline of the receiving object to reflect the fork of control in the sending object in Figure 2. In Figure 5 the alternatives are shown using the alternative fragment. Although the figure doesn't show a branching lifeline, this same branching is implied by the alternatives in the fragment, in their separate boxes.
- (d) To get the horn to sound five times for every forward movement in the sequence diagram in Figure 3 you would add an iteration clause to the second, nested message: `*[j := 1..5] sound()`. In Figure 6 you would add the extra iteration by including a fragment within the existing fragment.

2.4 Summary of section

This section has discussed how messages should be used to initiate the tasks involved in a use case:

- UML provides suitable notations to enable conditional messages to be expressed in sequence diagrams
- UML provides suitable notations for iterative message sending and for constraining the number of iterations
- in this module we will adopt the notation based on guarded messages.

3

Design issues

Having extended our modelling techniques, we now look at some of the key issues that we need to deal with during design.

3.1 Communication between the user interface and business model

The interaction diagrams that you encountered in Unit 6 were used to explore interactions between objects within a model. Each of them was initiated by a single message coming from a user interface. For the checking in message in the hotel system, you might be tempted to make the interface send a message such as one of these:

checkIn("Jill Smith"), passing a name as a string

checkIn("res23"), passing some sort of identifier.

The disadvantage of both these messages is that the peculiarities of the interface affect the design of the business model. They are both attempts at identifying objects by their value (using what is called an *external identifier*) but in an object model every object already has its own identity, so why not use that? An object model does not force a need to identify objects by using external values. Using external identifiers strongly couples the rest of the system to choices made in the user interface, an effect to be avoided.

In order to decouple the choices made in user dialogues from the rest of the system, the first messages of the main interactions should contain parameters that are references to objects – for example, *checkIn(jill)*, where *jill* is an object of class *Guest*, as opposed to *checkIn("Jill")*, where the identifier is a string.

Remember that we have been using concrete examples to construct interaction diagrams. Inside the class *Hotel*, for example, the above operation may be defined as:

checkIn(newResident : Guest)

Clearly, in order to pass an object identifier as a parameter, we need some way for the user interface to obtain the necessary object reference. A common way of doing this is for the interface to know all *Guest* objects and display them in a list from which the user can make a choice. Even though what is visible in the list are strings representing the guests' names, when a user selects a guest from the list the value obtained is the reference to a *Guest* object, not the name of an object.

Alternatively the user might enter the customer's name, swipe their card or scan the bar code on a voucher. Whatever method is employed, an important design issue is that this process of locating the relevant business objects should take place in the user interface and *not* in the business model. It is

then possible to change the way customers are identified without needing to make any alterations to the business model.

Because the business model only deals with the business objects, and the identification of the relevant objects is localised in the user interface, only the latter will be affected by an alteration to how identification is performed. The business model is insulated from such changes.

If the interface offers something like the pick list described above, how will it populate the list? It will need to ask the business model for a list of all *Guest* objects. There will therefore need to be additional operations in the business model, such as *getAllGuests()*, which are there solely to support the user interface. This shows that even with the best design decisions, there will be points where user-interface decisions are coupled to the business model. The goal is to minimise the degree of such coupling.

We also need to consider how the business objects will be created. Clearly there must be operations in the business model for this purpose. Object creation must be initiated from the user interface because information about new customers, rooms, reservations and so on originates there. In contrast to the messages that initiate use cases, object-creation messages *will* include external data such as customer names and reservation numbers.

Different system architectures may place different constraints on where such objects are created and destroyed.

So, for example, in *Hotel* we might have an operation such as the following, which creates a new *Guest*:

```
createGuest(lastname : String, firstname : String, address : String)
```

Once business objects have been created using operations similar to this, they can be assigned to variables or saved in a suitable collection. Subsequently the user interface will use the business objects directly and operations in the business model will use these objects rather than using *String* objects or other data to find the objects.

3.2 Checking preconditions

As discussed in subsection 3.5 in Block 1 Unit 3, use cases specify how the system changes by specifying a precondition and a postcondition. The precondition says what can be assumed about the initial state. The postcondition says what must be true afterwards. For example, a *BankAccount* might have a debit operation with a precondition that the amount to be debited is greater than zero, and that the account holds sufficient funds to perform the debit operation without exceeding the overdraft limit.

But whose responsibility is it to ensure that the precondition really is true? Should the operation assume that the precondition is true because the caller has invoked the method? Should the operation check that the precondition is true because it does not want to rely on possibly untrustworthy client code outside its control? And what should happen if the precondition is *not* true?

There are two possible strategies for checking the precondition.

- In **defensive programming** the checking of the precondition is done by the operation itself, that is, the receiver of the message (see Figure 7).

Each operation is made responsible for checking whether the precondition is met and it is safe to proceed – and for taking appropriate action if the precondition is *not* met. The operation will not go ahead if the precondition is false.

- In **design by contract** the responsibility is shifted to the caller of the operation, that is, the sender of the message (see Figure 8). The operation can therefore assume that the precondition is true and does not have to worry about either checking it or deciding what to do when the precondition is false. In Block 3 Unit 10 you will see how design by contract affects implementation.

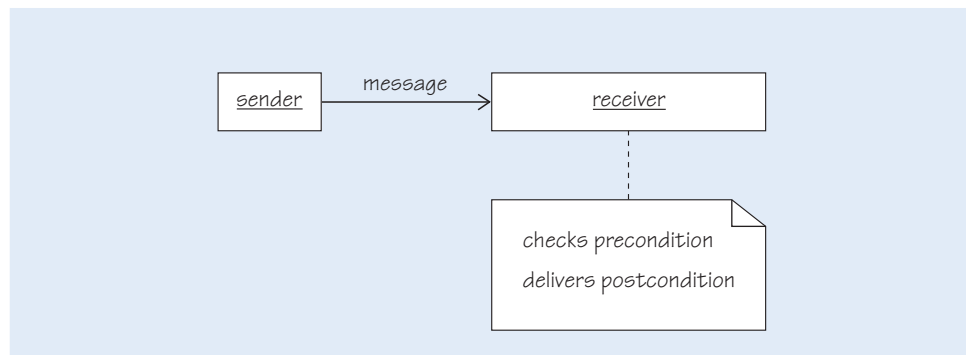


Figure 7 Defensive programming

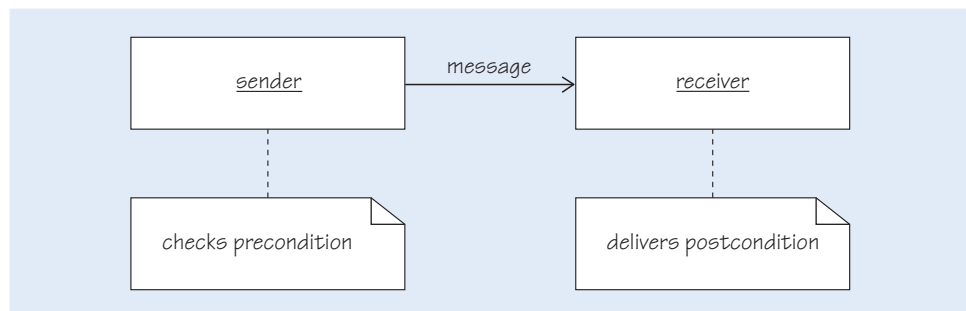


Figure 8 Design by contract

With defensive programming the checking of the precondition is part of the same operation that will also deliver the postcondition, so the relationship between the two may be easier to understand. The main disadvantage of defensive programming is that conditions are repeatedly checked in different places. For example, suppose the debit operation generates a cheque-printing request for the sum involved. The receiver of the request is likely to check all over again that the sum of money is greater than zero. Defensive programming can also create difficulties when state changes are made. For example, if in the bank example the debit operation checked the request, approved it and then debited the account, the cheque-printing operation should not verify the balance again.

What should happen in design by contract if the sender of a message is faulty and does not check the preconditions as it ought? In that case, the receiver

will no longer guarantee the postcondition because the sender has not honoured its side of the contract. It is difficult to define the failure behaviour, because the condition is not being checked.

Be aware that the strategies can be combined. The receiver can check certain preconditions that it thinks are critical and leave others to be checked by the caller. Alternatively it could check all preconditions as a strategy to help with testing failure behaviour.

3.3 Strategies for implementing use cases

You have seen that when you construct interaction diagrams, the first two design decisions that have to be made are the following.

- Which message should be sent from the interface?
- To which object should the interface send the message?

In the check-in scenario we had to consider whether to send a message to the *Hotel* or to the object *jill : Guest*. Hence we were faced with deciding which classes should have the operations that correspond to the identified use cases. Below we discuss three strategies that can be adopted for making this decision.

One central class

Some designers like to minimise the knowledge the interface must have of the business model by making the interface send all messages to a single object – usually some general object like a *System* or *Hotel*. In the case of the hotel chain model each message could initially be sent to the *Hotel*, which would then forward the message to whatever object is to do the work.

From the interface's point of view, it has to know the existence of only one object. Each use case will become a method of the *Hotel* object. There is good traceability from use cases to code because every use case links to a method in the same *Hotel* class.

The main disadvantage arises when the one central class becomes overloaded with use cases. One possible solution to this problem is to divide the software system into several packages. Each package could still make use of one class to respond to messages from the interface. The focus of each package would be on the key concepts within each domain, which you will see in the final section of this unit. In the case of the hotel chain, for example, one package could deal with reservations, another with checking in and out, another with payment, and so on. This is a form of architectural design, a topic we return to in Block 3 Unit 9.

Actor classes

Other designers are of the opinion that a clearer structure will result if the message is sent to a software object corresponding to the real-world actor who initiated the operation. For example, when a person arrives at a hotel they initiate the checking in process. They talk to a receptionist, who uses a

keyboard to stimulate a user interface to send a message to an object representing interaction with the real-world actor, as opposed to the *Guest* object that represents information about the guest. Traceability is rather different from that for one central class. To locate the code for a use case, the class corresponding to the initiating actor must be searched. There is a complication when there are two actors that can initiate an interaction, as in the case of making a reservation illustrated in Figure 9 in Block 1 Unit 3.

Use cases as classes

In a third strategy, interface messages are sent neither to a system object like a *Hotel* nor to the object corresponding to an actor. Instead, a new class is defined for each use case. There might be classes such as *CheckerIn*, *CheckerOut* and *ReservationMaker*. Each would have a method with a name like *run*, with a suitable number of arguments. The user interface would then create a single instance of the appropriate class, initialise it suitably and then send the *run(...)* message.

For example we might send the message *run(jill)* to a *CheckerIn* object, which assumes that we have previously identified the right guest with the aid of another class called, perhaps, *GuestIdentifier*.

Not surprisingly, this strategy employs what are known as **use case objects**. Following this strategy is one way to improve reuse. For example the core concepts or business objects in the hotel domain are the rooms in each hotel and the guests that occupy them. These are likely to remain the core concepts as you move from one hotel chain to another.

What are likely to change are the rules and workflows that apply in different hotel chains at different times. One hotel chain may force reservations before allowing anyone to check in, while others may have different rules. The same hotel chain may even change its way of working – probably the reason why they want a new software system, in fact. As a result, the user interface is the part of a software system most likely to change.

Working with use case objects lets you change or even replace the software to implement a given scenario, minimising the effects upon the core concepts. For example, if you changed the means of identifying each guest, the change would affect the implementation of the *GuestIdentifier* class but not that of any of the business objects. The *CheckerIn* class would be affected only if the interface in the *GuestIdentifier* class had been changed.

Advocates of such use case objects appreciate the traceability they provide from each use case to a class. Each use case class contains information relevant to the use case alone, so each use case can be understood in isolation. If you want to collect management information about the use case, such as frequency of use or timing histories, having an object provides a natural place to put such functionality.

This is an example of a software pattern known as the **command pattern**. You will look in general at the idea of software patterns in Block 3 Unit 9. A possible disadvantage of the approach is the large number of extra classes that must be defined – one for each use case. Another possible disadvantage is that

many of the use case classes can be similar, resulting in duplicated code and more difficult maintenance.

SAQ 2

- (a) What is the main disadvantage of defensive programming?
- (b) Which of the three strategies for implementing use cases is being used when we place methods such as *createGuest(lastname : String, firstname : String, address : String)* in the `Hotel` class?
- (c) Are use case objects consistent with the twin aims of high cohesion and low coupling (terms introduced in Block 1 Unit 1)?

Answer

- (a) The main disadvantage of defensive programming is that the same constraints within each precondition may be checked repeatedly in different operations.
- (b) We are following the strategy of using one central class here.
- (c) Yes and no. Yes, because high cohesion asks us to ‘do one thing and do it well’. A `CheckerIn` class should only support the one activity of checking in. No, because having an extra class for each use case introduces more coupling through the extra associations involved. However the introduction of these new classes limits the impact of a change to a software system.

Exercise 1

Suppose you are designing the graphical user interface (GUI) part of a hotel system. You need to be able to check in people known to the system as well as new people. What operations on the business model might be needed to support a usable GUI?

Solution

Checking in unknown people would involve the creation of a new *Guest* object, requiring an operation *createGuest(firstName, lastName, address)*.

To check in a known person, you need to find the business object that corresponds to that person. This mapping might be done by having a pick list of all people known to the system, which would require a business-model operation that returns a collection of guests, for example *getAllGuests()*.

Exercise 2

Recall the library system in Exercise 4 in Block 1 Unit 3. Suppose you wanted to minimise the dependency of the user interface on the rules and concepts of the library domain. Which messages should be sent from the user interface and to which object should the user interface send its messages?

Solution

A solution can be found from the first of the three strategies discussed above. The messages sent from the user interface correspond to each of the library’s

use cases. Look at Figure 11 of Block 1 Unit 3, from which you can identify six messages originating from the user interface:

- *reserveBook(...)*
- *borrowCopyOfBook(...)*
- *returnCopyOfBook(...)*
- *updateCatalogue(...)*
- *enrolNewMember(...)*
- *browseCatalogue(...)*.

Each one would be sent to an instance of one central class, typically a general object like a *System* or a *Library*. Although borrowing books and returning books might share common operations, all six messages might be needed to maintain the traceability between the requirements and your implementation.

Exercise 3

Assume the business model shown in Figure 9. Redraw the sequence diagram of Figure 12 in Unit 6 so that use case objects are used, according to the classes and operations shown in Table 1. You may assume that an instance of the use case class *CheckerIn* has already been created before the message *run(s)* is sent.

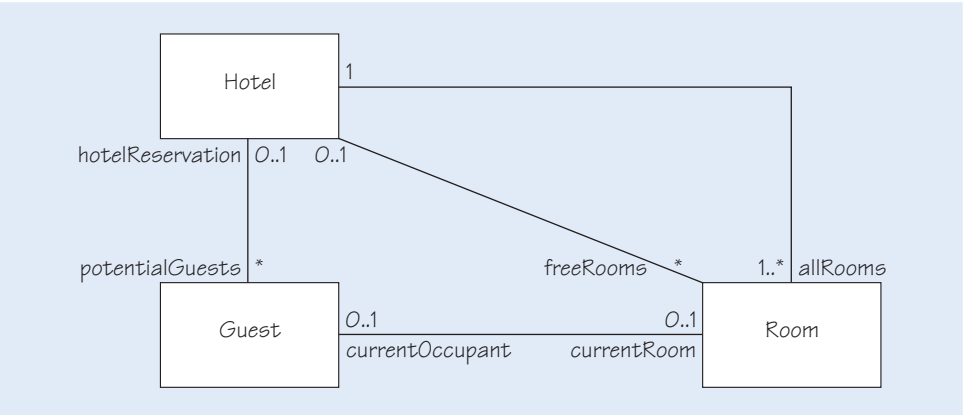


Figure 9 Hotel business model (without the *UserInterface* and *CheckerIn* classes)

Table 1 Operations for the check-in use case, using a *CheckerIn* class

Class	Operations
<i>Hotel</i>	<i>identifyGuest(s : String)</i> <i>findAFreeRoom()</i>
<i>Room</i>	<i>accept(g : Guest)</i> <i>setOccupant(g : Guest)</i>
<i>Guest</i>	<i>setRoom(r : Room)</i>
<i>CheckerIn</i>	<i>run(s : String)</i>

Solution

Our solution is shown in Figure 10.

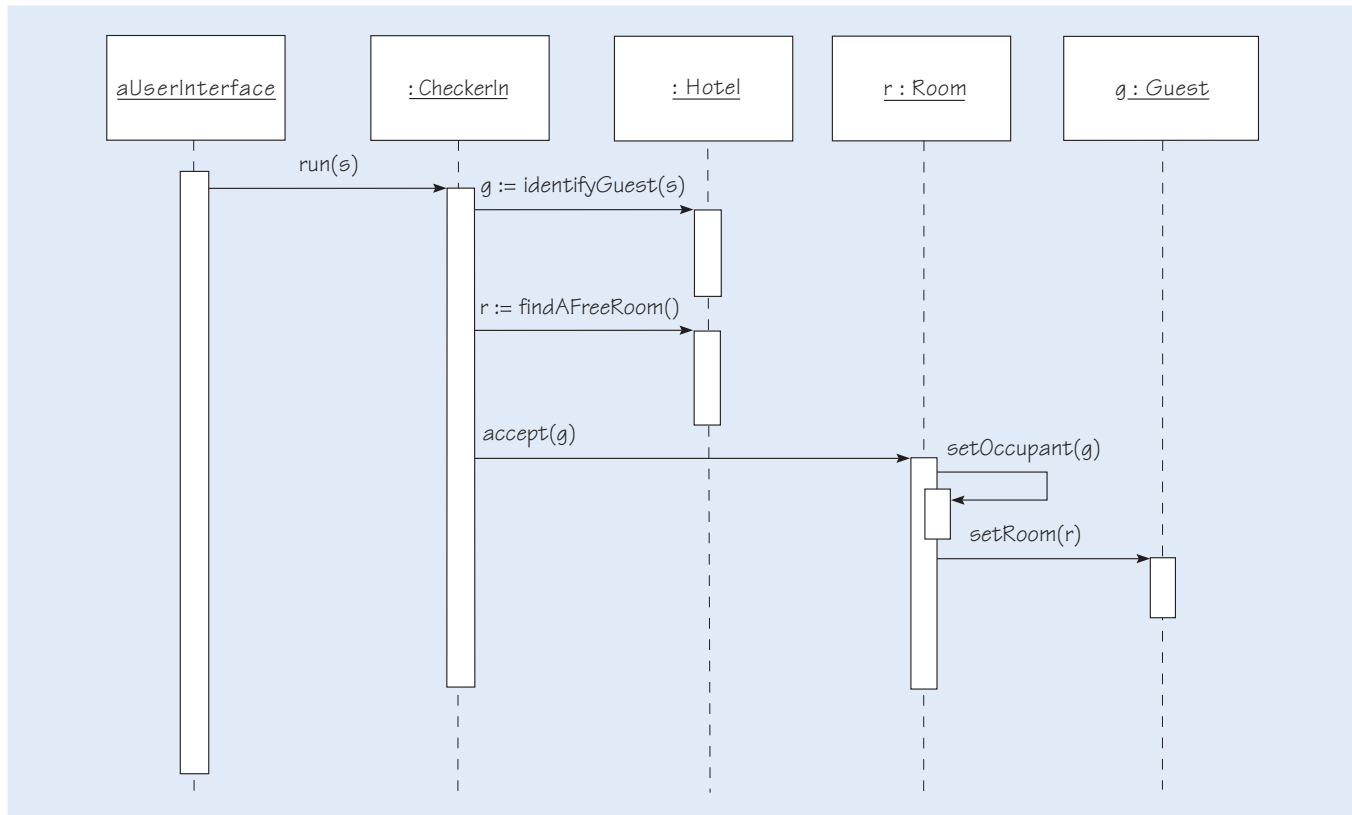


Figure 10 Using a use case object for the check-in use case

A reference to the current room must be kept in the instance of class *Guest*. There are two options for sending the message *setRoom(r)* to *g* – either from *r : Room* or from *: CheckerIn*. The first option has been used in Figure 10, to minimise the coupling that would otherwise have been introduced by using a *CheckerIn* object and to follow our earlier advice that one of the classes should be responsible for maintaining the consistency of the relationship (see also subsection 5.2 of Unit 6).

3.4 Summary of section

There are a variety of design philosophies, each affecting the design in different ways.

- Other than when invoking object-creation operations, user interfaces should communicate with the business model entirely in terms of business objects and not via external names or identifiers. The need for the interface to do the mapping from external identifiers to objects may require the business model to support extra queries that are not actually needed by any use case, such as *getAllGuests()*.

- There are two different strategies relating to the locations of the operations that enforce preconditions. Design by contract is generally preferred to defensive programming, in that checking conditions once reduces redundant code and the approach makes us think about which piece of code is responsible for what. The quality of code will, of course, depend on careful design and implementation, and there may be a case for defensive checks in some circumstances.
- There are three strategies for choosing which objects in a software system are to receive messages from the user interface: one central class, actor classes or use case classes.

Introduction to state machines

In this section we will describe a technique for modelling the life history of an object of a class as part of the dynamic behaviour of a system. That is, you will be shown how to model the changes within an object as a result of receiving a message. This technique can also be used to model the behaviour of real objects in a domain – in this case, changes occur as a result of events in that domain.

In the world that you are familiar with, things happen and they may do so at the most unexpected and possibly inconvenient times. These happenings are known as **events** and they can be modelled as part of the behaviour of a system. An event is an occurrence of a phenomenon at a certain moment in time.

The occurrence of the event itself is assumed to have no duration. For example, the keyboard strokes that were generated when writing this text are a sequence of events performed by the user of a word-processing software system. At some point, this text was printed as a result of another sequence of events.

Events in the real world, or the sending of messages between objects in a system, are the kinds of events we are interested in. In this section you will see a technique for modelling the behaviour of an object (in a domain or in a system) over time.

A **state machine** is a model that shows how an object changes from state to state in response to events. The state machine notation is based on David Harel's **statecharts**, a well-known formalism.

It is important to understand the difference between *states* and *events*. A **state** represents some condition or situation in the life of an object. For example, a car's motor might be *running* or a window might be *shut* or *open* – these are states.

A state is not an action, but is instead a description of something about the object that will remain true until something happens. A car's motor might *start* (an event) to put it into the *running* state. A person might *fall* – this is a description of an action, which is not a state. However we could say that the person is *falling*, a state that will continue until some other event happens (such as landing on the pavement).

4.1 State machine examples and diagrams

A familiar real-life example of a state machine is a retractable ballpoint pen, which has a button that you press to extend the ballpoint when it is retracted, or to retract it when it is extended. The point has two states, *extended* and *retracted*, as shown in Figure 11. The figure shows the features common to all state machine diagrams:

- *states* (in this case, two – *extended* and *retracted*), represented by the round-cornered rectangles containing their names
- *events* (in this case, only one – *button clicked*), given as labels on transition arrows
- *transitions* (in this case, one from *retracted* to *extended* and one from *extended* to *retracted*), represented by arrows and identified by the events that give rise to them. Transitions occur as a response to an event, and cause a change of state.

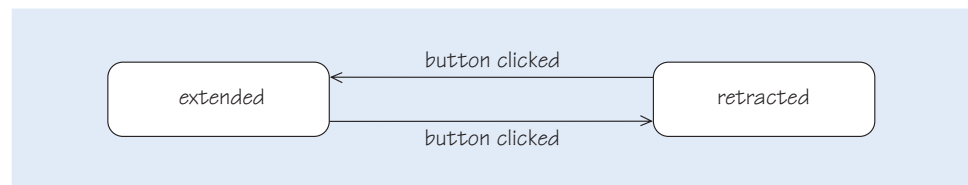


Figure 11 Two simple states for a ballpoint pen

Notice also that the response of the object to the event (what state the object transitions to) depends on what state the object is in – it is state-dependent. Figure 12, a description of starting and turning off a car motor, further illustrates the ideas of states, events and transitions. The ignition-key system in most cars has (at least) three positions: ‘off’, ‘run’ and ‘start’. Furthermore most cars will not permit you to turn the key from the ‘run’ position to the ‘start’ position more than once. You must turn the key to the ‘off’ position before you can turn the key to the ‘start’ position again.

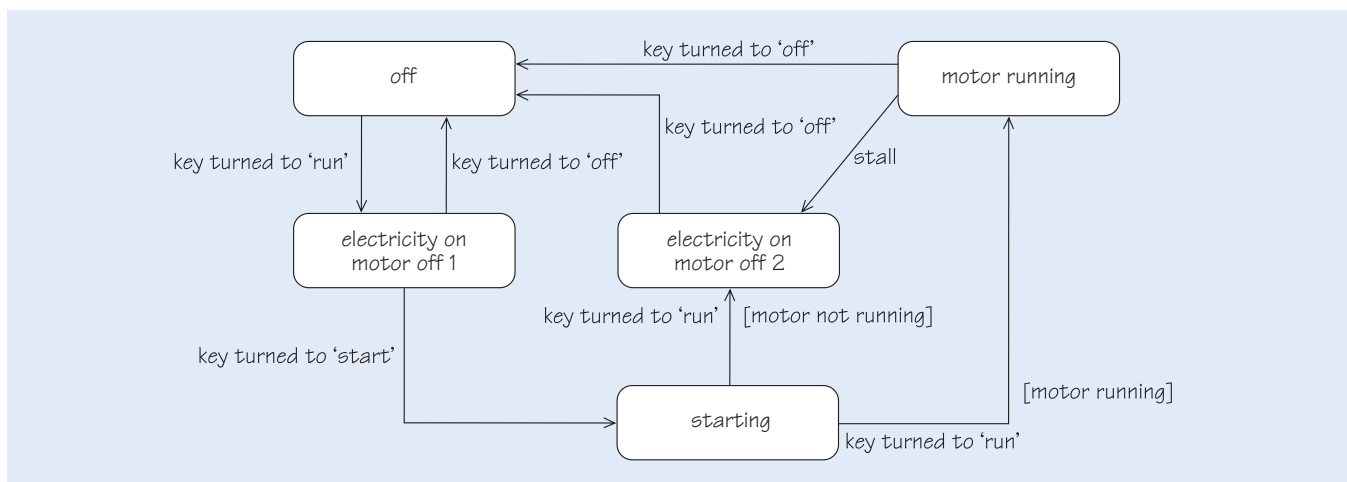


Figure 12 State machine diagram for starting and turning off a car motor

Figure 12 shows the states that the car's motor system can be in. The car starts in the *off* state, with nothing running and nothing powered. If the event *key turned to 'run'* occurs, the car enters the state *electricity on motor off 1*: the system has power, but the motor is not running. At this point, the user can either (a) turn the key to the 'off' position and transition back to the *off* state or (b) turn the key to the 'start' position to attempt to start the motor, transitioning into the *starting* state.

After a time the user will turn the key back to the 'run' position and either the motor will be running or it will not. If the motor is not running, the car transitions into the *electricity on motor off 2* state, where the user can leave the key at 'run' (and run down the battery) or turn the key to 'off' to go to the *off* state to try again. If the motor is running, the car transitions to the *motor running* state. The motor will stay running until either the key is turned to the 'off' position or the motor stalls.

There are two *electricity on motor off* states because we want to model when it is possible to turn the key to start and when it is not. In the first state, it is possible to turn the key to start. In the second state, it is not.

The text in between the square brackets [...] is a guard. A guard is used to choose which transition to take if the same event appears on more than one transition, as does the event *key turned to 'run'* when transitioning out of the *starting* state. Guards are discussed further below.

The concept of a state machine is well established in computing. A state machine is a mathematical model of a system that represents the legal internal configurations of the system. Each valid configuration reflects past inputs and determines what the system does on subsequent events. The current internal configuration of a system is its state.

Change takes place through transitions from one state to another. Transitions are *fired* or *triggered* by events. Which particular transition is fired depends on the current state.

In UML the term 'state machine' refers to the technique used to model behaviour in terms of states and transitions. A UML state machine diagram is a graphical representation of a state machine showing states, transitions and events. Referring back to Figure 11, the states are *extended* and *retracted*. Each state has one transition leaving it, fired by the event *button clicked*. In Figure 12, the states are *off*, *motor running*, *electricity on motor off 1*, *electricity on motor off 2* and *starting*.

Unlike a sequence or communication diagram, a state machine diagram (sometimes just called a state diagram) does not represent interactions between objects. Instead, a state machine models the life history of a single object:

- its initial state when first created
- the different states it can be in while it exists
- how it can move from state to state in response to events – the state transitions that are possible.

If the object is destroyed at some point, the state diagram will also include an exit state.

Initial state

One question that must be answered is what state an object enters when it is first created. This only applies when modelling objects from their creation onwards – sometimes we do not care about the object’s creation but only about its operation. In UML a solid black circle is used to show what is known as the **initial state**. However this initial state is not a true state, but is instead a way of indicating which of the real states the object will enter first.

Figure 13 shows how a state machine diagram can represent the behaviour of an object upon receiving a message and how that behaviour depends on the values of that object’s attributes and links. Recall the hotel example used in Unit 5, where we considered the problem of room occupancy. A room can either be occupied or unoccupied. Figure 13 shows how a state machine for instances of the class *Room* can be represented in a state machine diagram. A room becomes occupied when a particular guest checks in to the hotel. It becomes unoccupied when that guest checks out.

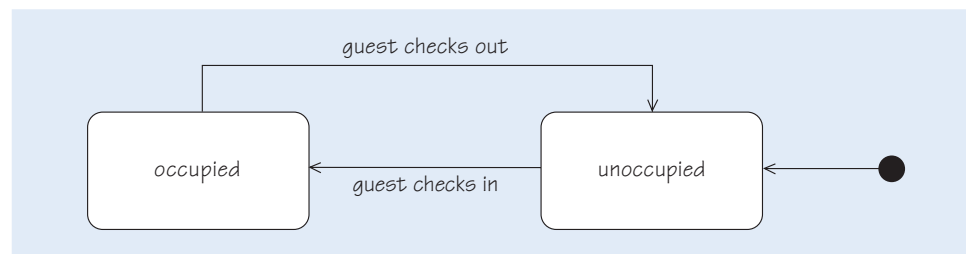


Figure 13 Two simple states for a *Room* object

Figure 13 shows the initial state when a *Room* object is created to be *unoccupied*. So when each *Room* object is created, the values of its attributes and links must be set so that it is in the *unoccupied* state.

The events that fire the transitions have been given simple names – *guest checks out* and *guest checks in* – to indicate which events give rise to the transitions. However we can do better than this by using information gleaned when constructing interaction diagrams. For example, if you look at Figure 14 you will see that there is an exchange of messages between *Hotel*, *Room* and *Guest* that results in a particular guest occupying a particular room. In other words, this exchange of messages results in a change of state of the room.

Final state

We have shown that a state machine starts with a single initial state, but what do we use to show that processing has completed or that an object’s life cycle has ended? UML uses a **final state**, signified by a black circle surrounding a solid black circle, to show that processing has been terminated. An initial state has only one outgoing transition, whereas a final state can have many incoming transitions but no outgoing transitions. As a consequence a state machine diagram shows no more than one initial state, but there can be zero, one or more final states as an object may terminate its activities in different ways.

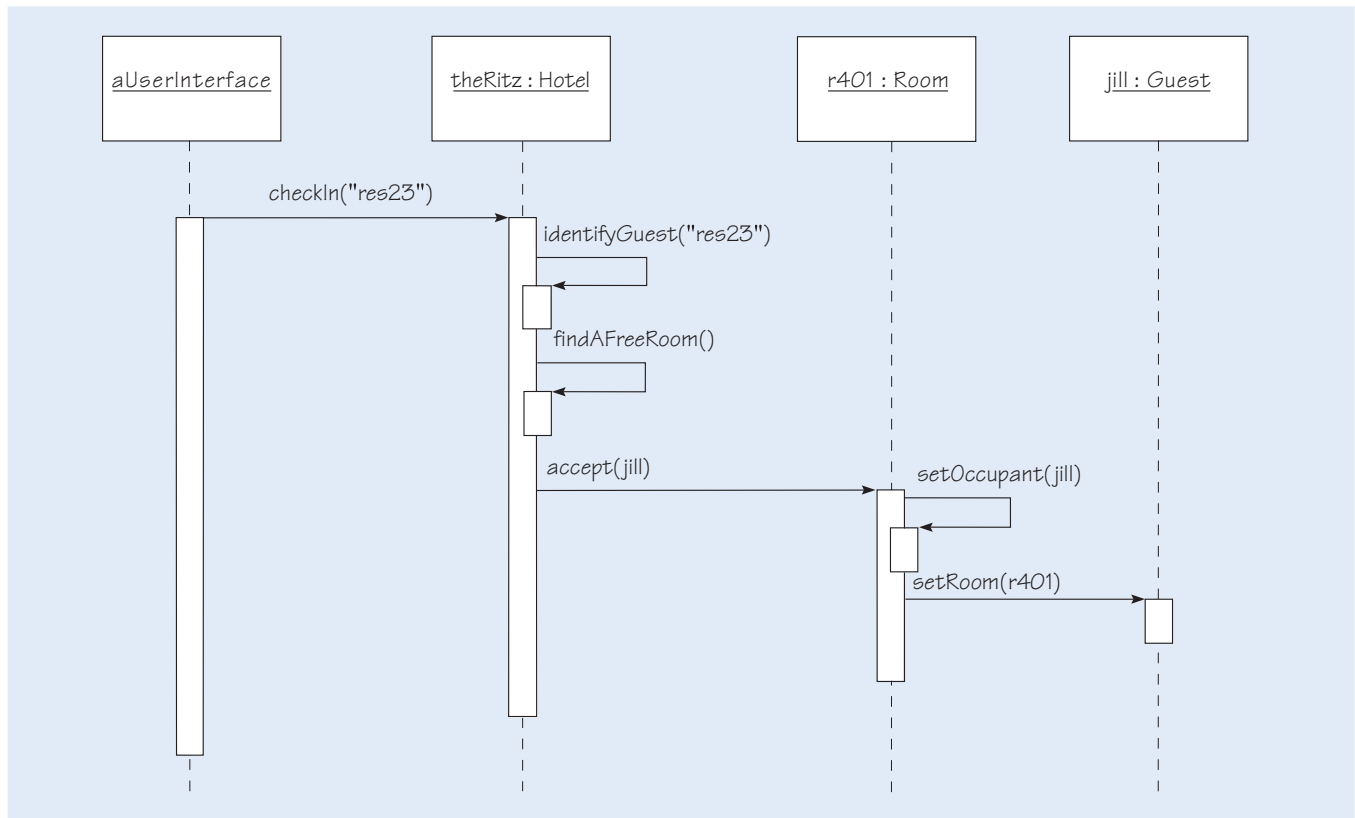
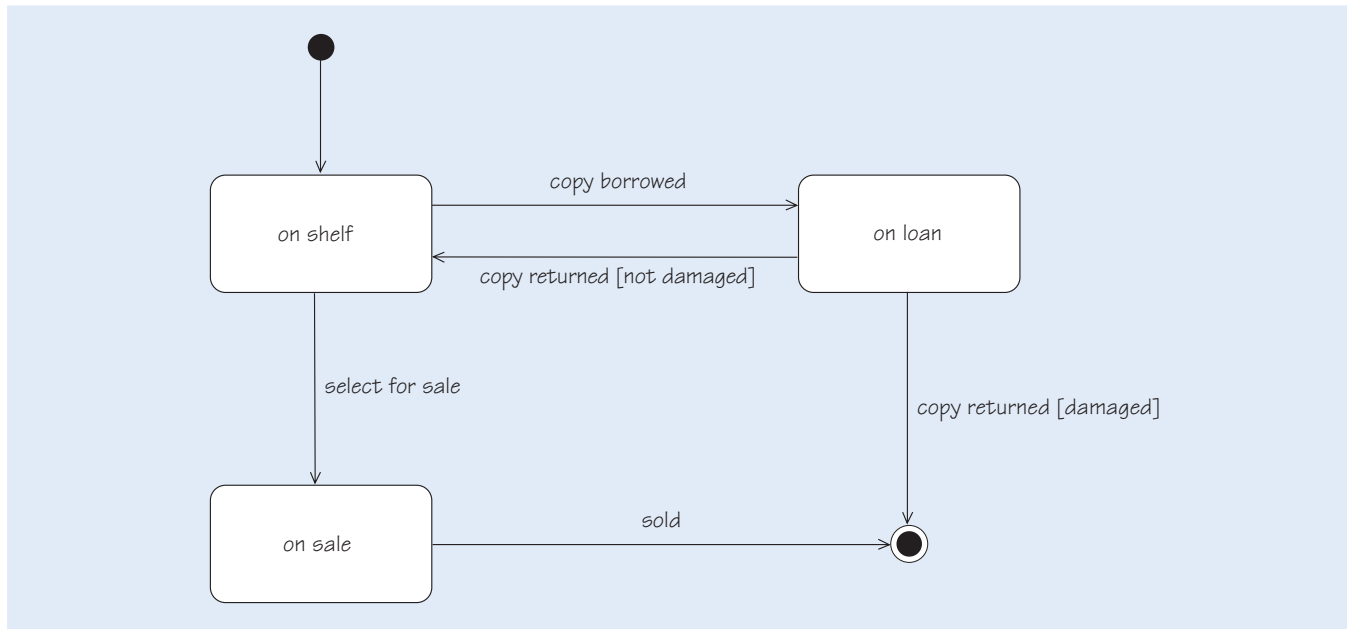


Figure 14 Sequence diagram for checking a guest in to a room

In a simple state machine, initial and final states signify the creation and the completion of the activity of an object. Suppose, for example, you were asked to model the behaviour of the books in a lending library. Newly purchased copies of a book are put on the shelf for lending to the library's members. Damaged copies are removed from stock. At certain times of the year, each library is allowed to select copies of books to go on sale to the public. Figure 15 shows a simple state machine that follows the life history of each instance of a *Copy* object. There are two possible transitions that lead to the final state for each *Copy* object. Once a physical copy of a book has been removed from the library's shelves, the corresponding *Copy* object can be destroyed.

Figure 15 Life history of a *Copy* object**SAQ 3**

- Figure 15 shows a simple state machine diagram for the class *Copy* in a library model. What should happen if the *copy returned* message arrives when an object of the class *Copy* is in the state *on shelf* (assuming that we are not using design by contract)?
- Give an example of an architectural decision that would provide a general solution to the problem of unexpected messages.

Answer

- Something has gone wrong. The state of the system does not describe the fact that the copy of the book is on loan, so it cannot be returned. There must be some means of indicating or recording an error, such as a message to a log file.
- At an architectural level we might introduce a single object of a class *Error*, which is globally accessible to objects in the software system. Such an object would be responsible for reporting errors due to unexpected messages, for example.

SAQ 4

- What does a final state signify in a state machine?
- In what ways does a final state differ from an initial state?

Answer

- Final states are used to show the point or points where the object in question has finished processing. Its activity has been completed.

- (b) There can be zero, one or more final states but at most one initial state. A final state can have several incoming transitions and no outgoing transitions, but an initial state has no incoming transitions and only one outgoing transition.

Events, actions and transitions

The receipt of a message is always an event. It might result in a change of state of the receiver object, or it might not – it could simply return some information but not change anything. An event, when it occurs, has consequences. One kind of consequence is a change of state.

Other consequences are recorded in the action part of the label on the transition. That is, instead of labelling a transition just with a simple name, we also record the **actions** (consequences) that result from the occurrence of the event.

Figure 16 shows a revised version of the state machine diagram of Figure 13, which you might have created after constructing the interaction diagram in Figure 14. In the checking in process, a *Room* object is sent the message *accept(aGuest)*. This is an event that causes the *Room* object to respond by sending itself the message *setOccupant(aGuest)*, an action. The result is that the *Room* object changes state from *unoccupied* to *occupied*. Actions are atomic, in that any intermediate changes they make are not observable in terms of the state machine.

aGuest indicates that the argument will be an object from the class *Guest*.

Figure 16 illustrates the notation for labelling transitions in UML: *event / action*. So you can see that transitions from state to state are not the only expressions of behaviour in a state machine.

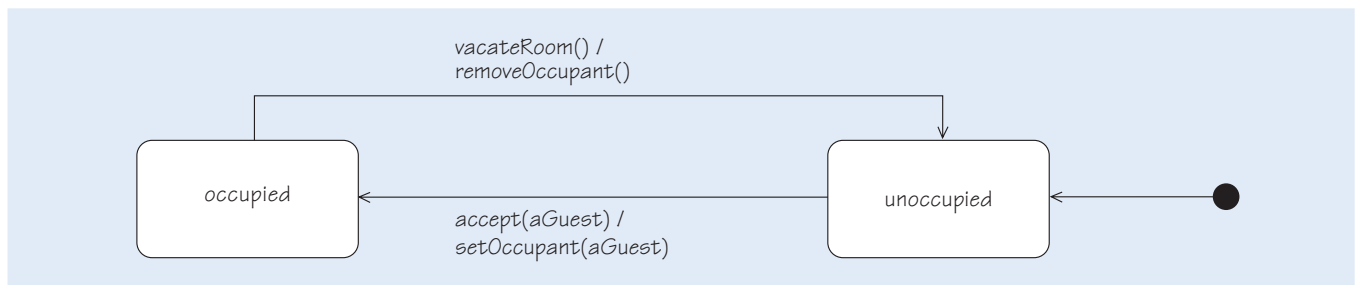


Figure 16 Adding detail to the state machine diagram for a *Room* object

A single event may often lead to a sequence of actions. In the check-in example there are two actions taken by each *Room* object, so the transition from *unoccupied* to *occupied* in Figure 16 should be labelled as follows:

accept(aGuest) / setOccupant(aGuest); aGuest.setRoom(self)

The second action – *aGuest.setRoom(self)* – is a message sent to another object. In this case it is the one identified by the reference passed to the *Room* object from the message *accept(aGuest)*. The association between *Guest* and *Room*, which you saw in Figure 8 of Unit 5, is navigable in both directions.

We need a return from the *aGuest.setRoom(self)* message to confirm that a guest is linked to a room.

If there were more messages to be sent, they could be added to the action sequence following the event. Each action is separated from the next in the list by a semicolon. The order in which the individual actions take place is defined by the order in which they appear in the list.

SAQ 5

- (a) Why might two objects of the same class respond differently to the same message?
- (b) What is the most common form of event that causes a transition between two states? How is it shown in a state machine diagram?
- (c) What is the difference between an event and an action?
- (d) What is the main constraint on the kinds of action that may be shown in a state diagram?
- (e) What is an action sequence?

Answer

- (a) An object's behaviour will in general be affected by the values of its attributes, which are part of the object's state. If two objects have different values for the same attributes, they are in different states, and therefore might respond differently to the same message.
- (b) The receipt of a message is the most common form of event that causes a transition between states. An event is used to label the transition between the states. In a state machine diagram, a transition is labelled with the name of the relevant message, which includes any arguments for that message.
- (c) An event is something done to the object, such as sending it a message. An action is something that the object does, such as sending a message to itself or to another object. An action is an object's reaction to an event.
- (d) Actions should only refer to things that the object 'knows' about. For example, they can refer to attributes, operations and links of the object, as well as to the parameters of the message that caused the transition. An action cannot refer to the state or attributes of another object unless there is some way for these other attributes to be known in a short time without requiring any state changes.
- (e) An action sequence is an ordered series of individual actions that are associated with a particular event. They are written as a list separated by semicolons, and are performed sequentially in left-to-right order. Like actions, action sequences are atomic.

4.2 Guards

Sometimes an event might take place but you only want a transition to fire under certain conditions. As you have seen in Block 1 Unit 3 and earlier in

this unit, UML provides the concept of a *guard* to deal with conditional behaviour.

In a state machine a guard is evaluated when an event occurs. The transition and any associated actions will only take place if the guard evaluates to true. For example, if we were following a defensive programming strategy, the object itself would want to ensure that guests could check in to rooms only if they had existing reservations, rather than depending on the sender to have made the checks. In this case, we could rewrite the *accept(aGuest)* transition in Figure 16 using the square bracket notation for denoting conditions as follows:

```
accept(aGuest) [reservation exists] / setOccupant(aGuest); aGuest.setRoom(self)
```

If no reservation were to be found for the guest, the actions would not take place and the *Room* object would not move from the *unoccupied* to the *occupied* state. Be aware that a guard condition is a form of query about the system and its state – it must not change any values, and it must not have any side effects.

Sometimes the same event can give rise to different transitions and actions depending on the prevailing conditions. Such situations are common where loops are formed – for example keeping track of how many rooms are available in a hotel, or the number of copies of a video on loan. We can illustrate this by considering what happens when a hotel becomes full. If we identify two simple states *full* and *not full* for the class *Hotel*, what happens as guests check in and out?

Figure 17 illustrates this situation. The hotel has a limit to the number of rooms that it can let to guests. At some point, the hotel may find that it cannot accept any more guests. Figure 17 shows that guests can continue to check in until that limit is reached. The hotel is not full provided that there is at least one room that can be occupied. Checking out is not a problem in this simple example. Both instances of the event *checkOut(aGuest)* lead to the *not full* state. However an occurrence of the event *checkIn(aGuest)* will lead to *either* the *not full* state *or* the *full* state, depending on whether or not the *Hotel* has more than one room unoccupied. Therefore the guard conditions in Figure 17 are mutually exclusive: [*last room*] or [*not last room*].

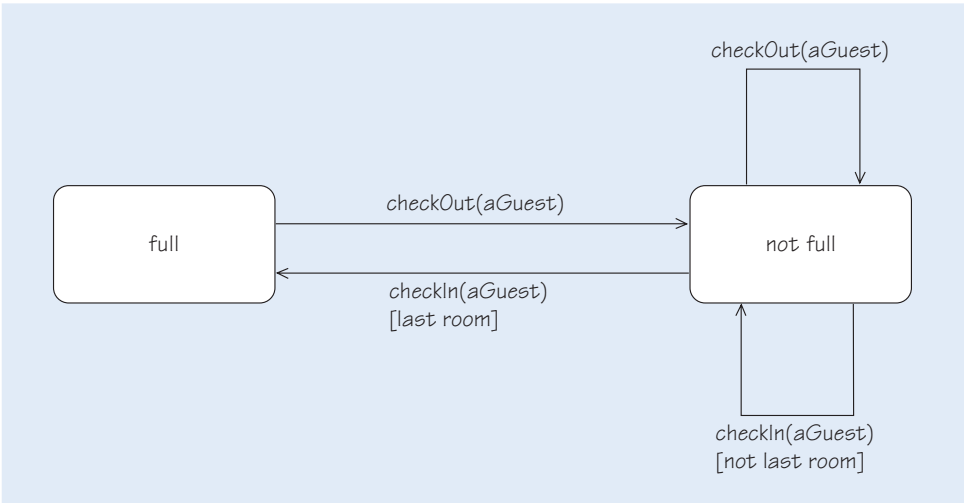


Figure 17 Simple state machine diagram for the *Hotel* class

A self-transition can also be represented as a curved arc as long as it starts and ends in the same state.

Figure 17 contains two instances of a special kind of transition, known as a **self-transition**, which originates in one state and returns to the same state. They are commonly found in state machines that model loops or cycles.

From the perspective of checking in and checking out, the state machine in Figure 17 is incomplete because it does not deal with the *empty* state. If every guest in the hotel checks out, the hotel is empty.

Figure 18 shows one way to follow the checking in and out of guests into a hotel that might be full, empty or somewhere in between. The maximum number of rooms in a hotel is denoted by *hotelLimit*. A simple counter called *availableRooms* keeps track of the number of unoccupied rooms in the hotel.

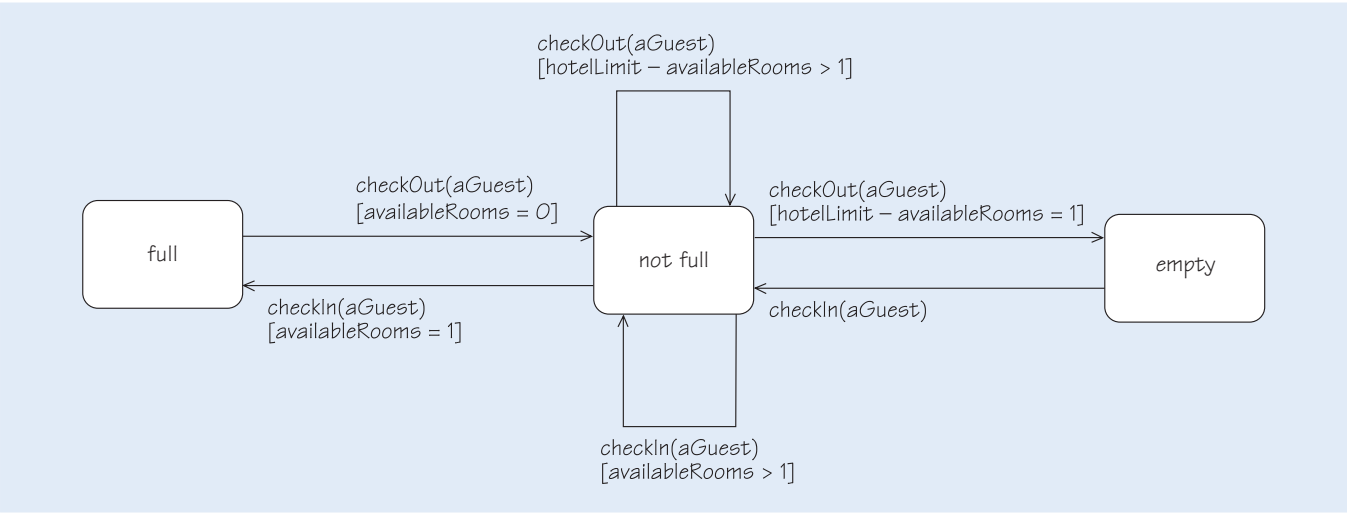


Figure 18 Full hotel or empty hotel?

If the same event is used on more than one transition, you must make sure that no more than one guard can evaluate to true. UML says that only one transaction will fire no matter how many guards are active, but it does not say

which one. To avoid any possibility of ambiguity, just one transition should be able to fire in any given situation.

For example, which transition will fire if the value of *hotelLimit* is 100 and the value of *availableRooms* is 2 when the event *checkIn(aGuest)* occurs, and the *Hotel* object is in the *not full* state? First, we do not need to worry about the *hotelLimit* under these conditions. There are two possibilities: either the *Hotel* object moves to the *full* state or it remains in the *not full* state. As the value of *availableRooms* is greater than 1, it is the self-transition that is triggered.

In a real hotel chain there are likely to be a number of complications and you would need to revise your state machine accordingly.

SAQ 6

- What is a guard and how does it protect a transition?
- Look at Figure 17, which only considers whether an object of class *Hotel* is full. Why are there no guards on the transitions for the *checkOut(aGuest)* event?
- Why were guards introduced on the transitions for the *checkOut(aGuest)* event in Figure 18?

Answer

- A guard is a Boolean condition that is applied to a transition – the guard must be either true or false. A guarded transition can only take place when the specified guard is true.
- A guest who is checking out cannot make the hotel full, so no guard is needed for the *checkOut(aGuest)* event.
- The example in Figure 17 is incomplete. Figure 18 deals with both extremes – when the hotel is full and when it is empty – as well as the possible configurations between full and empty. This results in *checkOut(aGuest)* occurring in three transitions, two of which (from *not full* to *empty* and from *not full* to *not full*) require guards to ensure mutual exclusion between them.

4.3 Exceptions

There are two other possible transitions that are not shown in Figure 18. The state machine diagram does not show what happens when:

- the event *checkIn(aGuest)* occurs and the *Hotel* object is in the *full* state
- the event *checkOut(aGuest)* occurs and the *Hotel* object is in the *empty* state.

You could view the diagram in Figure 18 as showing the normal processing, omitting the exceptional cases. Certainly we would not expect the *checkOut(aGuest)* event to occur if the *Hotel* were empty – if it did, we would consider it an error. Similarly if the *checkIn(aGuest)* event occurred when the *Hotel*

was full, this would be another error, given that an instance of the class *Guest* should not be able to check in without a reservation.

However if it were decided that the hotel system should cater for people requesting a room without having a reservation, the *checkIn(aGuest)* event occurring in the *full* state would undoubtedly occur in real life and should be captured as part of normal processing.

The question here is how best to deal with error situations. One useful guideline is to model only what happens if the error is in some sense *expected* and a special response to the error is needed. For example, you might want the user interface to ask the receptionist if the person was in fact checking out if someone tries to check in to a full hotel.

4.4 Entry and exit events and actions

In UML, activity diagrams and state machine diagrams are closely related. They have a number of common model elements, such as the use of an initial state, signified by a black circle. However there are some important differences, most notably between actions and activities. An *action* is an executable atomic computation that results in a change of state in a model or, more generally, leads to the return of a value. As an action is atomic it cannot be interrupted by an event, so an action is expected to run to completion.

In contrast, an **activity** is a substructure, usually a substate machine consisting of a sequence of states that may be interrupted by other events. Because an activity is itself a state machine, it can lead to a number of actions depending upon the prevailing conditions.

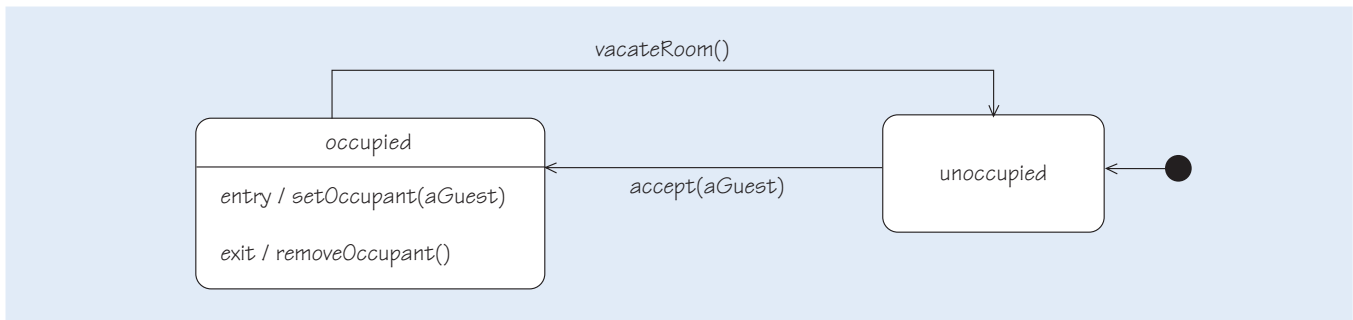
The notion of activities implies that states may be nested within other states, which is true. A single state can have a substructure that relates to an activity of interest to you as a developer.

UML provides ways (outside the scope of this course) of showing an activity on a state machine diagram, which avoids or defers the need to add complexity to your state machine by revealing all its substructure. However it may be necessary to show that one or more states do contain important activities. You may find that all transitions into a state share some actions. You can avoid putting these actions on every transition by using two special actions for state machines in UML:

- an *entry* event, which is triggered every time an object enters a state
- an *exit* event, which is triggered every time an object leaves a state.

The use of entry and exit events and their associated actions allows you to reduce the complexity of some state machines. Instead of writing the same action on two or more transitions leading to or from the same state, that action can be associated with the state's entry or exit event. For example, Figure 16 can be redrawn by adding an action to both an entry and an exit event, as shown in Figure 19. The notation used for an entry or exit event is the word 'entry' or 'exit' followed by a slash and the action or action sequence. The two state machine diagrams in Figures 16 and 19 are equivalent.

Entry and exit events are not allowed to have guards.

Figure 19 Using entry and exit events for a *Room* object

Whenever you use an entry event, you must remember that all transitions into the state will cause that entry event to occur. This means that the actions associated with the entry event must be appropriate for *all* incoming transitions. Similarly the actions on an exit event must be appropriate for *all* outgoing transitions. In the case of a self-transition both the entry and the exit events will occur, so you must take care to ensure that their actions are consistent with each other. While your chosen entry and exit events might be fine individually for non-self-transitions, they may be inappropriate for self-transitions.

UML also provides a way of dealing with the situation in which an event should cause an object to return again to the same state but without performing the entry and exit events. An **internal transition** is a special transition that does not involve a change of state. In contrast to a self-transition, an internal transition does not cause the execution of any entry or exit actions. An internal transition is typically used to handle an interrupt where you want some action to take place that does not affect the current state.

The details of such a scenario are not relevant here. You would, however, be advised to model such a requirement with one or more interaction diagrams. For example, `hotelLimitChanged()` might just signal a change in the value for each upper limit or you might prefer to pass in the new value as an argument to the method.

Figure 20 Internal transition within a *Hotel* object

Like entry and exit events, internal transitions are written inside the state to which they refer. Use a suitable event name followed by the action that follows it. For example, when a hotel's upper limit to the number of available rooms changes, the number of available rooms should be revised. Figure 20 shows how we might use an internal event to achieve the change in the *not full* state. In effect, a *Hotel* object receives the message `hotelLimitChanged()` – an event – and responds by sending the message `resetAvailableRooms()` to itself – an action.

SAQ 7

- (a) What is an entry event, and how does it contribute to the maintenance of a state diagram?
- (b) What is the benefit of using an internal event as opposed to a self-transition?

Answer

- (a) An entry event can be used where there are multiple transitions, with the same actions, leading to a particular state in a given diagram. An entry event occurs every time an object enters the state that it annotates. Entry events reduce the risk of introducing errors, because the action sequence is written once (associated with the entry event of the state) rather than many times (on each of the transitions leading to that state).
- (b) When there are entry and exit events that might interfere with a self-transition, an internal event is useful because the entry and exit events are not triggered.

4.5 When to use state machines

See Block 3 Unit 11 on the subject of testing.

As with any other kind of model, you should try to design your system so that each state machine for a class is as simple as possible. As a state machine becomes more complex, it becomes harder for client objects of other classes to use it correctly. For example, if a *Copy* object in Figure 15 is in the state *on loan*, is it an error for it to receive a *copy borrowed* message?

With a small number of states and transitions, it may be easy to gain some confidence about the state of the intended recipient of a message. As the number of potential pathways through a state machine grows, the more conditional statements you will need to code and test, which makes it harder to verify the behaviour with respect to the software specification.

State machines have several benefits.

- They are a means of elaborating the potential operations within a class – identifying which messages affect which attributes and considering the consequences for each object belonging to a given class. A state machine can help the developer understand the behaviour of a particular object over one or more use cases. As you saw above, for example, you might model the possible states of a *Room* object in a hotel system for the checking in and checking out of guests. Then at some later point you may have to deal with the cleaning and maintenance of each room, which would add complexity to the simple state machine shown in Figure 14.
- They help you understand the behaviour of an object over its lifetime. The more often an attribute of an object changes, the more likely you are to want to model the object's behaviour in a state machine. If more than one attribute changes often, the need to model the object's behaviour by a state machine increases accordingly. For example, the system may be asked to monitor the fact that a room must be cleaned regularly. So *clean* or *dirty*

could be added to the list of states for a room. However an increase in the number of states increases the potential complexity of the behaviour of the object because of interactions between the states. For example, the hotel management may not want a room to be cleaned while it is occupied.

- They help you understand how an object or a system must respond to events. Like communication and sequence diagrams, they help you to understand object interactions. In practice you should expect to use a combination of techniques.
- They are a means of ensuring correctness. In a formal development process you may be required to produce a state machine for each class as part of a given requirement to ensure a level of correctness for the behaviour of the objects within a software system.

A state machine is useful when it helps you to understand what is going on and to verify that understanding with the domain experts. It is also useful for unit testing as it shows the behaviour we need a class to exhibit. At this level of detailed design, however, it is sometimes easier to implement a class rather than produce more and more detailed diagrams.

We have now reached a point in the development of a software system where we are close to producing the code in our chosen programming language.

SAQ 8

- Identify three problems associated with complex state diagrams that might arise when designing classes.
- Suppose the class *Copy* included the attributes *returnDate*, *libraryNumber* and *classification*. Which of these attributes are significant for a state machine for the class *Copy* that has two states called *on shelf* and *on loan*? Explain your choice.
- Suppose you implemented your design for a class without using a state diagram. Are there any subsequent activities where a state machine might help you as a developer? Briefly explain your answer.

Answer

- Classes with complex state machines can cause three kinds of problem:
 - it is harder to write the eventual code for such classes because there are likely to be many conditional tests to identify the actual state
 - it is harder to test the classes because of the number of choices of pathway through the conditional tests
 - it is much harder for external code to use a class correctly without some means of ascertaining the actual state of objects belonging to complex classes.
- State machines help model what might happen when a particular object receives a given message because the behaviour of an object is influenced by the values of its attributes. A state machine tells you about the life history of an object. Therefore you can use the

frequency of change of an attribute during an object's lifetime as a means of choosing which attributes are significant when constructing a state machine.

In the class *Copy*, for example, the attribute *returnDate* is likely to change often, as it reflects the transition between the states *on shelf* and *on loan*. However attributes such as *libraryNumber* and *classification* will only change as a result of some reorganisation within the library (probably set during the creation of instances of the class *Copy*). The borrowing and returning of each copy by the library's members do not affect them. Among other things, you might consider an additional attribute that records the actual date when a copy is returned (*actualReturnDate*, say) if you need to investigate the notion of fines for overdue books.

- (c) There are situations where you might need to develop state machines retrospectively. For example, you might benefit by preparing a state machine when testing individual classes to demonstrate that the behaviour of an object over its lifetime satisfies the relevant requirements for its class – usually for the purposes of verification. In addition, a proposed change to a software system might introduce a need to prepare one or more state machine diagrams to show how an object's state would be affected.

Exercise 4

Suppose you were developing software for a telephone system. When looking for the objects with 'interesting behaviour' you selected landline telephone handsets because they represent an interface for users. There are two basic states to consider – *idle* and *active*. They reflect the basic activities of making a call to someone. Either you have picked up the handset and are ready to dial, or you have completed your call. Draw a simple state machine diagram that illustrates this behaviour, giving events and actions.

Solution

Figure 21 shows the transitions between *idle* and *active* for a telephone:

event: *handset lifted*; action: *sound dial tone*

event: *handset replaced*; action: *disconnect*

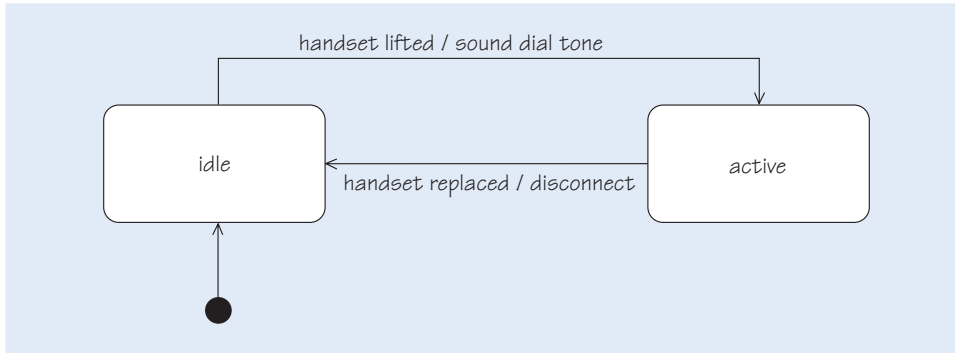


Figure 21 Two basic states for a telephone

Having modelled the simple fact that you are either using a telephone or you are not, your next model might consider the behaviour required to reach another telephone in order to communicate with another person. For example, you need to dial the appropriate number and find out whether the number is busy.

4.6 Final note on state diagrams

It is possible to model quite complex behaviour within a single state diagram. However such effort by the modeller must be justified – one way to do that is to look for those objects that are dependent upon their internal state for their response to a given event. For example, if an object always responds in the same way to a given event, it is unlikely that you will need to produce a state diagram – it is **state-independent**.

Those objects that react differently to a given event are **state-dependent**. One useful clue is the need for some conditional logic for the success or failure of an operation, such as payment authorisation when you use a credit card in a supermarket.

In general, state-dependent behaviour is found in classes that must perform control and coordination tasks. For example, you will need to investigate state-dependent behaviour for software systems that control manufacturing processes and telecommunications, where there are a number of devices involved, such as switches, temperature sensors and valve controllers.

Administrative systems can also have complex state-driven behaviour. For example, purchase and expense authorisation systems may have many levels of signature and approval, depending upon the amount involved.

You have already seen how interaction diagrams are used to model the flow of messages between objects, which are the most common form of internal event in the object-oriented paradigm. In other words, you can see how objects communicate in order to fulfil certain tasks. The need to model state-dependent behaviour is likely to be revealed as you model object collaborations when, for example, you encounter the need to deal with conditional behaviour – is this credit card valid or not?

SAQ 9

What characteristics of events suggest that an object should respond to them when they occur?

Answer

An object should normally respond to:

- events that are external, such as those from a point-of-sale terminal
- change or time events that require some response
- certain changes, such as a high or low temperature in a process-control system, to avoid problems or even disasters.

Exercise 5

Figure 15 shows a state diagram for an object of the class *Copy* in the lending-library example. How might the diagram be amended to show that a copy becomes overdue if it is not returned within 3 weeks?

Solution

You can modify the state diagram in Figure 15 in a number of ways. The *Copy* object must still be in the *on loan* state. There are several choices, each one depending on a given library's rules for overdue books.

For example, if it is not necessary to record an overdue book because there is no penalty attached to it, you can use an internal transition or a self-transition that is triggered by a time event: *after(3 weeks) / notify member*. Which kind of transition you use would depend on the existence of any entry and/or exit actions.

If there is a penalty for being overdue, you might use a composite state for *on loan* that requires at least one substate, *overdue*, which is entered by a change event:

when(currentDate – issueDate > 3 weeks)

Alternatively you may defer calculating any fine until the book is returned, which would lead to an increase in the number of guarded transitions that respond to the *return* event.

Exercise 6

UML lets us describe five things about a transition. What are they?

Solution

The five parts of a transition are:

- 1 source state
- 2 target state
- 3 event trigger
- 4 action (or action sequence)
- 5 guard.

In practice, you may not need all five parts in every case.

4.7 Summary of section

State machines are used to model the dynamic behaviour of an object or a system. Because state machines are modelling the behaviour of an object, they will usually be involved in the behaviour of more than one use case.

- A state machine in UML contains a number of states that are connected by transitions. The transitions between states are caused by events that may give rise to a number of actions. Initial and final states are helpful when recording the full life history of an object. The transition from an initial state leads to the particular state that an object enters after its creation or initialisation. A final state is reached when an object is about to be destroyed or otherwise stops responding to events.
- In a simple state machine diagram, the states are shown as boxes with rounded corners and the transitions between them are shown as arrows. Each transition may be labelled with an event and an associated action or sequence of actions. An initial state is shown with a solid black circle, from which there is a single transition without an event label to the first real state in the life of an object. In contrast, there may be more than one transition leading to an object's final state, shown with a black circle surrounding a solid black circle.
- Entry and exit events can be written inside a state. These can be used where there are multiple transitions, with the same actions, leading to or from a particular state. An entry event occurs every time an object enters the state that it annotates. Similarly an exit event occurs every time an object leaves the state.
- Actions or action sequences can also be written inside a state to avoid the firing of any entry or exit actions that apply to that state.
- State machines are a means of elaborating the operations of a class. It is possible to identify the messages that affect an attribute of that class and the actions that arise as a consequence.
- Whenever a specific event can lead to different transitions, it is essential to identify the conditions under which each transition occurs. This is achieved in a state machine diagram by the use of guards.

As a result, you can use state machines to identify what is needed to implement an operation of a class.

5

Overall design process

We have now seen some techniques that are particularly appropriate for designing the business logic, that is, those parts of the system that would be allocated to the application domain of a layered architecture (as seen in Block 1 Units 1 and 4).

An obvious question is, what about the design of the other layers? Later in the module we look at some aspects of the design of user interfaces (the presentation layer). A full treatment is beyond the scope of this unit. The design of ways of supporting persistent data (the infrastructure layer) is again beyond the scope of this unit.

Even given a focus on the business logic, there is more to design than the allocation of responsibilities using interaction diagrams and considerations of state. When we looked at requirements earlier in the module we mentioned that the non-functional requirements depend on the architecture or structure of the system. For example, in order to ensure the system you produce is maintainable, you need to structure it in an understandable way. We will look at a mechanism for doing this in the following subsection.

You also need to think about consistency. Even if you assume you have gathered the correct requirements for the system, you need to check that the various artefacts you have produced from the requirements onwards are consistent. If your design is inconsistent then, assuming consistent requirements, you must have introduced one or more errors that can be removed by removing the inconsistency. To check two artefacts are consistent you need to ensure that they match in ways you would expect. For example, given a use case have you allocated an operation to implement it?

Many consistency checks are relatively straightforward, although potentially time consuming. For large models consistency checking can become laborious and tool support might be highly desirable. The specific consistency checks that would be used in a system's development depend on the particular design process. If you have followed a systematic design process, you should have developed consistent models, and avoided the need for extensive checks subsequently. That is, you have drawn an interaction diagram for an existing scenario in an existing use case, you have made sure that an interaction diagram is consistent with the structural model, and so on. We will revisit this topic in the context of the case study.

Once you have a complete design careful thought needs to be given to its implementation. In a fully agile approach you would be coding from the user stories. In a plan-driven approach you will have produced rich enough design models that you can implement systematically. In both cases, however, there are other considerations that need to be taken into account. For example, there may be existing software products that will meet some or all of the requirements. Or there may be software packages that could be tailored to your needs, one of the most obvious being a database package.

Some other options are to outsource the production of the system to another company or even to produce it through an open source process. In this latter case software is produced by a collective of volunteers, often motivated by the nature of the particular product to be developed – for example, people may feel motivated to contribute to a freely available text editor as they consider one should be available.

In any case, the outsourcing and open source options come with a complex range of economic, political and practical issues that are interesting in their own right, but beyond the scope of this module. In the rest of this section we present a useful technique and notation that allows you to take a broader view of the software system as a whole, in particular when taking architectural decisions during design.

5.1 Packages

UML provides a way of grouping model elements into a unit called a **package**. We will see below how a package helps to structure the implementation of a design. A UML package is a much more general idea than a Java package. A package can be used to group almost anything. Use cases can be grouped into packages, for example – and so can other packages.

Each package defines a **namespace**. Any two distinct model elements in the same namespace (package) must always have different names, although elements in different namespaces (packages) can have the same name. This is exactly the same idea that lets us have two Java classes with the same name, so long as they belong to different packages.

One package can be associated with another – if some of the model elements in the first package are associated with model elements in the second – and in a similar way a package can depend on another package. However a package does not have an interface and it cannot have instances, so packages cannot *interact* with one another.

There are three main reasons for partitioning a software system:

- to manage size and complexity
- for information hiding
- for logical decomposition.

It is common for businesses to operate management information systems (MIS) that consist of a number of separate packages to support their commercial activities. Figure 22 illustrates such a system, which contains four basic packages. It reflects a typical company that makes things (*Manufacturing*), which it stores (*Stock Control*) and then sells (*Order Processing*) to its customers. The income and expenditure are recorded in a series of transactions (*Accounts*).

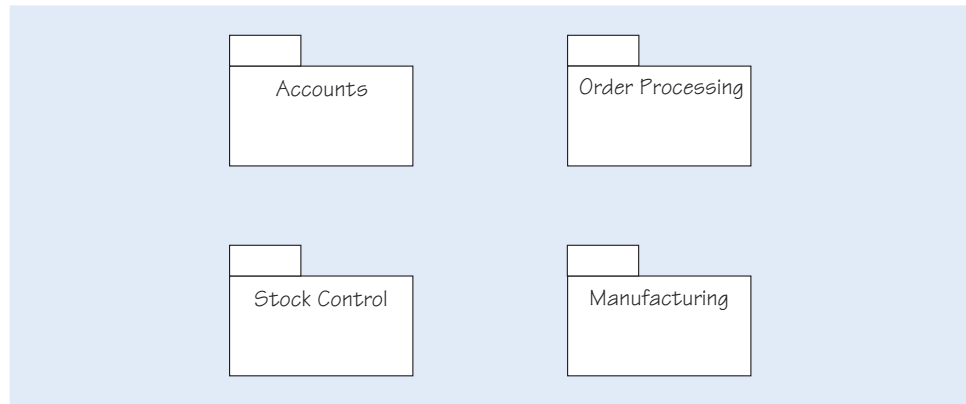


Figure 22 Typical management information system

The notation for a package is a large rectangle with a smaller rectangle attached to its top edge – just like the tabs you use with the folders inside a filing cabinet. The name of each package can be written either inside the large rectangle, as in Figure 22, or inside the smaller of the two rectangles.

Although Figure 22 does not show the dependencies between the packages, there will be a need for objects in each package to collaborate with one another. For example, customers place orders that must be made, delivered and then paid for. At the same time, the company pays for the raw materials and its employees' time taken to produce those orders.

In UML, a package is not the same as a class, so it cannot have any associations or take part in any interactions. However a class in one package can have an association with a class in another package – according to certain rules, as we explain below.

By implication, there is a root package that contains the entire model of a software system.

As packages are a simple grouping mechanism for model elements, one package can contain other packages. You can either draw one package inside another or represent the package hierarchy in a tree structure. For example, the *Accounts* package of Figure 22 may contain two other packages: one for accounts received and the other for accounts payable (income and expenditure). Also, Figure 22 can be redrawn to show one large package, called *Management Information System*, containing the original four packages. However the internal structure of each package is normally drawn in a separate diagram.

A package must have a unique name within the model and we include the package name to distinguish between model elements. For example, you could place the class *Order* inside the *Order Processing* package in Figure 22, so that its unique name would be *OrderProcessing::Order*.

Notice the convention of removing the spaces between package names.

Customers are typically part of the accounts received by a manufacturing company, so the class *Customer* would be identified as *Accounts::AccountsReceived::Customer*, which you can see in Figure 23.

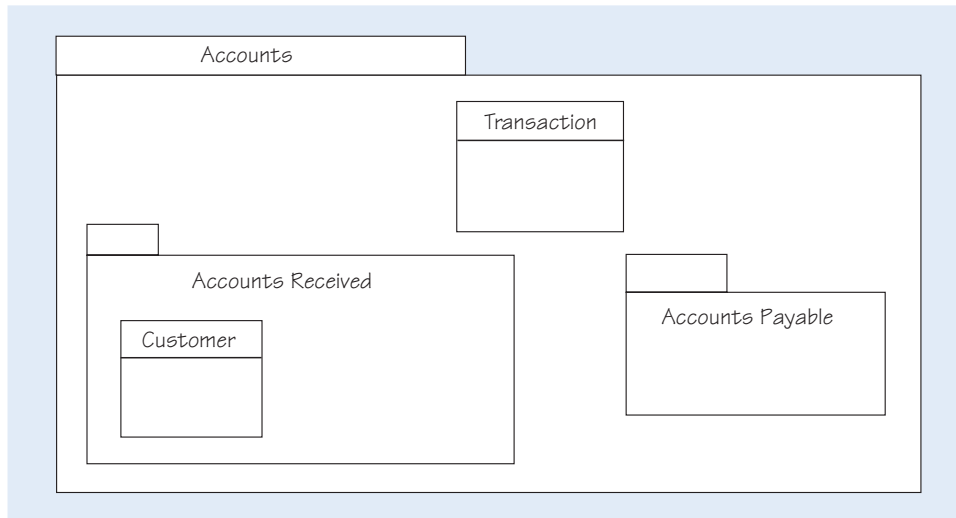


Figure 23 Nested packages

Once you partition your software system into packages, you need a way of allowing elements in different packages to ‘see’ each other.

The class *Customer* in Figure 23 has no difficulty in forming associations with any other class that is inside the same *Accounts Received* package, or with the class *Transaction* because it is inside the same *Accounts* package that also contains the *Accounts Received* package. However the class *Customer* is not able to ‘see inside’ the *Accounts Payable* package, because there is no shared namespace. In accounting terms, this barrier is quite helpful because you should not confuse income with expenditure.

If you do want to overcome the namespace problem, UML allows you to import the contents of one package into another. In Figure 23, for example, you can draw a dependency arrow from *Accounts Received* to *Accounts Payable*, and label it with the stereotype «import». As a result, elements inside *Accounts Received* can refer to all the publicly visible elements of *Accounts Payable*, just as if they were all in the same package. The target package name does not need to be used for imported elements. Any private elements in *Accounts Payable* will remain unavailable.

An alternative mechanism is to use the stereotype «access», which defines a different form of dependency. Importing is a way of creating a larger namespace. In contrast, access dependency is similar to a ‘special bridge’ between the source and target packages. With access dependency, elements of *Accounts Received* can ‘see’ the public elements of *Accounts Payable*, for example. However elements in the source package must include the target package name to access its elements, such as *Purchase* or *Supplier* in the accounting example.

If another package were to import or access the *Accounts Received* package, that package would not have access to elements of the *Accounts Payable* package. That package would need to either import or access the *Accounts Payable* package directly. In Figure 23, for example, *Order Processing* would need to explicitly import or access the contents of both *Accounts Received* and

UML allows you to mark individual elements of a package as public (+), private (–), protected (#) or package (~). Protected features can be accessed only by operations within the same class or its children. The other terms are obvious.

Accounts Payable, if there were a need to communicate with elements of both packages.

You can also use packages to present different views of a software system, such as its architecture. Figure 24 illustrates a three-layered architecture that we introduced in Block 1 Unit 1.

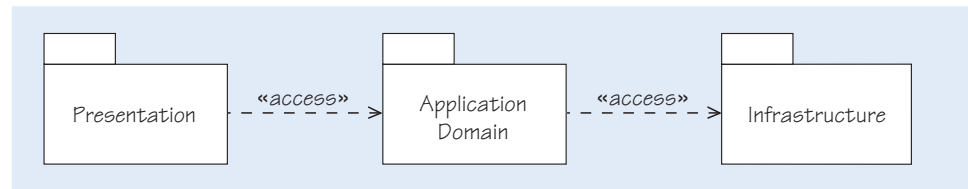


Figure 24 Three-layered architecture

The presentation (representing the top layer in Figure 2 of Block 1 Unit 1) translates the user actions into system requests. In the management information system of Figure 22, you would expect to see the classes that deal with the rules of order processing in the application domain. These classes would then interact with the infrastructure layer to access and store data, for example. The separation of a software system into three layers or tiers has the following advantages:

- developers can be allocated to different layers to facilitate parallel development or the use of particular skills such as interface design
- reuse becomes possible in certain areas, such as business services
- distribution of the resultant software can be considered in response to certain performance requirements.

Packages also define something that can be tested, as you will see in Block 3 Unit 11.

A well-structured package will have a clear boundary and contain model elements that share a given purpose. That is, a well-structured package displays high cohesion. In addition, the contents of a package and their visibility should be such that other packages can only see those elements that they need in order to do a specific job. That is, a well-structured package displays low coupling.

As you saw in Block 1 Unit 1, high cohesion and low coupling are prerequisites for useful components. However a package lacks several things: it has no interface, cannot be instantiated, and cannot participate in interactions. A more general concept without these restrictions, that of software components, is introduced in Block 3 Unit 10.

SAQ 10

- Do packages contain only classes?
- Which modelling principle would lead you to form packages?
- Explain how it is possible to have two separate elements that share the same name.
- The stereotypes «import» and «access» allow you to ‘see’ through the ‘wall’ around a package. Which one can give rise to a naming problem? Which one provides the simpler traceability?

- (e) Suppose you use `«access»` to ‘see’ into a package. Can you see everything inside that package? Briefly explain your answer.

Answer

- (a) No – they can contain any model element from UML. Among other things, you can include classes, use cases, associations and even other packages.
- (b) You should use a package to raise the abstraction level, suppressing any detail in a particular diagram that is not necessary to understand what the model is showing.
- (c) If you place the two elements in separate packages, they can share the same name. This is because each model element is identified by its name and the name of the smallest package that contains it.
- (d) The `«import»` stereotype can give rise to a naming problem, because an `«import»` adds the names of the elements in the target package to the source package. The result is that, if an imported name clashes with an existing name, you have to qualify the imported elements with their package name in order to distinguish them. The `«access»` stereotype allows simpler traceability, because other packages must use the full name of an accessed element, such as *PackageName::ElementName*.
- (e) There is no guarantee that you can ‘see’ all the elements of the package. It is possible to control the visibility of elements within a package in the same way that you can control the visibility of the attributes of a class. An element of the accessed package can be marked as private, making it invisible to other packages that import this package and indicated with a ‘-’ character. Alternatively, it can be marked as protected, making it invisible to importers of the package but visible to child packages, and indicated with a ‘#’ character.

5.2 Summary of section

In this section we have reviewed the overall area of software design, and you have learnt that model elements of almost any sort can be grouped into coherent units called packages.

- Packages help you organise models in a way that lets you better represent and communicate your understanding of a problem.
- Packages can help you construct large software systems by splitting them up into smaller, more manageable chunks.

A package defines a namespace. No two elements in the same package may have the same name, but elements in different packages may have the same name and are distinguishable by their respective package names.

An element inside a package is only visible from other packages if it is declared public. For the element to be visible from another package, it must be imported to that package, using either `«import»` or `«access»`.

Packages can be associated with, or depend upon, other packages. Ideally, packages should exhibit high cohesion and low coupling.

Summary

6

As you get closer to implementing your design, it is likely that you will alternate between static and dynamic modelling in revising your design to satisfy the users' requirements. It is possible to follow each scenario using a set of sequence diagrams, so that each one illustrates what will happen in a given set of circumstances for a given set of objects. You may then need to modify your class model as you try to find the best software solution to the problem. In practice, there will be a number of design issues that influence the kind of software system that you can implement.

A good model will need to be:

- a correct representation of the requirements – otherwise your effort will be wasted during the implementation activity
- consistent – otherwise the teams of developers on a large project will not be able to integrate the packages on which they have worked
- sufficiently complete – otherwise you may fail to model an important requirement.

In practice, you will experience a tension between the need to produce a good model and the needs of the project that you are working on. Models alone are not enough to produce a good software system, as you will see in Block 3 Unit 9.

On completion of this unit you should be able to:

- discuss the merits of three strategies for the implementation of use cases
- explain how packages promote modularisation within a large software system
- understand optional, alternative optional, alternative and loop fragments within sequence diagrams
- identify alternatives that influence where you might place the checking of the preconditions for an operation
- identify the main components of a state machine
- identify when a state machine can contribute to the development of software
- use UML notation for conditional and iterative message sends in sequence diagrams.

Unit 8 The case study: part 2

Contents

1	Introduction	193
2	Domain modelling – the conceptual model	198
	2.1 Identifying concepts, associations, multiplicities and attributes	198
	2.2 Summary of section	206
3	Analysis	207
	3.1 Constraints	207
	3.2 System operations	212
	3.3 Specifying operations	213
	3.4 Summary of section	225
4	Design – assigning responsibilities	226
	4.1 Value identities	226
	4.2 Verifying and fulfilling preconditions and postconditions	227
	4.3 Structural design model	239
	4.4 Summary of section	242
5	Consistency	243
6	Summary	246
	References	247

Introduction

In this unit we continue the work started in Block 1 Unit 4, where we concentrated on the two disciplines of domain modelling and requirements. There we focused on one iteration of the elaboration phase of the Unified Process (UP). In this unit we will continue to focus on that same iteration, complete the analysis of the case study, and produce the design for that iteration of the UP.

In Block 1 Unit 4, from our use case diagram, we identified the *make reservation*, *cancel reservation*, *check in guest* and *check out guest* use cases as a suitable set of requirements for the first iteration. They provide a usable slice of functionality. We also decided to focus during the first iteration on the main success scenarios. We continue working at the application domain layer of the three-layer architecture.

In this unit we will show that the analysis and design are driven from the requirements and will establish some basic traceability between the requirements and design. We begin by producing a structural domain model – the **conceptual model**. The conceptual model is just a class diagram in which the classes represent concepts from the problem domain rather than the software. This will allow us to represent sets of objects that are of relevance to the problem we are trying to solve.

Although logically it is sensible to think of domain modelling as preceding requirements and analysis, the existence of the requirements does not necessarily mean that we fully understand the domain. In fact this unit will show that, even for an area like hotel reservations and checking in, a lot of subtle issues need to be carefully considered. Nevertheless, where developers are working in a well-known domain, they may choose to skip domain modelling and start with what needs to be represented in software.

Units 5 to 7 used examples from a hotel system. A major difference in this case study is that we will conduct the requirements and analysis and create the diagrams systematically, with the whole process driven by the requirements given in Block 1 Unit 4. So, while the examples in Units 5 to 7 illustrate particular points, in the case study units we show the overall development approach.

In Block 1 Unit 4 we went from an initial problem statement, through a range of activities, to a set of detailed requirements. We currently have a set of use cases for the proposed system, along with a set of software requirements.

We will start by looking at the structure of our domain and identify which domain objects should be represented as classes, which attributes should be defined in the classes and which relationships exist between the classes. Remember that, although we are considering classes, they represent domain concepts in the **conceptual model**; we are not considering code as yet. We are still dealing with models, abstracting away from decisions related, for example, to how the system is going to be implemented.

Having completed the structural domain model – the conceptual model – we will move on to produce the **analysis model** as a first model of the software solution. As we already have a conceptual model, the analysis model will build on it and the requirements. Although classes in the analysis model may resemble those in the conceptual model, they represent what is needed in the software, in contrast to concepts, which represent the real-world domain. We may decide that some concepts from the real-world domain should not be represented in the solution or that we need classes in the solution that were not present in the domain.

As we carry out these steps we will produce an accompanying glossary, which defines each analysis class, its associations and attributes. In other words, the glossary documents the elements of each model.

We adopt from Jacobson et al. (1999) the following description of the characteristics of an **analysis class**, to indicate which characteristics we should identify in the analysis model.

- An analysis class focuses on handling functional requirements and postpones the handling of non-functional requirements until design and implementation.
- An analysis class seldom defines or provides any interface in terms of operations and their signatures.
- The behaviour of an analysis class may be outlined by responsibilities, although the detailed assignment of responsibilities is left until design.
- An analysis class defines attributes, although these attributes are at a fairly high level. The types of these attributes are often conceptual and recognisable from the problem domain. These attributes may become classes themselves in design and implementation.
- An analysis class is involved in relationships, although these relationships are still conceptual. Navigability is not important at this stage, although it will become essential when we come to design.

In our case then, the conceptual model we produce can be used as the basis for the analysis model. This just leaves us to consider constraints and then identify the **system operations** required from the use cases in order to complete our requirements and analysis. In identifying the system operations, there is an emphasis on their pre- and postconditions.

Moving into design, we can start deciding on the assignment of responsibilities to classes and recording these decisions on the class diagram. This will involve the following tasks:

- using interaction diagrams in verifying preconditions and fulfilling postconditions of the system operations – leading to identification of finer-grained operations
- assigning the operations to classes using GRASP patterns:
Expert – responsibility assigned to a class with information to fulfil that responsibility

Creator – responsibility given to classes that maintain, record and contain instances of the class of the newly created object

- revisiting the class diagram to record all the decisions taken; this will be our design model.

Figure 1 shows an overview of which parts of the **development process** we cover in this unit.

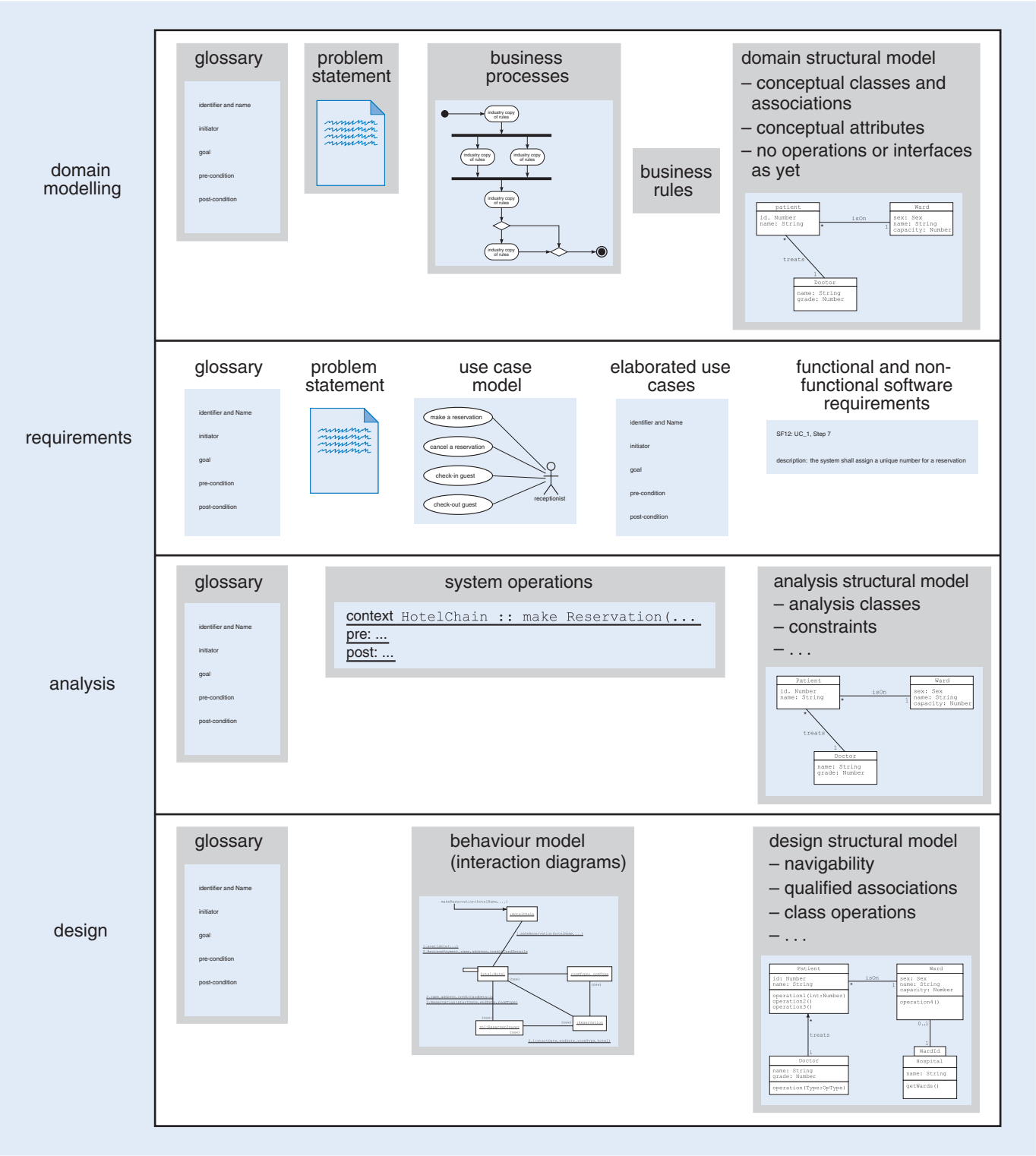


Figure 1 Parts of the development process covered in this unit

In this unit we take a systematic approach in going from software requirements via a structural model to a design. Remember, though, that a key criterion for constructing any artefact should be 'does it contribute to software

quality?'. Once we have developed a few use cases and got a strong grasp of the structure of our software, it may be that further use cases can be developed in a more agile way. For example, if a use case has similar functionality to an earlier one, we may be able to copy and modify code without updating the structural model.

Of course with this approach our model becomes a throw-away: when it has been used to design the system it is no longer treated as an accurate model to be used for system maintenance. The concerns that this unit considers still need to be addressed, but for more agile approaches this is done less formally.

For a 'big A' approach to agile (following an agile process model such as XP or Scrum, rather than being more agile within plan-driven development), based on user stories, a project team might go straight from the user stories introduced in Block 1 Unit 4 to code. In such a process the requirements would be elaborated as the code evolved but they would not necessarily be recorded, except that the code could be seen to embody them.

In the extreme case then, the material in this unit represents a process that might not occur in 'big A' agile. However, an agile team may well be familiar with modelling approaches from other projects and make pragmatic use of them during design. The choice of a software development process to follow, and in what ways it is appropriate to vary it, needs to be based on the nature of the project as well as the project team's experience and the context of the organisation, and it has implications for cost, delivery time and quality. You will see in this unit that we follow a slightly different process from that in earlier units.

2

Domain modelling – the conceptual model

Remember that at this stage we are still describing the world in which our problem exists, rather than describing a piece of software. We begin by using the four categories of objects from Unit 5 – tangible objects, roles, events and organisational units – to identify relevant candidate classes.

2.1 Identifying concepts, associations, multiplicities and attributes

We start by looking at the initial problem statement from Block 1 Unit 4 Section 3, identifying potential concepts relevant to the current iteration, to be represented in our model using the technique from Unit 5.

- Tangible objects: computer system, newspapers, debit/credit card, room.
- Roles: guest, customer, manager, receptionist.
- Business transactions: reservation, check in, check out, cancellation, no show, hotel full, add room type/room/hotel, delete room type/room/hotel, decorating, building work, reservation request, block booking, overflow.
- Organisational units: hotel chain, hotel, nearby hotel.

As usual when identifying concepts to be represented as classes there is a degree of arbitrariness in the answer. For example, we could have included hotel as a tangible object instead of, or in addition to, including it as an organisational unit. However, we did not feel that this would be useful later on.

In Block 1 Unit 4 we identified the use cases to be dealt with in the first iteration of development. In fact, from simple observation, we can see that they focus on the core functionality of the business of a hotel: checking guests in and out and handling reservations.

We will therefore start to build our conceptual model by thinking about guests, rooms and the events that determine the relationships between them. A class diagram represents a generalisation of all the possible configurations of objects in the domain.

A conceptual model plays such a central role that it is sensible to construct it systematically, justifying each step, in order to prevent any errors creeping in. For this reason, we will begin with the relationships between guests and rooms, and because both of these classes have direct relationships with a hotel, we will also consider the hotel class.

A first attempt to produce our conceptual model can therefore borrow from Unit 5, based on the same reasoning used there to give Figure 2.

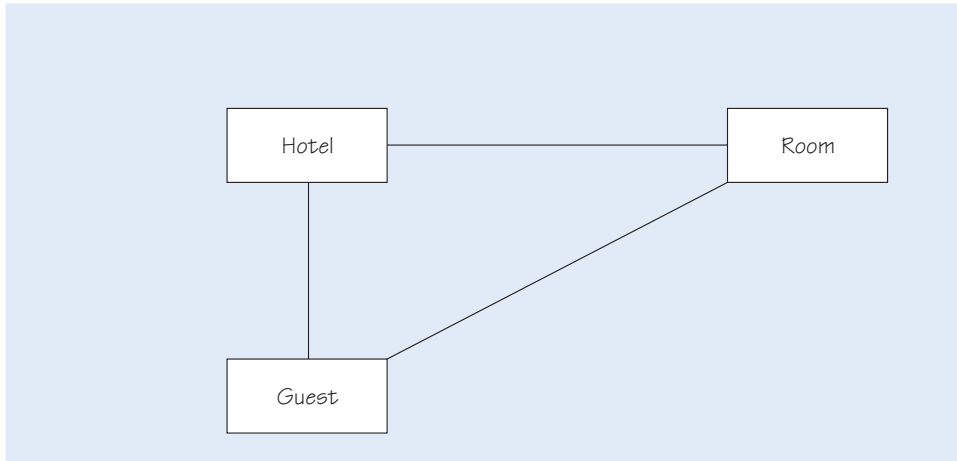


Figure 2 Some associations between *Guest*, *Hotel* and *Room*

The next step is to decide whether any other classes should be represented in the diagram, and establish how they relate to existing classes. We will do this by systematically working through the list of candidate classes given above.

We will use classes to represent the main concepts in our domain. In particular, those candidate classes that have important relationships with other classes will also become classes. As mentioned in Unit 5, Craig Larman (2004, p. 124) suggests the following: ‘If you do not think of some conceptual class X as a number or text in the real world, X is probably a conceptual class, not an attribute.’

Tables 1 to 4 show our decisions about the candidate classes listed above.

Table 1 Tangible objects as candidate classes

Candidate	Represent as class?
Computer system	Not part of the domain.
Newspapers	No. May record as an attribute, so that we know whether the guest should have one.
Debit/credit card	No. We will consider it to be an attribute of <i>Guest</i> for now, but may later need to use a class.
Room	Already included.

Table 2 Roles as candidate classes

Candidate	Represent as class?
Guest	Already included.
Customer	No. Subsume under <i>Guest</i> (but see Exercise 1).
Manager	No. No information needs to be represented about managers for this iteration. Although we may need to represent managers in a later iteration, we will defer this for now to concentrate on the core classes associated with this iteration.
Receptionist	No. No information needs to be represented about receptionists for this iteration.

Table 3 Business transactions as candidate classes

Candidate	Represent as class?
Reservation	Yes, as a reservation exists over a period of time.
Check in	No. Check in alters the relationship between guests and rooms, but there is no other information that needs to be recorded.
Check out	No. Similar argument to that for check in.
Cancellation	No. Causes a reservation to be deleted and perhaps a credit card to be debited, but again no other information needs to be recorded.
No show	No. Similar argument to that for cancellation.
Hotel full	No. Can be represented as an attribute.
Add room type/room/hotel	No. Outside scope of existing iteration.
Delete room type/room/hotel	No. Outside scope of existing iteration.
Decorating	No. Outside scope of existing iteration.
Building work	No. Outside scope of existing iteration.
Reservation request	No. Synonym for reservation.
Block booking	No. Outside scope of existing iteration.
Overflow	No. No long-term information needs to be recorded beyond that for a normal check in.

Table 4 Organisational units as candidate classes

Candidate	Represent as class?
Hotel chain	Yes.
Hotel	Already included.
Nearby hotel	No. Hotels are already included. This information can sensibly be included as an association.

As a consequence of the decisions made in Table 3, we add *Reservation* to our list of classes for this iteration. From Table 4, we add *HotelChain*. Our classes are now:

Guest, Room, Hotel, HotelChain, Reservation.

We add entries for these classes to our (currently empty) glossary, as shown in Table 5.

Table 5 Initial glossary for the hotel system

Term	Category	Definition
<i>Guest</i>	Concept	A person who reserves a <i>Room</i> , stays in a <i>Room</i> and is responsible for the bill. The <i>Room</i> may be occupied also, for example by the <i>Guest</i> 's family.
<i>Room</i>	Concept	A physical room in a <i>Hotel</i> .
<i>Hotel</i>	Concept	One of the hotels in the <i>HotelChain</i> .
<i>HotelChain</i>	Concept	The company which manages the <i>Hotels</i> overall.
<i>Reservation</i>	Concept	A record of the fact that a particular <i>Room</i> has been booked for a given period by a given <i>Guest</i> .

Before we attempt all the work of identifying associations and drawing the initial class diagram, it is sensible to check whether the list of classes we have identified is complete. Because we are aiming to drive the requirements analysis and design of our system from the requirements collected in Block 1 Unit 4, we do this by checking that the class diagram can support the use cases we identified.

SAQ 1

Work through the steps of the four use cases for the first iteration of the system as identified in Block 1 Unit 4, considering whether there are any classes needed to support each step that we have not yet decided to include.

Answer

In UC1 *make reservation*, Step 2 refers to a type of room.

In UC3 *check in guest*, Step 6 refers to a bill.

We choose to add *RoomType* as a concept rather than as an attribute because, as you will see later, it participates in a number of associations. Sometimes decisions like this can simply be based on insight, whereas in other cases they will be taken following further iteration.

Other properties are mentioned, but in each case they can more sensibly be represented as an attribute of an object. For example, reservation number is an integer, a value of a basic data type.

We will regard things like bills and invoices as separate classes, outside the system, because they have lifetimes very different from those of, for example, reservations. The lifetime of a bill may even exceed the lifetime of an individual hotel. This is because bills are considered 'legal documents' from the accounting perspective, and thus fall under the regulation-mandated storage rules. As such, they will need to be handled by a separate accounting system.

Our classes are now:

Guest, *Room*, *Hotel*, *HotelChain*, *Reservation*, *RoomType*.

We add an entry for *RoomType* to the glossary, as shown in Table 6.

Table 6 Adding *RoomType* to the glossary

Term	Category	Definition
<i>RoomType</i>	Concept	The cost of a type of <i>Room</i> and the kind of accommodation, for example single, double, twin, family.

SAQ 2

Identify the associations needed to support the use cases of our first iteration.

Answer

We need to consider each class against every other class. For *Guest*, we consider relationships with each of the other five classes; then for *Room*, we need to consider only four other classes, as the *Guest–Room* pair has already been considered. Although we identify each relationship only once below, this does not imply directionality.

Guest: a *Guest* can occupy a *Room*, be a customer of the *HotelChain* and have *Reservations*.

Room: a *Room* belongs to a *Hotel*, can be allocated to *Reservations* and is of a *RoomType*.

Hotel: a *Hotel* belongs to a *HotelChain*, has *Reservations* and has *RoomTypes*.

HotelChain: see above.

Reservation: a *Reservation* is for a *RoomType*.

RoomType: see above.

We add entries to our glossary corresponding to the associations found in SAQ 2, as shown in Table 7.

Table 7 Adding associations to the glossary

Term	Category	Definition
<i>Guest–Room</i>	Association	A <i>Guest</i> can occupy a <i>Room</i> .
<i>Guest–HotelChain</i>	Association	A <i>Guest</i> can be a customer of the <i>HotelChain</i> (has a past, current or future reservation for some hotel of the chain).
<i>Guest–Reservation</i>	Association	A <i>Guest</i> can have a number of <i>Reservations</i> . These include current <i>Reservations</i> (i.e. those covering a future stay, but also including the period when the <i>Guest</i> is staying in the <i>Hotel</i>) and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.
<i>Room–Hotel</i>	Association	A <i>Room</i> belongs to a <i>Hotel</i> .

<i>Room–Reservation</i>	Association	A <i>Room</i> can be subject to a <i>Reservation</i> . A <i>Room</i> is subject to a <i>Reservation</i> from the time a <i>Guest</i> reserves it to the time that the <i>Guest</i> checks out.
<i>Room–RoomType</i>	Association	A <i>Room</i> is of a given <i>RoomType</i> .
<i>Hotel–HotelChain</i>	Association	A <i>Hotel</i> belongs to the <i>HotelChain</i> .
<i>Hotel–Reservation</i>	Association	A <i>Hotel</i> has a number of <i>Reservations</i> . These include current <i>Reservations</i> (i.e. those covering a future stay, but also including the period when the <i>Guest</i> is staying in the <i>Hotel</i>) and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.
<i>Hotel–RoomType</i>	Association	A <i>Hotel</i> contains a number of <i>RoomTypes</i> .
<i>Reservation–RoomType</i>	Association	A <i>Reservation</i> is for a given <i>RoomType</i> .

SAQ 3

Draw the initial class diagram for the domain model.

Answer

A conceptual model corresponding to our analysis so far is shown in Figure 3.

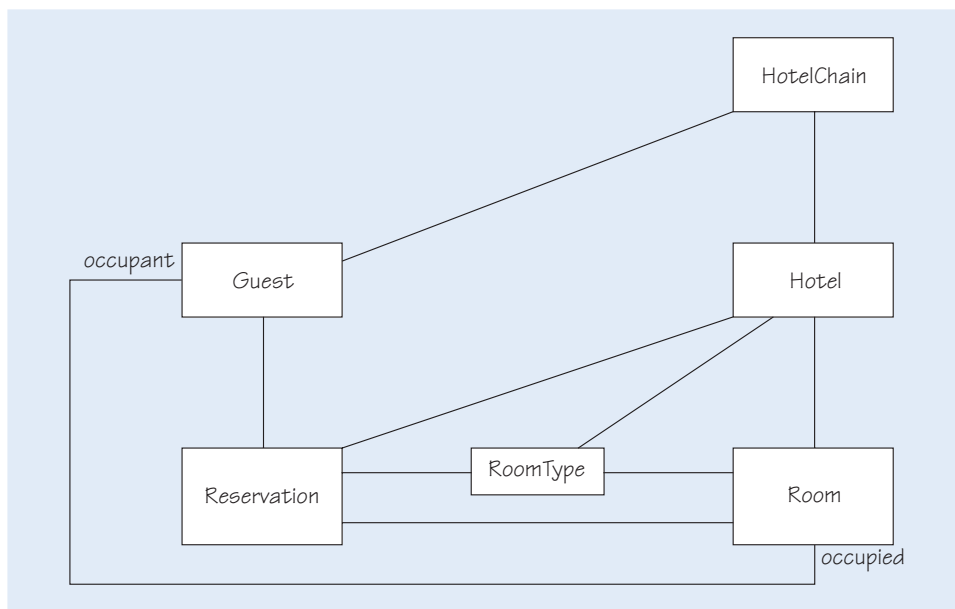


Figure 3 Conceptual model for the hotel system

We have also chosen to add a couple of role names, but this was not requested in the question. For the other associations, although we have quite a rich diagram, there are no pairs of classes that have more than one association directly between them, and the associations are mostly fairly obvious. In any case, they are identified in the glossary.

SAQ 4

- (a) Go through each step of each of the use cases for this iteration, checking that the relevant associations are present in the class diagram.
- (b) Add multiplicities to the diagram in Figure 3.

Answer

- (a) The relevant associations are indeed present.
- (b) The conceptual model in Figure 4 shows the multiplicities for each of the associations.

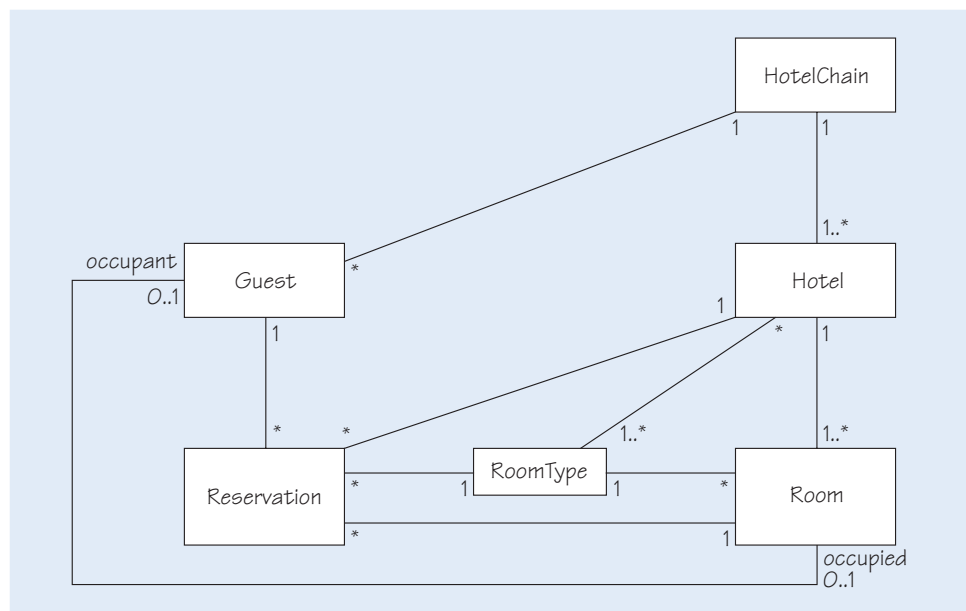


Figure 4 Conceptual model with multiplicities

SAQ 5

By working through the use cases for the current iteration step by step, add the attributes needed at this stage to the conceptual model in Figure 4, together with their corresponding data types.

Be aware that in analysis there are some basic data types, such as number, string, character, date and Boolean. Where none of these seems appropriate we use a **placeholder**, a made-up name such as *RoomKind*, employed temporarily, until we have sufficient information to decide what actual data type to use. This is an example of an attribute that could be represented as an enumerated type. We don't need the generality of a class to represent it.

Answer

The *make reservation* use case (UC1) suggests several attributes, as shown in Table 8.

Table 8 Attributes suggested by the *make reservation* use case

Step	Justification for adding attributes
2	Refers to selection of a desired hotel, presumably chosen by name: add the attribute <i>name</i> to <i>Hotel</i> .
6	Refers to identification, contact details and credit card details: add the attributes <i>name</i> , <i>addressDetails</i> , <i>creditCardDetails</i> and <i>id</i> (identity) to <i>Guest</i> (we will consider <i>creditCardDetails</i> to be an attribute for now but may later need to use a class).
7	Refers to a reservation number: add the attribute <i>number</i> to <i>Reservation</i> . In order to create a <i>Reservation</i> , we need to add <i>reservationDate</i> , <i>startDate</i> and <i>endDate</i> to <i>Reservation</i> . We also need to know the room number: add the attribute <i>number</i> to <i>Room</i> . We need to know the room type: add the attributes <i>kind</i> and <i>cost</i> to <i>RoomType</i> . We defer consideration of how these will be represented, but assume they will be single values rather than objects.

The other use cases suggested no attributes to us. The types that we have allocated to each attribute in the revised conceptual model of Figure 5 are just placeholders at this point in the development.

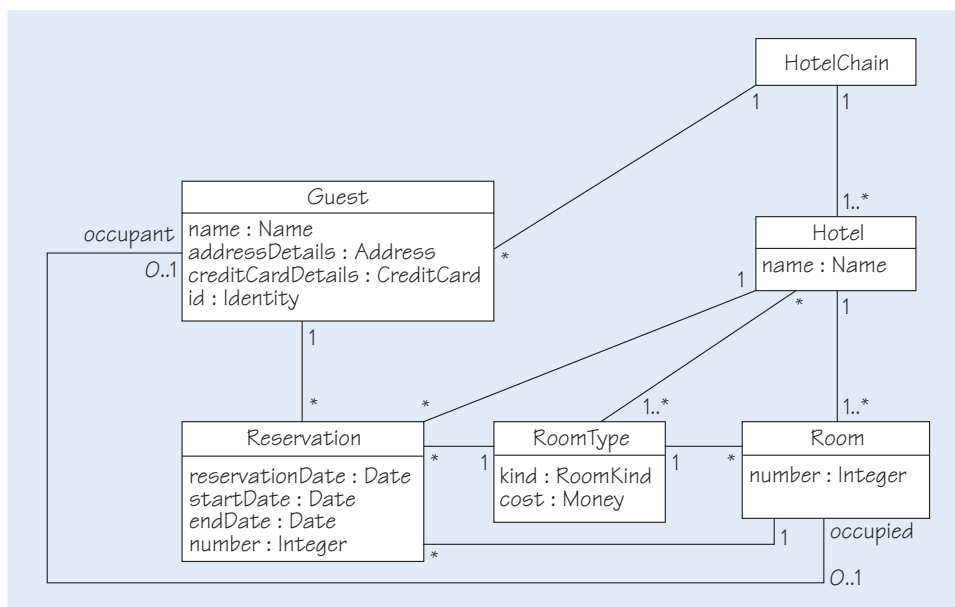


Figure 5 Conceptual model with attributes

We add the attributes identified in SAQ 5 to our glossary, as shown in Table 9. Further information will be added about the types later in the development when they become more than placeholders.

Attributes in a domain model are defined at a fairly high level of abstraction. Their types are, at this point, concepts from the problem domain that are considered to be basic as we do not want to specify how they are represented; examples are Address, CreditCard, Money. These attributes may become classes themselves in design and implementation.

Table 9 Adding attributes to the glossary

Term	Category	Definition
<i>name</i>	Attribute of <i>Hotel</i>	The name of a <i>Hotel</i> .
<i>name</i>	Attribute of <i>Guest</i>	The full name of a <i>Guest</i> .
<i>addressDetails</i>	Attribute of <i>Guest</i>	The address of a <i>Guest</i> .
<i>creditCardDetails</i>	Attribute of <i>Guest</i>	The credit card details of a <i>Guest</i> , sufficient to make a charge.
<i>id</i>	Attribute of <i>Guest</i>	A unique identifier for a <i>Guest</i> .
<i>reservationDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> was made.
<i>startDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> starts.
<i>endDate</i>	Attribute of <i>Reservation</i>	The date on which a <i>Reservation</i> ends (the last night).
<i>number</i>	Attribute of <i>Reservation</i>	A unique identifier of a <i>Reservation</i> .
<i>number</i>	Attribute of <i>Room</i>	The unique number identifying a <i>Room</i> in a <i>Hotel</i> .
<i>kind</i>	Attribute of <i>RoomType</i>	The details of a <i>Room</i> , in terms of the number of people it can accommodate and the sorts of beds it has (for example <i>double</i> , <i>twin</i> , <i>single</i> , <i>familyOf5</i> , <i>familyOf4</i> , ...).
<i>cost</i>	Attribute of <i>RoomType</i>	How much the <i>RoomType</i> costs per night. This is a simplification, as a <i>RoomType</i> may have different costs for different times of the year.

2.2 Summary of section

In this section you have seen how the concepts and associations in the **conceptual model** were selected. You have also seen how attributes were identified based on the initial problem statement and the requirements from Block 1 Unit 4.

Analysis

3

In analysis we take our conceptual model produced during domain modelling and add various pieces of information. First we need to decide whether all concepts correspond to analysis classes and whether there are any classes that need to be represented that were not identified as domain concepts. We can also revise our model as we think about its suitability for the task in hand. Furthermore, we may need to add association classes. The end result will be to produce an analysis model. We begin analysis by considering constraints on our conceptual model and then switch to a consideration of the system operations.

In this unit the approach we take to analysis and design differs from that in earlier units. The approach here is more systematic. During analysis we specify system operations corresponding to use cases. We do this by drawing object diagrams relating to the use case steps and then use these diagrams to draw up detailed preconditions and postconditions for system operations – these operations describe the behaviour of the system as a black box.

During design we allocate responsibilities to classes by analysing the pre- and postconditions for the system operations and adding operations that help check preconditions and fulfil postconditions. What we are essentially doing is refining a high-level specification by adding detail. Therefore, depending on the nature of particular projects, this approach may be more scalable than that in earlier units.

It is worth noting that this difference in approach is an example of where, although two pieces of development are using the same types of models, different processes are applied. This is a very common phenomenon in the software industry, where many variations exist. It would probably be an exception to find two companies following identical processes, since they usually adapt processes to suit their own situation. For example, different clients will want to work with software companies in different ways depending on their culture, and the culture and experience of the software company itself will be an important consideration.

Of course, the goal of a software process is to ensure software quality. This itself, however, may vary between industrial sectors since the key considerations in, say, the car industry would differ from those in, say, the financial sector.

3.1 Constraints

We will identify the constraints on the analysis model by considering association loops in the model. This is not an exhaustive exercise, but we will look at some loops that give us coverage of the model with only a small amount of overlap. Association loop invariants can be expressed from any class context that they touch; so there is a degree of choice in how they are expressed. We will also consider any constraints needed on attributes of classes.

For convenience Figure 5 is repeated as Figure 6.

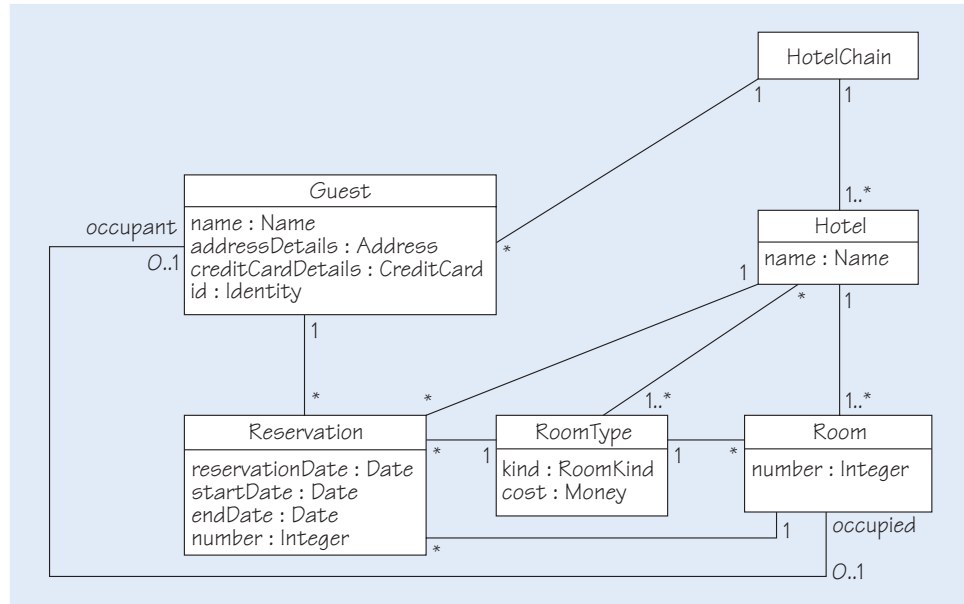


Figure 6 Conceptual model with attributes

Constraint for the loop *HotelChain*, *Hotel*, *Reservation* and *Guest*

We could have chosen any of the four classes to establish the constraint for this loop; we chose the *Hotel* class as a starting point and express it as follows.

The *Guests* associated with the *HotelChain*, to which the *Hotel* belongs, should include the *Guests* for whom the *Hotel* has *Reservations*. There is no inconsistency following a path from *Hotel* in one direction (to *Guest* through *HotelChain*) and following the path in the opposite direction (to *Guest* through *Reservation*). In OCL:

```

context Hotel inv:
  self.hotelchain.guest->includesAll(self.reservation.
    guest)
  
```

Constraint for the loop *Hotel*, *Room* and *RoomType*

The collection of *Rooms* in a *Hotel* is the same as the collection obtained by pooling the *Rooms* for all the *RoomType* objects linked to the *Hotel* concerned. We start again from class *Hotel* to express this association loop constraint. In OCL:

```

context Hotel inv:
  -- the Rooms of a Hotel are the Rooms of the RoomTypes
  of the Hotel.
  self.room = self.roomType.room
  
```

Remember that `self.guest` is a set. You may want to consult the OCL summary in Unit 5 subsection 6.5.

In fact, given the current multiplicities in the diagram, multiple *Hotels* can be associated with a given *RoomType*. This can be seen by the * multiplicity at the *Hotel* end of the *RoomType–Hotel* association. The original thinking here was that *RoomTypes* are shared across *Hotels*, but this leads to the situation where finding a *Hotel's Rooms* via its *RoomTypes* can result in our including *Rooms* from other *Hotels*. We will address this anomaly in Exercise 2 when we review the analysis model.

Constraint for the loop *Reservation*, *RoomType* and *Room*

The *Rooms* of a *RoomType* for a *Reservation* include the *Room* allocated to a *Reservation*:

```
context Reservation inv:
  self.roomType.room->includes(self.room)
```

Constraints for the loop *Reservation*, *Hotel* and *RoomType*

The *RoomTypes* of the *Hotel* of a *Reservation* include the *RoomType* of that *Reservation*:

```
context Reservation inv:
  self.hotel.roomType->includes(self.roomType)
```

Reservation also has a constraint on its attributes:

```
context Reservation inv:
  (self.endDate > self.startDate) and
  (self.startDate >= self.reservationDate)
```

Constraint for loop *Guest*, *Room* and *Reservation*

If a *Guest* occupies a *Room* in a *Hotel*, the *Room* is allocated to one of the *Reservations* of the *Guest*. However, there might not be any *Rooms* occupied by the *Guest*:

```
context Guest inv:
  self.room.notEmpty implies self.reservation.room-
  >includes(self.room)
```

Notice that this implication has a Boolean expression representing whether a guest occupies a room (or example `self.room.notEmpty`), as the constraint that the *Room* is allocated to one of the *Reservations* of the *Guest* makes sense only when it is true that the guest occupies a room.

Exercise 1

In the above conceptual models we have conflated the concepts of guest, reservation-maker and bill-payer. Is this realistic? Would the people playing these roles necessarily be the same person?

Solution

There are a number of problems with this simplification. In reality, a hotel should allow for the roles to be distinct. It should be possible to have guests recorded who neither pay a bill nor make a reservation. For example, partners and children may be staying in the hotel along with a person who reserves and/or pays the bill. In fact, the hotel would almost certainly want a complete picture of who is staying in the hotel for a range of reasons.

A separate issue is whether it should be possible for one person to reserve on behalf of another, or pay on behalf of another. On the one hand, allowing this gives more flexibility but on the other hand, having the credit card details of an actual occupier might make the sanction of charging for things such as damage stronger. There is also a modelling issue here because some people reserving rooms will also be guests and some not, and not all guests will make reservations – for example, partners as above. You might like to try drawing a simple class diagram to see the problems this poses for inheritance.

For the reasons identified in Exercise 1, we want to separate out the role of a guest from that of a person making reservations or paying bills.

Exercise 2

Are objects of the class *RoomType* shared across hotels? Does this present any problems?

Solution

The model permits the sharing of *RoomType* objects across hotels. This can be seen by the multiplicity at the *Hotel* end of the relevant association. However, this is problematic: if we try to find all the rooms a hotel has by navigating via the *RoomType* objects, we may include rooms from other hotels! In any case, there is an argument for making the *RoomType* objects associated with a *Hotel* unique to that hotel, as prices for a given type of room are likely to vary depending on location and local market conditions. This is what we decide to do.

Exercise 3

Does the model cover reservations for multiple rooms?

Solution

No. This can be seen from the multiplicities of 1 on the *Reservation–RoomType* and *Reservation–Room* associations.

The answers we came up with to the three exercises above suggest that our model should be adjusted. This iterative approach is to be expected in a real development, as stakeholders should be consulted when analysis produces artefacts that need to be checked against reality. We will also add an association class to allow us to record how many *Rooms* of a given *RoomType* are included in a given reservation. Our analysis class diagram is shown in Figure 7.

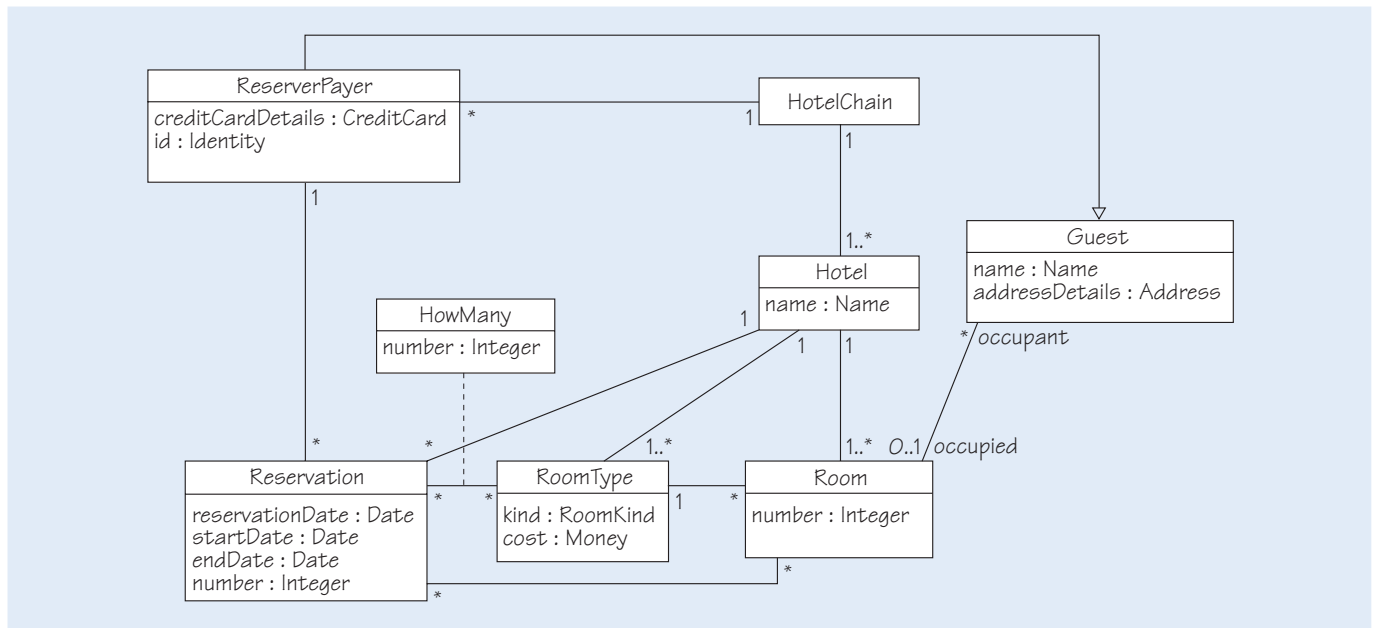


Figure 7 Analysis class diagram

Now we need to update the glossary. We need to remove the current entry for *Guest*, replacing it with two new entries for *Guest* and *ReserverPayer*, and add an entry for the association class *HowMany*. See Table 10. However, we will ignore the association class in the rest of the analysis and design, since it didn't form part of our use cases. In a real development we might clarify with the client whether to update the use cases.

Table 10 Updating concepts in the glossary

Term	Category	Definition
<i>Guest</i>	Concept	Somebody who might stay in a <i>Hotel</i> .
<i>ReserverPayer</i>	Concept	A <i>Guest</i> who reserves and/or pays for a <i>Room</i> .
<i>HowMany</i>	Concept	An association class to record the number of <i>Rooms</i> of a given <i>RoomType</i> in a <i>Reservation</i> .

We also need to remove the existing associations involving *Guest*, and represent the associations between *ReserverPayer*, *Reservation* and *HotelChain*, and the association between *Guest* and *Room*. See Table 11.

Table 11 Updating associations in the glossary

Term	Category	Definition
<i>Guest–Room</i>	Association	A <i>Guest</i> can occupy a <i>Room</i> .
<i>ReserverPayer–HotelChain</i>	Association	A <i>ReserverPayer</i> can be a customer of the <i>HotelChain</i> (has a past, current or future reservation for some hotel of the chain).
<i>ReserverPayer–Reservation</i>	Association	A <i>ReserverPayer</i> can have a number of <i>Reservations</i> . These include current <i>Reservations</i> (those covering a future stay, but also including the period when the <i>ReserverPayer</i> is staying in the Hotel) and past <i>Reservations</i> , which are recorded for reasons outside the current domain analysis.

Finally, we need to update the attribute entries to show how some of the attributes in *Guest* have been reallocated to the *ReserverPayer* class. We also record the attribute of the *HowMany* association class. See Table 12.

Table 12 Updating attributes in the glossary

Term	Category	Definition
<i>number</i>	Attribute of <i>HowMany</i>	The number of <i>Rooms</i> of a given <i>RoomType</i> in a <i>Reservation</i> .
<i>name</i>	Attribute of <i>Guest</i>	The full name of a <i>Guest</i> .
<i>addressDetails</i>	Attribute of <i>Guest</i>	The address of a <i>Guest</i> .
<i>creditCardDetails</i>	Attribute of <i>ReserverPayer</i>	The credit card details of a <i>ReserverPayer</i> , sufficient to make a charge.
<i>id</i>	Attribute of <i>ReserverPayer</i>	The identity of a <i>ReserverPayer</i> .

3.2 System operations

Now that we have developed the structural model of the domain (conceptual model) into a structural model of a software solution (analysis model), we can describe the effects of the use cases and detailed requirements. The main operations of the system can be identified from the use cases.

We start by defining the system operations, which describe the behaviour of the system as a black box. These operations define the external interface of the system – what any implementation will have to provide to the user, independently of how it is provided. (An implementation will probably provide a user interface for the interactions that take place in the scenarios of a corresponding use case.) An acceptable implementation is one that satisfies the specification of the system operations. Ultimately system operations can be used to construct tests of the system, as discussed in a later unit. In particular,

they allow us to test the application layer model independently of the user interface.

Use cases give an overview of the behaviour of the system; this may be enough in some cases. System operations add detail, and will be most useful for more complex use cases. For each use case, we have already defined a precondition and a postcondition; now that we also have a structural model, we can start detailing these as pre- and postconditions of operations, and specifying them in terms of the structure of the system. Initially, each use case will correspond to an operation for the whole system.

We will write specifications of operations in English, adopting the conventions that a precondition is described in the present tense and a postcondition is described in the future tense. The syntax for an operation specification is as follows:

```
context TypeName::operationName(parameter1 : Type1,
...) : Return_type
pre:
- - condition
post:
- - condition
```

An operation is defined within a *context* (*TypeName* is the class in which the operation is defined) and with a *name* (*operationName*), a series of *parameters* and their types, and the type of the *returned value* (when there is a returned value); this is the operation's **signature**.

The precondition is a constraint on the parameters of the operation and on the properties (values of attributes and links) of the context; the precondition has to be true before the operation is executed. The postcondition is a constraint on the state after the operation is executed, in relation to the state before the operation was executed; these states are represented by the properties of the context and the returned object (if any).

We will specify system operations for three of our use cases, omitting those for *check out guest*. Part of our rationale is that tackling the first three use cases is sufficient both to illustrate the overall approach and to establish the architecture of the system. For the same reasons, we will not address all the use cases to the same level of detail.

3.3 Specifying operations

As with use cases, there is no fixed way of deciding the level of granularity at which system operations should be defined. The main criterion is that they should be useful in understanding the effects of a business process on the system. We opt, initially, to define a single system operation for each use case corresponding to a main business process. Breaking the system operation into a set of more detailed operations will be done afterwards, when we make decisions about the allocation of responsibilities to classes.

At this point in the design it is convenient to allocate all the identified system operations to the **system class**. The system class in our example is *HotelChain*, which represents the whole system and has a single instance; it will coordinate the behaviour of other objects in the system in order to carry out the operations. A system class is just a convenient device to help us make progress with the system design. However, introducing such a class at this stage does not mean that it will still be present in the final system design, or that other classes will not be introduced to orchestrate part of the system behaviour.

In fact, in large systems the responsibility of the system class is typically distributed among a number of separate classes. At this point, however, it is a useful abstraction: a single point in the system from which all behaviours can be orchestrated and a single point of entry for all interactions of the system with the environment. The system class has one defining characteristic: there is only one instance of such a class. The system class keeps track of objects in the system via associations – either directly, as is the case for the class *Hotel* or indirectly through the associations that this class has with other classes in the system.

Table 13 reminds us of the description of the *make reservation* use case, as given in Table 1 of Block 1 Unit 4.

In order to specify the corresponding system operation, we need to look at the class diagram as defined in Figure 7. Although the class diagram allows for more than one room type for each reservation, the use case described above deals only with one reservation, and that reservation deals with only one room type. To keep it simple, we will specify a system operation that corresponds to the use case as defined above.

An operation to achieve the goal of this use case will need to have as its inputs the *Hotel*, the start and end dates, the *RoomType* and the *ReserverPayer* responsible for the reservation and its payment.

Table 13 The *makeReservation* system operation

Identifier and name	UC1 <i>make reservation</i>
Initiator	<i>Reserver or Receptionist</i>
Goal	A room in a hotel is reserved for a guest.
Precondition	None (that is, there are no conditions to be satisfied before carrying out this use case).
Postcondition	A room of the desired type will have been reserved for the guest for the requested period, and the room will no longer be free for that period.
Assumptions	The expected initiator is a reserver or receptionist, using a web browser to perform the use case. The guest is not already known to the hotel's software system (see step 5). The receptionist (if an actor) communicates details to the client when making a booking.
Main success scenario	
1	The reserver/receptionist makes a reservation request.
2	The reserver/receptionist selects the desired hotel, dates and type of room.
3	The hotel system provides the availability and price for the request; that is, one or more offers are made.
4	The reserver/receptionist accepts the offer.
5	The hotel system requests identification and contact details.
6	The reserver/receptionist provides identification and contact details of the guest for the hotel system's records, as well as credit card details.
7	The hotel system creates a reservation and gives it a number.
8	The hotel system allocates a room to the reservation.
9	The hotel system reveals the reservation number to the reserver/receptionist.
Extensions	
3.a.1	<i>No availability in this hotel.</i> The hotel system offers alternatives at a different hotel or hotels within a limited radius.
3.b.1	<i>No availability in this hotel or in any other hotels within a limited radius.</i> The hotel system informs the reserver and terminates the use case.
4.a.1	<i>Offer not accepted.</i> The hotel system terminates the use case.
6.a.1	<i>Guest already on record.</i> The guest is identified and (if needed) their record is updated and the scenario continues at step 7.

SAQ 6

Consider the *make reservation* use case described above and write the signature for a corresponding operation.

Answer

```
context HotelChain::makeReservation(hotel : Hotel,
startDate : Date, endDate : Date,
roomType : RoomType, reserverPayer : ReserverPayer)
```

We should now define the pre- and postconditions for the corresponding operation. To help specify them, let us look at two snapshots illustrating the

before and after states for the execution of this operation. Figure 8 illustrates a sample state of the system before the *makeReservation* operation is executed. Credit card details have been omitted from *ReserverPayer* objects as we have deferred the decision about their representation.

From now on, we will use *roomType* to refer to an instance of the class *RoomType*, and so on.

SAQ 7

Consider the snapshot in Figure 8 and draw a snapshot after the execution of the system operation *makeReservation*(*hotel1*, "8/4/2013", "10/4/2013", *roomType2*, *reserverPayer3*).

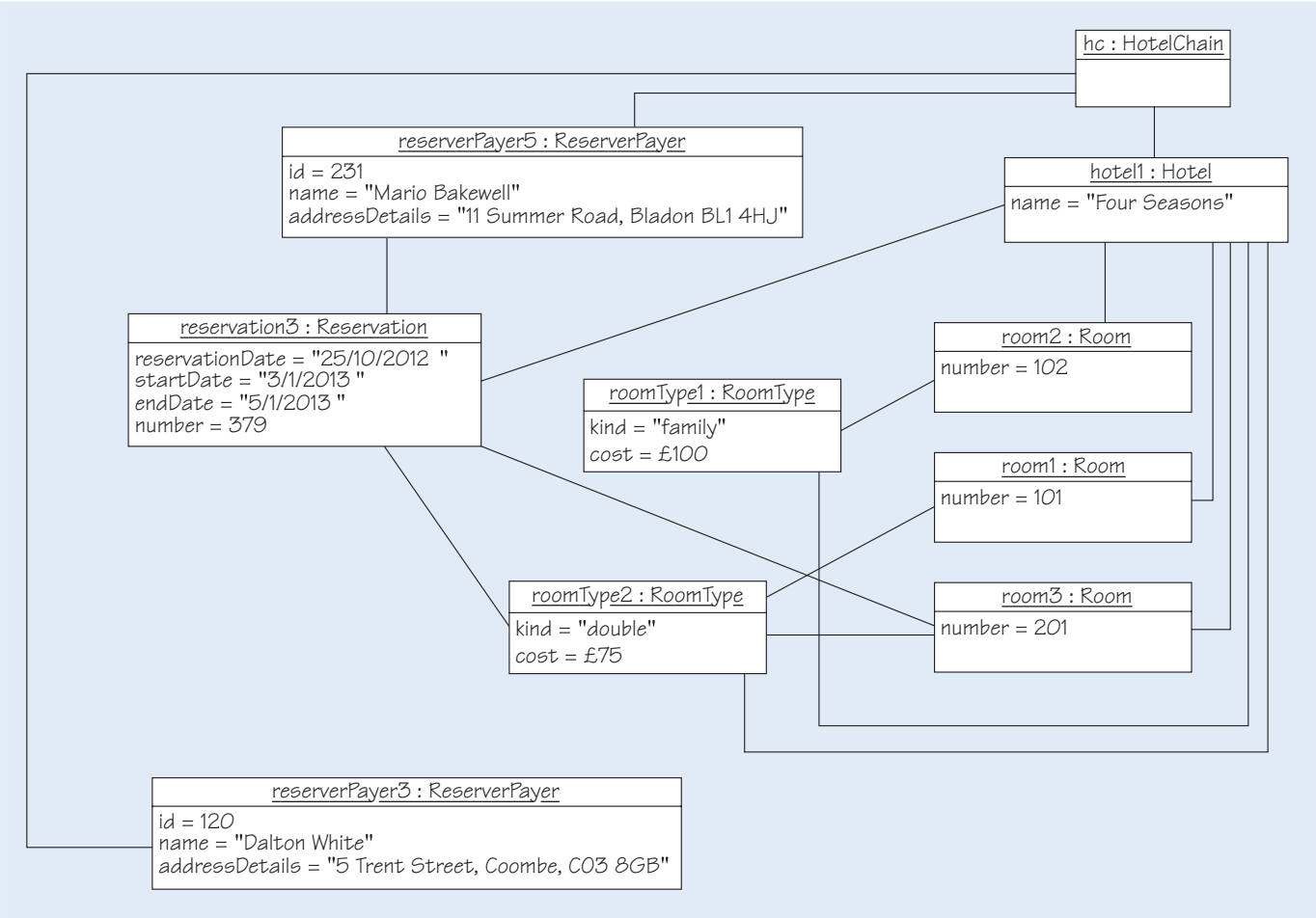


Figure 8 Snapshot illustrating a sample state of the system before the *makeReservation* operation is executed

Assume that the allocated room is *room1* (with room number 101).
Figure 8 is available on your module website.

Answer

Our answer is shown in Figure 9.

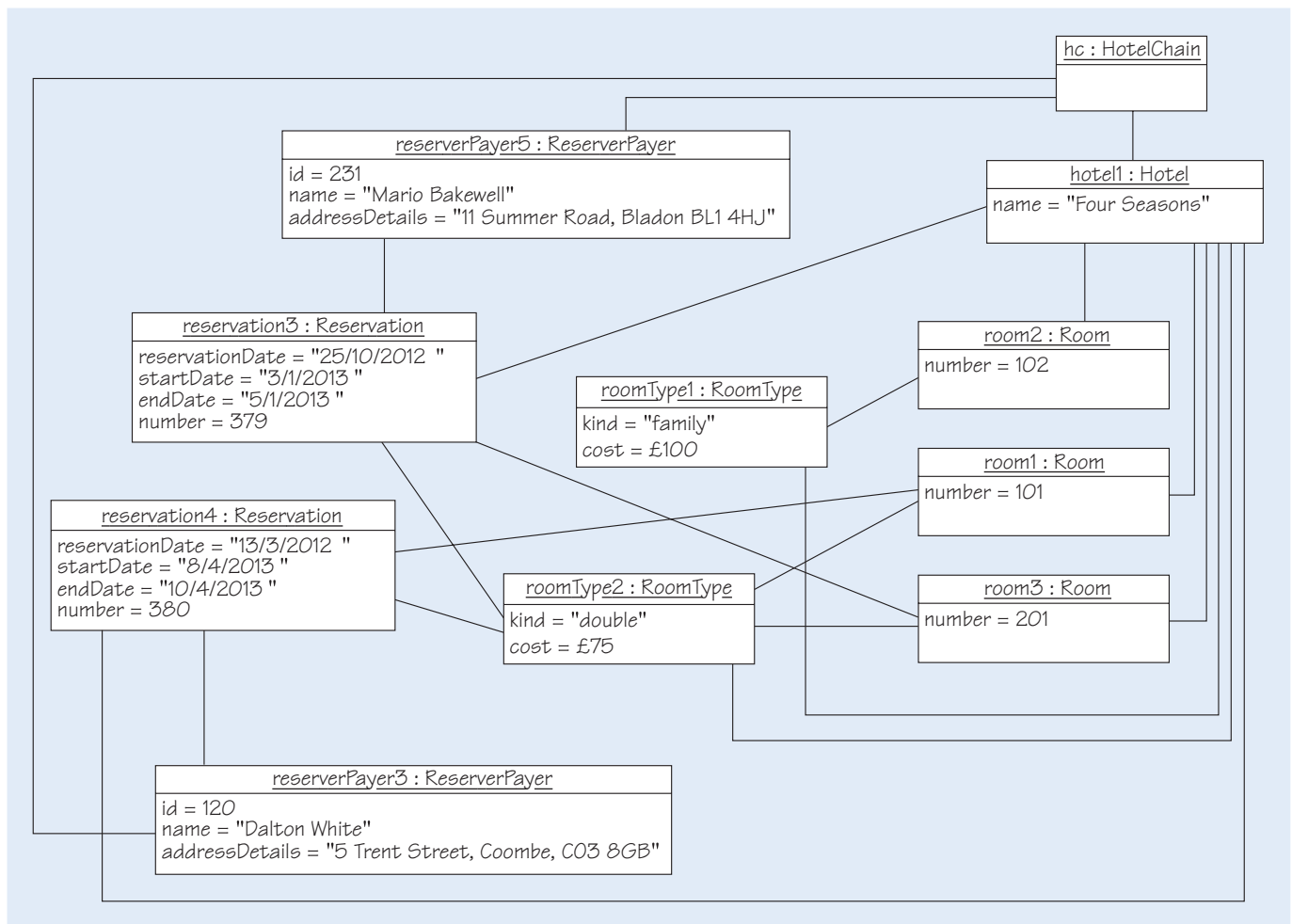


Figure 9 Snapshot illustrating the state after another reservation has been made: *makeReservation(hotel1, "8/4/2013", "10/4/2013", roomType2, reserverPayer3)*

The whole operation can now be specified by showing the pre- and postconditions. Let us look first at the preconditions.

```
context HotelChain::makeReservation(hotel : Hotel,
startDate : Date, endDate : Date, roomType : RoomType,
reserverPayer : ReserverPayer)
```

```
pre
```

- - startDate is before endDate
- - startDate is not in the past
- - roomType is one of hotel's RoomTypes

We have followed the convention (mentioned above) of writing the preconditions in the present tense. Also we have started to add more detail than we had in the use case, as we now have a class diagram with the initial representation of the software elements, and we can see how these are affected by the use case.

The model had a constraint about the dates of a reservation, and we need to impose this so that the *makeReservation* operation complies with the model. Also, the hotel must have the required room type. The structural model shows an association between *Hotel* and *RoomType*, but not all hotels have rooms of all types. Therefore, the precondition needs to stipulate that the hotel has rooms of the requested *roomType*.

We should now look at the snapshots before and after the operation is executed, as shown in Figures 8 and 9, and express the changes in the postcondition. We start by looking only at the changes that take effect when the main scenario of the use case is followed. The exception related to availability (we are addressing availability in one hotel only) will be dealt with later.

In writing the signature of the system operation with a parameter *reserverPayer*, we assume that at some point in the interaction either a new *reserverPayer* was created (as assumed in the main scenario, for example in a login interaction) or an existing *reserverPayer* was retrieved (as in exception 6.a.1 of the use case). Of course the details of an existing *reserverPayer* may need updating but we assume this is outside the scope of the current iteration.

From now on, for the rest of this unit, we assume there is an object called *HotelChain* representing the hotel chain.

```
context HotelChain::makeReservation(hotel : Hotel,
startDate : Date, endDate : Date, roomType : RoomType,
reserverPayer : ReserverPayer)
```

```
post:
```

- - a new Reservation object, reservation, will have been created (object creation)
- - hotelChain will be linked to reserverPayer
- - hotel will be linked to reservation
- - reserverPayer will be linked to reservation
- - reservation will be linked to roomType
- - reservation will be linked to a previously free Room

We now need to consider how to deal with the exceptions related to the availability of a room and to the guest being known to the hotel.

Room availability

At this stage in the analysis it is inappropriate to consider the details of how the availability of the requested room type will be represented (for example as an attribute of *Room* or by derivation from the reservations). However, we do need to decide who has the responsibility for recording the information about it. For the moment, let us assume that each hotel will know about the availability of a room of a specific type. We represent this by an operation assigned to *Hotel*: given a room type and a range of dates, it will determine the availability of a room of that type. At this point, we are not dealing with offers in other hotels or offers of rooms of other types.

The links are bidirectional; in analysis no decisions have been taken on the direction of associations.

SAQ 8

Discuss how you would apply GRASP to assign *Hotel* the responsibility for finding out the availability of a room of a specified type.

Answer

GRASP Expert would apply here in assigning the responsibility to the *Hotel* class, as *Hotel* knows about all its rooms, room types and reservations.

Here we see a blur between when analysis stops and design starts: in specifying system operations we need to take design decisions about responsibilities.

SAQ 9

Consider the operation that would determine the availability in a hotel of a room of a specific type, for given start and end dates, and write its signature.

Answer

```
context Hotel::available(startDate : Date,
  endDate : Date, roomType : RoomType) : Boolean
```

A reservation in a hotel can be made only if availability is confirmed. We therefore change the signature of *makeReservation* to return a string indicating whether a reservation has already been made and, if so, containing the reservation number:

```
context HotelChain::makeReservation(hotel : Hotel,
  startDate : Date, endDate : Date,
  roomType : RoomType, reserverPayer : ReserverPayer)
  : String
```

Completing the *makeReservation* operation

Now that we have addressed exceptions in the use case, we can complete the specification of the *makeReservation* operation as follows:

```
context HotelChain::makeReservation(hotelName : Name,
  startDate : Date, endDate : Date, roomType : RoomType,
  reserverPayer : ReserverPayer) : String

pre:
- - startDate is before endDate
- - startDate is not in the past
- - roomType is one of hotel's roomTypes

post:
- - if available(startDate, endDate, roomType)
  executed in hotel returned true then
- - a new Reservation object, reservation, will have
  been created (object creation)
- - hotelChain will be linked to reserverPayer
- - hotel will be linked to reservation
- - reserverPayer will be linked to reservation
- - reservation will be linked to roomType
```

- - reservation will be linked to a previously free Room
- - a string indicating the reservation number will have been returned
- - otherwise, a string indicating failure will have been returned

Look at Table 14 and recall the description of the *cancel reservation* use case in Table 2 of Block 1 Unit 4.

Table 14 The *cancelReservation* system operation

Identifier and name	UC2 <i>cancel reservation</i>
Initiator	<i>Receptionist</i> or <i>Reserver</i>
Goal	A guest’s reservation of a room at a hotel is cancelled.
Precondition	There is an active reservation (in the future and not cancelled) for a room at a hotel for a guest with a correct reservation number.
Postcondition	The reservation will have been cancelled, the allocated room will be freed, and a charge to the credit card will have been made if the cancellation was initiated within 24 hours of the start of the reservation.
Assumptions	The expected initiator is a receptionist or reserver.
Main success scenario	1 The receptionist/reserver requests the cancellation of a reservation. 2 The receptionist/reserver enters the reservation number, or the dates, name and address. 3 The hotel system retrieves the corresponding reservation. 4 The hotel system frees the allocated room. 5 The receptionist/reserver confirms the cancellation. 6 The reservation is cancelled and the hotel system informs the receptionist/reserver of the cancellation.
Extensions	6.a.1 If the cancellation was initiated within 24 hours of the beginning of the reservation, the hotel system charges the credit card whose details are kept in the reservation.

Exercise 4

Consider the *cancel reservation* use case described in Table 14.

- (a) Assume the situation as given in the snapshot in Figure 9 and draw a snapshot of the situation after the cancellation of the reservation with reservation number 379.
- (b) Write the signature of the corresponding system operation *cancelReservation*.
- (c) Write its pre- and postconditions.

Solution

- (a) Our after-cancellation snapshot is shown in Figure 10.

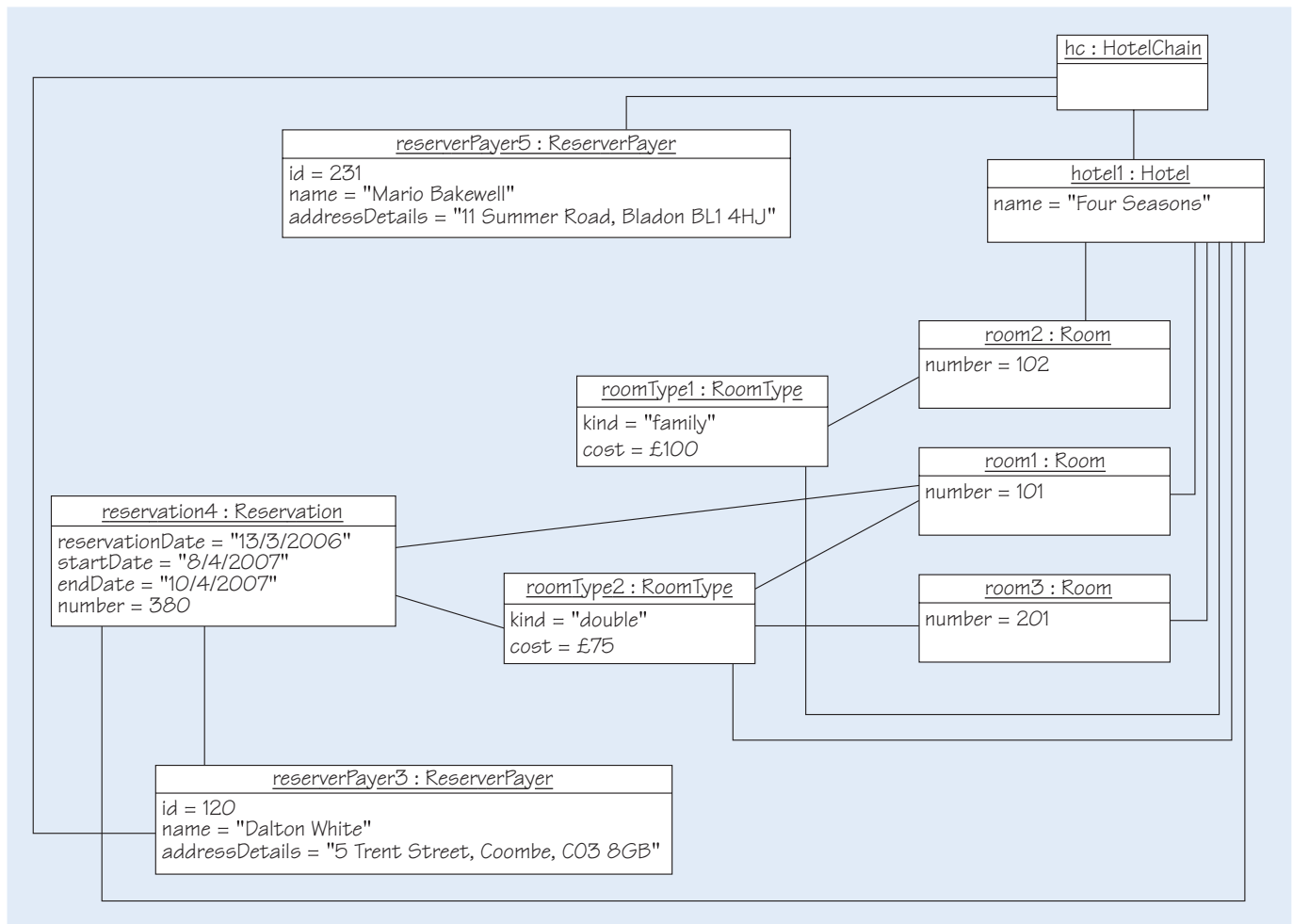


Figure 10 Snapshot illustrating a sample state after *cancelReservation* is executed

(b) The signature for *cancelReservation* is as follows:

context

```
HotelChain::cancelReservation(hotel : Hotel,
reservation : Reservation)
```

(c) The specification of pre- and postconditions is as follows:

context

```
HotelChain::cancelReservation(hotel : Hotel,
reservation : Reservation)
pre:
reservation is linked to hotel and reservation.
startDate is not in the past
post:
reservation will no longer be linked to hotel, to the
reserverPayer who made it, nor to the allocated room or
roomType
```

For this iteration, when a reservation is cancelled, the *ReserverPayer* that made the reservation is still kept. It is useful to have their details if they book again, and they may be a regular customer who will not want the inconvenience of supplying their details repeatedly.

Look at Table 15 and recall the description of the *check in guest* use case from Table 3 of Block 1 Unit 4.

Table 15 The *checkInGuest* system operation

Identifier and name	UC3 <i>check in guest</i>
Initiator	<i>Receptionist</i>
Goal	A guest takes up a reservation and occupies a room at the desired hotel.
Precondition	There is a room (of the desired type) reserved for the correct reservation number, for today, and the guest can pay for the room.
Postcondition	The guest will have been allocated to a room for the period identified in the reservation, and a bill will have been opened for the duration of the stay.
Assumptions	The guest is already known to the hotel system. The hotel is confident that the guest can pay. For example, the guest has a valid credit card.
Main success scenario	1 The guest provides a reservation reference number or the dates and name and address to the receptionist. 2 The receptionist enters the reference number or the dates and name and address to find the reservation. 3 The hotel system retrieves the details of the requested reservation. 4 The receptionist confirms the details of the room type and duration of the stay with the guest. 5 The hotel system allocates the reserved room to the guest. 6 The hotel system opens a bill for the guest. (It could be that there is a separate billing package, which must be notified upon check in.) 7 The receptionist issues a key to the guest.
Extensions	None

The use case as defined above allows for only one check in at a time. The class diagram treats *ReserverPayers* and other *Guests* differently; we will therefore look at the system operation for each case separately. We start by looking at a system operation that checks in a *ReserverPayer*.

Exercise 5

Consider the *check in guest* use case described above.

- (a) Assume the situation as given in the snapshot in Figure 9 and draw a snapshot of the situation after the check in for the reservation with reservation number 379.
- (b) Now assume that, for the same reservation and allocated room, the reserver’s partner is also to check in. Let us say that the partner is ‘Charlie Watson’ and lives at the same address. Draw a snapshot of the situation after execution of the check in.
- (c) Write the signature of the corresponding system operation, *checkInGuest*.

(d) Write its pre- and postconditions.

Solution

(a) Our after-check-in snapshot for a *ReserverPayer* is shown in Figure 11.

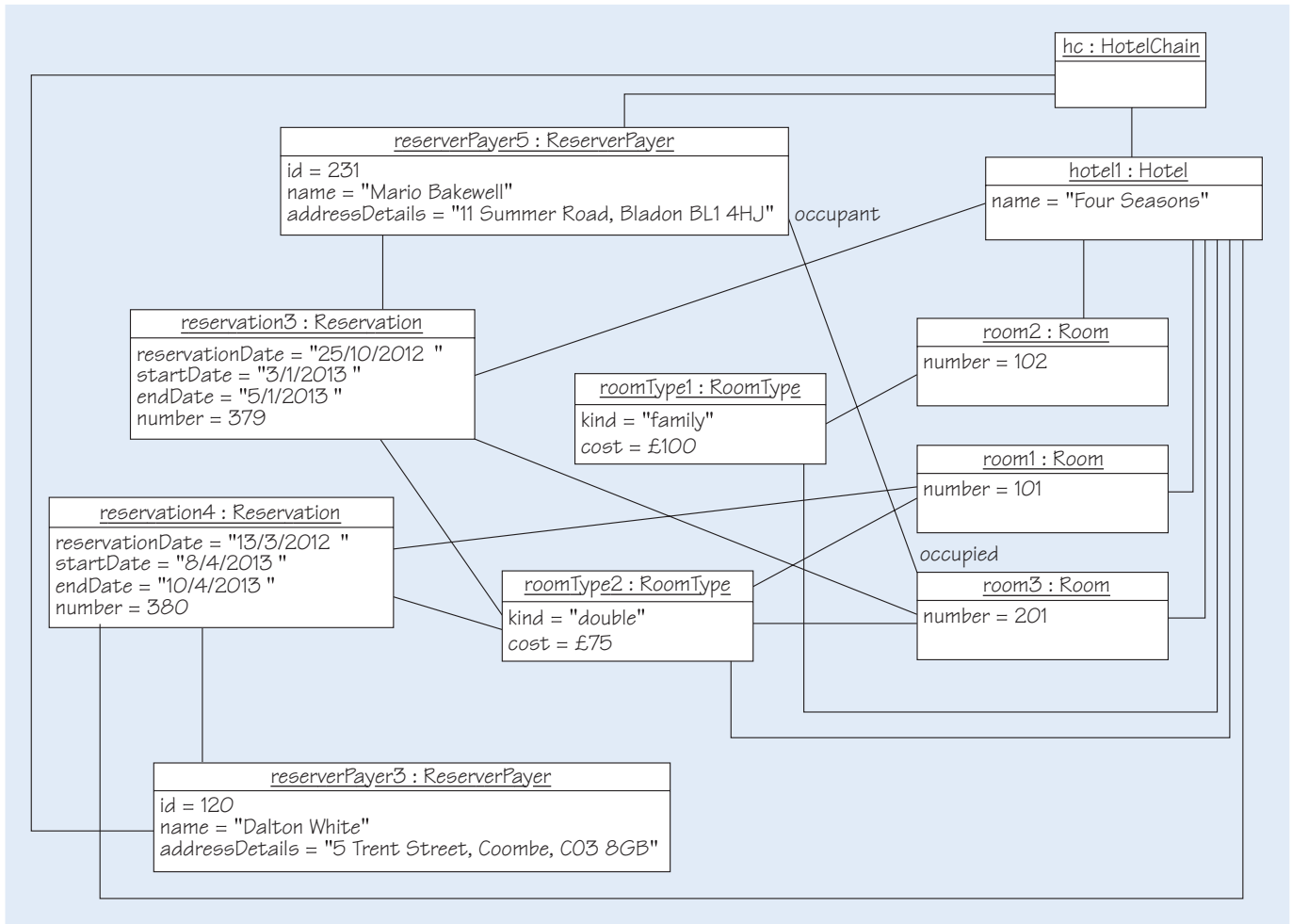


Figure 11 Snapshot illustrating a sample state of the system after a *ReserverPayer* checks in

(b) Our after-check-in snapshot for another *Guest* is shown in Figure 12.

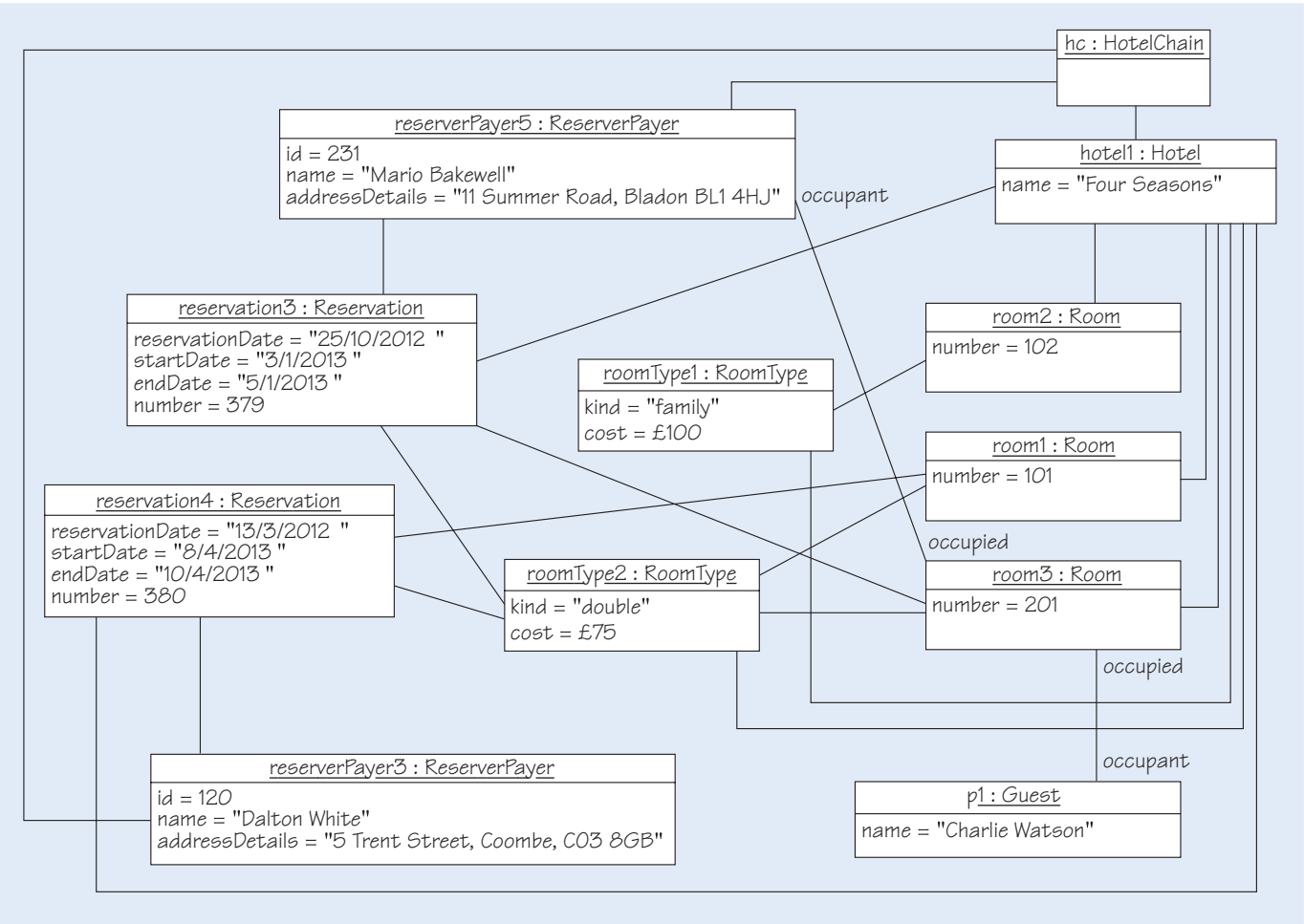


Figure 12 Snapshot illustrating a sample state of the system after another *Guest* checks in

(c) The signature for *checkInGuest* is as follows:

context

```
HotelChain::checkInGuest(hotel : Hotel,  
reservation : Reservation, guest : Guest)
```

(d) The specification of pre- and postconditions is as follows:

context

```
HotelChain::checkInGuest(hotel : Hotel,  
reservation : Reservation, guest : Guest)  
pre:  
reservation is linked to hotel  
- - a Room is linked to reservation, and the Room is  
of the RoomType for reservation  
post:  
Room linked to reservation will be linked to guest
```

3.4 Summary of section

In this section you have seen how to place constraints on the structural model. You have also seen how to relate the specifications of system operations to the use cases, and the effects on the structural model of the system.

4

Design – assigning responsibilities

In design, the analysis model defined during requirements analysis is refined in a number of ways to become the design model.

- More responsibilities are assigned to classes, by applying basic design principles and by analysing the pre- and postconditions of system operations.
- The navigation of associations is established, based on the way objects in the system need to collaborate to realise the system operations.
- Operations are added to the classes, based on the assigned responsibilities.

For the remainder of this unit, we will use the class diagram for the *HotelChain* introduced in subsection 3.1 as Figure 7 and reproduced here, with the addition of the *HotelChain* operations, as Figure 13.

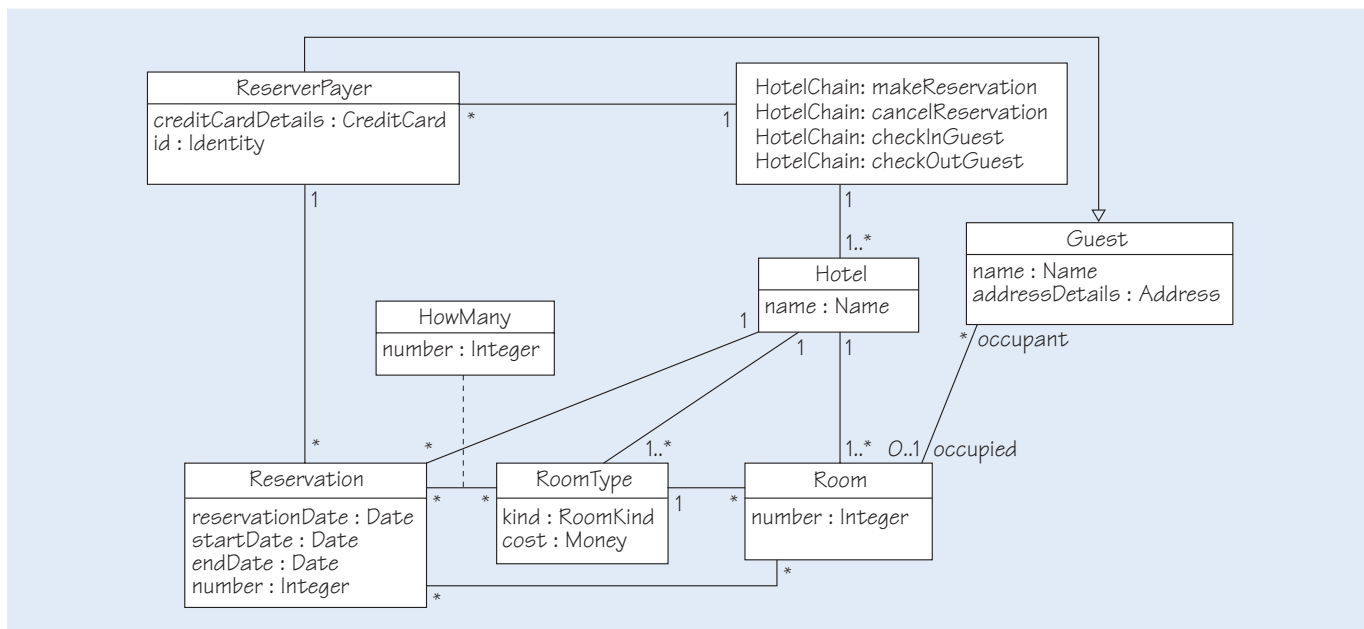


Figure 13 *HotelChain* class diagram

So far, we have assigned operations to the system class *HotelChain*. The system class has the responsibility for orchestrating the behaviour of the objects in the system and keeping track of them.

4.1 Value identities

So far we have been distinguishing between objects without saying anything about how this is done: one hotel is distinguished from another because the hotels have different object identifiers. We need a more meaningful way to distinguish between objects, one that can be used in software.

As you saw in Unit 5, many objects can be distinguished from each other by the values of one or more of their attributes: a *Reservation* by its reservation *number*, a *ReserverPayer* by its *id*, and so on. This form of identity is known as **value identity**. In Unit 5 you saw how to represent value identities on a class diagram by using the UML notation of qualified association: in a qualified association, a qualifier explicitly indicates the set of object properties that represents the identity of objects at the opposite end of the association.

In Figure 14 the class diagram has been redrawn to show value identities for the various classes: hotels are distinguished by *name*, rooms in a hotel by *number*, reservations by *number*, and *ReserverPayers* by *id*.

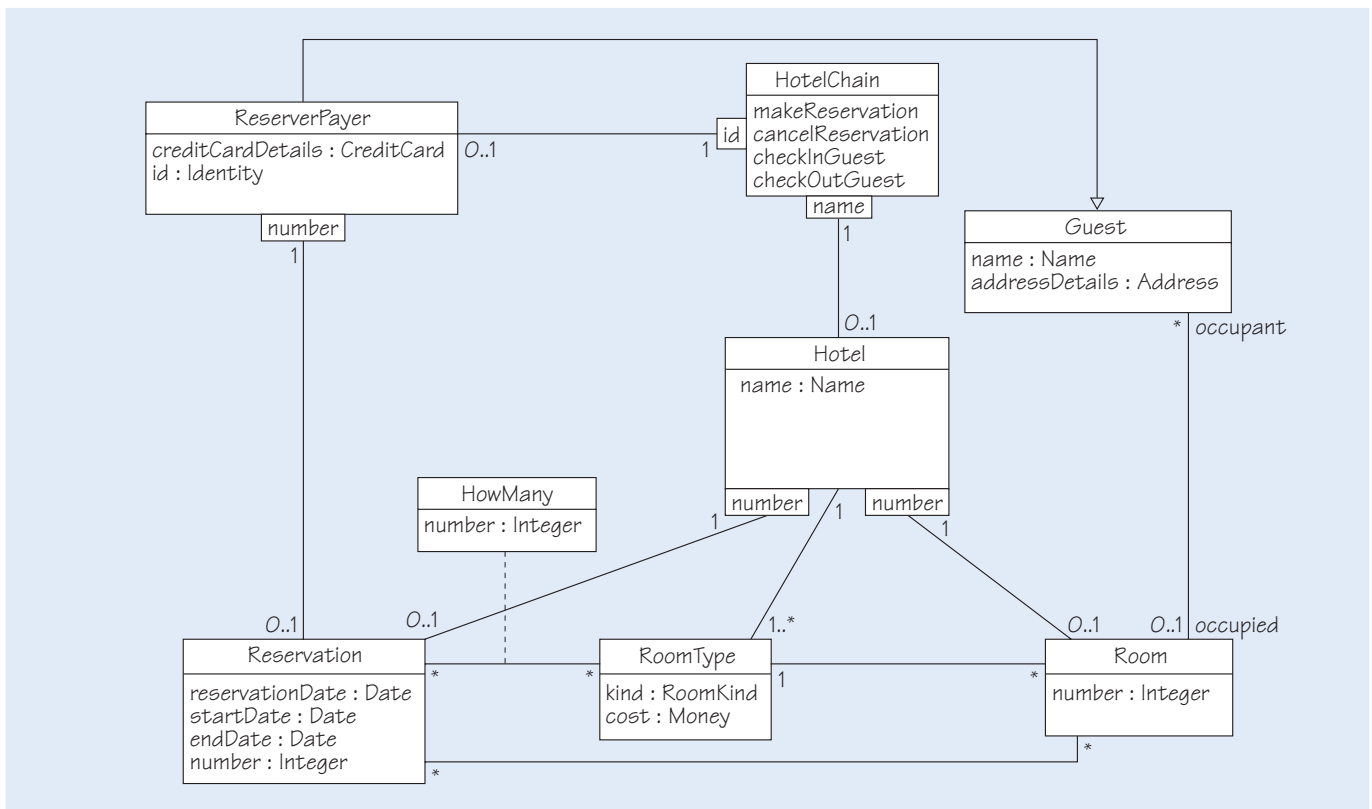


Figure 14 *HotelChain* class diagram with qualified associations

4.2 Verifying and fulfilling preconditions and postconditions

In subsection 3.3 we described system operations by giving their preconditions and postconditions. The precondition states what has to be true before the execution of the operation; the postcondition states what has to be true after the execution of the operation. By using pre- and postconditions, we can rigorously describe the conditions under which the service provided by an operation can be used, and the result obtained after the service is provided.

This captures the essential detail about *what* services are provided without needing to state *how* they are going to be provided.

Writing pre- and postconditions in a rigorous way also has the advantage of making us think about the requirements, clarifying any ambiguity and rectifying any omissions. A rigorous postcondition is also the first step for verification and testing. A properly implemented system will successfully make the postcondition of an operation true whenever that operation is executed, provided that the matching precondition is satisfied.

Verifying preconditions

Preconditions are verified, and postconditions fulfilled, by objects interacting with each other in a predefined way that may result in objects, and links between those objects, being created or destroyed. To describe how these objects need to interact to verify and fulfil the pre- and postconditions of each operation, we must first identify the objects that are involved and see how those objects and the links between them are affected.

Verifying preconditions may require consideration of:

- whether particular objects exist
- whether particular links exist
- the values of particular attributes.

Fulfilling postconditions may involve:

- creating particular objects
- creating particular links
- modifying the values of particular attributes
- returning particular values.

Let us look at the *makeReservation* operation as specified in subsection 3.3.

```
context HotelChain::makeReservation(hotelName : Name,
startDate : Date, endDate : Date, roomType : RoomType,
reserverPayer : ReserverPayer) : String

pre:
- - startDate is before endDate
- - startDate is not in the past
- - roomType is one of hotel's roomTypes

post:
- - if available(startDate, endDate, roomType)
executed in hotel returned true then
- -   a new Reservation object, reservation, will have
been created (object creation)
- -   hotelChain will be linked to reserverPayer
- -   hotel will be linked to reservation
- -   reserverPayer will be linked to reservation
- -   reservation will be linked to roomType
```



```

- - reservation will be linked to a previously free
Room
- - a string indicating the reservation number will
have been returned
- - otherwise, a string indicating failure will have
been returned

```

We start by focusing on the precondition of this operation. In order to identify the parties involved in the collaboration to verify the precondition, we need to identify:

- who initiates the collaboration (this is the actor of the corresponding use case; in the case of *make reservation* it is a *Reserver*)
- the object that orchestrates the collaboration (this is by definition the system object)
- other objects in the system with which the system object needs to collaborate in order to verify that the precondition of the operation is satisfied (to be able to identify these objects we need to look at the operation's definition).

By looking at the arguments and the precondition of the operation, we can see that the objects involved are the hotels, rooms and *RoomType* objects in the system. The hotels are linked to the system object through a qualified association, capturing the fact that the system object can perform a mapping between the value identity – the *name* of the hotel in this case – and the identity of the corresponding object. The details of this mapping will be addressed at implementation, when features of the target programming language will be used to realise the mapping. The mapping need not be considered any further in design: all the information needed is already captured in the qualified association.

Similarly, *Room* objects are linked to *Hotel* objects by qualified associations with the qualifier *number*, and so on.

For now, we adopt the convention that verifying the precondition is initiated by a message between the actor that triggers the corresponding use case and the system object, and that the message name is given by the system operation name prefixed by 'can'. In the final system, the actor–system interaction will occur through a user interface, which we have not considered yet. Therefore, this message is only an abstraction of what will actually happen when the system is completed.

We will now access objects by their value identities where qualified associations have been added to the class diagram in Figure 14.

```

context HotelChain::canMakeReservation
(hotelName : Name, startDate : Date, endDate : Date,
roomType : RoomType) : Boolean
post:
- - true will be returned if
- -   startDate is before endDate
- -   startDate is not in the past

```

- - there is a hotel with Name *hotelName*, and it has rooms of *roomType*)
- - otherwise, false will be returned

Exercise 6

Consider the operation *cancelReservation* rewritten now with value identities:

```
context HotelChain::cancelReservation
(hotelName : Name, reservationNumber : Number)

pre:
- - the Reservation with reservationNumber is linked
to the Hotel with hotelName
- - the start Date for the Reservation with
reservationNumber is not in the past

post:
- - the Reservation with reservationNumber will no
longer be linked to the Hotel with hotelName, to the
reserverPayer who made it, nor to the allocated room
```

Define an operation to verify its precondition.

Solution

The following operation will verify the precondition of *cancelReservation*:

```
context HotelChain::canCancelReservation
(hotelName : Name, reservationNumber : Number)
: Boolean

post:
- - true is returned if
- - there is a Hotel with hotelName, and
- - there is a reservation for that Hotel with
number reservationNumber and
- - the start Date for the Reservation with
reservationNumber is not in the past
```

The interactions required to process the *canMakeReservation* and *canCancelReservation* operations are sufficiently simple to be left until implementation.

Exercise 7

Consider the operation *checkInGuest* rewritten now with value identities. Be aware that check in is done guest by guest, and there is a slight difference between the way a *ReserverPayer* checks in and the way any other *Guest* checks in. *ReserverPayers* should already exist in the system, but information for other *Guests* is recorded only when they check in.

```
context HotelChain::checkInGuest(hotelName : Name,
reservationNo : Integer, roomNo : Integer, name : Name,
address : Address)
```

```
pre:
```

- - the Reservation with reservationNo is linked to the Hotel with hotelName
- - the room with roomNo is linked to the Reservation, and the Room is of the RoomType for the Reservation

```
post:
```

- - if name is not that of the ReserverPayer of the Reservation with reservationNo, then a new Guest will be created, and its attributes set; this new Guest or, otherwise, the ReserverPayer will be linked to the Room with roomNo

Define an operation to verify its precondition.

Solution

The following operation will verify the precondition of *checkInGuest*:

```
context HotelChain::canCheckInGuest(hotelName : Name,
reservationNumber : Number, roomNo : Integer) : Boolean
```

```
post:
```

- - true is returned if there is a Hotel with hotelName, and there is a Reservation with reservationNo for that hotel, and the Room with roomNo is of the roomType for the Reservation

Fulfilling postconditions

We can now consider the collaborations to satisfy the postcondition of a system operation. Once again, we will consider the *makeReservation* operation from *HotelChain* in order to illustrate the technique.

SAQ 10

Which parties are involved in satisfying the postcondition of a system operation? How can you identify them?

Answer

The parties are:

- the actor that initiates the corresponding use case
- the system object, which orchestrates the collaboration
- a set of objects within the system that are required to satisfy the postcondition; these can be identified by looking at the operation specification.

Consider once again the operation *makeReservation*, now using value identities. We have modified the postcondition to take into account the need to create a new *ReserverPayer* object in case it was not known to the hotel.

```

context HotelChain::makeReservation(hotelName : Name,
startDate : Date, endDate : Date,
roomType : RoomTypeKind, name : Name,
address : Address, creditCardDetails : CreditCard)
: String

pre:
- - startDate is before endDate
- - startDate is not in the past
- - there is a Hotel with Name hotelName, and it has
RoomTypes of RoomKind kind

post:
- - if available(startDate, endDate, roomType)
executed in the Hotel with hotelName returned true,
then
- - a new Reservation object, reservation, will have
been created (object creation)
- - if there is not a ReserverPayer with given
details in that Hotel, then
- - a new ReserverPayer object will be created
with attributes set to name, address,
creditCardDetails and a newly generated id
- - the new ReserverPayer object will be
linked to hotelChain
- - that Hotel will be linked to the ReserverPayer
- - that Hotel will be linked to reservation
- - the ReserverPayer will be linked to reservation
- - reservation will be linked to roomType
- - reservation will be linked to a previously free
Room
- - a string indicating the reservation number will
have been returned
- - otherwise, a string indicating failure will have
been returned

```

SAQ 11

By looking at *makeReservation*'s postcondition, identify the objects that are involved in the collaboration, and their links.

Answer

Besides the system object, there is a *Hotel* object, a *ReserverPayer* object (either pre-existing or newly created), a *RoomType* object, a *Room* object and the newly created *Reservation* object.

Fulfilling the postcondition of the operation *makeReservation* corresponds to the creation of objects and the creation of new links between objects, as you have seen in the snapshot in Figure 9. To show this, we choose to use communication diagrams rather than sequence diagrams, as the former clearly show the creation of new objects and new links with the *{new}* constraint. Sequence diagrams are better when we need to be clear about the temporal ordering of messages, although they can show object creation.

The responsibility for the fulfilment of *makeReservation* will ultimately lie with *Hotel*, as each hotel has the expertise required to achieve this operation (GRASP Expert): a hotel knows about the reservations and people who made them, and about rooms and room types.

We have already identified one operation of the class *Hotel* that is relevant to fulfil the postcondition of *makeReservation* – the operation *available*. Its operation signature is as follows:

```
context Hotel::available(startDate : Date,
endDate : Date, roomType : RoomType) : Boolean
```

We now need to discuss which class should take responsibility for the operation that creates a new *ReserverPayer*, and the operation that creates a new *Reservation*. By applying GRASP Creator, the creation of a new *ReserverPayer* should be assigned to *HotelChain*, as *HotelChain* records and holds information for all guests that ever come into contact with the hotel chain. As for a new reservation, the responsibility may be assigned to either *Hotel* or *ReserverPayer*: either of these classes may record information about a reservation. We decide to assign the responsibility to *Hotel*:

```
context HotelChain::createReserverPayer (Name,
address : Address, creditCardDetails : CreditCard)
: ReserverPayer

context Hotel::createReservation(startDate : Date,
endDate : Date, roomType : RoomType,
reserverPayer : ReserverPayer) : Reservation
```

Ultimately an object is created by a class operation, usually known as the **constructor** operation. We use the convention of naming these class operations that create new instances as *create*. The constructor operation returns the newly created instance, that is, an object identity for the newly created object. The operations *createReserverPayer* and *createReservation* will invoke the *create* class operations of *ReserverPayer* and *Reservation* respectively.

```
context ReserverPayer::create(name : Name,
address : Address, creditCardDetails : CreditCard)
: ReserverPayer
```

The class operation that creates a new *Reservation* will need to know not only the attributes of the new object, but also the object references of the objects that a new reservation should be linked to: the *ReserverPayer* and the *RoomType* for that reservation.

```

context Reservation::create(startDate : Date,
endDate : Date, roomType : RoomType,
reserverPayer : ReserverPayer) : Reservation
pre:
- - none
post:
- - a new Reservation is created and returned
- - its attributes are initialised to the given values
- - it is linked to the objects passed as parameters:
reserverPayer and roomType

```

The communication diagram of Figure 15 shows the interactions that fulfil the postcondition of *makeReservation* when *reserverPayer* is not known to the hotel, and when there is availability of the requested *roomType*. Notice the annotation of the new objects and the new links with the constraint *{new}*. Notice also the order in the messages, given by their numbering.

We have shown the need for a new link to the allocated room in the diagram, but controversially have left the details of how to achieve this as an implementation detail. Part of our rationale here is that the diagram provides sufficient detail to show our overall architectural approach.

By annotating each new link with the constraint *{new}* next to one of the ends of the link, we are implying a direction of navigation for the corresponding associations – from *Hotel* to *Reservation*, from *HotelChain* to *ReserverPayer*, from *Reservation* to *ReserverPayer*, and from *Reservation* to *RoomType*. As you will see in Exercise 8, some of these directions are quite arbitrary at the moment. The direction of the association between *ReserverPayer* and *Reservation* could be the opposite, and you may decide, at some point, that such an association should be navigated in both directions – a *ReserverPayer* being able to access its *Reservations*, and a *Reservation* being able to access its *ReserverPayer*.

There are naturally consequences to choosing the direction of navigation of an association, but these should be defined in the context of all design operations. These decisions have to do with the resulting amount of coupling and cohesion, as discussed in Unit 6.

Exercise 8

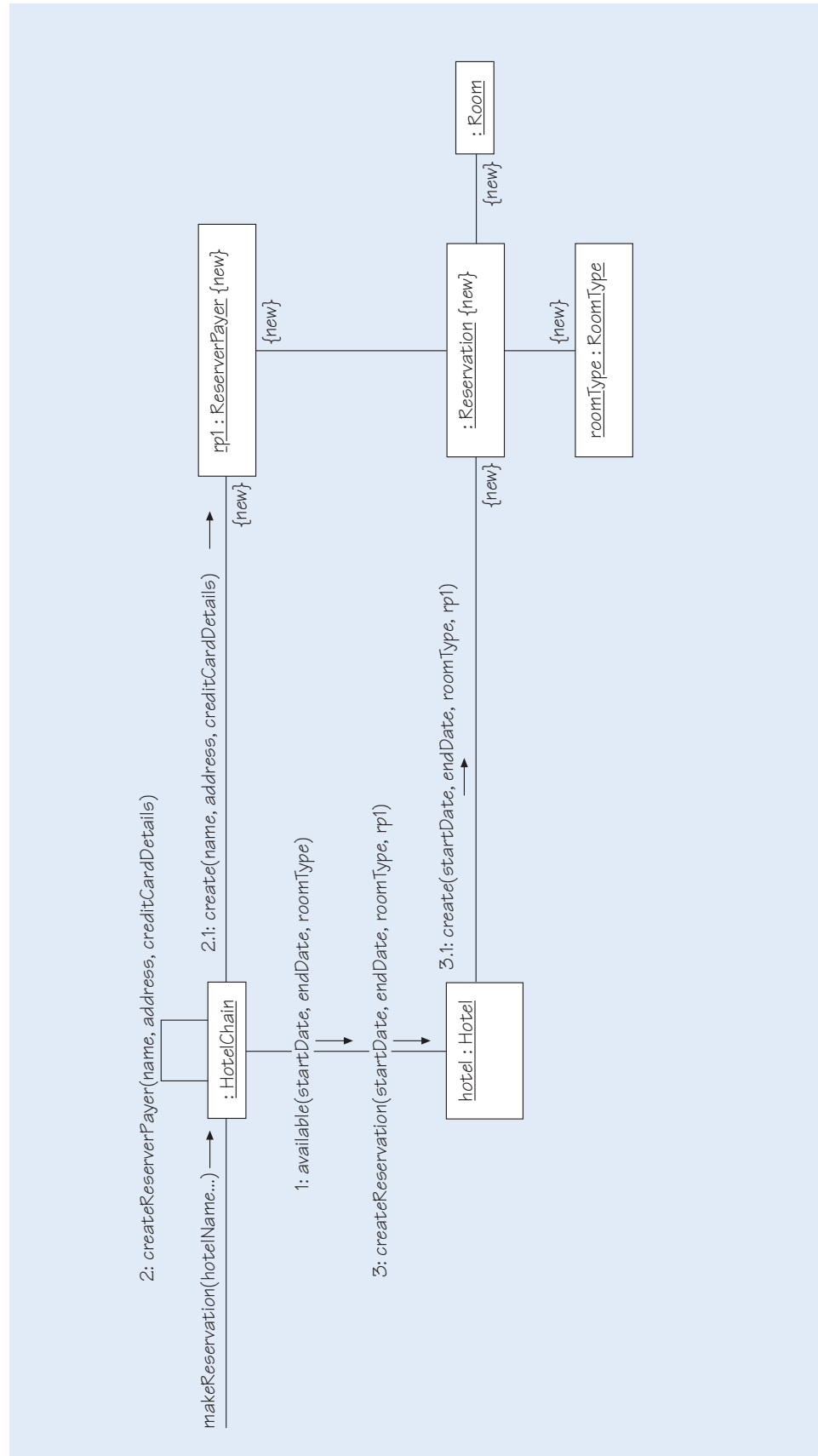
Redraw the communication diagram of Figure 15 assuming that the responsibility for creating a reservation is assigned to *ReserverPayer*.

Solution

Our solution is shown in Figure 16.

Exercise 9

Redraw the communication diagram of Figure 15 assuming that the *ReserverPayer* is known to the *Hotel*.

Figure 15 Communication diagram for the fulfilment of the postcondition of *makeReservation*

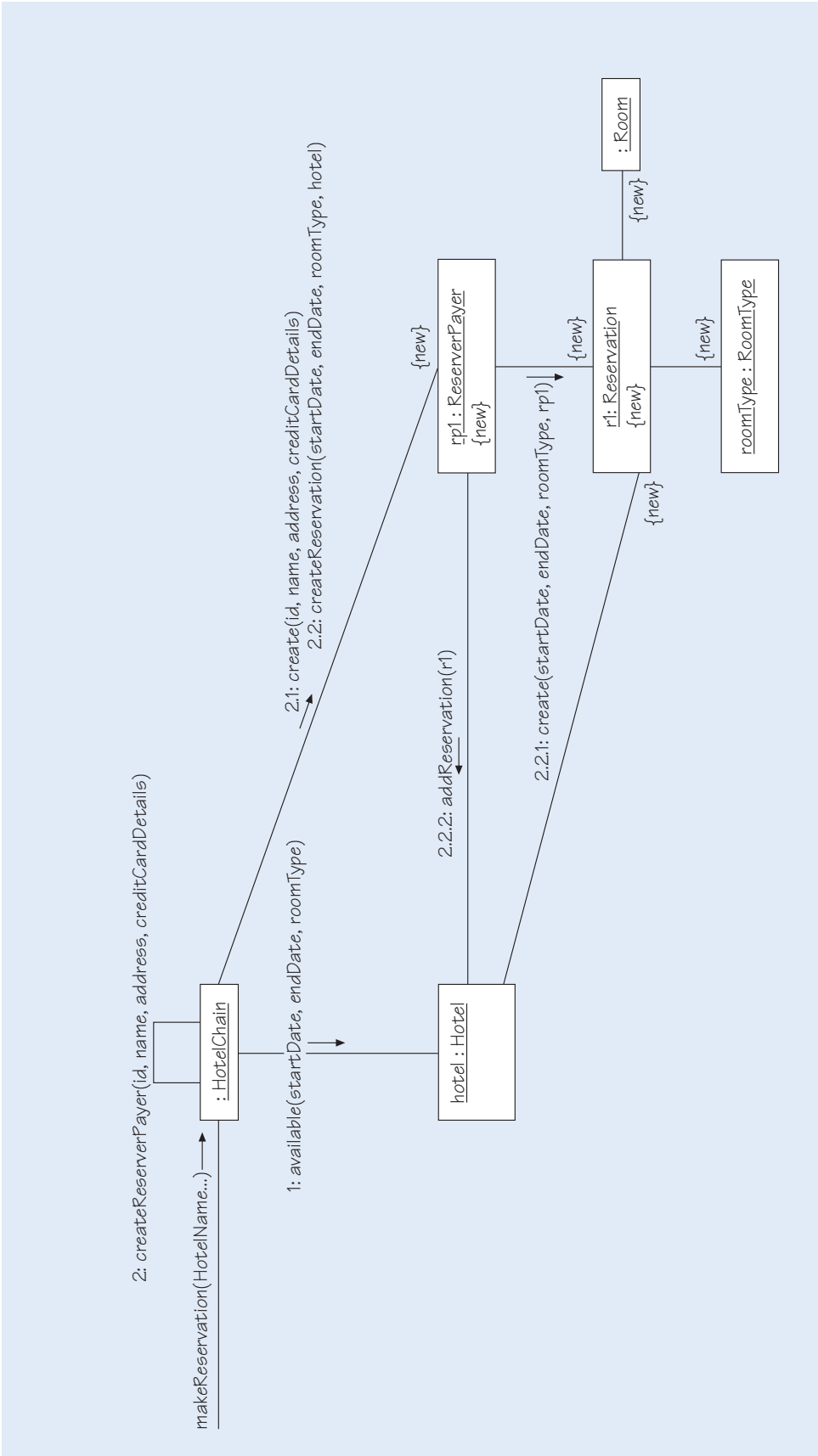


Figure 16 Revised communication diagram for the fulfilment of the postcondition of *makeReservation*

Solution

Our solution is shown in Figure 17.

Now consider the *checkInGuest* operation again:

```
context HotelChain::checkInGuest(hotelName : Name,
reservationNo : Integer, roomNo : Integer, name : Name,
address : Address)

pre:
- - the Reservation with reservationNo is linked to
the Hotel with hotelName
- - the Room with roomNo is linked to the Reservation,
and the Room is of the RoomType for the Reservation

post:
- - if name is not that of the ReserverPayer of the
Reservation with reservationNo, then a new Guest will
be created, and its attributes set; this new Guest or,
otherwise, the ReserverPayer will be linked to the Room
with roomNo
```

SAQ 12

By looking at *checkInGuest*'s postcondition, identify the objects that are involved in the collaboration.

Answer

Besides the system object, there is a *Hotel* object, either a pre-existing *ReserverPayer* object or a newly created *Guest*, a *RoomType* object, a *Room* object, and a *Reservation* object.

Exercise 10

- Which class should have the responsibility for creating a new *Guest* object, and why?
- Which new operations do you suggest should be added to fulfil the postcondition of *checkInGuest*?

Solution

- The responsibility for creating a *Guest* should be assigned to *Room*: it is the only class that is associated with *Guest*, and should therefore have this responsibility (GRASP Creator).
- The following operations need to be added:

```
Room::createGuest
Guest::create
```

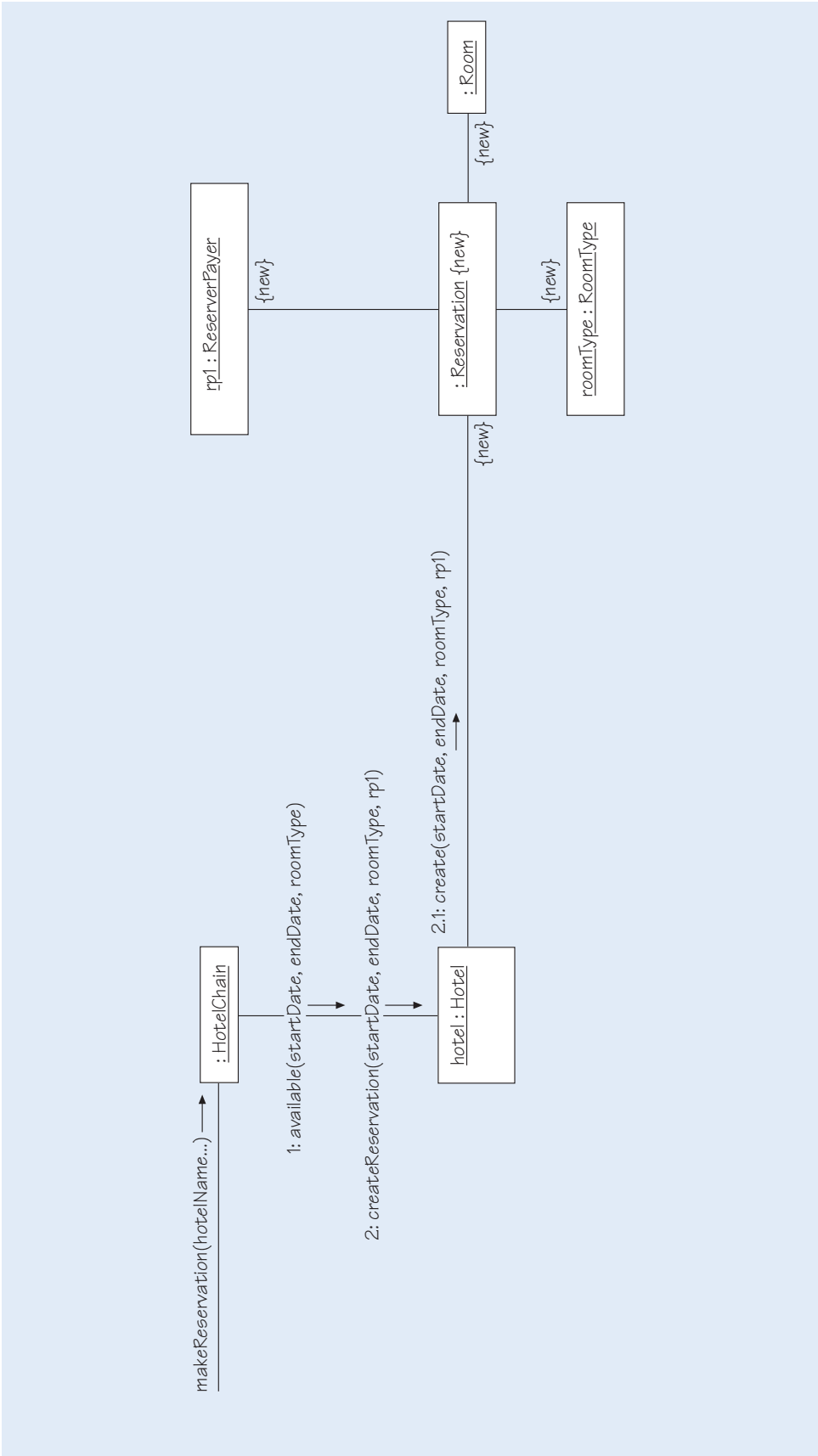


Figure 17 Revised communication diagram for `makeReservation`, assuming that the `ReserverPayer` is known to the `Hotel`

4.3 Structural design model

We can now redraw the class diagram to take into account the allocation of operations to classes that we have carried out so far, along with some tentative decisions in relation to the navigation of operations.

We need to add the operations below to Figure 14. See Figure 18, in which we have also indicated the **visibility** of operations. That is, we have used the UML notation of a + to indicate public methods and a – to indicate private methods. We need the system operations to be available to classes outside the package containing them, but where an operation is called from the same class it can be private.

HotelChain::makeReservation

HotelChain::cancelReservation

HotelChain::checkInGuest

HotelChain::checkOutGuest

HoteChain::createReserverPayer

HotelChain::canMakeReservation

HotelChain::canCancelReservation

HotelChain::canCheckInGuest

HotelChain::canCheckOutGuest

Hotel::createReservation

Hotel::available

Room::createGuest

ReserverPayer::create

Reservation::create

Guest::create

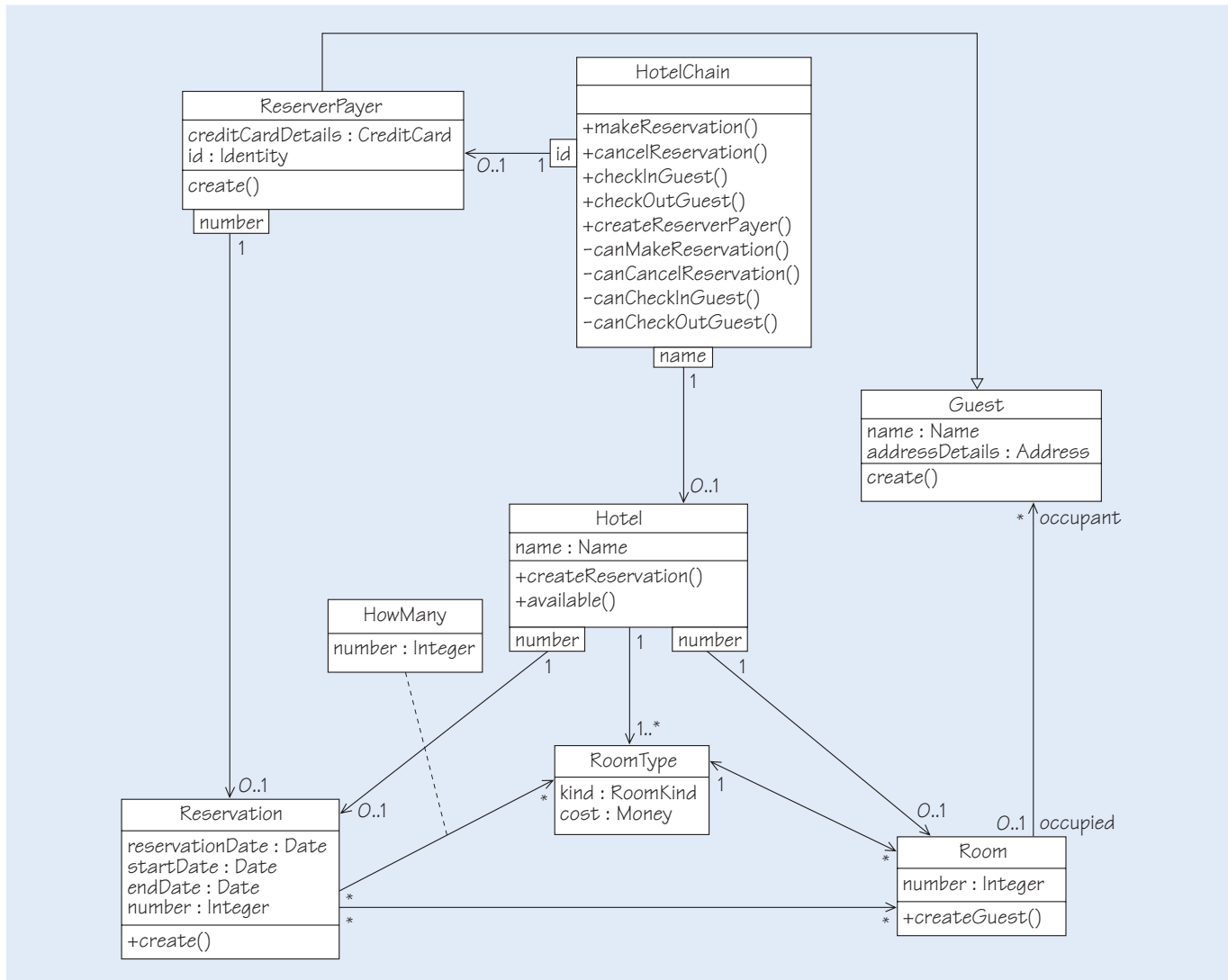


Figure 18 Structural design model with operations shown

It is interesting to consider whether we could have arrived at a similar design through the use of class-responsibility-collaboration cards, as introduced in Unit 6. Given familiarity with the domain, it is reasonable to think that we might have come up with similar operation names and allocated them to the classes in a similar way. One of the advantages of the cards is that the use of physical cards on a flat surface would allow the process to be carried out by a group, and perhaps draw on the expertise of the clients for the system.

At that stage there are two obvious possibilities. In an agile approach, we might code the system based on the cards representing the classes and user stories. This raises the question of whether the other design considerations we took into account, for example pre- and postcondition and association loop constraints, would be accounted for. This could be done during the coding process but it would require a degree of expertise on the part of the developers. Alternatively, the information on the cards could be used in

generating a class diagram, and some of the details refined using a similar approach to that taken in the main body of this unit.

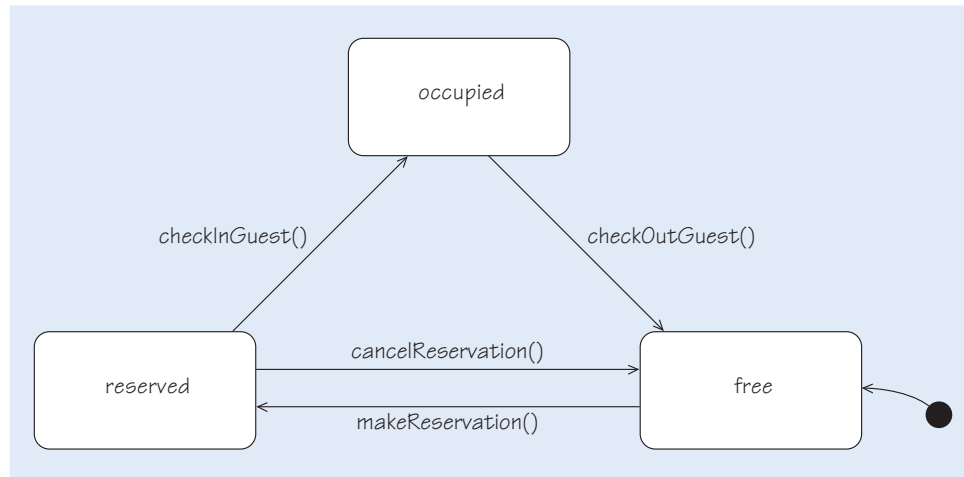
Exercise 11

We have now carried out the analysis and design activities for an increment of our system. One thing we have not considered is the state of the objects over more than one operation. What if we have a client who has worked in a hotel where the computer system managed to lose track of rooms? If they asked us to consider this problem we could think about using a state machine to model a typical room object.

- (a) Identify the states that a room might take.
- (b) Which of the operations specified in this unit would affect the state of a room?
- (c) Draw a state machine diagram to show the states of a room and how the room moves between them.
- (d) Explain why the state machine suggests that the problem mentioned above does not currently occur with the proposed design.

Solution

- (a) By looking at subsection 3.3 we can see the following:
 - ‘UC1 *make reservation*’ (Table 13) allocates a room to a reservation – let’s call this the reserved state
 - ‘UC2 *cancel reservation*’ (Table 14) frees an allocated room – let’s call this the free state
 - ‘UC3 *check in guest*’ (Table 15) allocates a room – let’s call this the occupied state
 - ‘UC4 *check out guest*’ (Table 4 in Block 1 Unit 4) also frees a room.
- (b) From (a) we can see that each of our use cases can affect the state of a room. This means that the corresponding operations allocated to the *HotelChain* class affect the state of the room. They do this using some of the other operations that we have allocated to the system. For example, *makeReservation* involves the *available* and *createReservationPayer* operations. The operations corresponding directly to the use cases are events in terms of state charts. The operations called will correspond to state machine actions, although in part (c) of this exercise we will opt to omit these from the diagram. In a more safety-critical situation we might be more formal and include them all. For now we are taking a slightly more agile approach.
- (c) See Figure 19.

Figure 19 State chart for a *Room* object

- (d) Figure 19 shows that a room moves between our three states in a sensible way. Of course this assumes we have taken proper account of the class model, use cases and their encoding as operations. Later in the development this conclusion would also be based on the assumption that the code for the implementation respects the system design. We might need to revisit the state diagram as we deal with further iterations of the system.

4.4 Summary of section

In this section you have seen how system operations were allocated to classes, and how their pre- and postconditions are fulfilled. You have also seen how navigability was added to the class diagram.

Consistency

5

In this case study we have carried out some analysis and design, to arrive at specifications for a set of operations that represent the functionality demanded by our requirements. These operations have now been assigned to appropriate classes, and some of the navigability of links that will be required has been tentatively established.

In this unit, once we arrived at an analysis class model, we carried out three main steps to decide how the requirements from the use cases could be supported by this structure.

- 1 We specified one system operation per use case (with pre- and postconditions).
- 2 Using interaction diagrams where appropriate, we designed operations to verify preconditions and fulfil postconditions of the system operations – leading to identification of finer-grained operations.
- 3 We assigned the operations to classes by revisiting the class diagram and also established navigability of links. This has led us to a design model.

In Unit 7 we considered the need for consistency. We can now think about this in the context of the case study. Given that we have followed a fairly systematic process to arrive at our design, you might believe that consistency is guaranteed. However, it is easy to make mistakes, and in a large project and under time pressure it might well be the case that operations, and indeed classes and links, get added in a more ad hoc way to ‘get the job done’.

Some consistency checks that we might apply are the following.

- 1 Do the system requirements in Block 1 Unit 4 all correspond to use case steps, and conversely is there at least one system requirement for each step?
- 2 Have we done enough at this stage to realise the use cases in our class diagram? In other words do the use cases link to the class diagram fully?
 - (a) Is there one system operation for each use case?
 - (b) Do the operations in the design class model correspond to operations identified in design either to verify preconditions or to assert postconditions of system operations? Where we built explicit dynamic models (interaction diagrams), the operations in the models should correspond to operations on the class diagram.
 - (c) Do the operations identified create only links that correspond to associations in the class diagram?
 - (d) Do the operations identified modify only attributes of the classes where they are defined, or that were passed as arguments to the operation?
- 3 We have used state machines to understand the behaviour of one of our classes. Any events in a state machine have to correspond to operations of the corresponding class.

One approach to consistency is the use of walkthroughs. For example, we can read through each step of a use case scenario to check that our design matches. This involves:

- going through the steps of the use case scenarios and the relevant operations and any corresponding dynamic models (interaction diagrams)
- following the paths of operations and associations in the design class diagram.

In this way we can check whether our design supports our use cases and revisit any decisions that have compromised consistency. There is a choice about whether we do these two steps together or separately. Early on in a development it might be helpful to work in the context of the use cases. Later on we might apply the checks independently, to reduce cost. If we find errors, we will still need to be sure we have dealt with them right across the development process.

Exercise 12

- Check that the model in Figure 15, showing the fulfilment of the postcondition of *makeReservation()*, is consistent with the class diagram in Figure 18.
- What would lead you to conclude that an interaction diagram and a class diagram were inconsistent?
- Having discovered an inconsistency, what would you do next?

Solution

- We need to consider each of the messages in the figure. Stepping through the messages on the model in Figure 15, we find six messages:

```
makeReservation(...)
1: available(...)
2: createReserverPayer(...)
2.1 create(...)
3 createReservation(...)
3.1 create(...)
```

Looking at the *makeReservation(...)* message we can see that it is sent to the *HotelChain* object, and that the *HotelChain* class exists in Figure 18 and supports the *makeReservation(...)* operation.

A similar argument applies for the message labelled 1, and the navigability indicated on Figure 15 allows a *HotelChain* object to send the message *available(...)* to a *Hotel* object. We can follow similar arguments for the remaining messages to show consistency. The new associations shown in the communication diagram are also consistent with the class diagram.

- There would be an inconsistency if we found a message in an interaction diagram being sent to a class and:
 - there was no corresponding class in the class diagram
 - or it didn't support the message

- (iii) or the message could not be sent because of navigability issues.
- (c) Given an inconsistency, we need to consider the source of the error. It could be that we have simply failed to allocate an operation in the class diagram or placed it in the wrong class. This would be easy to fix, but we need to make sure that such a fix does not lead to inconsistencies with other dynamic models. There could be a rational reason why the class diagram is the way it is and the dynamic modelling is wrong.

As well as correcting the dynamic model we should also check that it remains consistent with, for example, the use cases. If fixing one inconsistency leads to others we might like to consider whether our overall approach is sensible. Indeed, even if an inconsistency can easily be fixed it may be useful to consider if there is some wider lesson to be learned. Such a reflexive approach is often one of the differences between expert and inexperienced teams (Petre, 2009).

6

Summary

This unit has followed an iteration of domain modelling, through requirements and analysis to design. In Block 1 Unit 4 we looked into the requirements for this case study and decided on a subset for a first iteration. In this unit we proceeded by building a structural model of the domain, evolving it to a structural analysis model by taking decisions on what needs to be represented in a software solution and considering constraints. Finally we identified the system operations for the requirements chosen and carried out the allocation of responsibilities to classes taking design decisions.

On completion of this unit you should be able to:

- carry out domain modelling by identifying concepts, associations, multiplicities and attributes
- carry out analysis by taking decisions on what needs to be represented in a software solution and specifying constraints and identifying system operations
- carry out design by identifying further system operations, establishing pre- and postconditions, and assigning system operations to classes by taking design decisions.

References

Jacobson, I., Booch, G. and Rumbaugh, J. (1999) *The Unified Software Development Process*, Reading, MA, Addison-Wesley.

Larman, C. (2004) *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, Upper Saddle River, NJ, Prentice Hall. Also available online at <http://proquestcombo.safaribooksonline.com.libezproxy.open.ac.uk/book/software-engineering-and-development/uml/0131489062> (Accessed 13 May 2014).

Petre, M. (2009) 'Insights from expert software design practice' [online], *ESEC/FSE'09 Joint 12th European Software Engineering Conference (ESEC) and 17th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-17)*. The Netherlands, August 24–8. New York, ACM, pp. 233–42. Also available online at <http://oro.open.ac.uk/25994/> (Accessed 13 May 2014).

Index

Index entries in **bold** type indicate glossary entries.

access stereotype 183, 184–5

action sequences 167–8

actions 161, **167**, 172

entry and exit events and 172–4

events, transitions and 167–8

activation **99**

active objects 98, 100

activities **172**

activity diagrams 144, 172

actor classes 155–6

actors 10

additional operations 153

administrative systems 177

aggregations **38**

agile development 7, 35, 119, 180, 197, 240, 241

alt **148**

alternative fragment **148**, 149

analysis 77, 196, 207

analysis classes **194**, 207

analysis models **6**, 35–45, **194**

aggregation and composition 38

attributes on associations 42

case study 194, 207–25

constraints 207–12

specifying operations 213–24

system operations 212–13

derived associations 42–4

importance of time 36–7

interpreting attributes 35–6

navigability 39–40

qualified associations 40–1, 44–5

analysis structural models *see*

analysis models

analysis tools 80–1

application domain 184

architectural design 155

assertions **79**, 79–80

see also invariants;

postconditions; preconditions

assigning responsibilities 96–7

case study 226–42

assignment statements 115, 117

association classes **42**, 207

association loops 61–6, 67–8

case study 207–12

association names 22, 26–7

associations 11, 14, 15, 18–27

aggregation and composition 38

assumptions about meaning 23

attributes on 42

between a class and itself 25–7

bidirectional 124–5, 128–9, 132

constraints across 58–9

derived 42–4

hotel model 19, 20–1, 30–1,

103, 202–4, 212

importance of time 36–7

life history and 36–7

link manipulation 125–9

managing 124–5

multiplicities *see* multiplicities

navigability 39–40

navigation expressions 23–5, 26–7

qualified 40–1, 44–5, 227

recursive 25–6, 26–7

several associations between a pair of classes 25

unidirectional 124–5, 127, 132

assumptions about meaning 23

asterisk symbol (*) 20, 147

attributes 11, 14, 15, **18**, 168, 194

on associations 42

case study 204–6, 212

derived 43

interpreting 35–6

ballpoint pen 162

bank account model 40–1, 57

constraints across associations 59

specialisation 47–8, 48–9, 51–2

behaviour

constraints on 86–7, 88–9

of an object over its lifetime 12–13, 174–5

benefits of contracts 81, 83, 88

bidirectional associations 124–5, **128**, 128–9, 132

Boolean expressions 69, 146

branching lifelines 145–6, 150–1

business-critical systems 81

business model–user interface

communication 94–5, 152–3

business objects 152–3

business transactions **8**, 10, 198, 200

candidate concepts 8–11, 29–30

case study 198–202

car motor state machine diagram 162–3

cascade patterns 129–34, **130**

case study 189–246

analysis model 194, 207–25

constraints 207–12

specifying operations 213–24

system operations 212–13

conceptual model 193, 198–206

consistency 243–5

design model 194–5, 226–42

fulfilling postconditions 228, 231–8

structural design model 239–42

value identities 226–7

verifying preconditions 228–31

identifying concepts, associations, multiplicities and attributes 198–206

class diagrams **13**, 15–16, 240–1

consistency with interaction

diagrams 243, 244–5

hotel model 15–16, 28–9, 210–11, 226, 227, 239–40

role names 23–4

class modelling 6

class models **13**, 18

families 33–4

invoices 36

limitations 37

orchestra 32–3

video library 67

weaknesses in and sequence diagrams 105

class-responsibility-collaboration cards (CRC cards) 108–9, 240–1

classes

actor classes 155–6

associations 18–27

between a class and itself 25–7

- several associations between a pair of classes 25
- candidate concepts 8–11, 29–30, 198–202
- with complex state machines 175
- constraints on 56–8, 60–1
- convention for writing class names 14
- dividing functionality between 104
- encapsulating decisions in separate classes 104
- making a class responsible for link manipulation 128–9
- and objects 11
- one central class strategy 155, 157–8
- and properties 11
- showing how a class provides an operation 120
- subclasses 47–8, 49–50, 54–5, 57, 85–6
- superclasses 47–8, 49–50, 51
- system class 214
- and time 11–13, 32
- use case classes 156–7, 158–9
- command pattern 156**
- committee model 63
- communication diagrams 90, 91, 110–14, 116, 121, 144, 233**
 - creation and deletion 118–19
 - differences from sequence diagrams 110–11, 114
 - forks and cascades 129–31
 - lending library 121–3
- company model
 - constraints across associations 58–9
 - design decisions
 - forks and cascades 129–31, 132–3
 - link manipulation 125, 126, 128–9
 - qualified associations 44–5
- compiler 80**
 - complex constraints 65–6
 - complex interactions 144–51
 - complex state machines 175
 - complexity 57–8, 143
 - components, showing use of 120
- composition 38**
- concepts
 - hotel model 19, 27–8, 198–202, 209–10, 211
 - identifying 8–11, 27–8, 29–30, 198–202
- conceptual models 6, 120, 193**
 - case study 193, 198–206
 - see also* domain structural models
- conditional message sends 114, 144–6, 150–1
- consistency 180, 243–5
- constraining models 56–70
 - finding invariants using association loops 61–6
 - OCL operators 68–9
- constraints 56**
 - across associations 58–9
 - on behaviour 86–7, 88–9
 - case study 207–12
 - on classes 56–8, 60–1
 - on related objects 59–61
- constructor operation 233**
- contracts 78, 81, 84–7**
 - design by contract *see* design by contract (DbC)
 - enlarging a contract 82–4
 - production of quality software 79–80
 - in the real world 78, 81–4
 - subcontracting 82, 85–6
- control systems 177
- correctness 79, 175**
- creation of objects 117–19, 153
- critical systems 81
- data dictionary *see* glossary
- defects 80
- defensive programming 153, 153–4, 157, 169**
- delegation 103, 105–6
- delete* statement 118
- deletion of objects 117–19
- dependencies 57, 125
 - packages 183–4
- derived associations 42, 42–4**
- derived attributes 43
- derived elements 43
- design 77, 196, 207
 - assigning responsibilities in 96–7, 194–5, 226–42
 - issues 152–60
 - checking preconditions 153–5
 - communication between user interface and business model 152–3
 - strategies for implementing use cases 155–9
 - overall design process 180–6
- design by contract (DbC) 47, 57, 78, 78–89, 154**
 - checking preconditions 154
 - contracting, subcontracting and inheritance 84–7
 - contracts and quality of software 79–80
 - contracts in the real world 78, 81–4
 - quality improvement 80–1
- design decisions 124, 124–35**
 - constructing sequence diagrams 101–8
 - forks and cascades 129–34
 - link manipulation 125–9
 - managing associations 124–5
- design models 6, 35**
 - case study 194–5, 226–42
 - fulfilling postconditions 228, 231–8
 - structural design model 239–42
 - value identities 226–7
 - verifying preconditions 228–31
- design patterns 96–7, 120
- destroyed* constraint 118–19
- detail, levels of 119–23
- development process 195, 196**
- direction 39, 234
 - bidirectional associations 124–5, 128–9, 132
 - navigability 39–40, 124–5, 239, 240
 - unidirectional associations 124–5, 127, 132
- domain experts 6, 9, 19**
- domain structural models 6, 8–34, 196**
 - associations 11, 14, 15, 18–27

- attributes 11, 14, 15, 18
- class diagrams 13, 15–16
- class models 13, 18, 32–4
- classes and objects 11
- classes and properties 11
- classes and time 11–13, 32
- hotel model 27–9, 30–2
 - case study 193, 198–206
- identifying concepts 8–11, 27–8, 29–30
- identity 16–17
- object diagrams 13–15, 21, 23–4
- object models 13, 18, 19
- double colon 128
- dynamic analysis tools** 80, 80–1
- dynamic modelling** 78, 90, 90–109
 - assigning responsibilities in design 96–7
 - building a sequence diagram 92–5
 - class-responsibility-collaboration cards 108–9
 - design decisions *see* design decisions
 - focusing on a solution 90–2
 - sequence-diagram notation 97–100
 - see also* design by contract (DbC); interaction diagrams
- elaboration of operations within a class 174
- encapsulation 96, 104
- enlarging a contract 82–4
- entry events 172–4
- equality operator 64
- errors 171–2, 245
 - due to unexpected messages 166
 - programmer 128
 - syntax 80
- events** 161, 162–3, 175, 178
 - actions, transitions and 167–8
 - exit and entry events 172–4
 - identifying concepts 9–10, 11
- exclusive-or (*xor*) constraint 59, 61
- exit events 172–4
- external identifiers 105, 152
- external interface 212
- extra queries 159
- families model
 - class model 33–4
 - constraints on related objects 60–1
 - derived associations 42–3, 44
 - qualified associations 44–5
- filtering criteria 9–11
- final states** 164, 164–5, 166–7
- fork patterns** 129–34, 130
- formal technical reviews (FTRs)** 80–1
- fragments** 147, 147–50
- frames** (fragments) 147, 147–50
- full windows 17, 51–2
- general responsibility assignment software patterns** *see* GRASP
- generalisation** 46, 46–55, 61
 - and inheritance 51–2
 - substitutability test 46, 50–2, 53, 85
- generic instances 99, 100
- glossary** 8, 8–9
 - case study 194
 - associations 202–3, 212
 - attributes 206, 212
 - concepts 201, 202, 211
 - updating 211–12
- grammatical parse** 9
- graphical user interface (GUI) 51–2, 157
- GRASP** 96, 104, 128
- GRASP Creator 97
- GRASP Expert** 96, 96–7, 219
- guards 144–5
 - state machines 163, 168–71
- helper methods 132
- high cohesion 96, 157, 184
- hotel systems
 - association loop constraints
 - Guest, Room and Reservation* 209
 - Hotel, Room and RoomType* 208–9
 - HotelChain, Hotel, Reservation and Guest* 208
 - Reservation, Hotel and RoomType* 209
- Reservation, RoomType and Room* 209
- association loops 62
- associations 19, 20–1, 30–1, 103, 202–4, 212
- cancel reservation*
 - system operation 220–2
 - use case 193
 - verifying precondition 230
- case study *see* case study
- check in guest*
 - communication diagram 110, 121
 - fulfilling postcondition 237
 - main success scenario of the use case 101–4
 - sequence diagram 93–5, 97–107, 115–17, 158–9
 - system operation 222–4, 230–1, 237
 - use case 91–2, 93, 101, 193
 - verifying precondition 230–1
- check out guest*
 - sequence diagram 107
 - use case 91, 193
- class diagrams 15–16, 28–9, 210–11, 226, 227, 239–40
- communication diagrams 110, 121, 234–7, 238
- complex interactions 144–6
- constrained model of payments 60–1
- CRC cards 108–9
- design decisions
 - forks and cascades 133–4
 - link manipulation 126, 127–8
 - managing associations 124–5
- design issues 158–9
- domain structural model 27–9, 30–2
- Guest* class 12, 19, 27–8, 32, 103–4, 199, 200–1
- GUI 157
- Hotel* class 19, 27–8, 103–4, 199, 200–1
 - state machine 169–72
- HotelChain* class 200–1, 214, 218
- make reservation*

- fulfilling postconditions 231–7, 238
- system operation 214–20
- use case 193, 204–5
- verifying preconditions 228–30
- object diagrams 13–15
- Reservation* class 28, 200–1
- ReserverPayer* class 211, 218, 222–3, 232, 233
- role names 22–3
- room availability 218–19
- Room* class 19, 27–8, 32, 103–4, 199, 200–1
 - state machine 164, 165, 167–8, 169, 172–3
- RoomType* class 28, 29, 201–2
 - sharing across hotels 210
- sequence diagrams
 - billing guests 133–4
 - checking in a guest 93–5, 97–107, 115–17, 158–9
 - checking out a guest 107
 - conditional message sends 144–6
 - state machine 164, 165, 167–8, 169–72, 173
 - strategies for implementing use cases 155–7
- iconised windows 17, 51–2
- identity 16–17
 - value identities 226–7
- implementation class model 105
- import* stereotype 183, 184–5
- inequality operator 63, 64
- Information Expert (GRASP Expert) 96–7, 219
- infrastructure layer 180, 184
- inheritance 51–2, 60
 - subcontracting and 85–6
- initial state** 163, **164**, 165, 166–7
- instance names 98–9, 100
- interaction diagrams** **90**, 90–1, 110–23
 - complex interactions 144–51
 - consistency with class diagrams 243, 244–5
 - creation and deletion 117–19
 - design decisions *see* design decisions
 - levels of detail 119–23
 - message results and parameterised interactions 114–17
 - see also* communication diagrams; sequence diagrams
 - interfaces** **48**, 48–9
 - internal operations 120
 - internal transitions** **173**, 174
 - invariants** **56**, 57, 66, 79
 - finding using association loops 61–6
 - invoice class model 36
 - iterative message sends 147, 150–1
- Jacobson, I. 194
- Java 119, 125
- Larman, C. 18, 96, 199
- Law of Demeter** **131**, 131–2, 133–4
- lending library
 - analysis modelling 36–7
 - association loops 63–4
 - constraints on behaviour 86–7, 88–9
 - one central class strategy 157–8
 - prototypical interaction 121–3
 - qualified associations 44–5
 - specialisation 52–3
 - state machine 165–6, 178
- life histories 12, 36–7
 - state machines 163, 165–6, 175–6
- lifelines 98, 100
 - branching 145–6, 150–1
- lifetime, behaviour over 12–13, 174–5
- links** **14**, 119
 - manipulation 125–9
 - naming methods for 127–9
- loop** **148**
- loop fragment** **148**, 150
- loops
 - in associations 61–6, 67–8, 207–12
 - state machines 169–70
- low coupling 96, 157, 184
- main success scenario 101–4
- management information systems (MIS) 181–2, 184
- message number** **113**
- messages 90, 98, 99
 - conditional message sends 114, 144–6, 150–1
 - constructing a sequence diagram 101–2
 - iterative message sends 147, 150–1
 - Law of Demeter 131–2
 - message results and parameterised interactions 114–17
 - numbering 110–13, 121
 - unexpected 166
- mission-critical systems 81
- multiplicities** **15**, 19–21, **20**, 26–7
 - hotel model 20–1, 28–9, 30–1, 204
 - link manipulation 126
- multi-stage numbering system 113, 121
- mutually exclusive conditions 146, 150
- namespace** **181**, 183
- navigability** **39**, 39–40, **124**, 124–5, 239, 240
- navigation expressions**, 23–5, **24**, 26–7, 39, 60
- nested packages 182–3
- new* constraint 118–19, 234
- new* statement 117
- non-deterministic** behaviour **150**
- notes to express constraints 58–9, 60, 66
- noun identification 9–11, 29–30
- numbering of messages 110–13, 121
- Object Constraint Language (OCL)** **57**, 64
 - operators 68–9
- object diagrams** **13**, 13–15, 21, 95, 110
 - problems with association loops 67–8

- role names 23–4
 - see also* snapshots
- object models** 13, 18, 19
- objects
 - business 152–3
 - and classes 11
 - constraints on related objects 59–61
 - creation 117–19, 153
 - deletion 117–19
 - prototypical 115, 120
 - real-world 78
 - use case 156, 157, 158–9
- obligations of contracts 81–2, 83, 88
- one central class strategy 155, 157–8
- open source approach 181
- opt** 148
- optional fragment** 148
- orchestra class model 32–3
- organisational units** 8, 10, 198, 200
- outsourcing 181
- overall design process 180–6
 - packages 181–5
- packages** 155, 180, 181, 181–5
- parameterised interactions 114–17
- part-whole relationships 38
- passwords 39–40
- payments 54–5, 60–1
- petrol station conceptual model 29–30
- pick list 152, 153
- placeholder** 204
- plan-driven approach 180
- postconditions 78, 79, 153, 227–8
 - design by contract 79–80, 84–5, 85–6
 - constraints on behaviour 86–7, 88–9
 - strengthening postconditions 82–4
- hotel model
 - check in guest* use case 92, 101
 - fulfilling postconditions 228, 231–8
- system operations 213, 215–18, 219–20, 221, 224, 227–8
- sequence diagrams 93–5
- preconditions 78, 79, 227–8
 - checking 153–5
 - design by contract 79–80, 84–5, 85–6
 - constraints on behaviour 86–7, 88–9
 - weakening preconditions 82–3
- hotel model
 - check in guest* use case 92, 101
 - system operations 213, 215–18, 219, 221, 224, 227–8
 - verifying preconditions 228–31
- presentation layer 180, 184
- private methods 239, 240
- private package elements 183, 185
- procedural interactions 99, 100
- programmer errors 128
- properties, classes and 11
- protected package elements 183, 185
- prototypical interactions** 115, 115–17, 121–3
- prototypical objects** 115, 120
- public methods 132, 239, 240
- public package elements 183
- qualified associations** 40, 40–1, 44–5, 227
- quality improvement 80–1
- real-world contracts 78, 81–4
- real-world objects 78
- recursive associations** 25–6, 26, 26–7
- reflexivity 245
- related objects, constraints on 59–61
- requirements 193, 196
- requirements elicitation 8
- responsibilities, assigning *see* assigning responsibilities
- retrospective development 176
- returned value 115
- role names** 20, 21, 22–4, 60, 116
 - assumptions about meaning 23
 - using to name link-manipulation methods 127
- roles** 8, 10
 - hotel model 31, 198, 200
 - conflation of roles 209–10
- safety-critical systems 81
- self-transition** 170, 173, 174
- sequence diagrams** 90, 91–5, 97–108, 233
 - complex interactions 144–51
 - conditional message sends 114, 144–6, 150–1
 - iterative message sends 147, 150–1
 - UML fragments 147–50
 - creation and deletion 117–18
 - differences from communication diagrams 110–11, 114
 - differences between diagrams
 - drawn for use case scenarios and internal operations 120
 - making design decisions 101–8
 - notation 97–100
 - prototypical 115–17, 121–3
 - starting to build 92–5
 - time-ordering of messages 110–11, 112
- signature** 213
- snapshots** 13, 21, 110
 - specifying operations 215–16, 217, 218, 221, 223, 224
 - see also* object diagrams
- sorting algorithms 82–3
- specialisation** 46, 46–55, 61
 - interfaces 48–9
 - levels of 49–50
 - and substitutability 46, 50–2, 53, 85
 - superclasses and subclasses 47–8
- specific instances 98–9, 100
- specifications 15
 - system operations 207, 213–24
- state-dependent** behaviour 162, 177
- state-independent** behaviour 177
- state machines** 161, 161–79
 - case study 241–2
 - diagrams 163, 164, 165–6, 167, 170, 172–3, 177, 241–2

- entry and exit events and actions 172–4
- events, actions and transitions 167–8
- examples 162–8
- exceptions 171–2
- final states 164–5, 166–7
- guards 163, 168–71
- initial state 163, 164, 165, 166–7
- when to use them 174–7
- statecharts 161**
- states 161**, 162–3
 - final 164–5, 166–7
 - initial 163, 164, 165, 166–7
- static analysis tools 80**, 81
- stereotypes 51
- structural models 6–7, 90
 - analysis models *see* analysis models
 - design model 239–42
 - domain models *see* domain structural models
- subclasses 47**, 47–8, 57, 85–6
 - reasons for including 49–50, 54–5
- subcommunications 120
- subcontracting 82, 85–6
- subroutine semantics 100
- subset* constraint 58
- substitutability 46**, 50–2, 53, 85
- superclasses 47**, 47–8, 49–50, 51
- synchronous messages 98, 99
- syntax errors 80
- system class 214**
- system operations 194**, 207, 212–13
 - specifying 207, 213–24
 - structural design model 239–42
- tangible objects 8**, 10, 198, 199
- telephone state machine 176–7
- testing 80–1, 212–13
- three-layered architecture 184
- time
 - behaviour over 12–13, 174–5
 - classes and 11–13, 32
 - importance and analysis models 36–7
 - ordering of messages in interaction diagrams 110–14
- traceability 81, 155, 156
- transactions, business 8, 10, 198, 200
- transitions 162–3
 - events, actions and 167–8
 - five parts of 178–9
 - guards 163, 168–71
 - internal 173, 174
 - self-transitions 170, 173, 174
- trawling 8
- UML 35
 - naming associations 22–3
 - notation for multiplicities 20–1
 - packages 182, 183
 - state machines 163
 - whole–part relationships 38
- UML fragments 147–50
- unconditional interaction diagrams 114–15, 144
- unexpected messages 166
- unidirectional associations 124–5, 127, 132
- Unified Process (UP) 193
- use case modelling 6
- use case classes 156–7, 158–9
- use case objects 156**, 157, 158–9
- use case realisation 90**
- use case scenarios 120
- use cases 9, 79, 90, 91–2
 - constructing sequence diagrams 92–5, 97–108
 - strategies for implementing 155–9
 - system operations for 212–24
- user interface 95, 105
 - communication with business model 94–5, 152–3
- user stories 9
- value identities 226–7**, **227**
- video library 67–8
- visibility 239**
- walkthroughs 244
- whole–part relationships 38
- windowing systems 17, 51–2
- xor* constraint 59, 61

